
A HANDS-ON GUIDE FOR ABSOLUTE BEGINNERS

Build a Learning Management System from Scratch

Django REST Framework on the back, React on the front.
Every file explained, every command spelled out.

Python 3.10+

Django 5.2.7

Django REST Framework 3.15.2

Simple JWT 5.4.0

MySQL / MariaDB

Node 18+

React 19

Vite 8

Tailwind CSS 4

React Router 7

Who this is for. Someone who has never written a line of Django or React. Every term is explained the first time it appears, and every command is one you can copy and run.

What you end up with. A working web app where an admin hands out accounts, teachers build courses and grade work, students read and hand work in, and the server refuses anything the person signed in is not allowed to do.

How to use it. Read in order. Type the code yourself rather than pasting it. When something breaks, Chapter 14 lists the real error messages and what each one means.

Contents

Chapter 0 What You Are Building

What an LMS is	Tools you need
What you will build	How to read this book
The feature list	What this app deliberately does NOT do
The two halves	What you have now
The nine things the app stores	

Chapter 1 Setting Up Your Computer

1.1 What a terminal is	1.9 Installing the Python packages
1.2 What you are installing, and why	1.10 When mysqlclient refuses to install
1.3 Installing on macOS	1.11 Creating the database
1.4 Installing on Windows	1.12 Creating the Django project
1.5 Installing on Debian, Ubuntu and Mint	1.13 What got created, file by file
1.6 Checking that it all worked	1.14 Checkpoint: the rocket page
1.7 Making the folder layout	1.15 About the frontend folder
1.8 The virtual environment	What you have now

Chapter 2 Wiring Up Django: settings.py

2.1 What settings.py actually is	2.10 ROOT_URLCONF, TEMPLATES, WSGI_APPLICATION
2.2 Creating the project from nothing	2.11 DATABASES and environment variables
2.3 The top of the file: docstring, imports, BASE_DIR	2.12 AUTH_PASSWORD_VALIDATORS
2.4 SECRET_KEY, DEBUG, ALLOWED_HOSTS	2.13 Time: TIME_ZONE and USE_TZ
2.5 INSTALLED_APPS: what an app is	2.14 Email, password reset, static files, and the auto field
2.6 MIDDLEWARE and why the order matters	2.15 lms/urls.py: the two front doors
2.7 CORS_ALLOWED_ORIGINS and the same-origin rule	2.16 Start it up
2.8 REST_FRAMEWORK: locked by default	What you have now
2.9 SIMPLE_JWT: tokens, and why eight hours	

Chapter 3 The Database: models.py, Migrations and Django Admin

What a model is	Model 7: Assignment
Where models.py lives	Model 8: Submission
The two imports at the top	Model 9: Results
Model 1: Profile	Every field type you have met
Model 2: Teacher	Migrations
Model 3: Student	The four real migrations in this project
Model 4: Course	admin.py: a free web interface for your tables
on_delete=models.CASCADE, and why the delete dialog says what it says	createsuperuser, and the trap that catches everybody
Model 5: Enrollment	Try the models from the shell
Model 6: Lesson	What you have now

Chapter 4 Turning Rows Into JSON: serializers.py

First: what is JSON?	LoginSerializer
What a serializer does	ProfileUpdateSerializer
Create the file	ChangePasswordSerializer
The imports at the top	The password reset pair
ModelSerializer or Serializer?	How a validation error reaches the user
The eight resource serializers	What you have now
RegisterSerializer	

Chapter 5 The Endpoints: views.py and urls.py

What a view is	The finished urls.py
Where the URLs live: two files, not one	The hand-written views
HTTP methods, for people who have never met them	Every endpoint in one table
The generic views: eight resources without writing eight times	Trying it without the frontend
	The browsable API in a browser
	What you have now

Chapter 6 Who Can Do What: Roles and permissions.py

6.1 What a permission class actually is	6.9 SubmissionWrites and the one exception
6.2 Step one: give every account a role	6.10 Attaching the classes to the views
6.3 Step two: create permissions.py	6.11 The permission table, built from the code
6.4 role_of(user), line by line	6.12 What a refusal looks like on the wire
6.5 SAFE_METHODS: reading is open	6.13 The frontend copy: permissions.js
6.6 IsAdmin: the strictest class	6.14 Hands on: get yourself a 403
6.7 RoleWritePermission: the shared base	6.15 Keeping the two files in step
6.8 AdminWrites and TeachingStaffWrites	What you have now

Chapter 7 Starting the React App

What React is	src/main.jsx, line by line
What Vite is, and why it is not part of React	src/index.css and utility classes
Creating the app for real	JSX, and the three rules beginners trip on
Installing the rest	eslint.config.js
The npm scripts	The finished folder layout
vite.config.js	Checkpoint: get it running
index.html and the idea of a single page app	What you have now

Chapter 8 Talking to the Backend: api.js

Why every network call lives in one file	request(), line by line
BASE_URL and the frontend environment variable	The auth calls
Remembering who is logged in: localStorage	The resource() factory
Making Django's errors readable: readError()	flash.js: short messages that clean up after themselves
handleExpiredToken() and a custom browser event	The one message that has to survive a redirect
A short detour: Promises and async/await	Two things worth knowing about localStorage
	What you have now

Chapter 9 Signing In: Auth Context, Protected Routes and the Login Page

9.1 The problem: everybody needs to know who is signed in	9.6 Sidebar.jsx: the menu and the hole the pages fall into
9.2 auth.js: the box and the hook	Collapsing the sidebar
9.3 AuthContext.jsx: filling the box	9.7 Login.jsx: the form
9.4 ProtectedRoute.jsx: the bouncer	9.8 What happens when you press Log in
9.5 App.jsx: the route table	What you have now

Chapter 10 The Reusable Pieces: the components Folder

What a component actually is	Checkbox: a tick box with its label attached
Props: how a page tells a component what to say	Select: the dropdown
children: the stuff between the opening and closing tags	Textarea: the multi-line box
The rest spread pattern	Table: the shell, not the contents
The password box, and the eye button	ConfirmDialog: the pop-up that replaces window.confirm
styles.js: two strings that live in one place	ProtectedRoute
The looks object: one component, several appearances	index.js: the barrel file
IconButton: a button that is only an icon	The whole folder at a glance
Alert: the red and green banners	What you have now

Chapter 11 The Page Pattern, Built Once: Students

What already exists before we start

Create the file

The imports

The map at the top of the file

Step 1 - state

Step 2 - read the list when the page opens

Step 3 - reload()

The three form helpers

Step 4 - save

Step 5 - delete, in two halves

Step 6 - the JSX

Prove it works

The same page again, with different nouns

When the row only carries an id

The six steps, as a recipe

The eight pages at a glance

What you have now

Chapter 12 The Other Pages

12.1 Courses: one dropdown, and an id that has to become a name

12.2 Enrollments: two dropdowns, and a date you cannot type

12.3 Lessons: the shortest page in the book

12.4 Assignments: the two genuinely hard bits

12.5 Submissions: where "can add" and "can edit" split apart

12.6 Results: grading each submission exactly once

12.7 Dashboard: eight numbers, and where they come from

12.8 Accounts: creating logins, admin only

12.9 Profile: your own account, in two halves

12.10 NotFound: the route that catches everything else

What you have now

Chapter 13 The Whole Codebase, File by File

The chapter map used in the tables

The whole tree

Repo root

backend/

backend/lms/

backend/backend/

backend/backend/migrations/

frontend/

frontend/src/

frontend/src/components/

frontend/src/pages/

Two things on disk that the app never reads

Follow one click end to end: adding a teacher

A shorter trace: logging in

Glossary

What you have now

Chapter 14 Running It, Breaking It, Fixing It

14.1 The three things that must be running

14.2 Getting into a database that has nothing in it

14.3 A guided first run: filling the app with data

14.4 Troubleshooting: real errors and real fixes

14.5 Common beginner mistakes with this exact project

14.6 Where to go next

What you have now

What You Are Building

What an LMS is

LMS is short for **Learning Management System**. Think of a school office turned into software. In a real school office there is a filing cabinet with a drawer for teachers, a drawer for students, a drawer for courses, a register of who signed up for what, a shelf of lesson notes, a tray of homework that has been handed in, and a mark book with scores in it. An LMS is that filing cabinet, except it lives on a computer, several people can open it at the same time, and it will not let the wrong person change the wrong drawer.

Three kinds of people use the one you are about to build:

- An **admin** runs the place. They hand out login accounts, and they are the only ones who can add, change or remove teacher and student records.
- A **teacher** looks after teaching. They create courses, write lessons, set assignments, enrol students, and mark work.
- A **student** can read everything in the system and hand work in. They cannot change anything else.

The app keeps track of nine kinds of record: profiles, teachers, students, courses, enrollments, lessons, assignments, submissions, and results. You will meet all nine properly later in this chapter.

NOTE

This app is a teaching project, not a product you would sell to a school. Some things are deliberately left out so the code stays small enough to read in one sitting. There is an honest list of those at the end of this chapter, so you know from day one what you are and are not building.

What you will build

The finished app is a website with a sidebar down the left and a page on the right. Here is every screen, in the order the sidebar lists them.

THE LOGIN SCREEN

This is the first thing anybody sees. It is a small white card in the middle of a light grey page, with a graduation-cap icon, the words "Log in to LMS", a **Phone number** box, a **Password** box, and one indigo **Log in** button.

The surprise, and it catches everyone the first time: **you log in with a phone number, not a username and not an email address**. That is how the backend was written. It looks you up by the phone number stored on your profile. Under the button there is a line of small grey text that says "No account? Ask an admin to make you one." There is no sign-up link, because there is no public sign-up.

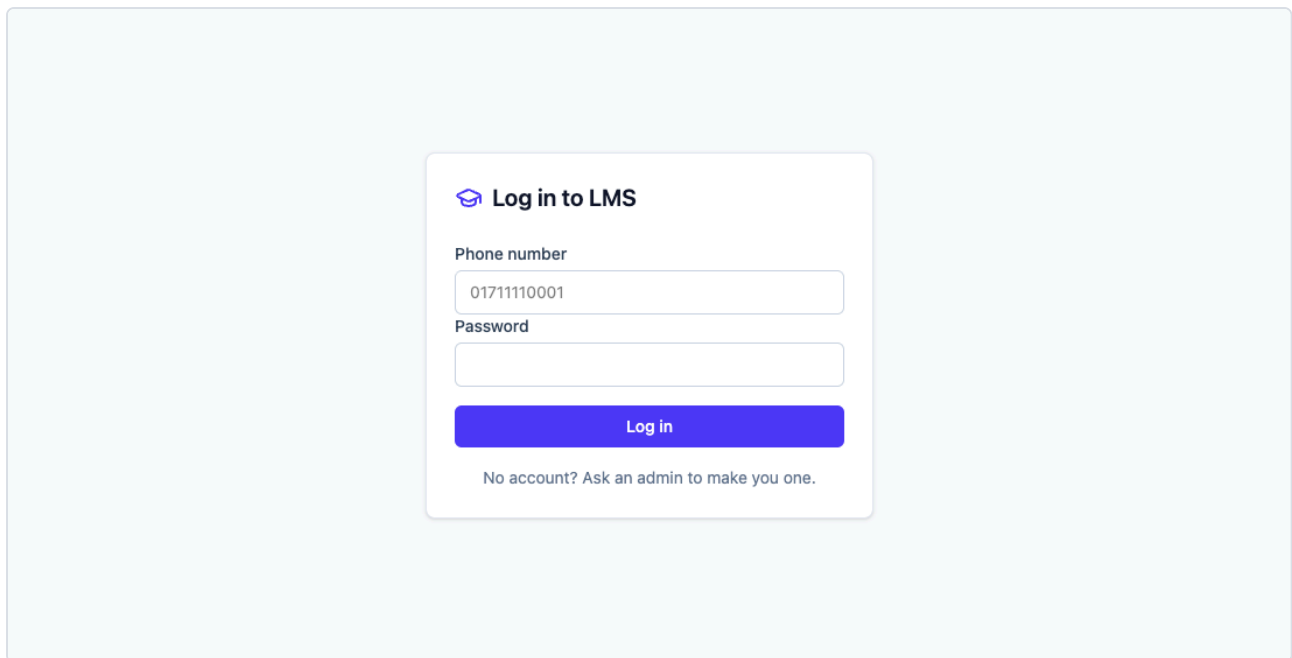
The image shows a login form titled "Log in to LMS" with a graduation cap icon. It contains two input fields: "Phone number" with the value "01711110001" and "Password" which is empty. Below the fields is a blue "Log in" button. At the bottom, there is a link that says "No account? Ask an admin to make you one."

Figure 0.1 The login screen. Phone number and password only. No sign-up link, because accounts are handed out by an admin.

If the phone number or the password is wrong, a red banner appears above the form with the sentence the server sent back: `Invalid phone or password`. The server deliberately does not say which of the two was wrong.

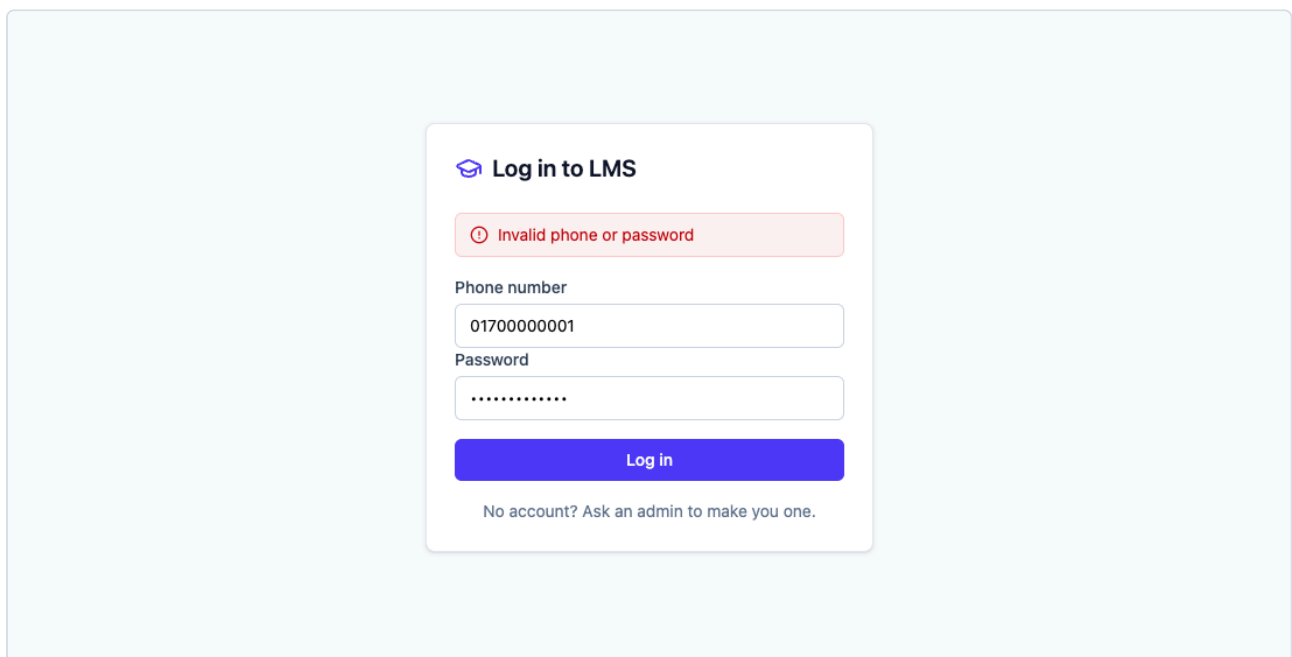
The image shows the same login form as Figure 0.1, but with a red error banner at the top that says "Invalid phone or password" with a warning icon. The "Phone number" field now contains "01700000001" and the "Password" field contains a series of dots. The "Log in" button and the bottom link remain the same.

Figure 0.2 The same screen after a failed attempt. One red banner, one message, and no clue about which half was wrong.

THE DASHBOARD

Once you are in, you land on the dashboard at `/`. The heading greets you by username ("Welcome back, nadia") and under it sits a grid of eight tiles: Teachers, Students, Courses, Enrollments, Lessons, Assignments, Submissions, Results. Each tile shows one big number, which is how many of that thing exist right now. Clicking a tile takes you to that page.

There is no "give me the counts" address on the backend, so the dashboard fetches all eight lists at once and counts them in the browser. While that is happening each tile shows a dash.

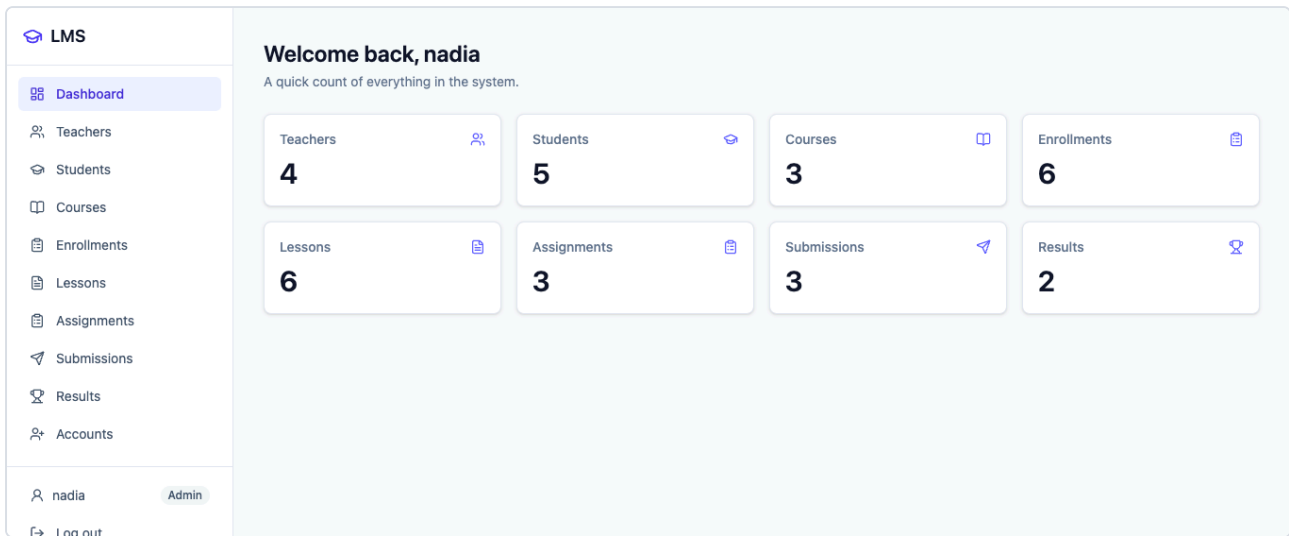


Figure 0.3 The dashboard as an admin. Eight tiles, one per record type, each showing a live count. Clicking a tile opens that page.

THE EIGHT RECORD PAGES

These are the heart of the app. They live at `/teachers`, `/students`, `/courses`, `/enrollments`, `/lessons`, `/assignments`, `/submissions` and `/results`. Every one of them is built the same way, which is good news for you: learn one and you have learned eight.

Each page has, top to bottom:

1. A title and a one-line subtitle. On the Teachers page those read "Teachers" and "The people who teach courses."
2. A red banner when something goes wrong, showing the exact message the server sent.
3. A green banner when something worked, saying for example `Teacher added.`, which clears itself after four seconds.
4. A card holding the table. Its header says how many rows there are ("3 teachers") and carries the **Add teacher** button.
5. A form that stays hidden until you press that button. The same form is reused for editing.
6. The table itself, with an **edit** (pencil) and **delete** (trash) button in the last column of each row.
7. A confirmation dialog that appears before any delete and names the row you are about to remove.

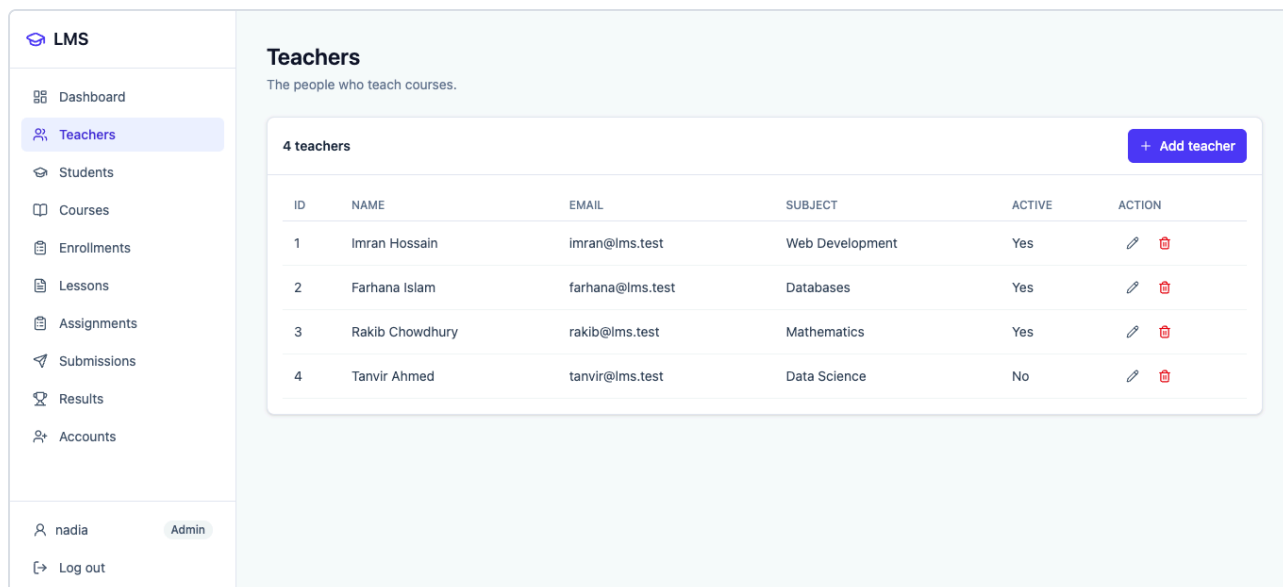


Figure 0.4 The Teachers page. Title, subtitle, a count in the card header, the Add teacher button, and one row per teacher with pencil and trash buttons.

Here is what each page holds. The last column of every table is headed "Action" and holds the two buttons.

PAGE	TABLE COLUMNS	WHO CAN CHANGE IT
Teachers	ID, Name, Email, Subject, Active, Action	admin
Students	ID, Name, Email, Roll no., Enrolled, Active, Action	admin
Courses	ID, Title, Description, Teacher, Action	admin, teacher
Enrollments	ID, Student, Course, Enrolled, Action	admin, teacher
Lessons	ID, Title, Description, Course, Action	admin, teacher
Assignments	ID, Title, Course, Lesson, Due, Action	admin, teacher
Submissions	ID, Assignment, Student, Content, Submitted, Action	admin, teacher (a student may add one)
Results	ID, Submission, Score, Feedback, Action	admin, teacher

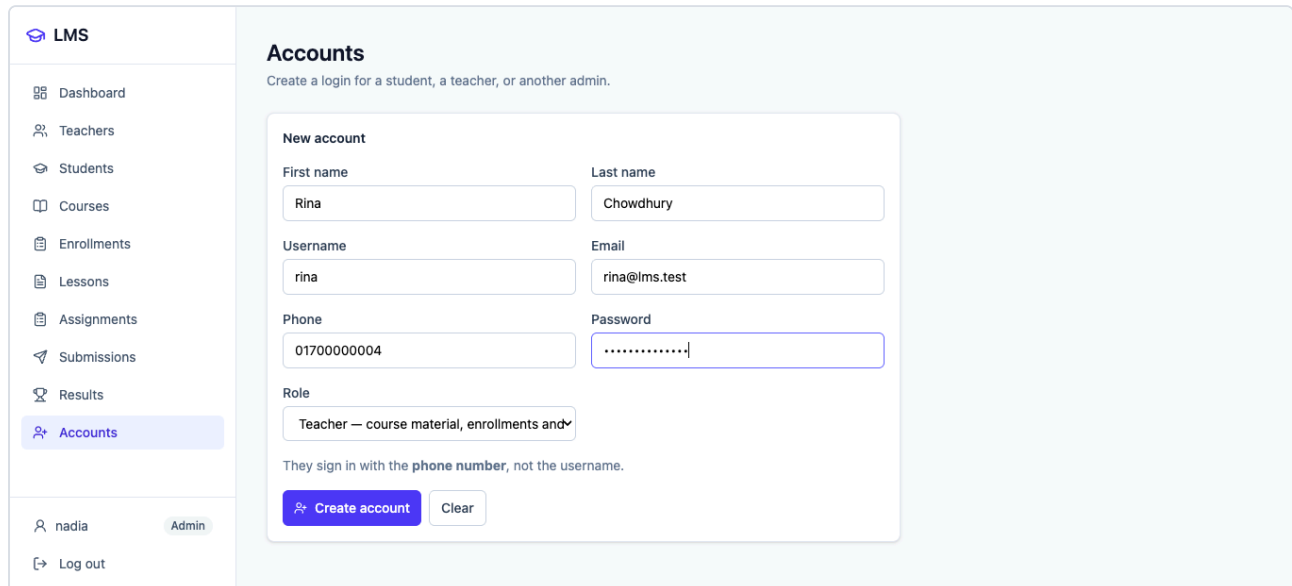
Two of the eight do a little extra work. **Assignments** narrows its Lesson dropdown to only the lessons belonging to the course you picked, and it converts the date you type into UTC before sending it. UTC is Coordinated Universal Time, the single world clock the server keeps every date in, so that two people in two countries mean the same moment. **Results** leaves out any submission that already has a result, because a submission can only be graded once.

ACCOUNTS (ADMIN ONLY)

At `/accounts` an admin creates login accounts: first name, last name, username, email, phone, password, and a **Role** dropdown with three options spelled out in plain words. After it saves, a green banner says something like "Account created for rina as a teacher. They log in with the phone number

01711110002."

This link is not even drawn in the sidebar for a teacher or a student. If one of them types the URL by hand the app sends them straight back to the dashboard. If they try to call the API by hand, the server refuses. **API** is short for Application Programming Interface, and here it just means the set of web addresses the backend answers on, meant for programs rather than people. The refusal comes back as a **403**, the code a web server uses for "you are signed in, but you are not allowed to do this".



The screenshot shows the LMS Accounts page. On the left is a sidebar with a menu: Dashboard, Teachers, Students, Courses, Enrollments, Lessons, Assignments, Submissions, Results, and Accounts (highlighted). Below the menu, it shows the user 'nadia' with an 'Admin' role and a 'Log out' button. The main content area is titled 'Accounts' with a subtitle 'Create a login for a student, a teacher, or another admin.' Below this is a 'New account' form. The form has fields for First name (Rina), Last name (Chowdhury), Username (rina), Email (rina@lms.test), Phone (01700000004), and Password (masked with dots). There is a Role dropdown menu currently set to 'Teacher — course material, enrollments and...'. Below the form, a note states 'They sign in with the phone number, not the username.' At the bottom of the form are 'Create account' and 'Clear' buttons.

Figure 0.5 The Accounts page. One form, a Role dropdown, and a green banner after a successful save telling the admin which phone number the new person logs in with.

YOUR PROFILE

At `/profile`, available to everyone whatever their role, there are two side-by-side forms. The left one edits your own first name, last name, email and phone. The right one changes your password, and it asks for your current password first. Changing your password signs you out on purpose and sends you back to the login screen with a green message explaining why.

Your username and your role are shown but cannot be edited. An admin sets those.

Figure 0.6 The profile page. Your details on the left, change password on the right, and a line saying your username and role are set by an admin.

THE 404 PAGE

Any URL the app does not recognise shows a small bordered card with a large "404" and the words "Page Not Found".

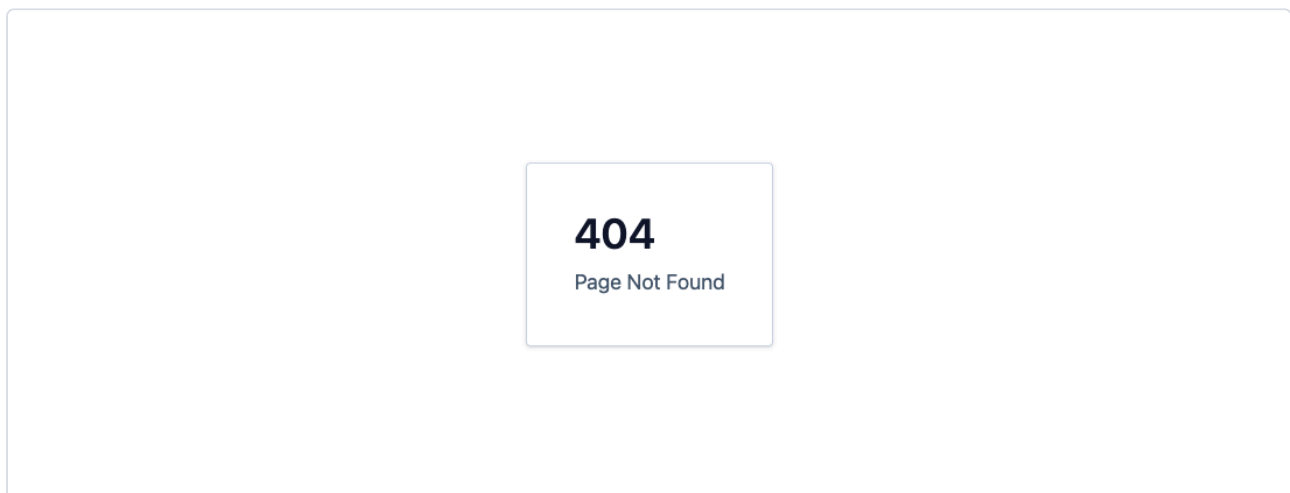


Figure 0.7 The 404 page. A bordered card, nothing else.

TIP

The whole app is thirteen page files in `frontend/src/pages/`. If you ever get lost, open that folder. `Login.jsx`, `Accounts.jsx`, `Profile.jsx`, `Dashboard.jsx`, `NotFound.jsx`, and the eight record pages. That is the entire user interface.

The feature list

Every item below is real and works in the code you are about to write. Nothing here is aspirational.

ACCOUNTS AND LOGIN

- Log in with a phone number and password. The server finds you through the phone number stored on your profile.
- Wrong details give one red message, `Invalid phone or password`, that does not reveal which half was wrong.
- Logging in hands your browser a **token**, a long string of characters that proves who you are. Every later request carries it.
- The token and your basic details are saved in the browser, so a page refresh does not log you out.
- Every page inside the sidebar is behind a guard. No token means an instant redirect to `/login`, and the page you wanted is remembered so you land there after logging in.
- An admin creates accounts on the Accounts page and picks the role at the same time.
- The sidebar shows your username with your role on a small grey pill next to it. That name is also the link to your own profile.
- A **Log out** button at the bottom of the sidebar throws the saved token away.
- Your profile page edits your own first name, last name, email and phone.
- Your profile page changes your own password, and it demands the old password before it will.

RECORDS

- Add, edit and delete teachers: name, email, subject, and an active yes/no.
- Add, edit and delete students: name, email, roll number, enrollment date, and an active yes/no.
- Add, edit and delete courses, each pointing at one teacher chosen from a dropdown.
- Add, edit and delete enrollments, joining one student to one course. The date is stamped by the server, so it appears in the table but never in the form.
- Every table shows the teacher's or student's real name rather than a bare number, because the page loads the other list too and joins the two together in JavaScript.
- The dashboard counts all eight lists in one go.

COURSE CONTENT

- Add, edit and delete lessons, each belonging to one course.
- Add, edit and delete assignments, each belonging to one course and one lesson, with a due date and time.
- On the Assignments form, choosing a course shrinks the lesson dropdown to that course's lessons only. Changing the course also clears a lesson that no longer fits.
- Due dates are stored in UTC and shown to you in your own local time.

GRADING

- Students hand in work on the Submissions page: pick the assignment, pick the student, type the answer.

- The time it was handed in is stamped by the server.
- Teachers and admins grade on the Results page: pick a submission, give it a score and written feedback.
- The submission dropdown only offers work that has not been graded yet.
- If you somehow grade the same submission twice, the page turns the server's raw complaint into a readable line: "That submission has already been graded. Edit its result instead."

SAFETY RAILS

- Three roles, with a fixed table of who may change what, checked on the server for every write.
- A refusal comes back as a 403 with a sentence explaining it, such as "Only an admin can change teacher and student records.", and the app shows that exact sentence in the red banner.
- The frontend hides buttons a role cannot use, so a student sees a dash instead of the pencil and trash icons. This is only politeness. The server is what actually stops them.
- Django REST Framework is set up so that a new address is locked by default. You have to opt a view out of that on purpose.
- Deleting always goes through a confirmation dialog that names the row and warns what else goes with it ("Their courses go too"). It closes on Cancel, on the Escape key, or on a click outside the card.
- A rejected or expired token clears the saved session, and the guard on the page sends you back to the login screen rather than leaving you on a blank one.
- The profile endpoint only ever edits the account the token belongs to, and its form on the server has no `role` field at all, so sending one is ignored. You cannot promote yourself.
- The backend only accepts browser requests from a short list of addresses: ports 5180 and 5173, on both `localhost` and `127.0.0.1`.

NOTE

The word **endpoint** means one address on the server that does one job.

`/api/teacher/` is an endpoint. `/api/login/` is another. You will build twenty-two of them.

The two halves

This project is two separate programs that talk to each other over the network. They live in two folders side by side:

```
lms/
  backend/    Django + Django REST Framework + MySQL
  frontend/   Vite + React + Tailwind + lucide-react
```

The **frontend** is what you see. It runs inside the web browser. It draws buttons and tables and forms, and it reacts when you click. It holds no data of its own for longer than the page is open.

The **backend** is what remembers. It runs on a server, it owns the database, and it decides what is allowed. It draws nothing.

The restaurant version: the frontend is the dining room and the waiter, the backend is the kitchen, and the database is the walk-in fridge. You never walk into the kitchen. You tell the waiter what you want, the waiter writes it down and carries it through, and food comes back. That is the whole idea, and it is the last time this book will mention restaurants.

The thing that carries messages between the two is **HTTP**, the same set of rules your browser uses to fetch any web page. A message going in is called a **request**. A message coming back is called a **response**. The requests here do not carry web pages, they carry **JSON**, which is just text arranged in the shape of a list or a lookup table, for example:

```
{"id": 1, "name": "Rina", "email": "rina@example.com", "subject": "Physics", "is_active": true}
```

Here is exactly what happens when you open the Teachers page and the table fills up:

BROWSER
(React, port 5180)

SERVER (Django, port 8001)

pages/Teachers.jsx

useEffect() runs on open

|

v

api.js

teachersApi.list()

|

v

fetch("http://127.0.0.1:8001/api/teacher/")

with header Authorization: Bearer <token>

|

|===== HTTP request =====>

backend/urls.py

path('teacher/', ...)

|

v

backend/views.py

TeacherListCreateView

(permission check happens here)

|

v

backend/serializers.py

TeacherSerializer

|

v

backend/models.py

Teacher.objects.all()

|

v

MySQL -> table backend_teacher

|

| rows

v

Model objects

|

v

Serializer turns them into JSON

|

<===== HTTP response ===== |

|

v

api.js returns a plain JavaScript array

|

```
    v
    setTeachers(...) -> React redraws the table
```

Read that diagram once now and again once you have the backend working. Every single feature in this app follows that path. The only things that change are the URL, the view, the serializer and the model.

NOTE

The two halves run as two separate programs on two different port numbers. Django on `http://127.0.0.1:8001`, the React dev server on `http://localhost:5180`. A dev server is a small program that serves your app to the browser while you are still working on it. You will need both running at once, in two terminal windows, plus MySQL. Three things, every time.

COMMON MISTAKE

Starting only the React side, seeing the login page appear, and assuming everything is fine. The login page is drawn entirely in the browser, so it renders happily with no backend at all. You only find out when you press **Log in** and get the red banner "Could not reach the server. Is Django running on `http://127.0.0.1:8001?`". That message is written into `api.js` for exactly this moment.

The nine things the app stores

All nine live in one file, `backend/backend/models.py`. A **model** is a Python class that describes one table in the database. One class, one table, one kind of record.

MODEL	ONE LINE
Profile	A login account's extra details: its phone number, which is how you log in, and its role (admin, teacher or student).
Teacher	A person who teaches: name, email, subject, and whether they are still active.
Student	A person who studies: name, email, roll number, the date they enrolled, and whether they are still active.
Course	A subject being taught: a title, a description, and the one teacher who runs it.
Enrollment	One student joined to one course, with the date it happened.
Lesson	One piece of a course: a title, a description, and the course it belongs to.
Assignment	Work that has been set: a title, a description, the lesson and the course it belongs to, and a due date and time.
Submission	Work a student handed in for one assignment: the text they wrote, and when it arrived.
Results	The mark for one submission: a score and written feedback. One per submission, never two.

Here is one of the nine, copied straight out of the file, so you can see how small a model really is:

backend/backend/models.py

```
class Course(models.Model):
    title = models.CharField(max_length=100)
    description = models.TextField()
    teacher = models.ForeignKey(Teacher, on_delete=models.CASCADE)

    def __str__(self):
        return self.title
```

Three fields and a name. `CharField` is short text with a length limit. `TextField` is long text with no limit. `ForeignKey` is the important one: it means "this course points at one teacher". `on_delete=models.CASCADE` means "if that teacher is deleted, delete this course too". That is why the delete dialog on the Teachers page warns you that their courses go with them.

And here is how the nine point at each other:

Django's built-in User (username, password, email, names)

|
| one-to-one
v

Profile phone + role (admin / teacher / student)

Teacher

Student

^

^

|

|

| teacher

| student

|

|

Course <----- Enrollment
^ ^
(course)

|

|

| | course

|

|

|

| +---- Lesson

|

|

^

| course | lesson

|

|

|

+----- Assignment

^

| assignment

Student

|

^

+----- Submission -----+ student

^

| one-to-one

|

Results score + feedback

Read an arrow as "points at". A Course points at a Teacher. An Assignment points at both a Lesson and a Course. A Submission points at an Assignment and a Student. Results points at exactly one Submission, and no Submission may have two.

Notice what is missing: there is no arrow between **Profile** and **Teacher**, and none between **Profile** and **Student**. A login account and a teacher record are two unrelated things in this app. That single gap is the cause of several limits listed at the end of this chapter, so keep it in mind.

NOTE

When the app sends a course to the browser, the teacher arrives as a plain number: `{"id": 1, "title": "Physics", "description": "Motion and forces", "teacher": 3}`. The teacher's name is not included. That is why the Courses page also downloads the full teacher list and matches the numbers up in JavaScript. Most pages load two or three lists for this reason.

Tools you need

Install these before Chapter 1. The last column gives a command you can type into a terminal. A **terminal** is a window where you type commands instead of clicking. On a Mac it is called Terminal, on Windows use PowerShell or the terminal built into VS Code.

TOOL	WHAT IT IS FOR	VERSION USED HERE	HOW TO CHECK IT IS INSTALLED
Python	The language the backend is written in	3.14.0 (3.10 or newer works)	<code>python3 --version</code>
pip	Installs Python packages. Comes with Python	25.3	<code>pip --version</code>
Django	The web framework that handles URLs, the database and the admin site. A framework is a large library that supplies the shape of the program and calls your code	5.2.7	<code>python -m django --version</code>
Django REST Framework	The add-on that turns Django into a JSON API	3.15.2	<code>pip show djangorestframework</code>
djangorestframework-simplejwt	Makes and checks the login tokens	5.4.0	<code>pip show djangorestframework-simplejwt</code>
django-cors-headers	Lets the browser page at port 5180 call the server at port 8001	4.9.0	<code>pip show django-cors-headers</code>
mysqlclient	The bit that lets Python speak to MySQL	2.2.7	<code>pip show mysqlclient</code>
PyJWT	Reads and writes the token format. simplejwt needs it, and it is pinned here too so its version cannot drift	2.10.1	<code>pip show PyJWT</code>
MySQL or MariaDB	The database where everything is stored	any recent version	<code>mysql --version</code>
Node.js	Runs the frontend build tools	25.9.0 (Vite 8 needs 20.19 or newer, or 22.12 or newer)	<code>node --version</code>
npm	Installs frontend packages. Comes with Node	11.12.1	<code>npm --version</code>
A code editor	Where you type the code. VS Code is free and fine	any	open it and see
A web browser	Where the app runs. Chrome, Firefox or Edge	any recent version	open it and see

The six Python packages are listed in one small file, which you will create in Chapter 1:

backend/requirements.txt

```
Django==5.2.7
django-rest-framework==3.15.2
django-rest-framework-simplejwt==5.4.0
django-cors-headers==4.9.0
mysqlclient==2.2.7
PyJWT==2.10.1
```

The `==` means "exactly this version". That is what makes the book's code and your machine agree.

The frontend packages are listed in `frontend/package.json`. Six of them sit under `dependencies`, meaning the app itself needs them at runtime. Vite sits under `devDependencies`, because it is a build tool you only need while working.

PACKAGE	WHAT IT DOES	VERSION IN PACKAGE.JSON
react, react-dom	Draws the interface and keeps it in step with the data	<code>^19.2.7</code>
react-router-dom	Decides which page shows for which URL	<code>^7.11.0</code>
tailwindcss, @tailwindcss/vite	Styling, written as short class names in the markup	<code>^4.3.3</code>
lucide-react	The icons: the graduation cap, the pencil, the trash can	<code>^1.27.0</code>
vite	The tool that runs the frontend while you work and builds it at the end	<code>^8.1.1</code>

The `^` in front of a version means "this version or any later one that only fixes and adds, never breaks". So npm may install 8.1.5 where the file says `^8.1.1`. That is expected, not a mistake.

CHECKPOINT Open a terminal and run these four commands one after another: `python3 --version`, `pip --version`, `node --version`, `mysql --version`. If all four print a version number rather than "command not found", your machine is ready and you can start Chapter 1. If one of them fails, install that tool before going further. Nothing later in the book will work around a missing one.

COMMON MISTAKE Installing `mysqlclient` on a fresh machine and getting a wall of red compiler errors. That package has to build itself against MySQL's own development files, so those need to be present first. Chapter 1 covers the fix for each operating system. If you hit it now, do not panic and do not switch database.

How to read this book

The chapters are in build order. Chapter 1 is an empty folder. The last chapter is a finished app. Every chapter assumes the ones before it are done and working. Do not skip ahead to the chapter that looks interesting, because the file it starts with will not exist yet.

Type the code yourself. Do not copy and paste. This is the single piece of advice in this book worth arguing about, so here is the reasoning. Typing is slow, and slowness is the point. You will make typos, the typos will produce error messages, and reading error messages is most of what programming actually is. Someone who pastes twelve files and hits one bug has no idea where to look. Someone who typed them has seen every line at least once.

Code blocks are labelled. The first line of most blocks is a comment naming the file, like `# backend/backend/models.py` or `// frontend/src/api.js`. Paths are always given from the top of the project folder, so `frontend/src/api.js` means the file `api.js` inside `src` inside `frontend`. When a file is long it is built in labelled pieces, and the text will say which part of the file you are looking at and whether it goes above or below what you already have.

Run things as you go. The **CHECKPOINT** callouts exist so you never build for an hour on a broken foundation. Each one is a command to run or a page to open, with a description of what you should see. If a checkpoint fails, stop there and fix it. The bug is nearly always in the thing you did last.

When something breaks. It will. That is not a sign you are bad at this, it is the normal texture of the work. Try three things, in this order:

1. Read the error message from the bottom up. The last line usually names the file and the line number.
2. Compare your file with the book's, character by character, on the line the error names. Look for a missing comma, a missing colon, a capital letter that should be lowercase, or a quote that was never closed.
3. Check that all three programs are actually running: MySQL, Django on port 8001, and Vite on port 5180.

If those three do not do it, go to the troubleshooting chapter at the back. It lists the errors this specific project produces, with the exact text you will see and what causes it. The Watchman timeout when you start `runserver`, the browser blocking every call because Vite opened on a port the backend does not have on its list (that block is called CORS, short for Cross-Origin Resource Sharing, the browser rule that stops a page on one address from calling a server on another unless that server says it is welcome), the `Invalid phone or password` you get from an account that has no profile row, and being thrown back to the login page when your token has run out. They are all in there.

TIP

Keep three terminal windows open the whole time, one for MySQL, one for the Django server, one for the Vite dev server, and never close them. Both servers watch your files and restart themselves when you save. Half of all beginner confusion is caused by editing code while the server that should be running it is not.

What this app deliberately does NOT do

A tutorial that only shows what works is not being straight with you. Here is what this app cannot do, and why, so you are not left hunting for a feature that was never built.

- **There is no public sign-up.** Nobody can make their own account. An admin creates every account on the Accounts page and chooses the role. The sign-up endpoint refuses anonymous callers with a 401 ("we do not know who you are") and refuses signed-in teachers and students with a 403 ("we know who you are, and no"), so there is no back door either. Visiting `/register` just bounces you to the login page.
- **There is no password-reset page in the frontend.** The backend has the two endpoints for it, but the email settings use the console backend, which means the reset link is printed into the terminal running Django instead of being posted to anyone, and there is no `/reset-password` screen for the link to open. So: if you know your password, you can change it on your profile page. If you have forgotten it, an admin has to reset it for you from Django's own admin site.
- **Teacher and Student rows are not linked to login accounts.** A `Teacher` record and a login account are two unrelated rows that happen to describe the same human. Nothing in the database ties them. Two things follow. First, when a student hands work in they pick their own name from a dropdown, so nothing stops them picking somebody else's. Second, "show me only my courses" and "show me only my grades" cannot be built, because the server has no way to work out which records are yours. Fixing it means adding a link from Teacher and Student to the account table, which is a larger change than adding roles was. This is also why a student is blocked from editing submissions at all: without ownership, "edit your own" and "edit anyone's" are the same thing.
- **There is no pagination.** Pagination is when a long list is served a page at a time, twenty rows now and twenty more if you ask. This API does not do that. Every list endpoint hands back one plain array with everything in it. Fine for a class of thirty, painful at ten thousand rows.
- **There is no filtering.** You cannot ask the server for "just the lessons in course 3". You ask for all the lessons and narrow them down in the browser. That is why the Assignments page downloads every lesson before it can shrink its dropdown.
- **There is no token refresh.** Your login token has a life span, set here to eight hours so it does not expire in the middle of a study session. There is no endpoint to renew it. When it dies the app clears your session and sends you back to the login screen.
- **There is no logout endpoint.** Pressing **Log out** throws the token away in your browser and nothing else. The server is never told. That token stays technically valid until it expires on its own. Good enough for a project on your own machine, not good enough for the real internet.

- **The Accounts page creates accounts but does not list them.** Seeing who exists, changing somebody's role, or switching an account off is done from Django's own admin site at `http://127.0.0.1:8001/admin/`, which Django gives you for free and which you will meet later in the book.

None of these are bugs to be fixed as you read. They are the edges of the project. Knowing where they are is what lets you build the rest with confidence.

What you have now

- A clear picture of the finished app: a login screen, a dashboard, eight record pages, an admin-only Accounts page, a profile page and a 404 page.
- The full feature list, grouped into accounts and login, records, course content, grading, and safety rails.
- The two-halves model: a React frontend that draws, a Django backend that remembers, and JSON over HTTP between them.
- The request path from a React page through `api.js`, over HTTP, into a Django URL, view, serializer and model, down to MySQL, and back again.
- The nine models and which one points at which, including the missing link between login accounts and teacher or student records.
- A checked list of every tool and version, and a command to prove each one is installed.
- An honest list of what this app will not do, so nothing later comes as a surprise.

Setting Up Your Computer

Nothing in this book will work until your computer has three programs on it: Python, Node.js and a MySQL-compatible database server. This chapter installs them, makes the two empty project folders, and finishes with a Django server running on your own machine showing its welcome page.

Take your time here. Setup is the part where most beginners give up, and it is also the part you only have to do once.

1.1 What a terminal is

A **terminal** is a window where you type commands instead of clicking buttons. You type one line, press Enter, and the computer does the thing and prints the answer back as text.

It looks unfriendly, but it is easier to work with than buttons once you get used to it. A button can only do the one thing someone drew on it. A typed command can be copied, pasted, saved and shared. Every instruction in this book is a command you can copy.

The line where you type is called the **prompt**. It usually ends with `$` on macOS and Linux, or `>` on Windows. When this book shows a command, do not type the `$` or the `>`.

OPENING A TERMINAL

macOS Press `Command` + `Space` to open Spotlight, type `Terminal`, press Enter. It also lives in Applications, inside the Utilities folder.

Windows Press the Start button and type `PowerShell`, then open **Windows PowerShell**. On Windows 11 you can also open the app called **Terminal**, which opens PowerShell inside it. Both are fine. This book calls it "PowerShell".

Linux (Ubuntu, Debian, Mint and friends) Press `Ctrl` + `Alt` + `T`. If that does nothing, open your applications menu and search for `Terminal`.

THE FIVE COMMANDS YOU NEED TODAY

COMMAND	MACOS / LINUX	WINDOWS POWERSHELL	WHAT IT DOES
Where am I?	<code>pwd</code>	<code>pwd</code>	Prints the folder you are currently in
What is in here?	<code>ls</code>	<code>ls</code>	Lists the files and folders
Go into a folder	<code>cd frontend</code>	<code>cd frontend</code>	Moves you into <code>frontend</code>
Go back up one	<code>cd ..</code>	<code>cd ..</code>	Moves you to the folder above
Make a folder	<code>mkdir lms</code>	<code>mkdir lms</code>	Creates a folder called <code>lms</code>

The terminal always has a "current folder", meaning the folder your commands act on. Almost every command in this book only works if you are standing in the right one, so when a command fails, the first question to ask is "which folder am I in?" and the answer is `pwd`.

TIP

If any folder in your path has a space in its name, wrap the whole path in double quotes. The machine this book was written on keeps the project at `/Users/tausif/Projects/Ostad/Batch 08/lms`, and `cd /Users/tausif/Projects/Ostad/Batch 08/lms` fails because the terminal reads `Batch` and `08/lms` as two separate things. Typing `cd "/Users/tausif/Projects/Ostad/Batch 08/lms"` works.

1.2 What you are installing, and why

PROGRAM	WHAT IT IS	WHAT THIS PROJECT USES IT FOR
Python	A programming language	The whole backend. Django is written in Python
pip	Python's package installer	Downloads Django and the other Python libraries. Comes with Python
Node.js	A program that runs JavaScript outside a browser	Builds and serves the React frontend
npm	Node's package installer	Downloads React, Vite, Tailwind. Comes with Node.js
MySQL (or MariaDB)	A database server	Stores every teacher, student, course, lesson and grade
Git	Version control	Optional but recommended. Saves snapshots of your work

A **package** is a bundle of ready-made code that somebody else wrote and published, so you do not have to write it yourself. A **package installer** downloads packages for you.

A **database server** is a separate program that runs quietly in the background and holds your data in tables. Django does not store data itself. It sends instructions to the database server and reads the answers back.

MariaDB is a drop-in replacement for MySQL made by the original MySQL developers. Anything in this book that says MySQL works on MariaDB too. Pick whichever is easier to install on your system.

1.3 Installing on macOS

macOS ships with an old Python and no package manager. Install **Homebrew** first. Homebrew is a program that installs other programs from the terminal.

```
/bin/bash -c "$(curl -fsSL
https://raw.githubusercontent.com/Homebrew/install/HEAD/install.sh)"
```

It will ask for your login password and may ask you to install Apple's command line developer tools. Say yes to both. Those developer tools include the C compiler, which is the program that turns C source code into something your machine can run. You will need it later in this chapter.

When Homebrew finishes it prints two or three lines starting with `eval` and asks you to run them. Run them. That step is what puts `brew` on your **PATH**, the list of folders your terminal searches when you type a command name.

Now install everything:

```
brew install python node mysql pkg-config
```

Start the database server, and tell macOS to start it again after every reboot:

```
brew services start mysql
```

NOTE

`brew install mysql` installs both the MySQL server and the client library headers that the Python package `mysqlclient` needs to build. If you skip this and install only a client later, see section 1.10.

1.4 Installing on Windows

Windows 10 and 11 come with **winget**, Microsoft's command line installer. Open PowerShell and run these one at a time:

```
winget install --id Python.Python.3.13 -e
winget install --id OpenJS.NodeJS.LTS -e
winget install --id Oracle.MySQL -e
```

`-e` means "exact match", so winget installs the package with exactly that id rather than guessing from a search.

The MySQL install opens a graphical setup wizard. Work through it and pay attention to two screens:

1. **Type and Networking.** Leave the port at `3306`. The project expects `3306`.
2. **Accounts and Roles.** It asks for a **root password**. Write it down somewhere. You need it in section 1.11 and there is no way to recover it.

At the end, let the wizard configure MySQL as a Windows service so the database starts with the computer.

COMMON MISTAKE

After installing anything with winget, close PowerShell and open a new one. A terminal reads the list of installed programs when it opens and never re-reads it, so `python --version` in the old window still says "not recognised" even though Python is now installed.

If `winget` is not available on your machine, download the installers by hand from python.org, nodejs.org and dev.mysql.com. On the Python installer's first screen, tick **Add python.exe to PATH** at the bottom before pressing Install. Almost every "python is not recognised" problem on Windows comes from that unticked box.

1.5 Installing on Debian, Ubuntu and Mint

Update the package list first, then install:

```
sudo apt update
sudo apt install python3 python3-venv python3-pip
sudo apt install nodejs npm
sudo apt install mariadb-server
```

`sudo` means "run this as the administrator". It will ask for your password.

Start the database:

```
sudo systemctl start mariadb
sudo systemctl enable mariadb
```

`start` runs it now. `enable` makes it run again after a reboot.

NOTE

The `nodejs` package in older Ubuntu releases can be several versions behind. The frontend in this book is built with Vite 8 (`vite` is pinned to `^8.1.1` in `frontend/package.json`), which needs a recent Node. If `node --version` prints something below 20, install Node from NodeSource or use `nvm`, the Node Version Manager, instead of apt.

1.6 Checking that it all worked

This is the first place to stop and prove something.

```
python3 --version
node --version
npm --version
mysql --version
```


On Windows, type `python --version` rather than `python3 --version`. Windows does not create the `python3` name.

You should get four version numbers back. They do not have to match the ones below, they just have to exist.

CHECKPOINT Four commands, four version lines, no "command not found". The machine used to write this book answers `Python 3.14.0`, `v25.9.0`, `11.12.1` and `mysql from 12.0.2-MariaDB, client 15.2` in that order. Anything from Python 3.10 upwards is fine for Django 5.2. If one of the four prints an error, fix that one now. Nothing later in the book will paper over it.

1.7 Making the folder layout

Decide where your projects live. A folder called `Projects` inside your home folder is a good, boring choice.

```
cd ~
mkdir Projects
cd Projects
```

`~` is shorthand for your home folder on macOS and Linux. On Windows PowerShell, `cd ~` works the same way.

Now make the project and its two halves:

```
mkdir lms && cd lms
mkdir backend frontend
```

`&&` means "and then". It runs `mkdir lms`, and if that worked, runs `cd lms`.

COMMON MISTAKE Using `&&` in the older Windows PowerShell 5.1, which does not understand it and answers `The token '&&' is not a valid statement separator`. Run the two commands on separate lines instead, or use the newer Terminal app on Windows 11.

You now have this, and nothing else:

```
lms/
  backend/    empty for now, Django goes here
  frontend/   empty for now, React goes here
```

Everything in the rest of this book happens inside one of those two folders. The split matters: the backend and the frontend are two separate programs that talk to each other over the network, and they are started separately, stopped separately and installed separately.

TIP

Keep two terminal windows open from here on, one parked in `backend` and one parked in `frontend`. You will be running a server in each of them, and a running server takes over its terminal until you stop it.

1.8 The virtual environment

Imagine every project you own shares one toolbox. Project A needs a 10mm spanner. Project B needs a 12mm spanner that happens to have the same label. Whichever one you put in the toolbox last wins, and the other project breaks the next time you open it.

Python has exactly that problem. Installed packages go into one shared place per Python installation. This project needs `Django==5.2.7`. Some other tutorial you follow next month will want Django 6. Installing that one would silently break this one.

A **virtual environment** is a private toolbox for one project. It is an ordinary folder holding its own copy of Python and its own installed packages. While it is switched on, `pip install` puts things in there and nowhere else. Delete the folder and the project's packages are gone, with nothing else disturbed.

Create one inside `backend`:

```
cd backend
python3 -m venv .venv
```

`python3 -m venv` means "run Python's built-in `venv` tool". `.venv` is the name of the folder it creates. The leading dot keeps it hidden in file listings.

Switch it on:

```
source .venv/bin/activate
```

On Windows PowerShell:

```
.venv\Scripts\activate
```

Your prompt changes. It now starts with `(.venv)`:

```
(.venv) ~/Projects/lms/backend $
```

That prefix is the whole point. It is how you know which toolbox is open. When you are done for the day:

```
deactivate
```

The prefix disappears and you are back on the system Python.

COMMON MISTAKE

Forgetting to activate. You open a fresh terminal tomorrow, type `python manage.py runserver 8001`, and get `ModuleNotFoundError: No module named 'django'`. Django is not missing. You are just standing outside the toolbox. Run `source .venv/bin/activate` and try again. This is the single most common confusion in the whole book, and the fix is always the same.

NOTE

Windows may refuse the activate script with a message about execution policies being disabled. Run `Set-ExecutionPolicy -Scope CurrentUser RemoteSigned` once, answer `Y`, and then activate again.

THE .VENV FOLDER MUST NEVER BE COMMITTED

Git works by recording snapshots of your files in a **repository**, which is just the folder Git is watching plus its history. `.venv` holds hundreds of megabytes of downloaded code that anyone can recreate from `requirements.txt` in one command. It also contains absolute paths pointing at your own home folder, so it would not even work on someone else's machine.

The project keeps a `.gitignore` at the top level, next to `backend` and `frontend`. A **.gitignore** is a plain text list of things Git should pretend it cannot see. Create it now at `lms/.gitignore` and put exactly this in it:

```

# Python
__pycache__/
*.py[cod]
*.pyc

# Django
db.sqlite3
db.sqlite3-3
media/
staticfiles/

# Env
.env
venv/
.venv/
env/

# Editors
.vscode/
.idea/
.DS_Store
frontend/node_modules/

```

That is the finished file, line for line as it is in the project. Three of the lines are worth understanding now:

- `.venv/`, `venv/` and `env/` cover the three names people give virtual environments, so the folder never gets committed whatever you called it.
- `__pycache__/` and `*.pyc` are the compiled files Python writes next to your source. They are generated, so they are noise in a repository.
- `frontend/node_modules/` does the same job for the frontend as `.venv/` does for the backend. `node_modules` is where npm puts its downloaded packages.

If you want Git in your life, and you should, run this once from inside `lms`:

```
git init
```

1.9 Installing the Python packages

You never install Python packages one by one from memory. You write down exactly which ones and which versions in a file called `requirements.txt`, and let `pip` read it. Anyone who copies your project runs one command and ends up with the same packages you have.

Create `backend/requirements.txt` and type these six lines into it:

```
Django==5.2.7
djangorestframework==3.15.2
djangorestframework-simplejwt==5.4.0
django-cors-headers==4.9.0
mysqlclient==2.2.7
PyJWT==2.10.1
```

`==` pins an exact version. `Django>=5.2` would let pip pick anything newer, and newer sometimes means different. Pinning is why this book's code will still run for you.

Here is what each one is for. Do not worry about the details yet, every one of them gets a chapter of its own:

PACKAGE	WHAT IT DOES
Django	The web framework, meaning the large library that supplies most of the structure of a website. Models, the database layer, URL routing, the admin site
djangorestframework	Adds JSON API tools to Django. JSON is a plain text format for sending data, and an API is a set of web addresses a program can call. Serializers, API views, the browsable API
djangorestframework-simplejwt	Login tokens. Turns a phone number and password into a token the frontend carries. The login view in this project really does look people up by phone, not by username
django-cors-headers	Lets the browser talk to a server on a different port. Without it every API call from React is blocked
mysqlclient	The driver Django uses to speak to MySQL. A driver is the small piece of code that knows how to talk to one particular kind of database
PyJWT	The library that actually signs and reads the login tokens

With the virtual environment active, install them all:

```
pip install -r requirements.txt
```

`-r` means "read the requirements from this file". pip downloads each package, plus anything those packages themselves depend on.

Check what landed:

```
pip list
```

Package	Version
-----	-----
asgiref	3.12.1
Django	5.2.7
django-cors-headers	4.9.0
djangorestframework	3.15.2
djangorestframework-simplejwt	5.4.0
mysqlclient	2.2.7
pip	25.3
PyJWT	2.10.1
sqlparse	0.5.5

Nine entries from six requested. `asgiref` and `sqlparse` are things Django itself needs, so pip fetched them without being asked. `pip` is in the list because pip lives inside the virtual environment too.

CHECKPOINT Run `python -m django --version`. It must print `5.2.7`. If it prints a different number, you are on a different Django than this book and some code will not match. If it prints `No module named django`, your virtual environment is not active. Look for `(.venv)` at the start of your prompt.

1.10 When mysqlclient refuses to install

This is the wall. More beginners stop here than anywhere else, so read this section even if your install worked.

`mysqlclient` is not pure Python. It is a thin Python wrapper around a C library that ships with MySQL. To install it, pip sometimes has to compile that C code on your machine, and compiling needs two things present: a C compiler, and the MySQL **client headers**, which are small files describing how to call MySQL from C.

When one of them is missing, `pip install -r requirements.txt` ends in a wall of red text. Ignore all of it except the last few lines. If you can see the words `mysql_config`, `pkg-config`, or `Can not find valid pkg-config name`, you have this problem and not some other one.

Fix on macOS

```
brew install mysql-client pkg-config
```

Homebrew deliberately does not put `mysql-client` on your PATH, because its command names collide with the full MySQL server's. You have to point at it yourself before installing. On Apple Silicon Macs:

```
export PATH="/opt/homebrew/opt/mysql-client/bin:$PATH"
export PKG_CONFIG_PATH="/opt/homebrew/opt/mysql-client/lib/pkgconfig"
pip install -r requirements.txt
```

On older Intel Macs, Homebrew lives at `/usr/local` instead, so swap `/opt/homebrew` for `/usr/local` in both lines.

`export` sets an environment variable, which is a named value the programs you start can read. It lasts for this terminal window only and is forgotten when you close it, which is fine, because you only need it during the install.

Fix on Debian and Ubuntu

```
sudo apt install default-libmysqlclient-dev build-essential pkg-config
pip install -r requirements.txt
```

`default-libmysqlclient-dev` is the headers. `build-essential` is the C compiler and friends. `pkg-config` is the small tool that tells the compiler where the headers ended up.

Fix on Windows

Windows almost never needs a compiler here, because `mysqlclient` publishes ready-made **wheels** for Windows. A wheel is a package that is already compiled, so pip just unzips it. If pip is trying to compile, it usually means pip is too old to recognise the wheel:

```
python -m pip install --upgrade pip
pip install -r requirements.txt
```

COMMON MISTAKE

Reacting to the red text by installing a different package, usually `pymysql` or `mysql-connector-python`, because a search result suggested it. Do not. This project's settings set the database engine to `django.db.backends.mysql`, which expects `mysqlclient`, and swapping the driver means changing settings you have not written yet. Fix the headers instead.

1.11 Creating the database

The database server is running, but it is empty. It needs a **database** (a named container for your tables) and a **user** (a login that is allowed to touch that container).

Open the MySQL command prompt as the administrator account, called `root`:

```
mysql -u root -p
```

`-u root` picks the user, `-p` makes it ask for the password. On a freshly installed MariaDB on Linux, root may have no password yet, in which case use `sudo mysql` instead.

You now have a different prompt, `mysql>`. Type these three lines, each ending with a semicolon:

```
CREATE DATABASE lms_db;
CREATE USER 'lms_user'@'127.0.0.1' IDENTIFIED BY 'lms_user_pw';
GRANT ALL PRIVILEGES ON lms_db.* TO 'lms_user'@'127.0.0.1';
```

Reading that line by line:

- `CREATE DATABASE lms_db;` makes the container. Every table this project creates lives inside `lms_db`.
- `CREATE USER 'lms_user'@'127.0.0.1' IDENTIFIED BY 'lms_user_pw';` makes a login named `lms_user` with the password `lms_user_pw`, usable only from `127.0.0.1`, which means "this same computer".
- `GRANT ALL PRIVILEGES ON lms_db.* TO 'lms_user'@'127.0.0.1';` gives that login full control of everything inside `lms_db`, and nothing at all outside it. The `.*` means "every table in this database".

Leave the prompt with:

```
exit
```

WHY NOT JUST CONNECT AS ROOT?

Because `root` can drop every database on the server, change other users' passwords, and read data that has nothing to do with this app. If the LMS ever has a bug that lets a stranger send it SQL, the damage stops at whatever the connecting user is allowed to do. `lms_user` can wreck `lms_db` and cannot touch anything else. That is the whole idea: give a program the smallest set of permissions that lets it do its job.

There is a practical reason too. Your root password is personal to your machine. `lms_user` / `lms_user_pw` is the same for everyone following this book, so the project's settings can carry those values as defaults and just work.

COMMON MISTAKE

Creating the user as `'lms_user'@'localhost'` instead of `'lms_user'@'127.0.0.1'`. MySQL treats those as two different users. `localhost` means a connection over a local socket file, `127.0.0.1` means a connection over the network to this machine. The project's settings default the database host to `127.0.0.1`, so a grant for `localhost` produces `Access denied for user 'lms_user'@'127.0.0.1'` and you will stare at it for an hour convinced the password is wrong.

NOTE

If you want different names or a different password, you can. The project reads five environment variables and falls back to the values above when they are not set: `DB_NAME` falls back to `lms_db`, `DB_USER` to `lms_user`, `DB_PASSWORD` to `lms_user_pw`, `DB_HOST` to `127.0.0.1` and `DB_PORT` to `3306`. Change the SQL and set the matching variables, or keep the SQL exactly as printed and skip the extra work.

1.12 Creating the Django project

Two commands make the entire backend skeleton. The names in them are not decoration. Every import in this codebase depends on them, so type them exactly as printed.

Make sure you are in `backend` with the virtual environment active:

```
cd backend
source .venv/bin/activate
django-admin startproject lms .
```

`django-admin` is a command that came with Django. `startproject` builds the outer shell of a Django site. `lms` is the project's name.

The trailing dot is important. It means "build it right here in the current folder". Without it, Django creates one more folder level and you end up with `backend/lms/manage.py`, which does not match this project and will break every path in this book.

Now the app:

```
python manage.py startapp backend
```

A Django **project** is the site as a whole: settings, the list of URLs, the entry points for a web server. A Django **app** is one feature area inside it: models, views, and the code that does the actual work. A project can hold many apps. This one holds exactly one, and it is called `backend`.

YES, `BACKEND/BACKEND/` IS MEANT TO LOOK LIKE THAT

You are now standing in a folder called `backend`, which contains a folder called `backend`. The full path to the app's models file is:

```
lms/backend/backend/models.py
```

The outer `backend` is the folder you made by hand in section 1.7, holding the whole Python side of the project. The inner `backend` is the Django app that `startapp` just created. They are unrelated things that happen to share a name.

This trips up everybody once. Here is the complete picture after both commands:

```
lms/
  backend/          <- you made this by hand
    .env/           <- your virtual environment
    manage.py       <- from startproject
    requirements.txt <- you wrote this
  lms/              <- from startproject: the project
    __init__.py
    settings.py
    urls.py
    asgi.py
    wsgi.py
  backend/          <- from startapp: the app
    __init__.py
    admin.py
    apps.py
    models.py
    tests.py
    views.py
    migrations/
      __init__.py
  frontend/         <- still empty
```

Both names are wired into real code. Line 9 of `backend/manage.py` names the project:

```
os.environ.setdefault('DJANGO_SETTINGS_MODULE', 'lms.settings')
```

`lms.settings` is Python for "the `settings` file inside the `lms` folder". Name the project something else and that line points at nothing.

The app names itself in its own config file, `backend/backend/apps.py`, which is the whole file:

```
from django.apps import AppConfig

class BackendConfig(AppConfig):
    name = 'backend'
```

Later, the project's URL file will hand every address starting with `/api/` over to the app by that same name. This is the finished `backend/lms/urls.py`:

```
urlpatterns = [
    path('admin/', admin.site.urls),
    path('api/', include('backend.urls')),
]
```

Right now your freshly generated `urls.py` has only the `admin/` line. The `api/` line arrives in a later chapter, once there is something to point it at. It is printed here so you can see that `'backend.urls'` is the app folder's name in string form. Rename the folder and that string stops resolving.

COMMON MISTAKE

Running `python manage.py startapp backend` from the wrong folder. `manage.py` only exists inside `lms/backend/`, so if you get `can't open file 'manage.py': No such file or directory`, run `pwd` and `cd` to the folder that has `manage.py` in it.

1.13 What got created, file by file

Thirteen files and three folders appeared. None of them are magic. Here are the ones that matter.

MANAGE.PY

The one command you will type more than any other. It is a small wrapper that loads your settings and then hands the rest of your command line to Django. This is `backend/manage.py` in full:

```
#!/usr/bin/env python
"""Django's command-line utility for administrative tasks."""
import os
import sys

def main():
    """Run administrative tasks."""
    os.environ.setdefault('DJANGO_SETTINGS_MODULE', 'lms.settings')
    try:
        from django.core.management import execute_from_command_line
    except ImportError as exc:
        raise ImportError(
            "Couldn't import Django. Are you sure it's installed and "
            "available on your PYTHONPATH environment variable? Did you "
            "forget to activate a virtual environment?"
        ) from exc
    execute_from_command_line(sys.argv)

if __name__ == '__main__':
    main()
```

That is the finished file. It is never edited. Note the error message Django wrote for you in the middle of it: *"Did you forget to activate a virtual environment?"* Django knows how often that happens too.

`execute_from_command_line(sys.argv)` is what turns `python manage.py runserver 8001` into "run the development server" and `python manage.py migrate` into "apply database changes". Every `manage.py` command in this book goes through that one line.

LMS/INIT.PY

Empty. Zero bytes. Its only job is to tell Python "this folder is a package you can import from", which is what makes `lms.settings` a valid thing to write. `backend/__init__.py` and `backend/migrations/__init__.py` are empty for the same reason. Do not delete them.

LMS/SETTINGS.PY

The project's control panel. One long file of variables that Django reads at startup. In the finished project it decides which database to use (`DATABASES`), which apps are installed (`INSTALLED_APPS`), how long a login token lasts (`SIMPLE_JWT`), and which web addresses are allowed to call the API (`CORS_ALLOWED_ORIGINS`).

Right now it is the stock version Django generated, and it points at a SQLite file rather than your MySQL database. Switching it over is the first job of the next chapter.

Four things in it are worth knowing about before you open it:

SETTING	WHAT IT MEANS
<code>SECRET_KEY</code>	A long random string Django uses to sign cookies and tokens. It is fine on your laptop, and it must never be published for a real site
<code>DEBUG = True</code>	Shows a detailed yellow error page when something breaks, with the exact line that failed. Very useful while learning, dangerous in public
<code>ALLOWED_HOSTS</code>	The list of hostnames the site will answer to. The finished project sets it to <code>['127.0.0.1', 'localhost']</code>
<code>INSTALLED_APPS</code>	Every app Django should load. Your new <code>backend</code> app has to be added to this list by hand, which is a later chapter

NOTE

The comment block at the top of a generated `settings.py` names the Django version that wrote it. In this project it reads `Generated by 'django-admin startproject' using Django 6.0.6.` even though `requirements.txt` pins `Django==5.2.7`, because the file was first generated on a newer Django and then pinned back. That mismatch has one real consequence, and the project writes it down at the bottom of the same file:

```
# The existing migrations were generated on Django 6, where BigAutoField is the
# default primary key. Django 5.2 falls back to AutoField unless this is set,
# which makes `makemigrations` want to rewrite every primary key even though the
# database columns are already bigint.
DEFAULT_AUTO_FIELD = 'django.db.models.BigAutoField'
```

Your own generated header will say whatever version you installed. That is normal and nothing is wrong.

LMS/URLS.PY

The site's table of contents. It maps a web address to the code that answers it. Fresh from `startproject` it knows about exactly one address, `/admin/`, the built-in Django admin site. Everything else in this project gets added to this file or to a file it points at.

LMS/WSGI.PY AND LMS/ASGI.PY

Two entry points for real web servers to hand requests to Django. You will not touch either one in this book, because `runserver` does that job during development. They matter the day you put the site on the internet. Here is `backend/lms/wsgi.py`, all of it:

```
"""
WSGI config for lms project.

It exposes the WSGI callable as a module-level variable named ``application``.

For more information on this file, see
https://docs.djangoproject.com/en/6.0/howto/deployment/wsgi/
"""

import os

from django.core.wsgi import get_wsgi_application

os.environ.setdefault('DJANGO_SETTINGS_MODULE', 'lms.settings')

application = get_wsgi_application()
```

`asgi.py` is the same file with `asgi` in place of `wsgi` throughout. WSGI is the older, simpler way for a web server to call Python. ASGI is the newer one that also handles long-lived connections such as WebSockets. Django ships both so you can choose later.

BACKEND/MODELS.PY

Where you describe your data. A **model** is a Python class that Django turns into a database table: one class becomes one table, one attribute becomes one column. Straight out of `startapp` it is one import and a comment. The real project file starts like this, with one extra import the finished models need:

```
from django.db import models
from django.contrib.auth.models import User

# Create your models here.
```

That comment is still sitting on line 4 of the real project file, with nine model classes underneath it.

`Profile`, `Teacher`, `Student`, `Course`, `Enrollment`, `Lesson`, `Assignment`, `Submission` and `Results` all get built in a later chapter.

BACKEND/VIEWS.PY

Where you write what happens when a request arrives. A **view** is a function or class that takes an incoming web request and returns a response. In this project the responses are JSON, because the frontend is a separate React app. Generated, it is one import and a `# Create your views here.` comment.

BACKEND/ADMIN.PY

Django ships a complete data-editing website for free at `/admin/`, but it only shows models you register there. This file is where you register them. Generated it is one import and a comment. In the finished project it is 75 lines that make all nine models editable, and it is the fastest way to put test data into the app before the React frontend exists.

BACKEND/APPS.PY

Configuration for this one app. Shown in full in the previous section. Django may also write a `default_auto_field` line into this file when it generates it; this project keeps that decision in `settings.py` as `DEFAULT_AUTO_FIELD` instead, which is why the real file is only five lines long.

BACKEND/TESTS.PY

A place for automated tests. Generated and untouched in this project, so `backend/backend/tests.py` is still these three lines:

```
from django.test import TestCase

# Create your tests here.
```

BACKEND/MIGRATIONS/

A **migration** is a recorded change to the database structure, written as a Python file. When you add a field to a model, Django writes a migration describing the change, and `migrate` runs it against the real database. That way the database and your models can never drift apart, and another developer gets the same tables by running the same migrations.

Right now the folder holds only the empty `__init__.py`. By the end of the book it holds four migrations:

```
0001_initial.py
0002_course_enrollment_lesson_assignment_submission_and_more.py
0003_profile_role.py
0004_existing_accounts_become_admins.py
```

They are numbered because they run in order. You never write them by hand; `python manage.py makemigrations` writes them for you from your models.

1.14 Checkpoint: the rocket page

Everything above was preparation. Now prove Django is alive.

From `lms/backend`, with `(.venv)` on your prompt:

```
python manage.py runserver 8001
```

The terminal prints something close to this and then appears to hang:

```
Watching for file changes with StatReloader
Performing system checks...

System check identified no issues (0 silenced).
August 04, 2026 - 05:40:15
Django version 5.2.7, using settings 'lms.settings'
Starting development server at http://127.0.0.1:8001/
Quit the server with CONTROL-C.
```

It has not hung. A server runs until you stop it, so it holds this terminal window for as long as it is up. The date line shows today's date on your machine.

You will probably also see a warning in red about unapplied migrations. That is expected. The database has no tables yet, and `migrate` is the next chapter's job.

Open a browser and go to **`http://127.0.0.1:8001/`**.

You get a page with a green rocket taking off and the words "The install worked successfully! Congratulations!" That page only ever appears when Django is running, has valid settings, and has no URL configured for `/`. All three are true right now.

CHECKPOINT The rocket page at `http://127.0.0.1:8001/`. If you see it, Python, pip, the virtual environment, Django and your project layout are all correct. Press `Ctrl + C` in the terminal to stop the server. It stops instantly and gives you your prompt back.

`127.0.0.1` is the address every computer uses for itself, also spelled `localhost`. `8001` is the **port**, a numbered door on that address. Django's development server picks 8000 by default, but this project runs it on 8001, because that is the port `frontend/src/api.js` calls. The React frontend will later sit on port 5180, set in `frontend/vite.config.js`, so the two can run side by side.

TIP If `runserver` dies with `pywatchman.SocketTimeout: timed out waiting for response`, Django's auto-reloader is trying to use Watchman and Watchman is not answering. Start it with `python manage.py runserver --noreload` (you then have to restart it yourself after editing Python files), or raise the timeout by setting `DJANGO_WATCHMAN_TIMEOUT=20` before the command.

COMMON MISTAKE Closing the terminal window instead of pressing `Ctrl + C`. On some systems the server keeps running in the background, and the next `runserver` fails with `Error: That port is already in use.` The cure is to find and stop the old one, or start the new one on another port with `python manage.py runserver 8001`.

1.15 About the frontend folder

`frontend` is still empty, and it stays that way for a few chapters. Vite creates the whole React project in one command when the time comes, and there is no point generating it before there is an API for it to call.

For now, all you needed from this chapter was `node --version` and `npm --version` answering with real numbers, which you checked in section 1.6.

NOTE If you ever run this project on Django's own default database, a `backend/db.sqlite3` file appears next to `manage.py`. The finished settings point the database engine at MySQL, so nothing reads it, and the root `.gitignore` lists `db.sqlite3` so it never reaches Git. If one shows up, you can delete it; the app will not notice.

What you have now

- A terminal you can open, and the five commands needed to move around in it
- Python, Node.js, npm and MySQL or MariaDB installed and verified by version number
- The folder `lms/` with empty `backend/` and `frontend/` inside it
- A virtual environment at `backend/.venv`, plus the knowledge that `(.venv)` on the prompt means it is switched on
- `backend/requirements.txt` with six pinned packages, all installed, `python -m django --version` answering `5.2.7`
- A database called `lms_db` and a user `lms_user` that can reach it from `127.0.0.1` and nothing else
- A Django project called `lms` and a Django app called `backend`, living at `backend/lms/` and `backend/backend/`
- A running development server showing the rocket page at `http://127.0.0.1:8001/`

Wiring Up Django: settings.py

In Chapter 1 you met the two halves of this project. Now you build the first real file of the backend and read it line by line.

Everything in this chapter lives in one file:

```
lms/
  backend/
    manage.py
  lms/
    settings.py    <-- this chapter
    urls.py        <-- also this chapter
```

By the end of it, `python manage.py runserver 8001` will start a working Django server with no error text, connected to MySQL, answering in JSON (JSON is the plain-text format programs use to send data to each other), and willing to answer the React app running on a different port.

2.1 What settings.py actually is

Django is a web framework. A framework is a big pile of code someone else wrote, which does the boring parts of a website for you: listening for requests, talking to a database, checking passwords. To do any of that it has to know a few things about *your* project. Which database? Which folder holds your code? Should error pages show the full crash report, or hide it?

Django answers all of those questions by reading one Python file.

`settings.py` is a plain Python file full of variables written in CAPITAL LETTERS. Django imports that file once when the server starts, reads the variables, and remembers them. That is the whole mechanism. There is no magic format, no XML, no config wizard. If you can write `DEBUG = True` in Python, you can configure Django.

How does Django know which file to read? Look at `manage.py`, the small script that sits next to the `lms/` folder:

backend/manage.py

```
def main():
    """Run administrative tasks."""
    os.environ.setdefault('DJANGO_SETTINGS_MODULE', 'lms.settings')
```

`DJANGO_SETTINGS_MODULE` is the name of the settings file written the way Python imports things: `lms.settings` means "the file `settings.py` inside the folder `lms`". Every Django command you run through `manage.py` sets that first, then starts Django.

NOTE

This project has two folders that both matter and one of them is confusingly named. `backend/lms/` is the **project** folder, and it holds `settings.py` and the top-level `urls.py`. `backend/backend/` is the **app** folder, and it holds `models.py`, `views.py`, `serializers.py`. When this book says "open `settings.py`", it always means `backend/lms/settings.py`.

COMMON MISTAKE

Editing `backend/backend/` when you meant `backend/lms/`. If you add a setting and Django appears to ignore it completely, check the path of the file you just saved. Only `backend/lms/settings.py` is read.

2.2 Creating the project from nothing

If you are following along from an empty folder, these are the exact commands that produce the layout above.

First make the folders and a virtual environment. A virtual environment is a private copy of Python that belongs to this project only, so the libraries you install here do not mix with libraries other projects on your computer use.

```
mkdir -p lms/backend
cd lms/backend
python -m venv .venv
source .venv/bin/activate
```

On Windows the last line is `.venv\Scripts\activate` instead. Once it works, your terminal prompt gets a `(.venv)` in front of it. That prefix is how you know the virtual environment is switched on. You have to switch it on again in every new terminal window.

Now install the libraries this project uses. The real list lives in `backend/requirements.txt`:

`backend/requirements.txt`

```
Django==5.2.7
django-rest-framework==3.15.2
django-rest-framework-simplejwt==5.4.0
django-cors-headers==4.9.0
mysqlclient==2.2.7
PyJWT==2.10.1
```

If you are starting from an empty folder, save those six lines into a file called `requirements.txt` next to where `manage.py` will land. `pip` is Python's installer. `pip install -r requirements.txt` reads that file and installs every line. The `==` pins an exact version so that the code in this book behaves the same on your machine as it does on mine.

```
pip install -r requirements.txt
```

Then create the Django project and the app:

```
django-admin startproject lms .  
python manage.py startapp backend
```

The dot at the end of the first command matters. Without it, `django-admin` would make an extra wrapper folder. With it, `manage.py` lands in the folder you are standing in and the project package `lms/` is created beside it.

CHECKPOINT Run `ls` (or `dir` on Windows). You should see `manage.py`, a folder `lms`, a folder `backend`, and `.venv`. If `manage.py` is one level deeper than you expected, you left off the dot. Delete what was made and run it again.

COMMON MISTAKE Installing `mysqlclient` fails with a compiler error on a fresh machine. That package is not pure Python, it builds against MySQL's client library, so MySQL's development files have to exist before `pip` can build it. Install MySQL (or MariaDB) first, then run the `pip install` again. Everything else in `requirements.txt` installs without any of that.

2.3 The top of the file: docstring, imports, BASE_DIR

Open `backend/lms/settings.py`. The first thing in it is a docstring, which is a block of text in triple quotes that Python ignores. `startproject` wrote it:

backend/lms/settings.py

```
"""
Django settings for lms project.

Generated by 'django-admin startproject' using Django 6.0.6.

For more information on this file, see
https://docs.djangoproject.com/en/6.0/topics/settings/

For the full list of settings and their values, see
https://docs.djangoproject.com/en/6.0/ref/settings/
"""
```

That says Django 6.0.6, but `requirements.txt` pins Django 5.2.7. Both are true. The project was first generated on a machine with a newer Django, then pinned to 5.2.7 afterwards. The docstring is just text and was never updated. It has one real consequence, which shows up at the very bottom of the file in section 2.14.

Next come the imports and one calculated value:

backend/lms/settings.py

```
import os
from datetime import timedelta
from pathlib import Path

# Build paths inside the project like this: BASE_DIR / 'subdir'.
BASE_DIR = Path(__file__).resolve().parent.parent
```

`startproject` only writes the `pathlib` line. The other two were added later in this project, because settings further down need them: `os` for reading environment variables in the database and email blocks, and `timedelta` for the token lifetimes.

`BASE_DIR` is worth understanding, because Django uses it everywhere. `__file__` is the path of this settings file. `.resolve()` turns it into a full path with no shortcuts. `.parent` goes up one folder, and `.parent.parent` goes up two. So starting from `backend/lms/settings.py`, one `.parent` gives `backend/lms/`, and the second gives `backend/`. `BASE_DIR` is the folder that holds `manage.py`. Anywhere you need a path later you can write `BASE_DIR / 'something'` and it works no matter where the project was copied to.

2.4 SECRET_KEY, DEBUG, ALLOWED_HOSTS

These three sit together under a warning comment that `startproject` writes:

```
backend/lms/settings.py
```

```
# Quick-start development settings - unsuitable for production
# See https://docs.djangoproject.com/en/6.0/howto/deployment/checklist/

# SECURITY WARNING: keep the secret key used in production secret!
SECRET_KEY = 'django-insecure-@hg+%+(*w2m0jh=8$ksnpsoe+dv2jb+py3@1b(3vhohjb7cx8!'
```

```
# SECURITY WARNING: don't run with debug turned on in production!
DEBUG = True
```

```
ALLOWED_HOSTS = ['127.0.0.1', 'localhost']
```

SECRET_KEY is a long random string Django uses as the ingredient for anything that needs to be unforgeable: session cookies, password reset links, and in this project the login tokens. Think of it as the master key to a rubber stamp. Django stamps a value, and later it can tell the difference between a stamp it made and a stamp somebody else drew by hand, because only Django knows the key. If an attacker learns your `SECRET_KEY`, they can mint a password reset link for any account. Django generated this one during `startproject` and prefixed it with `django-insecure-` on purpose, as a reminder that it was generated for local use.

DEBUG is a switch for how much Django tells you when something breaks. With `DEBUG = True`, a crash gives you a yellow error page in the browser with the exact line of code that failed, the values of local variables, and the full settings. That is a big help while learning. It is also a full description of your server, handed to whoever hit the URL.

ALLOWED_HOSTS is a list of the host names your site is allowed to answer to. Every request a browser sends carries a few lines of extra information called headers, and one of them, the `Host` header, holds the address that was typed in. Django refuses any request whose `Host` is not on this list. `startproject` leaves the list empty, and Django then quietly permits `localhost` and `127.0.0.1` while `DEBUG` is `True`. This project spells the two out instead, so the list keeps working if `DEBUG` is ever switched off.

Say this plainly: **a checked-in `SECRET_KEY` and `DEBUG = True` are fine for learning and wrong for a real deployment**. On a real server you read the key out of an environment variable instead of typing it in the file, you set `DEBUG = False`, and you put your real domain in `ALLOWED_HOSTS`. Nothing in this book runs on a public server, so the file stays as it is. Just do not copy it onto one.

NOTE

`127.0.0.1` and `localhost` are two names for the same thing: your own computer. `127.0.0.1` is the numeric address, `localhost` is the word that maps to it. They behave identically for you but they are different *text*, and later in this chapter that difference causes real trouble, which is why both are listed in two places in this file.

2.5 INSTALLED_APPS: what an app is

Django splits a project into **apps**. An app is a folder of Python files that belong together: some database tables, some views, sometimes an admin page. Django only looks inside a folder if that folder is named in `INSTALLED_APPS`. An app that is not listed does not exist as far as Django is concerned. Its tables are never created and its admin pages never appear.

Here is the real list:

backend/lms/settings.py

```
INSTALLED_APPS = [  
    'django.contrib.admin',  
    'django.contrib.auth',  
    'django.contrib.contenttypes',  
    'django.contrib.sessions',  
    'django.contrib.messages',  
    'django.contrib.staticfiles',  
    'backend',  
    'rest_framework',  
    'rest_framework_simplejwt.token_blacklist',  
    'corsheaders',  
]
```

The first six lines are Django's own apps, written for you and switched on by `startproject`. `django.contrib.auth` is the one you will meet most: it provides the `User` table, password hashing (storing a scrambled version of a password so the real one is never written to disk), and the permission machinery. `django.contrib.admin` is the ready-made admin website at `/admin/`.

The last four lines were added by hand. This is what each one is and how it got onto your computer.

ENTRY	WHAT IT IS	HOW YOU INSTALL IT
'backend'	Your own app, all the LMS models and views	<code>python manage.py startapp backend</code>
'rest_framework'	Django REST Framework, the layer that turns models into a JSON API	<code>pip install djangorestframework==3.15.2</code>
'rest_framework_simplejwt.token_blacklist'	The table of cancelled login tokens	<code>pip install djangorestframework-simplejwt==5.4.0</code>
'corsheaders'	Adds the response headers that let the React app call this API	<code>pip install django-cors-headers==4.9.0</code>

A few notes on those.

'backend' is listed by its plain folder name because that is what the app is called. `startapp` also generated `backend/apps.py`, which is where an app can describe itself:

backend/backend/apps.py

```
from django.apps import AppConfig

class BackendConfig(AppConfig):
    name = 'backend'
```

'rest_framework' is the import name of the library whose *pip* name is `djangorestframework`, with no dashes and no capitals. The pip name and the import name being different is normal in Python and catches everybody once.

'rest_framework_simplejwt.token_blacklist' is unusual: it is not the whole library, it is one sub-app inside it. The main part of Simple JWT needs no entry in `INSTALLED_APPS` at all. This sub-app is listed because it brings two database tables with it, which record which login tokens have been issued and which of them have been cancelled. This project uses them when somebody changes their password. Because it is an app with tables, adding this line is what makes `migrate` create those tables.

A word on that command, since it turns up in every callout from here on. A **migration** is a small Python file describing one change to the database, such as "make a table called `backend_teacher` with these columns". `python manage.py makemigrations` writes those files by looking at your models, and `python manage.py migrate` runs them against the real database.

'corsheaders' also needs a middleware entry, which is the next section.

COMMON MISTAKE

Adding an app to `INSTALLED_APPS` and not running `python manage.py migrate` afterwards. If an app brings tables, they are only created when you migrate. The symptom is a crash saying a table does not exist, with a name like `token_blacklist_outstandingtoken` in the message.

2.6 MIDDLEWARE and why the order matters

Middleware is a stack of small pieces of code that every request passes through on the way in, and every response passes through on the way back out. Picture it as a row of security desks between the front door and your view function. Each desk can inspect the request, change it, or turn it away. On the way out, each desk gets a look at the response and can add to it.

The list runs top to bottom on the way in, and bottom to top on the way out.


```
MIDDLEWARE = [
    'django.middleware.security.SecurityMiddleware',
    'corsheaders.middleware.CorsMiddleware',
    'django.contrib.sessions.middleware.SessionMiddleware',
    'django.middleware.common.CommonMiddleware',
    'django.middleware.csrf.CsrfViewMiddleware',
    'django.contrib.auth.middleware.AuthenticationMiddleware',
    'django.contrib.messages.middleware.MessageMiddleware',
    'django.middleware.clickjacking.XFrameOptionsMiddleware',
]
```

Every line here except the second was written by `startproject`. The one added line is `'corsheaders.middleware.CorsMiddleware'`, and it is deliberately placed **second, right after `SecurityMiddleware` and above everything else.**

That position is not decoration. Two reasons:

1. `CorsMiddleware` has to run **above** `CommonMiddleware`. `CommonMiddleware` is the one that redirects a URL missing its trailing slash. If a redirect is generated before the CORS headers are attached, the browser gets a redirect with no CORS headers on it and blocks the whole thing. The response never reaches your JavaScript, and the error you see says nothing about redirects.
2. A browser sends a "preflight" request before certain calls, an `OPTIONS` request that asks the server "am I allowed to send this?". `CorsMiddleware` answers that itself and returns straight away. For it to do that, it must be reached before middleware that might reject the request for some unrelated reason.

COMMON MISTAKE

Putting `'corsheaders'` in `INSTALLED_APPS` and forgetting `CorsMiddleware` in `MIDDLEWARE`, or pasting the middleware line at the bottom of the list. Both look fine at a glance and both produce the same symptom: the API works perfectly in a terminal, and every call from the React page fails.

2.7 CORS_ALLOWED_ORIGINS and the same-origin rule

CORS stands for Cross-Origin Resource Sharing. This is the setting that confuses beginners more than any other in the file, so here is the browser rule it exists to work around.

A browser will happily load a page from anywhere. But JavaScript running on a page is only allowed to read the response from a request it sent to the **same origin** as that page. An origin is three things stuck together: the scheme, the host, and the port.

```
http://localhost:5180
^^^^  ^^^^^^^^^  ^^^^^
scheme host      port
```

Two addresses are the same origin only if all three match exactly. This rule is what stops a page on a random website from quietly calling your bank's API using the cookies in your browser and reading the answer.

Now look at this project. Vite, the tool that serves the React app while you are working on it, runs at `http://localhost:5180`. Django runs at `http://127.0.0.1:8001`. Different host text, different port. Two different origins. So when the React page calls the API, the browser blocks JavaScript from reading the reply, unless Django says it is fine.

The way Django says it is fine is by putting a header on the response, `Access-Control-Allow-Origin`, naming the origin that is allowed. `django-cors-headers` is the library that adds that header, and `CORS_ALLOWED_ORIGINS` is the list it checks:

backend/lms/settings.py

```
# The Vite dev server runs on 5180 (see frontend/vite.config.js). 5173 is kept
# here because that is Vite's own default and the port the README used to name.
CORS_ALLOWED_ORIGINS = [
    'http://localhost:5180',
    'http://127.0.0.1:5180',
    'http://localhost:5173',
    'http://127.0.0.1:5173',
]
```

Four entries, two ports times two ways of writing "this computer". Both spellings are here because the browser compares the origin as **text**. If you open the app at `http://127.0.0.1:5180` and only `http://localhost:5180` were listed, the browser would compare the two strings, find them different, and block the call. Listing both means it does not matter which one you type in the address bar.

The port `5180` comes from the frontend's own config:

frontend/vite.config.js

```
// Which port to open on. Other projects on this machine already use
// 5173 to 5175, so this one uses 5180 to stay out of their way.
// If 5180 is busy too, Vite quietly picks the next free number and tells
// you which one in the terminal.
server: {
  port: 5180,
},
```

Read that last comment again, because it is the setup for the next callout.

COMMON MISTAKE

Vite starts on a port that is not 5180, and suddenly every request in the browser fails. This happens when something else on your machine already holds 5180. Vite does not stop and complain, it takes the next free port and prints it. The line in your terminal reads `Local: http://localhost:5181/` instead of `5180`. The app loads, the login page draws fine, and then every single API call fails.

How to spot it. Two clues together are conclusive. First, the browser console shows a message containing the words `has been blocked by CORS policy` and names your origin. Second, and this is the giveaway, the same request works perfectly from a terminal:

```
bash curl -i http://127.0.0.1:8001/api/teacher/
```

`curl` is a program that fetches a URL from the command line. It returns a normal reply here (a 401, since you sent no token, which still proves Django answered). `curl` is not a browser and does not enforce the same-origin rule, so it never cares about CORS. **Backend fine in curl, broken in the browser** is the signature of a CORS problem, every time. It is not a Django bug, not a broken URL, and not a login problem.

How to fix it. Either free port 5180 and restart `npm run dev` so Vite gets the port it asked for, or add the port Vite actually chose to `CORS_ALLOWED_ORIGINS`, both spellings, and restart Django. Changing `settings.py` needs a Django restart to take effect.

COMMON MISTAKE

Writing an origin with a trailing slash or a path, like

`'http://localhost:5180/'`. `django-cors-headers` checks this at startup and Django refuses to run, printing an error that contains `should not have path` and the id `corsheaders.E014`. An origin is scheme, host and port only. Nothing after the port.

TIP

The frontend chooses its API address in one line, `frontend/src/api.js` line 8, which reads the environment variable `VITE_API_URL` and falls back to `http://127.0.0.1:8001/api`. If you ever move Django to a different port, that is the other end of the same wire.

2.8 REST_FRAMEWORK: locked by default

Django REST Framework, DRF for short, is the library that turns Django models into a JSON API. An API is a set of URLs that return data instead of web pages, meant to be read by another program rather than by a person. One of those URLs is called an **endpoint**.

DRF is configured through a single Python dictionary named `REST_FRAMEWORK`. A dictionary is a set of names paired with values, written inside curly braces. Everything DRF does looks in here for its defaults, and any individual view can override them.

backend/lms/settings.py

```
REST_FRAMEWORK = {
    "DEFAULT_AUTHENTICATION_CLASSES": (
        "rest_framework_simplejwt.authentication.JWTAuthentication",
    ),
    # Without this, DRF's default is AllowAny, so any view added without its
    # own permission_classes would be public. Failing closed means a new view
    # is locked until somebody decides otherwise; the two views that really
    # are public (register, login) say so with AllowAny.
    "DEFAULT_PERMISSION_CLASSES": (
        "rest_framework.permissions.IsAuthenticated",
    ),
}
```

Two separate ideas live here, and beginners mix them up constantly.

Authentication answers "who is this?" `DEFAULT_AUTHENTICATION_CLASSES` is set to `JWTAuthentication`, which means: on every request, look at the `Authorization` header, find a token there, work out which user it belongs to, and attach that user to the request. If there is no token, the request simply has no user. Authentication never refuses anybody. It only identifies them.

Permission answers "is this person allowed to do this?" `DEFAULT_PERMISSION_CLASSES` is set to `IsAuthenticated`, which refuses anybody that authentication could not identify.

The phrase for that arrangement is **failing closed**. DRF ships with `AllowAny` as its default, which means a view you write and forget to protect is open to the internet. Flipping the default to `IsAuthenticated` reverses it: a view you write and forget about is **locked**, and it stays locked until you deliberately open it. The cost of forgetting changes from "leaked the whole student table" to "got a 401 while testing". That is a far better mistake to make.

As it happens, every view in `backend/views.py` today names its own `permission_classes`, so none of them fall back to this default. That does not make the setting pointless. It is the safety net for the next view somebody adds in a hurry.

The views that genuinely need to be public say so, one line each:

backend/backend/views.py

```
class LoginView(APIView):
    """Login view using phone and password."""

    permission_classes = [AllowAny]
```

`permission_classes` written on a view replaces the default from `settings.py` for that view alone.

NOTE

The comment in `settings.py` says register and login are the public pair. That was true when the comment was written. Registration was later locked down to admins only, so `RegisterView` now carries `permission_classes = [IsAdmin]`, and the genuinely public views are `LoginView`, `PasswordResetRequestView` and `PasswordResetConfirmView`. The comment is left as it is in the real file, and this book quotes the real file. Roles and `IsAdmin` get a chapter of their own later.

TIP

A **401** and a **403** are different answers and the difference is useful. 401 means "I do not know who you are", which almost always means the token was missing or expired. 403 means "I know exactly who you are and you are not allowed", which means a permission class refused you. When debugging, read the number before reading anything else.

2.9 SIMPLE_JWT: tokens, and why eight hours

A **token** is a small string the server hands you when you log in, and that you attach to every later request to prove it is still you.

The cloakroom is the analogy that fits. You hand over your coat, you get a numbered ticket. The cloakroom does not remember your face and does not need to. Later you show the ticket and you get your coat. Anybody holding that ticket gets that coat, so do not drop it, and the ticket is only good for tonight.

A JWT, JSON Web Token, is that ticket with two extra properties. It carries the information inside itself, such as which user id it belongs to, so the server does not have to look anything up. And it is signed with your `SECRET_KEY`, so a ticket somebody drew by hand does not verify.

Simple JWT hands out two tickets at login: a short-lived **access** token for everyday requests, and a longer-lived **refresh** token whose only job is to obtain a fresh access token when the first one expires.

backend/lms/settings.py

```
# The library default access token lifetime is 5 minutes, which forces a
# re-login mid-task during development. There is no token refresh endpoint,
# so the access token is widened instead.
SIMPLE_JWT = {
    "ACCESS_TOKEN_LIFETIME": timedelta(hours=8),
    "REFRESH_TOKEN_LIFETIME": timedelta(days=7),
    "AUTH_HEADER_TYPES": ("Bearer",),
}
```

`timedelta` is the `datetime` helper imported at the top of the file. `timedelta(hours=8)` is a length of time, not a moment in time.

Read the comment, because it explains a design decision you would otherwise find puzzling. Simple JWT's own default access lifetime is **five minutes**. That is a sensible number for a finished product, because a finished product has a refresh endpoint: the frontend notices the expiry, quietly swaps the refresh token for a new access token, and the user never sees it happen. **This project has no refresh endpoint**. So with the library default, you would be thrown back to the login page five minutes into reading a page of this book. Widening the access token to eight hours makes one login last a working day. The refresh token's seven days is the library's own default, left alone because nothing uses it except the blacklist.

`AUTH_HEADER_TYPES` is the word that must appear in front of the token in the request header. With `"Bearer"`, the header the frontend builds looks like this:

```
Authorization: Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...
```

"Bearer" means "whoever bears this". Exactly the cloakroom ticket. The frontend builds that header in one line:

`frontend/src/api.js`

```
headers.Authorization = `Bearer ${token}`;
```

COMMON MISTAKE

Sending the token without the word `Bearer`, or with a different word such as `Token` or `JWT`. Simple JWT will not recognise the header, so authentication finds no user, so the permission class refuses you, and you get a 401 while staring at a header that plainly contains a valid token.

2.10 ROOT_URLCONF, TEMPLATES, WSGI_APPLICATION

Three lines from `startproject` sit between the JWT block and the database block. They need one sentence each.

`backend/lms/settings.py`

```
ROOT_URLCONF = 'lms.urls'
```

The first file Django consults when a request arrives, to work out which view handles this URL. Section 2.15 opens it.

backend/lms/settings.py

```
TEMPLATES = [
    {
        'BACKEND': 'django.template.backends.django.DjangoTemplates',
        'DIRS': [],
        'APP_DIRS': True,
        'OPTIONS': {
            'context_processors': [
                'django.template.context_processors.request',
                'django.contrib.auth.context_processors.auth',
                'django.contrib.messages.context_processors.messages',
            ],
        },
    ],
]
```

Templates are HTML files with holes in them that Django fills in. This project's user interface is React, so it builds almost no HTML of its own. This block stays because Django's admin site and DRF's browsable API pages are built from templates, and they need it. `'DIRS': []` is empty because there is no project-level template folder. Leave the whole thing alone.

backend/lms/settings.py

```
WSGI_APPLICATION = 'lms.wsgi.application'
```

This names the object a web server hands requests to. WSGI is the agreed way a Python web application and a web server talk to each other. The value points at the variable `application` inside `backend/lms/wsgi.py`, which `startproject` also wrote. A production server would load it, and so does `runserver`, so this line is live even while you are only working locally. `startproject` wrote `lms/asgi.py` alongside it, which is the same idea for async servers, and this project does not use that one.

2.11 DATABASES and environment variables

A database is the program that stores your data on disk and answers questions about it. This project uses MySQL.

backend/lms/settings.py

```
DATABASES = {
    'default': {
        'ENGINE': 'django.db.backends.mysql',
        'NAME': os.getenv('DB_NAME', 'lms_db'),
        'USER': os.getenv('DB_USER', 'lms_user'),
        'PASSWORD': os.getenv('DB_PASSWORD', 'lms_user_pw'),
        'HOST': os.getenv('DB_HOST', '127.0.0.1'),
        'PORT': os.getenv('DB_PORT', '3306'),
    }
}
```

'default' is the name of this connection. Django supports several at once, and 'default' is the one everything uses unless told otherwise.

'ENGINE' is the translator. Django writes queries in a general way and the engine turns them into the dialect the actual database speaks. `django.db.backends.mysql` is the MySQL translator, and it is the reason `mysqlclient` is in `requirements.txt`. `startproject` writes `django.db.backends.sqlite3` here by default, which is why a stray `backend/db.sqlite3` can appear if you run `migrate` before this block is edited. Once it says `mysql`, nothing reads that file.

The other five entries introduce a pattern worth learning properly.

An **environment variable** is a named value that belongs to your terminal session rather than to any file. Your operating system hands the set of them to every program you start. They are the standard way to give a program a secret without writing the secret down in a file that gets committed to Git along with the code.

`os.getenv('DB_PASSWORD', 'lms_user_pw')` means: look for an environment variable called `DB_PASSWORD`, and if there is not one, use `'lms_user_pw'`. The second argument is the fallback. So this project works with zero configuration if you set up the database exactly as the README describes, and works with a different password without editing a single line of Python.

SETTING	ENV VAR	DEFAULT	WHAT IT IS
NAME	DB_NAME	lms_db	The database name inside MySQL
USER	DB_USER	lms_user	The MySQL account Django logs in as
PASSWORD	DB_PASSWORD	lms_user_pw	That account's password
HOST	DB_HOST	127.0.0.1	Which machine MySQL runs on
PORT	DB_PORT	3306	MySQL's port, 3306 is the standard one

The defaults line up with the SQL in the README, run once inside MySQL before your first `migrate`:


```
CREATE DATABASE lms_db;  
CREATE USER 'lms_user'@'127.0.0.1' IDENTIFIED BY 'lms_user_pw';  
GRANT ALL PRIVILEGES ON lms_db.* TO 'lms_user'@'127.0.0.1';
```

To override any of them, set the variable in the same terminal before starting Django:

```
export DB_PASSWORD='my-real-password'  
python manage.py runserver 8001
```

On Windows `cmd` that is `set DB_PASSWORD=my-real-password` instead.

COMMON MISTAKE

Setting an environment variable in one terminal window and starting `runserver` in a different one. The variable belongs to the session you typed it in, and nowhere else. The new window sees only the defaults, so Django tries `lms_user_pw` and MySQL refuses the login. The error text names access denied for `lms_user`, which reads as though the password in the file is wrong when the real problem is which terminal you are standing in.

CHECKPOINT

With MySQL running and the database created, run `python manage.py migrate`. You should see a list of lines beginning `Applying`, ending with `OK`, covering Django's own tables, the token blacklist tables, and this project's four `backend` migrations. If instead you get `Can't connect to MySQL server`, MySQL is not running. If you get `Access denied`, the user or password does not match.

2.12 AUTH_PASSWORD_VALIDATORS

When somebody sets a password, Django runs it past a list of validators. Each validator either says nothing or raises an error with a sentence explaining the refusal, and that sentence is what the user ends up reading.

backend/lms/settings.py

```
AUTH_PASSWORD_VALIDATORS = [
    {
        'NAME': 'django.contrib.auth.password_validation.UserAttributeSimilarityValidator',
    },
    {
        'NAME': 'django.contrib.auth.password_validation.MinimumLengthValidator',
    },
    {
        'NAME': 'django.contrib.auth.password_validation.CommonPasswordValidator',
    },
    {
        'NAME': 'django.contrib.auth.password_validation.NumericPasswordValidator',
    },
]
```

These four are Django's defaults, untouched. Here is what each one rejects:

VALIDATOR	REJECTS
UserAttributeSimilarityValidator	A password too much like the user's own username, first name, last name or email. Somebody called <code>rahim</code> cannot use <code>rahim123</code> .
MinimumLengthValidator	Anything shorter than 8 characters.
CommonPasswordValidator	Anything on a bundled list of the 20,000 most common passwords. <code>password</code> , <code>qwerty</code> , <code>letmein</code> and thousands of others are all refused.
NumericPasswordValidator	A password made only of digits. <code>84726194</code> is long enough and is not a common password, and is still refused, because digits alone are quick to guess by trying every combination.

These run wherever this project sets a password: the admin creating an account, a user changing their own password on the profile page, and the password reset flow. The refusal comes back as a normal validation error and the React page shows the sentence in its red banner.

TIP

These are the four rules that make `python manage.py createsuperuser` argue with you. (A superuser is an account that can do everything, including sign in to `/admin/`.) It is not broken. Pick a password of at least 8 characters that is not all digits and is not similar to the username, or answer `y` when it asks whether to use it anyway.

2.13 Time: TIME_ZONE and USE_TZ

backend/lms/settings.py

```
LANGUAGE_CODE = 'en-us'

TIME_ZONE = 'UTC'

USE_I18N = True

USE_TZ = True
```

`LANGUAGE_CODE` and `USE_I18N` control translation of Django's own text. Nothing in this project is translated, so they stay as generated.

The other two matter a great deal later.

`USE_TZ = True` tells Django to treat every date and time as **timezone-aware**, and to store all of them in the database in UTC. UTC, Coordinated Universal Time, is the world's reference clock. It has no daylight saving, it never shifts, and every local time in the world is some fixed offset from it.

`TIME_ZONE = 'UTC'` is the project's own timezone, used when Django needs to display a time without being told which zone to use.

Why this matters here: the LMS stores due dates.

backend/backend/models.py

```
due_date = models.DateTimeField()
```

That line is on the `Assignment` model. An assignment due date is a moment in time, and the people using this app may not sit in the same country. Storing everything in UTC means "the deadline" is one unambiguous instant, whoever is looking at it. The conversion to a human's local clock happens at the edges, in the browser. The frontend does exactly that: `frontend/src/pages/Assignments.jsx` turns the value from the `datetime-local` input into UTC before sending it, and back into local time when showing it. A teacher in Dhaka and a student in London see their own wall clock, and the database holds one number.

Two other fields in the same models file are filled in by the server rather than by a form. One is on `Enrollment` and one is on `Submission`:

backend/backend/models.py

```
enrollment_date = models.DateField(auto_now_add=True)
submitted_at = models.DateTimeField(auto_now_add=True)
```

`auto_now_add=True` means Django sets the value once, at creation, from the current time, and ignores anything you send for that field.

NOTE

`TIME_ZONE = 'UTC'` has one visible side effect you will notice within a minute. Django sets the process timezone from this setting, so the timestamp `runserver` prints in its startup banner is UTC, not your wall clock. If your clock says 11:40 and Django says 05:40, nothing is wrong. Your machine is six hours ahead of UTC.

COMMON MISTAKE

Assuming a due date shown as `2026-08-04 18:00` in the database is 6pm for the student. It is 6pm UTC. Always convert at the moment of display, never by adjusting the stored value.

2.14 Email, password reset, static files, and the auto field

Four small blocks close the file.

Password reset link and its expiry:

backend/lms/settings.py

```
FRONTEND_PASSWORD_RESET_URL = os.getenv("FRONTEND_PASSWORD_RESET_URL",
"http://localhost:5173/reset-password")

PASSWORD_RESET_TIMEOUT = 60 * 60 # 1 hour
```

`FRONTEND_PASSWORD_RESET_URL` is not a Django setting. It is a name this project invented, and it can do that because `settings.py` is just a Python file: any variable in it is readable through `django.conf.settings`. The reset view uses it to build the link that goes in the email:

backend/backend/views.py

```
reset_link = (
    f"{settings.FRONTEND_PASSWORD_RESET_URL}"
    f"?uid={uid}&token={token}"
)
```

`PASSWORD_RESET_TIMEOUT` is a real Django setting, and it is measured in seconds. `60 * 60` is written as a multiplication instead of `3600` so that a reader can see it means sixty seconds times sixty minutes. After an hour, the token in that link stops verifying and the link is dead.

Email:

```
backend/lms/settings.py
```

```
EMAIL_BACKEND = os.getenv("EMAIL_BACKEND",
    "django.core.mail.backends.console.EmailBackend")
DEFAULT_FROM_EMAIL = os.getenv("DEFAULT_FROM_EMAIL", "no-reply@example.com")
EMAIL_HOST = os.getenv("EMAIL_HOST", "smtp.gmail.com")
EMAIL_PORT = int(os.getenv("EMAIL_PORT", "587"))
EMAIL_HOST_USER = os.getenv("EMAIL_HOST_USER", "asif1920001@gmail.com")
EMAIL_HOST_PASSWORD = os.getenv("EMAIL_HOST_PASSWORD", "")
EMAIL_USE_TLS = os.getenv("EMAIL_USE_TLS", "True") == "True"
EMAIL_USE_SSL = os.getenv("EMAIL_USE_SSL", "False") == "True"
```

`EMAIL_BACKEND` is the one that decides what "sending an email" means. The default here is `console.EmailBackend`, which does not send anything at all. **It prints the whole message to the terminal where Django is running.** That is the right choice for a tutorial project: you can watch the reset flow work end to end without owning a mail server, without spamming anybody, and without a password sitting in a file.

The remaining lines are the settings the real SMTP backend would need if you switched `EMAIL_BACKEND` over. SMTP is the protocol mail servers speak. `EMAIL_HOST_PASSWORD` defaults to an empty string on purpose, so the file carries no secret.

Note the two conversions. `int(os.getenv("EMAIL_PORT", "587"))` is there because environment variables are always text and a port has to be a number. `os.getenv("EMAIL_USE_TLS", "True") == "True"` is there for the same reason: the string `"False"` counts as true in Python, so comparing against `"True"` is what turns the text into a real true-or-false value.

TIP

To watch a password reset actually happen, send a `POST` request to `/api/password-reset/` with an email address that exists, then look at the terminal running `runserver`. The full message appears there, link included. Copy the link out of the terminal. That is the whole flow. The README lists this as a known gap: the link points at a `/reset-password` page the frontend does not have yet.

Static files:

```
backend/lms/settings.py
```

```
STATIC_URL = 'static/'
```

The URL prefix for CSS and images Django itself serves. Here it matters only for the admin site's own stylesheets and DRF's browsable API pages.

And the last line, which has a story:

backend/lms/settings.py

```
# The existing migrations were generated on Django 6, where BigAutoField is the
# default primary key. Django 5.2 falls back to AutoField unless this is set,
# which makes `makemigrations` want to rewrite every primary key even though the
# database columns are already bigint.
DEFAULT_AUTO_FIELD = 'django.db.models.BigAutoField'
```

Every Django model gets an automatic `id` column, a whole number that counts up and identifies each row. That column is the table's **primary key**, the value used to point at one specific row.

`DEFAULT_AUTO_FIELD` decides what size that number is. `AutoField` is a 32-bit integer, up to about 2.1 billion. `BigAutoField` is 64-bit, which is a number nobody is going to run out of.

Remember the docstring at the top saying Django 6.0.6. The migration files in `backend/backend/migrations/` were generated then, and Django 6 uses `BigAutoField` by default, so every `id` in those files is written as `models.BigAutoField(...)` and the real MySQL columns are `bigint`. The project then pinned Django 5.2.7, whose fallback default is the smaller `AutoField`. Without this line, Django 5.2 compares what it thinks the models should be against what the migrations say, disagrees about every single primary key, and `makemigrations` offers to generate a migration shrinking all of them. The columns in the database are already the right size. Setting the default explicitly makes Django's expectation match reality, and `makemigrations` goes quiet.

CHECKPOINT Run `python manage.py makemigrations`. The correct answer at this point is `No changes detected`. If instead you see a proposed migration full of `AlterField` operations on `id` columns, `DEFAULT_AUTO_FIELD` is missing or misspelled. Do not accept that migration. Fix the setting instead.

2.15 lms/urls.py: the two front doors

`ROOT_URLCONF = 'lms.urls'` pointed at this file. It is the first thing Django consults for every request, and in this project it is very short.

`startproject` generates it with a long comment block explaining the options and an empty `urlpatterns`. Only two lines were added:

backend/lms/urls.py

```
from django.contrib import admin
from django.urls import include, path

urlpatterns = [
    path('admin/', admin.site.urls),
    path('api/', include('backend.urls')),
]
```

`urlpatterns` is a list, checked from top to bottom. The first entry whose pattern matches the start of the URL wins.

`path('admin/', admin.site.urls)` hands everything beginning with `admin/` to Django's built-in admin website. That is `startproject`'s own line, kept because this project relies on the admin heavily: it is where you give the first superuser a Profile, where you change somebody's role, and the quickest way to add test data.

`path('api/', include('backend.urls'))` is the added line and the interesting one. `include()` means "chop `api/` off the front of the URL and hand what is left to that other file". So a request for `/api/teacher/5/` arrives here, matches `api/`, and `backend/urls.py` is asked about `teacher/5/`.

That is why the app's own URL file never mentions `api/`. Here are the first few lines of it:

`backend/backend/urls.py`

```
urlpatterns = [
    path('login/', LoginView.as_view(), name='login'),
    path('register/', RegisterView.as_view(), name='register'),
    path('profile/', ProtectedView.as_view(), name='protected'),
```

`login/` in that file is reached at `/api/login/` in the browser. The prefix is added once, here, and every route in the app inherits it. Change `'api/'` to `'v1/'` in this one file and all twenty-two routes in `backend/urls.py` move at once. That file gets a chapter of its own later.

COMMON MISTAKE

Leaving the trailing slash off a URL. Every path in this project ends with `/`. `http://127.0.0.1:8001/api/teacher` without the slash still works for a `GET` (the request a browser makes when it just fetches a page), because `CommonMiddleware` redirects it, but a `POST` (the request that sends data in) can lose its body in that redirect. Always include the trailing slash.

2.16 Start it up

Everything in this chapter is in place. Time to prove it.

Make sure MySQL is running, your virtual environment is active, and you are in the `backend/` folder next to `manage.py`. First ask Django to check itself over without starting anything:

```
python manage.py check
```

The only correct output is one line:

```
System check identified no issues (0 silenced).
```

That single line means a lot: `settings.py` parsed as valid Python, every app in `INSTALLED_APPS` was found and imported, the middleware paths all exist, and `django-cors-headers` approved every entry in `CORS_ALLOWED_ORIGINS`. Now start the server:

```
python manage.py runserver 8001
```

CHECKPOINT The terminal should print something very close to this, with today's date and no red text anywhere:

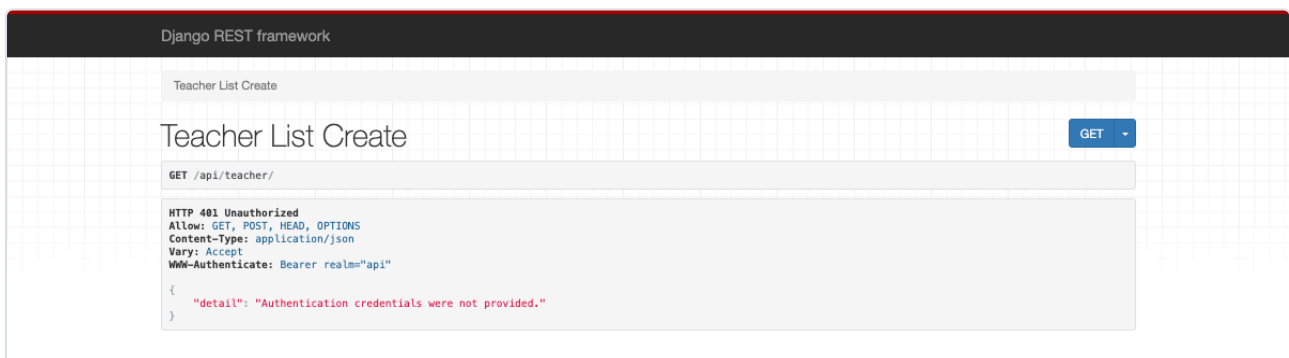
```
``` Watching for file changes with StatReloader Performing system checks...
```

```
System check identified no issues (0 silenced). August 04, 2026 - 05:40:47 Django
version 5.2.7, using settings 'lms.settings' Starting development server at
http://127.0.0.1:8001/ Quit the server with CONTROL-C. ```
```

Check three things in that banner. The version says **5.2.7**, matching `requirements.txt`. The settings module says `lms.settings`, which is the file you just spent a chapter on. And the check line says **no issues**. The paragraph underneath about this being a development server is a normal warning, not an error, and it always appears.

Remember section 2.13: that clock is UTC, so it will not match your wall clock unless your country happens to sit on UTC.

Now open **`http://127.0.0.1:8001/api/teacher/`** in a browser. You should get a DRF page saying authentication credentials were not provided, with the status **401 Unauthorized**.



**Figure 2.1** The DRF browsable API page at `/api/teacher/`, refusing an anonymous visitor with a 401

That is not a failure. It means the URL routing found the view and the view refused a visitor it could not identify. The class doing the refusing on this particular URL is `AdminWrites`, from `backend/permissions.py`, which `TeacherListCreateView` names itself; it requires a signed-in user before it looks at anything else. `DEFAULT_PERMISSION_CLASSES` from section 2.8 is the same idea applied to any view that does not name one.



If you get a 404 instead, check `lms/urls.py`. If you get a 200 with a list of teachers, something is wrong with permissions: check that `REST_FRAMEWORK` is spelled correctly in `settings.py` and that the view still has its `permission_classes`.

If `runserver` dies with `pywatchman.SocketTimeout: timed out waiting for response`, that is the auto-reloader failing to reach a file-watching tool called Watchman, not a problem with anything in this chapter. Start it with `python manage.py runserver --noreload`, which means you restart Django yourself after each edit, or raise the timeout with `DJANGO_WATCHMAN_TIMEOUT=20`.

**TIP**

`settings.py` is read once, when the server starts. The auto-reloader restarts Django when you save a `.py` file, so most edits take effect on their own, but if you ever change a setting and nothing seems different, stop the server with Ctrl+C and start it again before hunting for a bug.

## What you have now

- A Django project at `backend/lms/` and an app at `backend/backend/`, created with `startproject` and `startapp`
- A virtual environment with Django 5.2.7, DRF, Simple JWT, django-cors-headers and mysqlclient installed from `requirements.txt`
- `INSTALLED_APPS` carrying your app plus the three added libraries, and `MIDDLEWARE` with `CorsMiddleware` in the one position that works
- A CORS allow list that lets the Vite dev server on 5180 or 5173 talk to Django, and the knowledge that "works in curl, blocked in the browser" always means this setting
- An API that fails closed: any view added without its own `permission_classes` needs a valid JWT
- Login tokens that last 8 hours instead of 5 minutes, sent as `Authorization: Bearer <token>`
- A MySQL connection whose five values can be overridden with environment variables, and no secrets required to run locally
- All times stored in UTC, ready for assignment due dates
- Password reset emails that print to your terminal instead of going anywhere
- Two URL routes, `admin/` and `api/`, with everything else deferred to `backend/urls.py`
- A server that starts clean and answers `/api/teacher/` with a 401

# The Database: `models.py`, Migrations and Django Admin

In Chapter 2 you got an empty Django project talking to a MySQL database. Right now that database holds nothing you care about. This chapter fills it.

By the end you will have nine tables, a way to change them safely as you learn, and a ready-made web page where you can add and edit rows without writing a single line of HTML.

## What a model is

A **model** is a Python class that becomes a database table.

That is the whole idea. Three words are worth learning once and then forgetting about:

IN PYTHON	IN THE DATABASE	PLAIN ENGLISH
a class, like <code>Teacher</code>	a table called <code>backend_teacher</code>	the shape: "a teacher has a name, an email and a subject"
a class attribute, like <code>name = models.CharField(...)</code>	a column called <code>name</code>	one piece of information
an object, like <code>Teacher(name="Rahim")</code>	a row	one actual teacher

You write the class. Django writes the **SQL** (the language databases speak) that creates the table. You never type `CREATE TABLE` in this book.

### NOTE

A **table** is a grid of data, like one sheet in a spreadsheet. A **column** is a heading across the top ("name", "email"). A **row** is one line of data under those headings. If you have used Excel you already know what a table is.

## Where `models.py` lives

Your Django app is called `backend` and it sits inside the `backend` folder of the project. Yes, the name appears twice. The outer one is a folder on disk, the inner one is the app.

```
from the lms/ folder
cd backend
ls backend/
```

You should see `models.py`, `admin.py`, `views.py`, `serializers.py`, `permissions.py`, `urls.py`, `apps.py`, `tests.py` and a `migrations/` folder. `models.py` is the file this chapter starts with. Open it.

#### TIP

Every command in this chapter is run from the `lms/backend` folder, the one that holds `manage.py`. If a command says "No such file or directory: `manage.py`", you are in the wrong folder.

## The two imports at the top

Open `backend/backend/models.py` and make sure the first lines look like this.

`backend/backend/models.py`

```
from django.db import models
from django.contrib.auth.models import User

Create your models here.
```

`models` is the toolbox. Everything you write below comes out of it: `models.Model`, `models.CharField`, `models.ForeignKey`.

`User` is a model Django wrote for you. It already has a username, a password (stored scrambled, never in plain text), a first name, a last name, an email and an "is this person a superuser" flag. You are not going to rebuild any of that. You are going to attach your own extra fields to it.

Django created this file with the first import and the `# Create your models here.` comment. The comment is still in the real file. Leave it. Everything else in this chapter gets added below it.

---

## Model 1: Profile

**Why it exists:** Django's `User` knows a username and a password. This app logs people in with a **phone number** and gives each person a **role**. `Profile` is where those two extra facts live.

Start with the role constants.

backend/backend/models.py

```
class Profile(models.Model):
 ADMIN = "admin"
 TEACHER = "teacher"
 STUDENT = "student"

 ROLE_CHOICES = [
 (ADMIN, "Admin"),
 (TEACHER, "Teacher"),
 (STUDENT, "Student"),
]
```

`ADMIN`, `TEACHER` and `STUDENT` are plain Python strings with names. They are here so that the rest of the code can write `Profile.ADMIN` instead of typing `"admin"` by hand in twenty places. Type `"adm"` by accident and Python runs happily and the app misbehaves. Type `Profile.ADMN` and Python stops immediately and tells you. That is the entire benefit, and it is worth it.

`ROLE_CHOICES` is a list of pairs. In each pair the **left** value is what gets stored in the database, and the **right** value is what a human sees in a dropdown. So the database holds `"teacher"` and the admin page shows `Teacher`.

Now the three fields.

```
backend/backend/models.py (inside class Profile)
user = models.OneToOneField(User, on_delete=models.CASCADE)
phone = models.CharField(max_length=20, unique=True)
Anyone can register, so a new account gets the least privileged role.
Promote people from /admin/ once you know who they are.
role = models.CharField(max_length=10, choices=ROLE_CHOICES, default=STUDENT)
```

Three new things appear here, so meet all three.

**CharField** stores a short piece of text. `max_length` is not optional: the database needs to know how many characters to reserve. Twenty characters is plenty for a phone number, ten is plenty for the word `"teacher"`.

**unique=True** tells the database that no two rows may hold the same value. Two people cannot register the same phone number. If someone tries, the save is refused. This matters more than it looks: the login code finds you by phone, so if two accounts shared one phone number, login would not know which account you meant.

**OneToOneField** is a link to exactly one row in another table, and that row can only be claimed once. One `User` gets one `Profile`, and one `Profile` belongs to one `User`. Never two profiles for one user.

`choices=ROLE_CHOICES` limits what may be stored, and makes the Django admin show a dropdown instead of a free text box.

`default=STUDENT` means: if nobody says what the role should be, store `"student"`. The role with the fewest permissions is the safe one to fall back to.

#### NOTE

The comment in the real file says "Anyone can register". That was true when the comment was written. It is not true any more. `RegisterView` in `views.py` is admin-only now, and its own docstring says so: "There is no public sign-up." Comments go stale. Trust the code, and fix comments when you notice them.

Finally, `__str__`.

```
backend/backend/models.py (inside class Profile)
def __str__(self):
 return f"{self.user.username} - {self.phone} ({self.role})"
```

`__str__` (two underscores each side) answers one question: "when something needs to print this object as text, what should it say?" Without it, Django shows `Profile object (1)`, which tells you nothing. With it, you see `rahim - 01711223344 (admin)`.

You will see that string in the Django admin list and in every dropdown that points at a profile.

## PROFILE FIELDS

FIELD	TYPE	WHAT IT MEANS	BLANK / NULL / DEFAULT
<code>user</code>	<code>OneToOneField(User)</code>	the login account this profile belongs to	required
<code>phone</code>	<code>CharField(max_length=20)</code>	the number you log in with	required, <code>unique=True</code>
<code>role</code>	<code>CharField(max_length=10)</code>	<code>admin</code> , <code>teacher</code> or <code>student</code>	<code>default="student"</code>

Django also adds an `id` column for you, automatically, to every model. You never write it. It is a whole number that counts up, and it is how every other table points at this one.

## Model 2: Teacher

**Why it exists:** it is the record of a person who teaches, with the subject they teach.

backend/backend/models.py

```
class Teacher(models.Model):

 name = models.CharField(max_length=100, blank=True, null=True)
 email = models.EmailField(blank=True, null=True)
 subject = models.CharField(max_length=100)
 is_active = models.BooleanField(default=True)

 def __str__(self):
 return f"{self.name} - {self.subject}"
```

**EmailField** is a **CharField** that also checks the text looks like an email address before saving. It sets its own **max\_length** of 254, which is the longest an email address is allowed to be.

**BooleanField** stores true or false, nothing else. **default=True** means a new teacher is active unless you say otherwise.

Now the pair that confuses everybody: **blank=True, null=True**.

- **null=True** is about the **database**. It says the column is allowed to hold nothing at all.
- **blank=True** is about **forms and validation**. It says a form may be submitted with this box left empty.

They are separate because they answer different questions, and you usually want both together on an optional text field, which is what happens here. **name** and **email** are optional. **subject** is not: it has neither flag, so the database refuses a teacher with no subject.

#### COMMON MISTAKE

Setting **null=True** on a text field but forgetting **blank=True**. The database will accept nothing, but the form still shouts "This field is required." Both, or neither, on optional text.

## TEACHER FIELDS

FIELD	TYPE	WHAT IT MEANS	BLANK / NULL / DEFAULT
name	CharField(max_length=100)	the teacher's name	blank=True, null=True
email	EmailField	contact address	blank=True, null=True
subject	CharField(max_length=100)	what they teach	required
is_active	BooleanField	still teaching here?	default=True

**\_\_str\_\_** returns something like **Md. Rahim - Physics**. That exact string is what fills the **Teacher** dropdown on the Courses page in Django admin, which is why it is worth putting the subject in there: two teachers named Rahim are told apart by their subject.

## Model 3: Student

**Why it exists:** the record of a person who studies here.

backend/backend/models.py

```
class Student(models.Model):
 name = models.CharField(max_length=100, blank=True, null=True)
 email = models.EmailField(blank=True, null=True)
 enrollment_date = models.DateField()
 is_active = models.BooleanField(default=True)
 roll_number = models.CharField(max_length=100, blank=True, null=True)

 def __str__(self):
 return f"{self.name} - {self.email}"
```

**DateField** stores a calendar date and nothing else: year, month, day. No hours, no minutes. It is written as `2026-08-04`, which is year, then month, then day.

Notice what is *not* on this `DateField`: no `auto_now_add`. So `Student.enrollment_date` is a date **you** type in. It is required. Leave it out and the save fails.

### COMMON MISTAKE

There are two fields called `enrollment_date` in this project and they behave in opposite ways. `Student.enrollment_date` is typed by a human and appears in the Students form. `Enrollment.enrollment_date` is stamped by the server and never appears in a form. Same name, different rules. You meet the second one two models from now.

## STUDENT FIELDS

FIELD	TYPE	WHAT IT MEANS	BLANK / NULL / DEFAULT
name	CharField(max_length=100)	the student's name	blank=True, null=True
email	EmailField	contact address	blank=True, null=True
enrollment_date	DateField	the day they joined the institution	required, typed by hand
is_active	BooleanField	still studying here?	default=True
roll_number	CharField(max_length=100)	the institution's own id for them	blank=True, null=True

`__str__` returns `Fatima Akter - fatima@example.com`. That is what you will see in the **Student** dropdown when you create an enrollment or a submission.

## Model 4: Course

**Why it exists:** a subject being taught, owned by one teacher.

backend/backend/models.py

```
class Course(models.Model):
 title = models.CharField(max_length=100)
 description = models.TextField()
 teacher = models.ForeignKey(Teacher, on_delete=models.CASCADE)

 def __str__(self):
 return self.title
```

**TextField** is text with no length limit. Use **CharField** for a line, **TextField** for a paragraph. A course title is a line. A course description is a paragraph.

**ForeignKey** is the important one. It is a link from this row to **one** row in another table, and unlike **OneToOneField**, the other row can be linked to many times. One teacher, many courses. This is how the whole app is stitched together.

What actually lands in the database is a column called `teacher_id` holding a number: the `id` of a row in the teacher table. In Python you write `course.teacher` and get the whole **Teacher** object. But the **API** (the set of web addresses the React app calls to read and write data) sends **JSON**, a plain-text format made of names and values, and there the teacher is just the number: `{"teacher": 3}`. Both are the same link seen from different sides. The project README lists this under "Things that surprise people", because the React pages have to fetch `/api/teacher/` as well and match the numbers up in JavaScript.

`__str__` here returns just `self.title`, so a course prints as `Physics 101`.

### COURSE FIELDS

FIELD	TYPE	WHAT IT MEANS	BLANK / NULL / DEFAULT
<code>title</code>	<code>CharField(max_length=100)</code>	course name	required
<code>description</code>	<code>TextField</code>	what the course covers	required
<code>teacher</code>	<code>ForeignKey(Teacher)</code>	who teaches it	required, <code>on_delete=CASCADE</code>



## NOTE

`Course` mentions `Teacher` by writing the class name directly. Python reads the file from top to bottom, so `Teacher` has to be defined **above** `Course`. That is why the models appear in this order in the real file. Move `Course` above `Teacher` and Python fails with `NameError: name 'Teacher' is not defined` before Django even starts.

## on\_delete=models.CASCADE, and why the delete dialog says what it says

Every `ForeignKey` and `OneToOneField` must answer one question: **if the row I point at is deleted, what happens to me?** `on_delete` is that answer. This project gives the same answer everywhere: `models.CASCADE`, meaning "delete me too."

Follow it through with real data from this app.

You have a teacher, Rahim. Rahim teaches two courses. Each course has lessons and assignments. Students are enrolled in those courses, have handed in submissions, and some of those submissions have been graded.

Now you delete Rahim. Here is what goes, in order:

```
Teacher: Rahim
|-- Course: Physics 101 (teacher -> CASCADE)
| |-- Lesson: Newton's laws (course -> CASCADE)
| |-- Assignment: Problem set 1 (course -> CASCADE)
| |-- Submission by Fatima (assignment -> CASCADE)
| |-- Result: 87 (submission -> CASCADE)
| |-- Enrollment: Fatima (course -> CASCADE)
|-- Course: Physics 201
 |-- ...the same again
```

One click removes all of it. The `Student` rows survive, because nothing links a student to a teacher. Everything hanging off the courses does not.

This is not a hidden detail. The React app warns about it in plain words. Here is the real text from three of the delete dialogs you will build later, with the name filled in at the moment the dialog opens:

PAGE	WHAT THE DIALOG SAYS
Teachers	Delete Md. Rahim? Their courses go too. This cannot be undone.
Students	Delete Fatima Akter? Their enrollments and submissions go too. This cannot be undone.
Courses	Delete Physics 101? Its lessons, assignments and enrollments go too. This cannot be undone.

Those sentences are true because of `on_delete=models.CASCADE` in `models.py`. The model decides what happens, the dialog just tells the truth about it. When you get to the chapter that builds `ConfirmDialog`, come back to this page and check that each warning still matches the model.

**TIP**

`CASCADE` is not the only choice. `models.PROTECT` refuses the delete and raises an error, and `models.SET_NULL` keeps the row and blanks the link. This project uses `CASCADE` everywhere because a lesson without a course, or a grade without a submission, would be meaningless data. Know that the alternatives exist, then move on.

## Model 5: Enrollment

**Why it exists:** it records that one student is taking one course. A student takes many courses; a course has many students. A join table like this is how you say that.

backend/backend/models.py

```
class Enrollment(models.Model):
 student = models.ForeignKey(Student, on_delete=models.CASCADE)
 course = models.ForeignKey(Course, on_delete=models.CASCADE)
 enrollment_date = models.DateField(auto_now_add=True)

 def __str__(self):
 return f'{self.student.name} enrolled in {self.course.title}'
```

`auto_now_add=True` means: when this row is created for the first time, the server writes today's date into it. Nobody types it. Nobody can send it. Updating the row later does not change it.

Three consequences, and all three show up in this app:

1. The field is **read-only** through the API. A **serializer** is the small class that decides which fields go out as JSON and which may come in; you will write them in the next chapter. `EnrollmentSerializer` lists `enrollment_date` in its `fields`, but Django REST Framework sees `auto_now_add` on the model and makes the field read-only anyway. Send `{"enrollment_date": "2020-01-01"}` and the server ignores it without complaining.
2. So the Enrollments page in React shows a date **column** in the table, but has no date **box** in the form. The code even says so: "enrollment\_date is auto\_now\_add on the server. Sending it is silently ignored, so it is shown in the table but never in the form."
3. The date is the server's date, not the browser's. That is the point. A user cannot backdate an enrollment by changing the clock on their laptop.

## ENROLLMENT FIELDS

FIELD	TYPE	WHAT IT MEANS	BLANK / NULL / DEFAULT
student	ForeignKey(Student)	who is enrolled	required, CASCADE
course	ForeignKey(Course)	in what	required, CASCADE
enrollment_date	DateField(auto_now_add=True)	the day the row was created	set by the server, read-only

`__str__` returns `Fatima Akter enrolled in Physics 101`, which reads like a sentence in the admin list.

---

## Model 6: Lesson

**Why it exists:** one unit of teaching inside a course.

backend/backend/models.py

```
class Lesson(models.Model):
 title = models.CharField(max_length=100)
 description = models.TextField()
 course = models.ForeignKey(Course, on_delete=models.CASCADE)

 def __str__(self):
 return self.title
```

Nothing new here, which is the point: once you know `CharField`, `TextField` and `ForeignKey`, most models are a rearrangement of those three.

## LESSON FIELDS

FIELD	TYPE	WHAT IT MEANS	BLANK / NULL / DEFAULT
title	CharField(max_length=100)	lesson name	required
description	TextField	the lesson content	required
course	ForeignKey(Course)	which course it belongs to	required, CASCADE

`__str__` returns the title, so a lesson prints as `Newton's laws`.

---

## Model 7: Assignment

**Why it exists:** work given out to students, with a deadline.

backend/backend/models.py

```
class Assignment(models.Model):
 title = models.CharField(max_length=100)
 description = models.TextField()
 lesson = models.ForeignKey(Lesson, on_delete=models.CASCADE)
 course = models.ForeignKey(Course, on_delete=models.CASCADE)
 due_date = models.DateTimeField()

 def __str__(self):
 return self.title
```

`DateTimeField` stores a date **and** a time. A deadline of "the 20th" is not enough; you need "the 20th at 23:59". In `backend/lms/settings.py` the project sets `TIME_ZONE = 'UTC'` and `USE_TZ = True`, so what goes into the database is a moment in UTC, the world clock that time zones are measured from. The React Assignments page converts in both directions: it turns the stored moment into the browser's local time for display, and calls `toISOString()` before sending, so the server always gets UTC back. That is frontend work you will do later, but it starts here, with the choice of `DateTimeField` over `DateField`.

Notice that `Assignment` points at **both** a lesson and a course. The lesson already knows its course, so this is a duplicate link. It makes the assignment list easy to filter by course without walking through the lesson, and it means the frontend can narrow the lesson dropdown to the chosen course. It also means the two can disagree if you are careless: nothing in the model stops you saving an assignment whose lesson belongs to a different course. The React form is what keeps them lined up.

### ASSIGNMENT FIELDS

FIELD	TYPE	WHAT IT MEANS	BLANK / NULL / DEFAULT
<code>title</code>	<code>CharField(max_length=100)</code>	assignment name	required
<code>description</code>	<code>TextField</code>	the task	required
<code>lesson</code>	<code>ForeignKey(Lesson)</code>	the lesson it belongs to	required, <code>CASCADE</code>
<code>course</code>	<code>ForeignKey(Course)</code>	the course it belongs to	required, <code>CASCADE</code>
<code>due_date</code>	<code>DateTimeField</code>	deadline, date and time	required, typed by hand

`__str__` returns the title, so an assignment prints as `Problem set 1`.

---

## Model 8: Submission

**Why it exists:** the work a student hands in for an assignment.

backend/backend/models.py

```
class Submission(models.Model):
 assignment = models.ForeignKey(Assignment, on_delete=models.CASCADE)
 student = models.ForeignKey(Student, on_delete=models.CASCADE)
 submitted_at = models.DateTimeField(auto_now_add=True)
 content = models.TextField()

 def __str__(self):
 return f"Submission by {self.student.name} for {self.assignment.title}"
```

`submitted_at` is the second `auto_now_add` field in the project, and it is the one where the reason is obvious. If a student could send their own `submitted_at`, every submission would arrive one minute before the deadline. The server stamps it. It cannot be sent and it cannot be edited afterwards.

Same three consequences as before: read-only in the API, shown as a column but never as a form box, and taken from the server's clock.

### SUBMISSION FIELDS

FIELD	TYPE	WHAT IT MEANS	BLANK / NULL / DEFAULT
<code>assignment</code>	<code>ForeignKey(Assignment)</code>	what is being answered	required, <code>CASCADE</code>
<code>student</code>	<code>ForeignKey(Student)</code>	who handed it in	required, <code>CASCADE</code>
<code>submitted_at</code>	<code>DateTimeField(auto_now_add=True)</code>	the moment the row was created	set by the server, read-only
<code>content</code>	<code>TextField</code>	the answer itself	required

`__str__` returns `Submission by Fatima Akter for Problem set 1`. That is the string Django admin puts in the **Submission** dropdown when you grade work.

## Model 9: Results

**Why it exists:** the score and feedback given to one submission.

backend/backend/models.py

```
class Results(models.Model):
 submission = models.OneToOneField(Submission, on_delete=models.CASCADE)
 score = models.FloatField()
 feedback = models.TextField()

 def __str__(self):
 return f"Results for {self.submission.student.name} - {self.score}"
```

**FloatField** stores a number that may have a decimal point: 87, 87.5, 9.25. A score of 87.5 is normal, so a whole-number field would be wrong here.

**OneToOneField** is the whole design of this model. It is a **ForeignKey** with **unique=True** welded on. One submission can be graded exactly once.

Try to grade the same submission twice and the database refuses. Django REST Framework turns that refusal into an HTTP **400 Bad Request** (the reply code that means "your request was not acceptable") with the message:

```
submission: results with this submission already exists.
```

That is real, and it is ugly, so the React Results page catches it and rewrites it:

frontend/src/pages/Results.jsx

```
if (problem.message.includes("already exists")) {
 setError("That submission has already been graded. Edit its result instead.");
} else {
 setError(problem.message);
}
```

The same page also leaves already-graded submissions out of the dropdown, so a user has to work hard to hit the error at all. Two layers: the model makes it impossible, the interface makes it unlikely.

#### NOTE

The model is called **Results**, plural, while every other model is singular. The web addresses follow it: **/api/results/** is plural while **/api/submission/** is singular. This is an inconsistency in the project, not a rule you should copy. It is left alone because renaming a model means renaming its table, and the README warns you about it instead. When you build your own project, keep every model singular.

## RESULTS FIELDS

FIELD	TYPE	WHAT IT MEANS	BLANK / NULL / DEFAULT
submission	OneToOneField(Submission)	the work being graded, once only	required, CASCADE
score	FloatField	the mark, decimals allowed	required
feedback	TextField	the teacher's comments	required

`__str__` returns `Results for Fatima Akter - 87.5`.

---

## Every field type you have met

Keep this near you. Nine models, and they use nine kinds of field between them.

FIELD TYPE	STORES	MUST YOU PASS ANYTHING?
CharField	short text, one line	yes, <code>max_length</code>
TextField	long text, any length	no
EmailField	text that must look like an email	no, <code>max_length</code> defaults to 254
BooleanField	true or false	no, but pass <code>default=</code>
DateField	a calendar date	no
DateTimeField	a date and a time	no
FloatField	a number with decimals	no
ForeignKey	a link to one row, reusable	yes, the model and <code>on_delete=</code>
OneToOneField	a link to one row, claimed once	yes, the model and <code>on_delete=</code>

And the options that appear on them:

OPTION	MEANING
<code>max_length=100</code>	the longest text the column accepts
<code>null=True</code>	the database column may be empty
<code>blank=True</code>	a form may leave the box empty
<code>default=True</code>	value used when nothing is given
<code>unique=True</code>	no two rows may share this value
<code>choices=...</code>	only these values allowed, shown as a dropdown
<code>auto_now_add=True</code>	the server stamps it on creation, read-only after
<code>on_delete=models.CASCADE</code>	delete me when the row I point at is deleted

## Migrations

You have written nine classes. The database still has nothing in it. Python code and database tables are two different things, and something has to carry changes from one to the other. That something is a **migration**: a file that records one step of change to the database.

Two commands do all of it.

### MAKEMIGRATIONS: WRITE DOWN THE PLAN

```
python manage.py makemigrations
```

Django compares your `models.py` against the migrations it has already recorded, works out what changed, and writes a new Python file into `backend/backend/migrations/`. It does **not** touch the database. It only writes the plan.

The output looks like this:

```
Migrations for 'backend':
 backend/migrations/0001_initial.py
 + Create model Student
 + Create model Teacher
 + Create model Profile
```

### MIGRATE: CARRY THE PLAN OUT

```
python manage.py migrate
```



Django looks at which migration files have already been applied, finds the ones that have not, turns each into SQL, and runs it against MySQL. Now the tables exist.

Django remembers what it has run in a table called `django_migrations`. That is how running `migrate` twice does nothing the second time, and how a teammate who pulls your code gets exactly the same tables by running one command.

**CHECKPOINT** Run `python manage.py showmigrations backend`. Every line should start with `[X]`, meaning applied. A line starting with `[ ]` is a migration file that exists but has not been run. Run `migrate` again.

## WHAT IS ACTUALLY INSIDE A MIGRATION FILE

A migration is ordinary Python. It has a name, a list of migrations it depends on, and a list of operations. Nothing magic. You can open one and read it, and you should.

**TIP** To see the real SQL a migration will run, without running it: `python manage.py sqlmigrate backend 0001`. Reading that output once, early, does more to explain what models are than any amount of prose.

## The four real migrations in this project

Look inside `backend/backend/migrations/`. There are four numbered files plus an empty `__init__.py` that just marks the folder as a Python package.

### 0001\_INITIAL.PY

The first migration. It created three tables: `Student`, `Teacher` and `Profile`. Here is its header and the `Profile` part.

`backend/backend/migrations/0001_initial.py`

```
class Migration(migrations.Migration):

 initial = True

 dependencies = [
 migrations.swappable_dependency(settings.AUTH_USER_MODEL),
]
```

`initial = True` marks this as the first migration for the app.

`dependencies` is a list of migrations that must run **before** this one.

`swappable_dependency(settings.AUTH_USER_MODEL)` means "whatever migration creates the user table has to have run first." It has to, because `Profile` points at `User` and you cannot point at a table that does not exist yet.

backend/backend/migrations/0001\_initial.py

```
migrations.CreateModel(
 name='Profile',
 fields=[
 ('id', models.BigAutoField(auto_created=True, primary_key=True,
serialize=False, verbose_name='ID')),
 ('phone', models.CharField(max_length=20, unique=True)),
 ('user', models.OneToOneField(on_delete=django.db.models.deletion.CASCADE,
to=settings.AUTH_USER_MODEL)),
],
),
```

Two things to notice.

There is `id`, the column you never wrote. `BigAutoField` means a large whole number that Django increases by one for each new row. `primary_key=True` means it is the column that identifies the row.

And there is **no role field**. At this point in the project's life, roles did not exist. That comes later, and that is the whole reason migrations are interesting.

## 0002\_COURSE\_ENROLLMENT\_LESSON\_ASSIGNMENT\_SUBMISSION\_AND\_MORE.PY

The long file name is Django's own doing: it stitches together the names of what the migration creates and gives up with `_and_more`. This one added the other six tables: `Course`, `Enrollment`, `Lesson`, `Assignment`, `Submission` and `Results`.

backend/backend/migrations/0002\_course\_enrollment\_lesson\_assignment\_submission\_and\_more.py

```
dependencies = [
 ('backend', '0001_initial'),
]
```

It depends on `0001`, because `Course` points at `Teacher` and `Teacher` was created there.

Look at how the links were written out:

backend/backend/migrations/0002\_course\_enrollment\_lesson\_assignment\_submission\_and\_more.py

```
 ('teacher', models.ForeignKey(on_delete=django.db.models.deletion.CASCADE,
to='backend.teacher')),
```

```
backend/backend/migrations/0002_course_enrollment_lesson_assignment_submission_and_more.py
```

```
 ('submission',
models.OneToOneField(on_delete=django.db.models.deletion.CASCADE,
to='backend.submission')),
```

`to='backend.teacher'` is the app label plus the model name, all lowercase. A migration never imports your model classes, because your classes change over time and a migration has to keep working as it was written. It refers to models by name instead.

### 0003\_PROFILE\_ROLE.PY

This is the migration that added roles, months after the first two. It changes one thing.

```
backend/backend/migrations/0003_profile_role.py
```

```
operations = [
 migrations.AddField(
 model_name='profile',
 name='role',
 field=models.CharField(choices=[('admin', 'Admin'), ('teacher', 'Teacher'),
('student', 'Student')], default='student', max_length=10),
),
]
```

`AddField`, not `CreateModel`. The table already existed; this added a column to it.

The `default='student'` is doing two jobs. Going forward, new profiles get `student`. And right now, at the moment this migration runs, every profile row that already exists gets `student` written into its brand new column, because a column cannot be left undefined.

Which caused a problem.

### 0004\_EXISTING\_ACCOUNTS\_BECOME\_ADMINS.PY

Read the header line of `0001` and then of `0003`. `0001` says `# Generated by Django 6.0.6 on 2026-06-09`, `0003` says `# Generated by Django 5.2.7 on 2026-08-03`. This is a real project that ran for a while, on more than one Django version, with real people using it.

Those people had accounts. Before roles existed, every account could do everything. After migration `0003` ran, every one of those accounts was suddenly a **student**, which meant they lost the ability to add a teacher, create a course or open the Accounts page. Everybody was locked out at once.

Migration `0004` fixes that. It is a **data migration**: a migration that changes rows rather than columns.

```
backend/backend/migrations/0004_existing_accounts_become_admins.py
```

```
from django.db import migrations

def make_existing_accounts_admins(apps, schema_editor):
 """Everyone who already had an account keeps full access.

 New accounts default to `student`, but the people who registered before
 roles existed were able to do everything, so locking them out on upgrade
 would be a surprise. Promote them once, here.
 """
 Profile = apps.get_model("backend", "Profile")
 Profile.objects.update(role="admin")
```

`Profile.objects.update(role="admin")` is one line of SQL under the covers: set the role column to `admin` on every row of the profile table.

Two details worth learning:

`apps.get_model("backend", "Profile")` is how a migration reaches a model. It gives you the model **as it was at this point in history**, not as it is in today's `models.py`. If you imported `Profile` from `models.py` directly, this migration would break the day someone renames a field, and old migrations must never break.

`schema_editor` is the second argument Django always passes. This function does not use it. Take it anyway, or Django cannot call your function.

Every data migration should also know how to undo itself.

```
backend/backend/migrations/0004_existing_accounts_become_admins.py
```

```
def back_to_student(apps, schema_editor):
 Profile = apps.get_model("backend", "Profile")
 Profile.objects.update(role="student")

class Migration(migrations.Migration):
 dependencies = [
 ("backend", "0003_profile_role"),
]

 operations = [
 migrations.RunPython(make_existing_accounts_admins, back_to_student),
]
```

`migrations.RunPython(forwards, backwards)` takes two functions. The first runs on `migrate`. The second runs if you ever roll back, with `python manage.py migrate backend 0003`. Pass only the first and Django refuses to reverse the migration at all.

#### NOTE

This file has no `# Generated by Django` header. Django cannot guess a data migration, so a human wrote this one. The way you start one is `python manage.py makemigrations backend --empty --name existing_accounts_become_admins`, which writes an empty migration with the right number, the right dependency and the right file name. Then you fill in the operations yourself.

#### COMMON MISTAKE

Editing a migration file that has already been applied to your database, or deleting one from the folder. Django tracks applied migrations by name in `django_migrations`, so a deleted file leaves a row pointing at nothing and `migrate` starts failing in confusing ways. Once a migration has run, treat it as finished. Change `models.py` and make a **new** migration instead.

## RUNNING ALL FOUR ON A FRESH DATABASE

```
python manage.py migrate
```

On an empty `lms_db` this runs Django's own migrations (users, sessions, admin, and the token blacklist tables that come with `rest_framework_simplejwt.token_blacklist`) followed by your four, in order.

#### CHECKPOINT

Connect to MySQL and run `SHOW TABLES;` inside `lms_db`. You should see nine tables whose names start with `backend_`: `backend_assignment`, `backend_course`, `backend_enrollment`, `backend_lesson`, `backend_profile`, `backend_results`, `backend_student`, `backend_submission`, `backend_teacher`. Django names a table by joining the app label and the lowercased model name with an underscore.

## admin.py: a free web interface for your tables

Django ships with an admin site: a set of web pages, already written, that can list, search, add, edit and delete rows in any model you tell it about. You get it for nothing. It is not the app you are building for your users, but it is how you will put test data in, fix a mistake, and change somebody's role.

It is switched on already. `django.contrib.admin` is the first entry in `INSTALLED_APPS` in `backend/lms/settings.py`, and `backend/lms/urls.py` maps it to `/admin/` with `path('admin/', admin.site.urls)`. All that is missing is telling it which of your models to show.

Open `backend/backend/admin.py`. Django left one line in it, `from django.contrib import admin`. Add the import of your models.

backend/backend/admin.py

```
from django.contrib import admin

from .models import (
 Assignment,
 Course,
 Enrollment,
 Lesson,
 Profile,
 Results,
 Student,
 Submission,
 Teacher,
)
```

The dot in `from .models` means "the `models.py` next to this file". The brackets let the names sit on separate lines, which keeps the list readable and easy to add to.

## THE SIMPLEST POSSIBLE REGISTRATION

The shortest way to put a model in the admin is one line per model:

```
backend/backend/admin.py (the simple version, not what the project uses)
admin.site.register(Teacher)
```

That already works. You get a list page and an add form. But the list would show one column, the `__str__` of each row, with no filters and no search box.

## THE DECORATOR, AND A MODELADMIN

To control how a model looks, you write a small class and attach it to the model. That attaching is what `@admin.register(...)` does.

backend/backend/admin.py

```
@admin.register(Teacher)
class TeacherAdmin(admin.ModelAdmin):
 list_display = ['id', 'name', 'email', 'subject', 'is_active']
 list_filter = ['is_active', 'subject']
 search_fields = ['name', 'email']
```

A line starting with `@` above a class is a **decorator**: a line that hands the thing below it to a function. `@admin.register(Teacher)` says "register `Teacher` in the admin, using the class below to configure it." It does exactly what `admin.site.register(Teacher, TeacherAdmin)` does, written more compactly. The project uses the decorator for all nine models.

`admin.ModelAdmin` is the class you inherit from. Every setting you can change is an attribute on it. This project uses four of them.

OPTION	WHAT IT DOES
<code>list_display</code>	which columns to show on the list page, left to right
<code>list_filter</code>	adds a filter sidebar on the right, one section per field
<code>search_fields</code>	adds a search box at the top and says which fields it searches
<code>list_editable</code>	lets you change a field <b>from the list page</b> and save many rows at once

On `TeacherAdmin`: the list shows five columns starting with the raw `id`, which is handy because the frontend deals in ids. The sidebar filters by active and by subject. The search box looks inside name and email.

## THE REST OF THE RESOURCE MODELS

Each one follows the same pattern, so read them as a group.

backend/backend/admin.py

```
@admin.register(Student)
class StudentAdmin(admin.ModelAdmin):
 list_display = ['id', 'name', 'email', 'roll_number', 'enrollment_date', 'is_active']
 list_filter = ['is_active']
 search_fields = ['name', 'email', 'roll_number']

@admin.register(Course)
class CourseAdmin(admin.ModelAdmin):
 list_display = ['id', 'title', 'teacher']
 list_filter = ['teacher']
 search_fields = ['title']
```

`list_display = ['id', 'title', 'teacher']` has a `ForeignKey` in it. The admin does not print a number for it. It prints the `Teacher.__str__`, so the column reads `Md. Rahim - Physics`. This is the first place your `__str__` methods pay for themselves.

`list_filter = ['teacher']` on a `ForeignKey` builds a sidebar listing every teacher, and clicking one narrows the list to that teacher's courses.

backend/backend/admin.py

```
@admin.register(Enrollment)
class EnrollmentAdmin(admin.ModelAdmin):
 list_display = ['id', 'student', 'course', 'enrollment_date']
 list_filter = ['course']

@admin.register(Lesson)
class LessonAdmin(admin.ModelAdmin):
 list_display = ['id', 'title', 'course']
 list_filter = ['course']
 search_fields = ['title']
```

`EnrollmentAdmin` has no `search_fields`, because an enrollment has no text of its own to search. Only leave options out when there is a reason.

`enrollment_date` is in `list_display` here even though it is `auto_now_add`. Showing a read-only value in a list is fine. It just will not appear on the add form, exactly as it does not appear on the React form.

backend/backend/admin.py

```
@admin.register(Assignment)
class AssignmentAdmin(admin.ModelAdmin):
 list_display = ['id', 'title', 'course', 'lesson', 'due_date']
 list_filter = ['course']
 search_fields = ['title']

@admin.register(Submission)
class SubmissionAdmin(admin.ModelAdmin):
 list_display = ['id', 'assignment', 'student', 'submitted_at']
 list_filter = ['assignment']

@admin.register(Results)
class ResultsAdmin(admin.ModelAdmin):
 list_display = ['id', 'submission', 'score']
```

`ResultsAdmin` is the smallest one in the file. `feedback` is a `TextField` and long text makes an ugly table column, so it is left off the list. It is still on the edit form. `list_display` controls the list only, never what you can edit.

## PROFILEADMIN, AND THE ONE ADMIN JOB THAT MATTERS

This is the first class in the real file, and it is the one you will actually use.



```
@admin.register(Profile)
class ProfileAdmin(admin.ModelAdmin):
 # `role` is editable straight from the list, because promoting someone who
 # has just registered is the one admin job this app really needs.
 list_display = ['user', 'phone', 'role']
 list_editable = ['role']
 list_filter = ['role']
 search_fields = ['user__username', 'phone']
```

That comment is in the real file, and it explains the design. Without `list_editable`, changing five people's roles means: click a person, change the dropdown, save, go back, click the next person, and so on. With it, the role column on the **list page** becomes a dropdown, you change all five, and press **Save** once.

Two rules govern `list_editable`, and both are satisfied here:

1. A field in `list_editable` must also be in `list_display`. `role` is in both.
2. It cannot be the **first** entry in `list_display`, because the first column is the link that opens the edit page, and a column cannot be a link and a form control at the same time. Here `user` is first, so `role` is safe in third place.

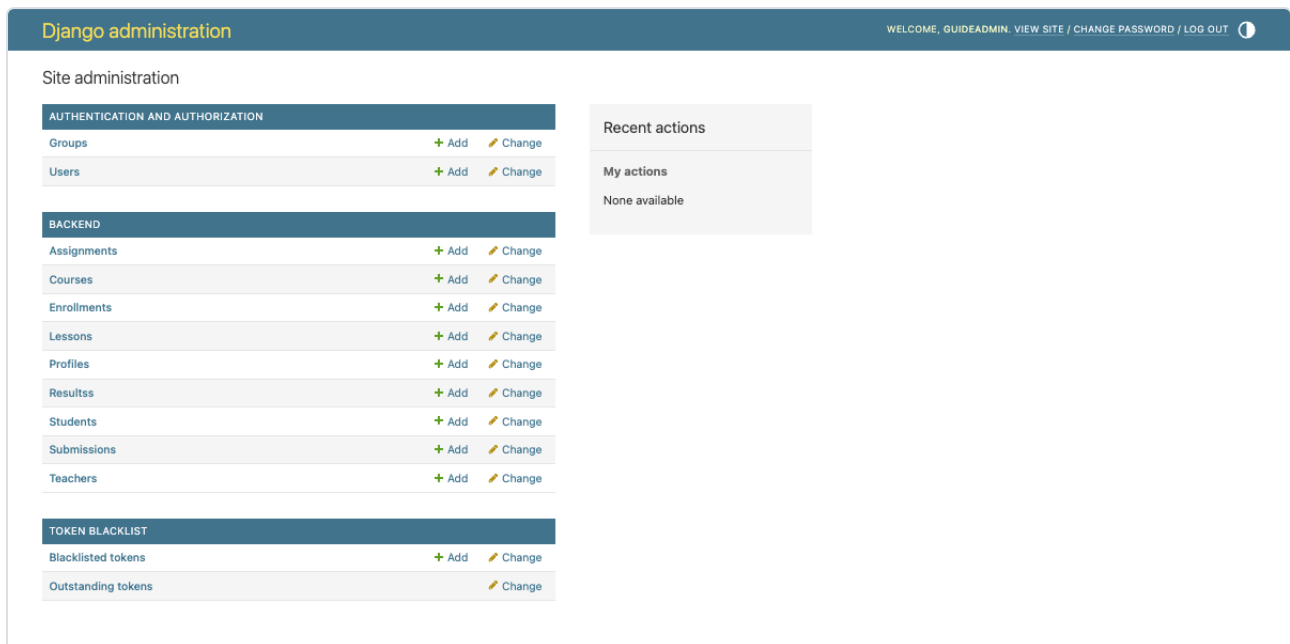
Break either rule and Django refuses to start, printing a system check error that names `list_editable` and tells you which rule you broke. That is a good error: it stops at boot rather than misbehaving later.

`search_fields = ['user__username', 'phone']` introduces the **double underscore**.

`user__username` means "follow the `user` link, then look at the `username` field on the other side." A single underscore would be a field literally named `user_username`, which does not exist. Two underscores means "go through the relation." You will meet this notation everywhere in Django.

`list_filter = ['role']` gives a sidebar with Admin, Teacher and Student, so "show me every teacher account" is one click.

**CHECKPOINT** Start the server with `python manage.py runserver 8001` and open `http://127.0.0.1:8001/admin/`. You should see a **BACKEND** section listing all nine models: Assignments, Courses, Enrollments, Lessons, Profiles, Results, Students, Submissions and Teachers. If a model is missing from that page, it is missing from `admin.py`.



**Figure 3.1** The Django admin home page. Every model registered in `admin.py` appears under the BACKEND heading, alongside Django's own Groups and Users.

## createsuperuser, and the trap that catches everybody

To open `/admin/` at all you need an account with admin rights. Django has a command for it.

```
python manage.py createsuperuser
```

It asks for a username, an email address (you may leave it blank) and a password twice. The password does not appear as you type. That is normal, not a broken keyboard.

Now here is the trap, and it is the single most common way to get stuck at the start of this project.

**A superuser can log into `/admin/` and cannot log into the React app.**

Read that twice, then look at why. The React login form asks for a **phone number**, not a username.

`LoginView` in `views.py` finds the account with `Profile.objects.get(phone=phone)`. And `createsuperuser` does not create a `Profile`. It creates a `User` and stops. So there is no phone number anywhere that points at your new superuser, and nothing for the login code to find.

You get `Invalid phone or password` no matter what you type, on an account you just made, with a password you are certain about.

The fix is three steps, straight out of the README.

### 1. Create the superuser.

```
python manage.py createsuperuser
```

**2. Give it a Profile by hand.** Open <http://127.0.0.1:8001/admin/backend/profile/add/>. Pick your new user in the **User** dropdown, type a phone number, set **Role** to **Admin**, and press Save.

**Figure 3.2** The Django admin "Add profile" form. Three fields: the User dropdown, the phone number you will log in with, and the Role dropdown showing Admin, Teacher and Student.

**3. Log in to the React app** at <http://localhost:5180/login> using that **phone number** and the password you set in step 1.

That account is now an admin in the app, and the Accounts page is open to you, so every other account can be created from the app itself. You only ever do this once per database.

#### COMMON MISTAKE

Typing your **username** into the React login form because that is what `createsuperuser` asked for. The form asks for a phone. The account is found through `Profile.phone` and only through `Profile.phone`. If you have no Profile row, no combination of characters will work.

#### NOTE

A superuser without a Profile is still treated as an admin by the permission code. `role_of()` in `backend/backend/permissions.py` returns `Profile.ADMIN` for any `user.is_superuser`, so the account is not powerless. The only thing missing is a phone number to log in with. That is what step 2 supplies.

#### TIP

Forgotten which accounts exist? The Accounts page in the React app creates accounts but does not list them. <http://127.0.0.1:8001/admin/backend/profile/> is where you see everyone, change a role, and search by username or phone.

## Try the models from the shell

Before moving on, prove to yourself that a model really is a table. Django gives you a Python prompt with everything already loaded.

```
python manage.py shell
```

Then, one line at a time:

```
from backend.models import Teacher, Course
t = Teacher.objects.create(name="Md. Rahim", email="rahim@example.com", subject="Physics")
t.id
Course.objects.create(title="Physics 101", description="Motion and forces.", teacher=t)
Teacher.objects.all()
```

`Teacher.objects.create(...)` builds the object and saves the row in one step. `t.id` shows the number the database gave it. `Teacher.objects.all()` returns a **QuerySet**, which is Django's word for a list of rows, and it prints each row using the `__str__` you wrote, so you should see `<QuerySet [Teacher: Md. Rahim - Physics]>`.

Type `exit()` to leave.

**CHECKPOINT** Refresh <http://127.0.0.1:8001/admin/backend/teacher/>. The teacher you created in the shell is in the list, with columns for id, name, email, subject and is active, and a filter sidebar on the right. The same row exists in three places now: in MySQL, in the shell, and on that page. That is one table, seen three ways.

## What you have now

- A `models.py` with nine models: `Profile`, `Teacher`, `Student`, `Course`, `Enrollment`, `Lesson`, `Assignment`, `Submission` and `Results`.
- Nine tables in MySQL, all named `backend_<model>`, created by `python manage.py migrate`.
- Four migration files that record every change so far, including a data migration that promoted the project's original accounts to admin instead of locking them out.
- A working understanding of `on_delete=models.CASCADE`, and the reason the delete dialogs you build later say "Their courses go too."
- Two `auto_now_add` fields, `Enrollment.enrollment_date` and `Submission.submitted_at`, which the server stamps and no form ever asks for.
- A one-to-one link on `Results.submission`, so a submission can only be graded once.
- An `admin.py` registering all nine models with useful columns, filters and search, and a role dropdown you can edit straight from the profile list.

- A superuser **with a Profile**, which means a phone number you can log in with once the React app exists.

# Turning Rows Into JSON: serializers.py

---

In Chapter 3 you built `models.py`. That gave you tables in MySQL and Python classes to talk to them. A `Teacher` object in Python is a fine thing, but a web browser has never heard of Python. Something has to stand between the database and the browser and translate.

That something is a **serializer**. This chapter builds the whole of `backend/backend/serializers.py`, one class at a time.

## First: what is JSON?

JSON is a way of writing data as plain text, so that two programs written in different languages can pass it back and forth. It has objects written with curly braces, lists written with square brackets, plus strings, numbers, `true`, `false` and `null`.

Almost every web **API** speaks it. An API is just a set of URLs that a program (rather than a person) calls to read or change data. Your Django backend is the API. The React app in the browser is the program calling it. JavaScript in the browser can turn JSON text into a real value with one function call.

Here is one teacher from this project, written as JSON:

```
{
 "id": 3,
 "name": "Rahim Uddin",
 "email": "rahim@example.com",
 "subject": "Mathematics",
 "is_active": true
}
```

And here is a list of two of them, which is what `/api/teacher/` actually returns:

```
[
 { "id": 3, "name": "Rahim Uddin", "email": "rahim@example.com", "subject": "Mathematics",
 "is_active": true },
 { "id": 4, "name": "Karim Ali", "email": "karim@example.com", "subject": "Physics",
 "is_active": true }
]
```

That is all JSON is. Text with a shape.

```
Pretty-print ☐
{
 "detail": "Authentication credentials were not provided."
}
```

**Figure 4.1** What `/api/teacher/` shows in a plain browser tab. A browser cannot attach the Authorization header, so the API answers 401 instead of the list above.

## What a serializer does

A serializer works in two directions, and they are not equally interesting.

**Direction one, going out.** You have a `Teacher` row from the database. The serializer reads its fields and hands you a plain Python dictionary, which Django REST Framework then writes out as the JSON above. This direction is called **serializing**. It is the easy half, because the data already came from your own database, so you know it is fine.

**Direction two, coming in.** A browser sends you a lump of JSON that claims to be a new teacher. You do **not** know it is fine. It might be missing `subject`. It might have a `subject` that is 4,000 characters long. It might have `email` set to the word "banana". The serializer takes that incoming data, checks every field against the rules, and either gives you back a clean `validated_data` dictionary you can safely save, or a set of error messages saying exactly what was wrong.

That second half, the checking, is the whole point. A serializer is not really a translator. It is a **gate**. Nothing untrusted gets past it into your database. Every single time you see a `validate_something` method later in this chapter, that is the gate doing its job.

### NOTE

"Validated" just means "checked". `serializer.validated_data` is the version of the request you are allowed to trust. `serializer.initial_data` is the raw version you are not. Only ever save from `validated_data`.

## Create the file

When you ran `python manage.py startapp backend` in Chapter 2, Django made you `models.py`, `views.py`, `admin.py` and a few others. It did **not** make `serializers.py`, because serializers come from Django REST Framework, not from Django itself. Django REST Framework is a separate add-on library, listed in `backend/requirements.txt` as `djangorestframework==3.15.2`. You create the file yourself.

```
from the backend folder, the one with manage.py in it
touch backend/serializers.py
```

On Windows, or if you prefer, just make a new empty file called `serializers.py` inside the `backend/backend/` folder, next to `models.py`.

## The imports at the top

Start with the imports. These are all the outside pieces the file uses. Type them in now and they will make sense as each class arrives.

backend/backend/serializers.py

```
from .models import Assignment, Enrollment, Lesson, Profile, Results, Submission, Teacher, Student, Course
from django.db import transaction
from rest_framework import serializers
from django.contrib.auth.models import User

from django.contrib.auth.password_validation import validate_password
from django.contrib.auth.tokens import default_token_generator
from django.utils.encoding import force_str
from django.utils.http import urlsafe_base64_decode
from rest_framework import serializers
from django.contrib.auth import get_user_model

User = get_user_model()
```

Line by line, in plain terms:

IMPORT	WHAT IT IS FOR
<code>from .models import ...</code>	Your own models from Chapter 3. The dot means "from this same folder".
<code>transaction</code>	Lets you say "do these two database writes together, or do neither".
<code>serializers</code>	The Django REST Framework toolbox. Everything in this file is built from it.
<code>User</code>	Django's built-in account model. It stores username, email, first name, last name and the hashed password.
<code>validate_password</code>	Django's own password rules (too short, too common, all digits, too close to your name).
<code>default_token_generator</code>	Makes and checks the one-time code in a password reset link.
<code>force_str</code> , <code>urlsafe_base64_decode</code>	Undo the encoding used to hide a user id inside a reset URL.
<code>get_user_model()</code>	Asks Django "which model is the account model?" and returns it.

Two things about this block are worth saying out loud, because they will confuse you otherwise.



`from rest_framework import serializers` appears **twice**, on line 3 and again on line 11. That is a leftover from when the password-reset code was pasted in. Importing the same thing twice in Python is harmless: the second one does nothing. Leave it or delete it, the file behaves the same either way.

`User` is also set twice: once imported directly on line 4, then reassigned on line 14 by `get_user_model()`. In this project both give you the exact same class, because the project uses Django's stock `User`. `get_user_model()` is the safer habit, because if you ever swap in a custom account model, code written that way keeps working.

**TIP**

`get_user_model()` must be called, with the brackets. `User = get_user_model` without brackets stores the function itself, and then `User.objects` fails with a confusing `AttributeError`.

## ModelSerializer or Serializer?

Django REST Framework gives you two base classes, and this file uses both.

`serializers.ModelSerializer` is for when the JSON maps onto a model. You tell it which model and which fields, and it works out the rest: which fields are required, what maximum length each one has, that `is_active` is a true/false, that `enrollment_date` is a date. It also writes `create()` and `update()` for you, so saving works with no extra code.

`serializers.Serializer` is the plain version. Nothing is worked out for you. You declare every field by hand, and if anything needs saving you write that yourself. You use it when the incoming JSON is **not** a model: a login attempt, a password change, a reset link. Those are actions, not rows.

Here is how the split falls in this project:

SERIALIZER	BASE CLASS	WHY
<code>RegisterSerializer</code>	<code>ModelSerializer</code>	Creates a real <code>User</code> row (and a <code>Profile</code> ).
<code>LoginSerializer</code>	<code>Serializer</code>	Checks a phone and password. Saves nothing.
<code>ProfileUpdateSerializer</code>	<code>Serializer</code>	Writes to two models at once, so no single model fits.
<code>ChangePasswordSerializer</code>	<code>Serializer</code>	An action. There is no "password change" table.
<code>PasswordResetRequestSerializer</code>	<code>Serializer</code>	Just an email address.
<code>PasswordResetConfirmSerializer</code>	<code>Serializer</code>	A uid, a token and two passwords. Not a row.
The eight resource serializers	<code>ModelSerializer</code>	Each one is exactly one model.

We will write the eight easy ones first.

#### NOTE

In the real repo these eight sit at the **bottom** of `serializers.py`, below all the auth classes. Python does not care what order classes appear in, as long as each one is defined before it gets used at runtime. Add them at the bottom to match the repo, or at the top while you learn. Either works.

## The eight resource serializers

These are the ones that carry the actual LMS data: teachers, students, courses, enrollments, lessons, assignments, submissions and results. Every one of them is four lines and follows the same shape.

Start with teachers and students:

```
backend/backend/serializers.py (near the bottom)
class TeacherSerializer(serializers.ModelSerializer):
 class Meta:
 model = Teacher
 fields = ['id', 'name', 'email', 'subject', 'is_active']
class StudentSerializer(serializers.ModelSerializer):
 class Meta:
 model = Student
 fields = ['id', 'name', 'email', 'enrollment_date', 'is_active', 'roll_number']
```

That inner `class Meta` is not a normal class. It is a small block of settings that Django REST Framework reads. Two settings matter here:

- **model** says which model in `models.py` this serializer is about. It goes and reads that model's field definitions to learn the rules.
- **fields** is the exact list of field names that appear in the JSON, in order. A field not on this list is invisible to the API: it is never sent out, and it is ignored if someone sends it in.

Compare `TeacherSerializer` with the `Teacher` model from Chapter 3 and you will see they line up one for one: `name`, `email`, `subject`, `is_active`. The extra one, `id`, is the **primary key**: the number that uniquely identifies one row in a table. Django adds it to every model automatically, and every serializer here lists it so the frontend knows which row is which when it wants to edit or delete one.

#### COMMON MISTAKE

putting a name in `fields` that is not on the model. Django REST Framework raises `ImproperlyConfigured` the first time that serializer is used, with a message like `Field name `phone_number` is not valid for model `Teacher` in `backend.serializers.TeacherSerializer`.` It names the bad field, the model and the serializer, but it does not list the valid names, so go and read your model.

Now the four that link to other tables:

backend/backend/serializers.py

```
class CourseSerializer(serializers.ModelSerializer):
 class Meta:
 model = Course
 fields = ['id', 'title', 'description', 'teacher']

class EnrollmentSerializer(serializers.ModelSerializer):
 class Meta:
 model = Enrollment
 fields = ['id', 'student', 'course', 'enrollment_date']
```

backend/backend/serializers.py

```
class LessonSerializer(serializers.ModelSerializer):
 class Meta:
 model = Lesson
 fields = ['id', 'title', 'description', 'course']

class AssignmentSerializer(serializers.ModelSerializer):
 class Meta:
 model = Assignment
 fields = ['id', 'title', 'description', 'lesson', 'due_date', 'course']
```

And the last two:

backend/backend/serializers.py

```
class SubmissionSerializer(serializers.ModelSerializer):
 class Meta:
 model = Submission
 fields = ['id', 'assignment', 'student', 'submitted_at', 'content']

class ResultSerializer(serializers.ModelSerializer):
 class Meta:
 model = Results
 fields = ['id', 'submission', 'score', 'feedback']
```

#### NOTE

the class is `ResultSerializer`, singular, but the model it wraps is `Results`, plural. That mismatch is real and it is easy to trip over when you import it in `views.py`. The URL is plural too: `/api/results/`, while every other resource is singular (`/api/teacher/`, `/api/student/`). `frontend/src/api.js` even has a comment about it: `// Note the naming: /submission/ is singular but /results/ is plural.`

## FOREIGN KEYS COME OUT AS PLAIN NUMBERS

Look at `CourseSerializer` again. The `teacher` field on the `Course` model is a `ForeignKey`, which means "this course points at one teacher". When a foreign key is listed in `fields` and nothing else is said about it, Django REST Framework represents it as **the id of the row it points at, as a plain integer**.

So a course is not this:

```
{ "id": 2, "title": "Algebra", "teacher": { "id": 3, "name": "Rahim Uddin" } }
```

It is this:

```
{ "id": 2, "title": "Algebra", "description": "Basic algebra", "teacher": 3 }
```

The same is true in the other direction. To create a course, the frontend sends `"teacher": 3`, not a name and not an object.

Here is every link in the project:

SERIALIZER	FIELD	POINTS AT	IN THE JSON
<code>CourseSerializer</code>	<code>teacher</code>	<code>Teacher</code>	<code>"teacher": 3</code>
<code>EnrollmentSerializer</code>	<code>student</code> , <code>course</code>	<code>Student</code> , <code>Course</code>	<code>"student": 1, "course": 2</code>
<code>LessonSerializer</code>	<code>course</code>	<code>Course</code>	<code>"course": 2</code>
<code>AssignmentSerializer</code>	<code>lesson</code> , <code>course</code>	<code>Lesson</code> , <code>Course</code>	<code>"lesson": 5, "course": 2</code>
<code>SubmissionSerializer</code>	<code>assignment</code> , <code>student</code>	<code>Assignment</code> , <code>Student</code>	<code>"assignment": 5,</code> <code>"student": 1</code>
<code>ResultSerializer</code>	<code>submission</code>	<code>Submission</code>	<code>"submission": 7</code>

**Why this matters for the React app, much later.** A number is not something you can show a human. Nobody wants a Courses table with a column reading "3". So the frontend has to fetch the teacher list as well and match the ids up itself. That is exactly what `frontend/src/pages/Courses.jsx` does, and the comment in the file says so:

frontend/src/pages/Courses.jsx

```
// A course sends {"teacher": 3}, not the teacher's name. There is no way to
// ask the API to expand it, so the teacher list is fetched as well and the
// two are joined here in JavaScript.
function findTeacherName(teacherId) {
 const teacher = teachers.find((item) => item.id === teacherId);
 if (teacher) {
 return teacher.name;
 }
 return "Unknown";
}
```

Two API calls instead of one, and a join done by hand in the browser. That is the price of keeping the serializers this simple. You will meet this pattern on the Courses, Enrollments, Lessons, Assignments, Submissions and Results pages, all six of which fetch two or three lists at once with `Promise.all`.

**CHECKPOINT** With the virtual environment active and MySQL running, open a Python shell that knows about your project and drive a serializer by hand:

```
bash python manage.py shell
```

```
```python
```

```
from backend.serializers import
TeacherSerializer s =
TeacherSerializer(data={'subject':
'Mathematics', 'name': 'Rahim Uddin'})
s.is_valid() True dict(s.validated_data)
{'name': 'Rahim Uddin', 'subject':
'Mathematics'}```
```

Now break it on purpose, by leaving out the one field the model insists on:

```
```python
```

```
s2 = TeacherSerializer(data={}) s2.is_valid() False s2.errors {'subject':
[ErrorDetail(string="This field is required.", code='required')]} ```
```

You never wrote "subject is required" anywhere. `ModelSerializer` read that off the `Teacher` model, where `subject = models.CharField(max_length=100)` has no `blank=True`. Press `Ctrl-D` (or type `exit()`) to leave the shell.

## RegisterSerializer

This is the biggest class in the file, so we will build it in five pieces. It creates an account. It is the only place in the whole project where a `User` row and a `Profile` row are made.

Remember the awkward shape from Chapter 3: Django's `User` holds the username, email, names and password, and your own `Profile` holds the phone number and the role. One account is two rows in two tables. `RegisterSerializer` has to hide that from whoever is calling.

### PIECE 1: THE FIELDS DJANGO REST FRAMEWORK CANNOT WORK OUT

backend/backend/serializers.py

```
class RegisterSerializer(serializers.ModelSerializer):
 phone = serializers.CharField(required=True, write_only=True)
 first_name = serializers.CharField(required=True)
 email = serializers.EmailField(required=True)
```

Three fields are declared by hand here, each for a different reason.

`phone` is not on the `User` model at all. It lives on `Profile`. So `ModelSerializer` would never invent it, and you must declare it yourself. `write_only=True` means "accept this coming in, but never include it in the JSON going out". That is not about secrecy here, it is about honesty: since `phone` is not a `User` field, there would be nothing to read back from the created `User` anyway.

`first_name` and `email` are on `User`, but on that model both are allowed to be empty. If you left them alone, `ModelSerializer` would happily accept an account with no name and no email. Redefining them with `required=True` overrides that. Now the request is rejected unless both are filled in.

#### NOTE

a field you declare by hand always wins over the one `ModelSerializer` would have generated. This is how you tighten rules for the API without changing the database.

### PIECE 2: THE ROLE FIELD

backend/backend/serializers.py

```
Only an admin can reach this endpoint, so letting the caller pick the
role is safe. Left out, the account is a student.
role = serializers.ChoiceField(
 choices=Profile.ROLE_CHOICES,
 required=False,
 default=Profile.STUDENT,
)
```

An **endpoint** is one URL the API answers on, together with the code behind it. This one is

```
/api/register/.
```

A `ChoiceField` accepts one value out of a fixed list and rejects everything else. The list is

`Profile.ROLE_CHOICES`, the same one you wrote on the model in Chapter 3, so `"admin"`, `"teacher"` and `"student"` are in and `"superadmin"` is out, with no extra code.

`required=False` plus `default=Profile.STUDENT` means: if the request does not mention a role, the new account is a student.

That comment above the field is doing real work, so read it carefully. Normally, letting whoever is signing up choose their own role would be a serious hole: anybody could ask to be an admin and the server would say yes. It is safe **here** and only here, because of one line in `views.py`:

```
backend/backend/views.py
```

```
permission_classes = [IsAdmin]
```

Nobody reaches this serializer unless they are already signed in as an admin. An anonymous request gets a 401 (the HTTP status for "you are not signed in"), and a signed-in teacher or student gets a 403 ("you are signed in, but not allowed"). By the time `role` is read, the only person who could have set it is somebody who already has every permission. There is nothing to gain.

Compare it with the comment on the model itself, which describes a different situation:

```
backend/backend/models.py
```

```
Anyone can register, so a new account gets the least privileged role.
Promote people from /admin/ once you know who they are.
```

That comment is from an earlier version of the project, when sign-up was public. The default is still `STUDENT`, which is the right default either way.

#### COMMON MISTAKE

copying this `role` field into a serializer behind a public endpoint. Then anyone can post `{"role": "admin"}` and hand themselves the keys. A field is only as safe as the permission class in front of it.

## PIECE 3: THE META BLOCK

backend/backend/serializers.py

```
class Meta:
 model = User
 fields = ['id', 'username', 'email', 'password', 'phone', 'first_name',
'last_name', 'role']
 read_only_fields = ['id']
 extra_kwargs = {'password': {'write_only': True}}
```

`model = User` says this serializer is built around Django's account model. `fields` lists everything the API talks about, including `phone` and `role`, which are not on `User`, and which you declared by hand above.

Two new settings appear:

`read_only_fields = ['id']` means the id goes out but never comes in. The database picks primary keys, not the caller.

`extra_kwargs` is how you adjust a field you did **not** declare by hand. `password` is generated automatically by `ModelSerializer` from the `User` model, so there is no line of yours to add `write_only=True` to. `extra_kwargs` reaches in and sets it anyway. The result is that a password can be sent in but is never sent back out.

### TIP

the two ways of setting a rule are `extra_kwargs` (for auto-generated fields) and redeclaring the field (for anything you want more control over). Use `extra_kwargs` when the change is small, redeclare when you need a different field type.

## PIECE 4: THE THREE CHECKS

These are the gate. Django REST Framework has a naming convention: a method called `validate_<field name>` is run automatically for that one field, after the basic type check has passed. It receives the cleaned value and must return it, or raise.

backend/backend/serializers.py

```
def validate_phone(self, value):
 if Profile.objects.filter(phone=value).exists():
 raise serializers.ValidationError("This phone number is already registered.")
 return value

def validate_username(self, value):
 if User.objects.filter(username=value).exists():
 raise serializers.ValidationError("That username is already taken.")
 return value
```



Both do the same thing: run a query, and if a row already exists with that value, refuse. `.exists()` is a query that only asks "is there at least one?", so the database does not have to hand back any rows.

The phone check matters more than it looks, because in this project **the phone number is the login**. `LoginView` looks the account up with `Profile.objects.get(phone=phone)`. Two accounts sharing a phone would make that lookup ambiguous, so the field is `unique=True` on the model too. This check exists to turn the database's blunt failure into a sentence a human can read.

backend/backend/serializers.py

```
def validate_password(self, value):
 # Run Django's own password rules, the same ones the reset flow uses.
 validate_password(value)
 return value
```

This one calls the `validate_password` you imported at the top of the file: Django's built-in checker, driven by `AUTH_PASSWORD_VALIDATORS` in `lms/settings.py`. In this project that list holds four rules: not too similar to your own name or email, at least 8 characters, not one of the thousands of most common passwords, and not all digits.

#### NOTE

yes, a method named `validate_password` calls a function named `validate_password`, and no, it is not calling itself. Python looks up a plain name inside a function body in the local scope, then the module scope. It never looks in the surrounding class. So the name resolves to the import from `django.contrib.auth.password_validation`, which is what you want. It reads alarmingly, but it works.

Here is the gate turning away a bad request, run in the shell:

```
>>> from backend.serializers import RegisterSerializer
>>> s = RegisterSerializer(data={'username': 'zzztest', 'email': 'z@z.com',
... 'password': 'abc', 'phone': '0170000zz',
... 'first_name': 'Z'})
>>> s.is_valid()
False
>>> s.errors
{'password': [ErrorDetail(string='This password is too short. It must contain at least 8 characters.', code='password_too_short'), ErrorDetail(string='This password is too common.', code='password_too_common')]}
```

Two complaints about one field, both from Django, none of them written by you. Notice that both messages are filed under `password`. That happens because this is a per-field check, and Django REST Framework knows which field it was running when the error appeared. Keep that in mind, because later in this chapter the same Django checker produces messages that are **not** filed under a field name.

## PIECE 5: CREATING THE TWO ROWS

backend/backend/serializers.py

```
@transaction.atomic
def create(self, validated_data):
 phone = validated_data.pop('phone')
 email = validated_data.pop('email')
 role = validated_data.pop('role', Profile.STUDENT)
 first_name = validated_data.pop('first_name', '')
 last_name = validated_data.pop('last_name', '')

 user = User.objects.create_user(
 username=validated_data['username'],
 email=email,
 password=validated_data['password'],
 first_name=first_name,
 last_name=last_name,
)
 Profile.objects.create(user=user, phone=phone, role=role)
 return user
```

`create()` is the method `ModelSerializer` would have written for you. You are replacing it, because the generated one would try to shove `phone` and `role` onto a `User` and fail.

`.pop()` takes a key out of a dictionary and gives you its value. Every popped key is one that must not be passed to `create_user`. A second argument to `.pop()` is a fallback for when the key is not there: `role` defaults to student, and both names default to an empty string.

`@transaction.atomic` is a decorator. A decorator is a line starting with `@` written directly above a function, and it wraps that function in extra behaviour. This one says: everything this function writes to the database counts as one unit. If any line raises an exception, every write inside is rolled back as if it never happened.

Think about what happens without it. `User.objects.create_user(...)` succeeds. Then `Profile.objects.create(...)` fails, say because two admins created the same phone number at the same moment and one slipped past `validate_phone`. Now you have a `User` with **no** `Profile`, which means no phone number, which means **no way to log in**, and no way for anyone to notice except by reading the database. With `@transaction.atomic`, the failed `Profile` takes the half-made `User` down with it and the caller gets a clean error. Two tables, one action, one decorator.

**Why `User.objects.create_user` and not `User(...)`?** This is the most important line in the class. `create_user` is Django's account factory, and it runs the password through a hashing function before storing it. A **hash** is a one-way scramble: the database ends up holding something like `pbkdf2_sha256$870000$....`, and there is no way back to the original text.

If you wrote `User(username=..., password='secret123')` and saved it, Django would store the literal string `secret123` in the password column. Two things then break. First, anybody who ever sees your database sees everybody's real password. Second, login stops working, because `LoginView` calls `user.check_password(password)`, which hashes what was typed and compares hashes. A hash never equals plain text, so every login attempt fails with "Invalid phone or password" while the password is obviously correct.

**COMMON  
MISTAKE**

creating users with `User.objects.create(...)` instead of `create_user(...)`. They are one word apart and the symptom (nobody can log in, ever) points nowhere near the cause.

## ONE MORE METHOD: TO\_REPRESENTATION

backend/backend/serializers.py

```
def to_representation(self, instance):
 data = super().to_representation(instance)
 data['role'] = instance.profile.role
 return data
```

`to_representation` is the going-out direction: model row in, dictionary out. Overriding it lets you adjust the JSON after the normal work is done.

`super().to_representation(instance)` runs the standard version and hands you the dictionary it built. That dictionary has `id`, `username`, `email`, `first_name` and `last_name`, all read straight off the `User`. It does **not** have a useful `role`, because `role` is not a `User` field. The one line after it goes and gets the real role from the `Profile` and puts it in.

`instance.profile` works because of how you declared the link in Chapter 3:

backend/backend/models.py

```
user = models.OneToOneField(User, on_delete=models.CASCADE)
```

With a `OneToOneField` and no `related_name`, Django creates a reverse accessor on the other side named after the model in lower case. So from a `User` you reach its `Profile` as `user.profile`.

The net effect: create an account and the response tells you the role you asked for, even though that value came out of a different table.

**COMMON  
MISTAKE**

later adding `related_name="account"` to that `OneToOneField`. The moment you do, `instance.profile` stops existing and register responds with a 500 error, the status that means the server code crashed. If you rename it, rename it here too.

## LoginSerializer

After all that, this one is a relief.

backend/backend/serializers.py

```
class LoginSerializer(serializers.Serializer):
 phone = serializers.CharField(required=True)
 password = serializers.CharField(required=True, write_only=True)
```

Two fields, and that is the entire class.

It is a plain `serializers.Serializer`, not a `ModelSerializer`, **because there is nothing to save**. Logging in does not create a row. It reads one. There is no model to point `Meta.model` at, so there is no reason to reach for `ModelSerializer` and no `create()` or `update()` to inherit.

What this class is for is the shape check. It guarantees that by the time `LoginView` runs, both `phone` and `password` were present and were strings. `write_only=True` on the password means it can never leak back into a response.

The actual work happens in the view, which you will write in Chapter 6:

backend/backend/views.py

```
serializer = LoginSerializer(data=request.data)
serializer.is_valid(raise_exception=True)
phone = serializer.validated_data['phone']
password = serializer.validated_data['password']
```

**CHECKPOINT** in `python manage.py shell`, prove the gate catches an empty request:

```
```python
```

```
from backend.serializers import
LoginSerializer l = LoginSerializer(data={})
l.is_valid() False l.errors {'phone':
[ErrorDetail(string='This field is required.',
code='required')], 'password':
[ErrorDetail(string='This field is required.',
code='required')]} ```
```

Two fields, two complaints, and the view never had to run.

ProfileUpdateSerializer

This one backs the "Your profile" page, where a signed-in person edits their own details. Start with its docstring, which is part of the real file and explains the design better than any paragraph could:

backend/backend/serializers.py

```
class ProfileUpdateSerializer(serializers.Serializer):
    """What you are allowed to change about yourself.

    Not on this list, on purpose:
        role      - changing your own would be a promotion. Admins only, in
                    Django admin.
        username  - it is how an admin finds you; the phone is what you log in
                    with, so there is nothing to gain by renaming yourself.
        password  - has its own endpoint, because it needs the old one first.
    """
```

A docstring is the string right under a `def` or `class` line. Python keeps it attached to the object, and tools show it as help. Here it is being used to record decisions.

Take the three exclusions one at a time, in beginner terms:

role is missing on purpose. If `role` were a field on this serializer, any student could send `{"role": "admin"}` to `/api/profile/` and promote themselves. Nothing else would stop them, because they genuinely are allowed to edit their own profile. The defence is simply that the field does not exist here, so the value is thrown away unread. Roles change in the Django admin site, by somebody who already is an admin.

username is missing because letting people rename themselves gains nothing and costs something. You log in with your phone, not your username, so a rename does not help you sign in. Meanwhile the username is how an admin picks you out of a list, and a person who renames themselves every week is a person nobody can find.

password is missing because changing a password safely needs one thing this endpoint does not ask for: the old password. That gets its own serializer and its own URL, next section.

NOTE

the React page says the same thing to the user, in the same words, so the interface and the server agree. `frontend/src/pages/Profile.jsx` reads: "Your username and your role are set by an admin, so they are not editable here."

THE FOUR FIELDS

backend/backend/serializers.py

```
first_name = serializers.CharField(required=False, allow_blank=True, max_length=150)
last_name = serializers.CharField(required=False, allow_blank=True, max_length=150)
email = serializers.EmailField(required=False)
phone = serializers.CharField(required=False, max_length=20)
```

Every field is `required=False`. That is because this endpoint is used with **PATCH**. HTTP methods are the verbs a browser sends with a URL: GET means "give me this", POST means "here is something new", PUT means "replace this whole thing", DELETE means "remove it", and PATCH means "change only what I mention". Sending just `{"first_name": "Rahim"}` must be allowed, and must not wipe out the email.

`allow_blank=True` on the two names says an empty string is a legitimate value. Some people have no surname, and someone who wants to clear the field should be able to. Note that `phone` and `email` do **not** have it. An empty phone would lock you out of your own account, and an empty email would break password reset.

The `max_length` values are copied from the models they end up on: 150 on `User.first_name`, 20 on `Profile.phone`. Because this is a plain `Serializer`, nothing is inherited, so limits have to be repeated here.

VALIDATE_PHONE

backend/backend/serializers.py

```
def validate_phone(self, value):
    value = value.strip()
    if not value:
        raise serializers.ValidationError(
            "Your phone number is how you sign in, so it cannot be blank."
        )

    me = self.context["request"].user
    if Profile.objects.filter(phone=value).exclude(user=me).exists():
        raise serializers.ValidationError(
            "Another account already uses that phone number."
        )
    return value
```

`.strip()` removes spaces from both ends, then the value is checked for emptiness. Django REST Framework already rejects a completely empty string here, since `allow_blank` was not set. This catches the sneakier case of `" "`, three spaces, which is not empty until you strip it, and it replies with a sentence that explains **why** rather than the stock "This field may not be blank."

Then the interesting part.

`self.context` is a dictionary the view hands to the serializer. In `views.py` it is filled in like this:

backend/backend/views.py

```
serializer = ProfileUpdateSerializer(
    data=request.data,
    context={'request': request},
)
```

So `self.context["request"].user` is the signed-in person, worked out from the **token** on the request. A token is a long string the server hands you when you log in; the browser sends it back with every later call so the server knows who is asking. Call that person `me`.

Now read the query out loud: find every `Profile` whose phone is the new value, **except mine**, and see if any exist. If one does, somebody else already has that number, so refuse.

Why `.exclude(user=me)` is not optional. Imagine you open your profile page, change your first name to "Rahim", and press Save. The React page sends all four fields, including your phone number, unchanged. Without the exclude, the query would be "is there any Profile with this phone?" and the answer would be **yes: yours**. You would be told "Another account already uses that phone number", about yourself, and you could never save anything again unless you also changed your phone every single time. The exclude removes your own row from the search so the check only ever finds genuine clashes.

VALIDATE_EMAIL

backend/backend/serializers.py

```
def validate_email(self, value):
    # Password reset looks an account up by email, so two accounts sharing
    # one would make that ambiguous.
    me = self.context["request"].user
    if User.objects.filter(email__iexact=value).exclude(pk=me.pk).exists():
        raise serializers.ValidationError(
            "Another account already uses that email address."
        )
    return value
```

The same idea, with two differences.

`email__iexact` is a Django lookup meaning "equals, ignoring capital letters". So `Rahim@example.com` counts as a clash with `rahim@example.com`, which is right, because email addresses are not case sensitive in practice and letting both exist would be a trap.

The exclude is written differently: `.exclude(pk=me.pk)` instead of `.exclude(user=me)`. That is not a style choice, it is forced by what is being searched. `validate_phone` searches `Profile` rows, and the way a `Profile` says "I am yours" is its `user` field. `validate_email` searches `User` rows, and the way a `User` says "I am you" is by being you, which you express with the primary key. `pk` is Django's universal name for the primary key column.

The comment gives the reason two accounts must not share an email: `PasswordResetRequestView` finds an account with `User.objects.filter(email=email, is_active=True).first()`. If two accounts matched, `.first()` would pick one arbitrarily, and half the time the reset link would arrive for the wrong account.

SAVE

backend/backend/serializers.py

```
@transaction.atomic
def save(self, **kwargs):
    me = self.context["request"].user
    data = self.validated_data

    for field in ("first_name", "last_name", "email"):
        if field in data:
            setattr(me, field, data[field])
    me.save()

    if "phone" in data:
        profile = Profile.objects.filter(user=me).first()
        if profile:
            profile.phone = data["phone"]
            profile.save()

    return me
```

Because this is a plain `Serializer` there is no `create()` or `update()` to inherit, so `save()` is written from scratch.

The loop walks three field names. `if field in data` is the PATCH rule in code: a field the caller did not send is simply not touched. Without that check, a request carrying only a phone number would blank out your first name, last name and email, because `validated_data` would not contain them.

`setattr(me, field, data[field])` sets an attribute whose name is held in a variable. `setattr(me, "email", "x@y.com")` does exactly what `me.email = "x@y.com"` does, but it works when the name is only known at runtime. It saves writing the same three lines three times.

Then `me.save()` writes the `User` row, and the phone, which lives on the other table, is handled separately. `.filter(...).first()` returns the `Profile` or `None`. The `if profile:` guard means an account that somehow has no profile does not crash the request.

`@transaction.atomic` is here for the same reason as in `RegisterSerializer`: two tables are written, and either both changes land or neither does.

COMMON MISTAKE

forgetting `context={'request': request}` when you build this serializer in a view. Every validator here starts with `self.context["request"]`, so without it you get a bare `KeyError: 'request'` and a 500 error, with nothing in the message about what is actually missing.

ChangePasswordSerializer

Changing a password is deliberately more work than changing a name.

backend/backend/serializers.py

```
class ChangePasswordSerializer(serializers.Serializer):
    current_password = serializers.CharField(write_only=True)
    new_password = serializers.CharField(write_only=True)
    confirm_password = serializers.CharField(write_only=True)
```

Three fields, all `write_only`, so none of them can ever appear in a response. All three are required: `required=True` is the default, and it has not been turned off.

backend/backend/serializers.py

```
def validate_current_password(self, value):
    # Knowing the old password is what makes this yours to change. Without
    # it, a borrowed browser tab would be enough to take the account.
    if not self.context["request"].user.check_password(value):
        raise serializers.ValidationError("That is not your current password.")
    return value
```

`check_password` takes what was typed, hashes it the same way the stored one was hashed, and compares. It returns `True` or `False`. It never decodes the stored password, because that is not possible.

The comment explains why this field exists at all. The caller already proved they hold a valid token, so why ask again? Because a token can be borrowed. Someone who sits down at an unlocked laptop has the token. Asking for the current password means they need something the real owner **knows**, not just something the laptop is holding. Without this field, walking past an unattended screen would be enough to take over the account for good.

```
def validate(self, attrs):
    if attrs["new_password"] != attrs["confirm_password"]:
        raise serializers.ValidationError({
            "confirm_password": "The two new passwords do not match."
        })

    if attrs["new_password"] == attrs["current_password"]:
        raise serializers.ValidationError({
            "new_password": "The new password has to be different from the old one."
        })

    validate_password(attrs["new_password"], user=self.context["request"].user)
    return attrs
```

`validate` with no field name on the end is the **cross-field** check. Django REST Framework runs it after every individual field has passed, and hands it `attrs`, a dictionary holding all of them at once. That is the only place you can compare one field against another.

Two rules live here, and neither could be written as a single-field validator:

1. **The two new passwords must match.** This is the reason forms ask you to type it twice: to catch a typo before it locks you out.
2. **The new password must differ from the old one.** Otherwise "change your password" quietly does nothing while telling you it worked.

Both raise with a **dictionary** rather than a bare string. That attaches the message to a named field, so the error comes back as `{"confirm_password": ["The two new passwords do not match."]}` instead of the generic `non_field_errors`. The frontend can then say which box to look at.

The last line runs Django's own rules again, and this time it passes `user=`. That extra argument matters: it switches on `UserAttributeSimilarityValidator`, which refuses a password that is too close to your username, email or name. Without a user to compare against, that check has nothing to do and quietly passes.

There is a catch worth knowing. `validate_password` is being called here inside `validate`, not inside a `validate_<field>` method, and it raises Django's own kind of error rather than Django REST Framework's. When that happens, Django REST Framework has no field name to file it under, so the messages come back under `non_field_errors` instead of under `new_password`.

NOTE

the ordering is guaranteed. Field validators run first, then `validate`. So by the time `validate` compares `new_password` with `current_password`, the current one has already been confirmed correct by `validate_current_password`. You never have to re-check it.

CHECKPOINT once the API is running (Chapter 6), sign in, go to Your profile, and change your password to `12345678`. You should get a red banner reading `This password is too common. This password is entirely numeric.` and no change is made. Two messages, no field name in front of them, because they arrived under `non_field_errors` as just described. They came from Django's `CommonPasswordValidator` and `NumericPasswordValidator`, through `validate_password`, through this serializer.

The password reset pair

Two serializers cover the "I forgot my password" flow. The first one is the smallest class in the file:

backend/backend/serializers.py

```
class PasswordResetRequestSerializer(serializers.Serializer):
    email = serializers.EmailField()
```

One field, `EmailField` checks the text at least looks like an address. The view takes it from here: it finds the matching account, builds a one-time link and emails it. Notice there is no check that the email exists, and no error if it does not. That is on purpose, and `PasswordResetRequestView` always replies with the same sentence: *"If an account exists with this email, a password reset link has been sent."* If it said "no such account", anyone could use the form to discover who has an account.

The second serializer handles the click on that link:

backend/backend/serializers.py

```
class PasswordResetConfirmSerializer(serializers.Serializer):
    uid = serializers.CharField()
    token = serializers.CharField()
    new_password = serializers.CharField(write_only=True)
    confirm_password = serializers.CharField(write_only=True)
```

`uid` is the account's id, encoded so it is not a bare number in a URL. `token` is a one-time code that proves the link came from your server and has not expired.

The link the view builds looks like this:

```
http://localhost:5173/reset-password?uid=Mw&token=cq2x1p-04a...
```

`Mw` is what the number `3` becomes after base64 encoding. Base64 is a way of writing data using only letters, digits and a couple of safe symbols, so it survives being pasted into a URL. It is **not** encryption. Anyone can decode `Mw` back to `3`. The security is in the token, not the uid.

backend/backend/serializers.py

```
def validate(self, attrs):
    if attrs["new_password"] != attrs["confirm_password"]:
        raise serializers.ValidationError({
            "confirm_password": "Passwords do not match."
        })

    try:
        uid = force_str(urlsafe_base64_decode(attrs["uid"]))
        user = User.objects.get(pk=uid, is_active=True)
    except Exception:
        raise serializers.ValidationError({
            "uid": "Invalid reset link."
        })
```

The type check comes first, since it is cheap. Then the uid is turned back into a user in two steps:

`urlsafe_base64_decode` undoes the encoding and gives you raw bytes, and `force_str` turns those bytes into a normal Python string.

The whole thing sits in `try` / `except Exception` because so many different failures are possible: the uid might not be valid base64, the decoded text might not be a number, there might be no user with that id, the account might be disabled. All of them mean the same thing to the person holding the link, so all of them produce the same message. That also avoids telling an attacker which part of a guessed link was wrong.

backend/backend/serializers.py

```
if not default_token_generator.check_token(user, attrs["token"]):
    raise serializers.ValidationError({
        "token": "Invalid or expired reset token."
    })

validate_password(attrs["new_password"], user=user)

attrs["user"] = user
return attrs
```

`default_token_generator.check_token(user, token)` is the real gate. Django built that token from the user's id, the time it was made, and, importantly, the current password hash. Three things follow from that, and none of them needed a database table:

- A token only works for the account it was made for.
- A token stops working once `PASSWORD_RESET_TIMEOUT` has passed. In this project `lms/settings.py` sets that to `60 * 60`, one hour.

- A token stops working the moment the password changes, because the hash it was built from is gone. So a link cannot be used twice.

Then Django's password rules run on the new password, with `user=user` so the similarity check has something to compare against. As in `ChangePasswordSerializer`, any complaint from that call comes back under `non_field_errors`.

The last line is a small trick worth knowing: `attrs["user"] = user` puts the loaded `User` object into `validated_data`. The serializer already did the work of finding it, so the view does not repeat the lookup. It just reads it:

backend/backend/views.py

```
user = serializer.validated_data["user"]
new_password = serializer.validated_data["new_password"]
```

NOTE

this flow is only half built in this project. The backend is complete: both URLs exist in `backend/urls.py` as `password-reset/` and `password-reset-confirm/`, both views work, and `frontend/src/api.js` even exports `requestPasswordReset` and `confirmPasswordReset`. What is missing is the React side. `App.jsx` has no `/reset-password` route and no page calls either function, so the link in the email lands on the 404 page. The default in settings points at `http://localhost:5173/reset-password`, while the Vite dev server in this project runs on port 5180 (see `frontend/vite.config.js`), so the address in the email is not one the React app answers at all. Finishing it is a good exercise once you have read the chapters on routing and forms.

TIP

you can still test the flow today without an email account. `lms/settings.py` defaults `EMAIL_BACKEND` to `django.core.mail.backends.console.EmailBackend`, which prints the email into the terminal running `runserver` instead of sending it. POST an email address to `/api/password-reset/` and the link, uid and token appear right there in your terminal, ready to paste into a POST to `/api/password-reset-confirm/`.

How a validation error reaches the user

You have now written a lot of `raise serializers.ValidationError(...)` lines. Follow one all the way to the screen so you know what happens to it.

Step 1, the serializer raises. Say an admin creates an account with the password `abc`.

`validate_password` calls Django's checker, which raises, and the message ends up on the `password` key.

Step 2, the view turns it into a response. Every view that uses one of these serializers calls it the same way:

```
backend/backend/views.py
```

```
serializer.is_valid(raise_exception=True)
```

`raise_exception=True` says: if anything failed, do not carry on, throw. Django REST Framework catches that throw for you and turns it into an HTTP response with status **400 Bad Request** and this body:

```
{ "password": ["This password is too short. It must contain at least 8 characters.", "This password is too common."] }
```

The keys are field names, the values are lists of messages, because one field can fail more than one rule.

Step 3, the frontend flattens it. The React app funnels every backend call through one function in `api.js`, and that function has to cope with three different error shapes, as its own comment records:

```
frontend/src/api.js
```

```
// Django REST Framework reports errors in a few different shapes:  
// {"detail": "..."} a permission or 404 error  
// {"error": "..."} the hand-written login view  
// {"phone": ["This field is required"]} a serializer rejecting a field  
// This turns all of them into one readable line.
```

For the field-errors shape it walks the object and builds a line per field:

```
frontend/src/api.js
```

```
const parts = [];  
for (const [field, value] of Object.entries(body)) {  
  const text = Array.isArray(value) ? value.join(" ") : String(value);  
  parts.push(field === "non_field_errors" ? text : `${field}: ${text}`);  
}
```

`non_field_errors` is the key Django REST Framework uses when an error does not belong to one named field: a `validate` method raising a bare string instead of a dictionary, or a Django error such as the one `validate_password` throws inside `validate`. Since that name means nothing to a user, it is printed without a prefix. Everything else gets `fieldname:` in front, which is why raising with `{"confirm_password": "..."} in ChangePasswordSerializer was worth the extra typing.`

Step 4, it becomes an exception in the page. The joined text is thrown:

frontend/src/api.js

```
if (!response.ok) {  
    throw new Error(readError(body, response));  
}
```

Step 5, the page catches it and stores it. Every page follows this pattern, this one from the Accounts page. `setError` is a React state setter, which means: remember this value and redraw the page:

frontend/src/pages/Accounts.jsx

```
} catch (problem) {  
    setError(problem.message);  
} finally {  
    setIsSaving(false);  
}
```

Step 6, the red banner appears. `<Alert>{error}</Alert>` renders nothing while `error` is an empty string, and a red box with a warning icon as soon as it is not. The `whitespace-pre-line` class in `Alert.jsx` is what makes the line breaks between multiple fields show up as separate lines rather than running together.

So the banner the admin sees reads:

```
password: This password is too short. It must contain at least 8 characters. This password  
is too common.
```

See Figure 0.5.

Six steps, from a `raise` in Python to red pixels in a browser, and the only place the actual rule was decided was your serializer.

TIP

when an error banner in the app says something puzzling, work backwards along this chain. The message is almost always a literal string, so search the backend for it (`grep -rn "the exact sentence" backend/` on Mac or Linux, or your editor's find-in-files box). If it is not in your code, it came from Django or Django REST Framework, and searching the exact sentence online will name the validator that produced it.

What you have now

- A finished `backend/backend/serializers.py`, about 240 lines, holding fourteen serializers.

- Eight `ModelSerializer` classes for teachers, students, courses, enrollments, lessons, assignments, submissions and results, each one a `Meta` block with `model` and `fields`.
- An understanding that foreign keys travel as plain integer ids, and that the React app will have to fetch the related list and join by hand.
- `RegisterSerializer`, which creates a `User` and a `Profile` together inside one transaction, hashes the password with `create_user`, keeps `phone` write-only, and adds `role` to the outgoing JSON with `to_representation`.
- `LoginSerializer`, a two-field shape check that saves nothing.
- `ProfileUpdateSerializer`, which lets people edit their own name, email and phone, refuses to touch `role`, `username` or `password`, and uses `.exclude()` so your own unchanged values never count as duplicates.
- `ChangePasswordSerializer`, which demands the current password and runs Django's own password rules on the new one.
- `PasswordResetRequestSerializer` and `PasswordResetConfirmSerializer`, a backend-complete reset flow waiting on a React page.
- A clear picture of how one `raise serializers.ValidationError(...)` becomes a red banner on screen, and why some messages carry a field name while others do not.

The Endpoints: `views.py` and `urls.py`

In Chapter 3 you built the models, which describe the tables in the database. In Chapter 4 you built the serializers, which turn a row of a table into JSON and back again. Neither of those things is reachable from the outside yet. Nothing happens when you type an address into a browser, because nothing has connected an address to any of that code.

This chapter connects them. When you finish it, the backend is a working API that answers real requests, and you will have talked to it from the terminal without writing a single line of React.

Two files do the work:

FILE	JOB
<code>backend/backend/views.py</code>	The code that runs. One class per endpoint.
<code>backend/backend/urls.py</code>	The address list. Which web address runs which class.

An **endpoint** is one address the API answers on, such as `/api/teacher/`. The word turns up on every page from here on, so it is worth fixing now: address plus the code behind it, together, is an endpoint.

What a view is

A **view** is the piece of code that runs when somebody asks for a web address.

That is the whole idea. A person (or the React app, or the `curl` command) sends a **request**: "GET me the address `/api/teacher/`". Django looks down a list of addresses, finds the one that matches, and calls the view attached to it. The view does some work, usually reading or writing the database, and hands back a **response**: a status number such as `200`, and a body, which in this project is always JSON.

NOTE

JSON (say it "jay-son") is just text in a shape both Python and JavaScript understand. `{"id": 1, "name": "Ada"}` is JSON. Django speaks Python dictionaries, the browser speaks JavaScript objects, and JSON is the common written form they pass back and forth.

In plain Django a view is often a function. In Django REST Framework (DRF, the library that turns Django into an API toolkit) a view is normally a **class**. A class is a template with named pieces inside it. You will see classes such as `TeacherListCreateView` below. The class holds three or four settings, and DRF supplies all of the machinery around them.

Where the URLs live: two files, not one

A **URL** is a web address: `http://127.0.0.1:8001/api/teacher/`. A **URL pattern** is a rule that says "an address that looks like *this runs that view*".

Django starts every request at one file, named by `ROOT_URLCONF` in the settings. Look at `backend/lms/settings.py`:

`backend/lms/settings.py`

```
ROOT_URLCONF = 'lms.urls'
```

So `backend/lms/urls.py` is the front door. Here is the real one, minus the comment block Django wrote at the top:

`backend/lms/urls.py`

```
from django.contrib import admin
from django.urls import include, path

urlpatterns = [
    path('admin/', admin.site.urls),
    path('api/', include('backend.urls')),
]
```

Five lines, and they decide the shape of every address in the project.

`urlpatterns` is a plain Python list. Django reads it from the top down and stops at the first rule that matches.

- `path('admin/', admin.site.urls)` sends anything starting with `admin/` to the built-in Django admin site.
- `path('api/', include('backend.urls'))` says: if the address starts with `api/`, chop that part off and hand the rest to the list in `backend/backend/urls.py`.

That second line is why every endpoint in this chapter begins with `/api/`. The app's own `urls.py` never writes `api/` anywhere. It writes `teacher/`, and the prefix is bolted on by the include.

Follow one request all the way through:

STEP	WHAT HAPPENS
1	Browser asks for <code>http://127.0.0.1:8001/api/teacher/5/</code>
2	Django removes the host, leaving the path <code>api/teacher/5/</code>
3	<code>lms/urls.py</code> matches <code>api/</code> , chops it off, passes <code>teacher/5/</code> on
4	<code>backend/urls.py</code> matches <code>teacher/<int:pk>/</code> , with <code>pk</code> set to <code>5</code>
5	<code>TeacherRetrieveUpdateDestroyAPIView</code> runs and returns JSON

TIP

Keeping the app's URLs in their own file is not just tidiness. If you ever wanted the whole API to live at `/v2/` instead of `/api/`, you would change one line in `lms/urls.py` and nothing else in the project.

COMMON MISTAKE

Leaving off the trailing slash. Every pattern in this project ends with `/`. A `GET` to `/api/teacher` (no slash) still works, because Django's `APPEND_SLASH` setting sends the browser a redirect to the slashed address. A `POST` to `/api/teacher` does not. Django cannot carry the body of a `POST` through a redirect, so with `DEBUG = True` (which is what this project ships with) it stops and shows you a `RuntimeError` reading "You called this URL via POST, but the URL doesn't end in a slash and you have APPEND_SLASH set." The same happens for `PUT`, `PATCH` and `DELETE`. Always type the slash.

HTTP methods, for people who have never met them

Every request carries a **method**, a single word saying what kind of action it is. There are many, but this project uses five.

METHOD	PLAIN MEANING	WHERE YOU SEE IT IN THE FINISHED LMS
<code>GET</code>	Read. Change nothing.	Opening the Teachers page loads the list. The Dashboard fetches every list so it can count the rows.
<code>POST</code>	Create something new.	The Save button when you are adding a teacher. Also the Log in button, and Create account .
<code>PUT</code>	Replace an existing thing, all fields at once.	The Save button when you are editing an existing row.
<code>PATCH</code>	Change part of an existing thing.	Save details on <i>Your profile</i> .
<code>DELETE</code>	Remove a thing.	The red trash icon, after you confirm in the dialog.

The address alone does not say what you want. `/api/teacher/5/` with `GET` reads teacher 5. The exact same address with `DELETE` removes teacher 5. Address plus method is the full instruction.

NOTE

DRF also treats `HEAD` and `OPTIONS` as reading. Together with `GET` these three are called the *safe methods*, because they never change data. The permission classes in this project lean on that. `backend/backend/permissions.py` contains this inside `RoleWritePermission`:

```
python if request.method in SAFE_METHODS: return True
```

That is how "anyone signed in may read, only some roles may write" is expressed in two lines.

The response carries a status number. The ones you will meet:

STATUS	MEANING HERE
200 OK	Read or update worked. Body has the data.
201 Created	<code>POST</code> worked. Body has the row that was made, including its new <code>id</code> .
204 No Content	<code>DELETE</code> worked. Body is empty on purpose.
400 Bad Request	You sent something the serializer rejected. Body names the bad fields.
401 Unauthorized	You sent no token, or a token that is no good.
403 Forbidden	Your token is fine, but your role is not allowed to do this.
404 Not Found	No row with that id, or no URL pattern matched.

The generic views: eight resources without writing eight times

The LMS has eight resources: teachers, students, courses, enrollments, lessons, assignments, submissions and results. Each one needs the same six things:

1. list them all
2. create one
3. read one by id
4. replace one by id
5. change part of one by id
6. delete one by id

Written by hand that is forty-eight blocks of near-identical code. DRF ships **generic views**, which are classes that already contain all of that behaviour. You inherit from one and fill in the blanks.

Two of them cover this whole project.

GENERIC VIEW	ADDRESS IT GOES ON	METHODS IT ANSWERS
ListCreateAPIView	the collection, /api/teacher/	GET (list) and POST (create)
RetrieveUpdateDestroyAPIView	one item, /api/teacher/5/	GET (read one), PUT, PATCH, DELETE

"Retrieve" means read one. "Destroy" means delete. The class names are long because they list exactly what they do, which is a mercy when you come back to the file in three months.

BUILDING THE FIRST PAIR

Open `backend/backend/views.py`. The imports at the very top of the real file are these:

```
# backend/backend/views.py (top of file, imports)
from django.shortcuts import render
from django.contrib.auth.models import User
from rest_framework.views import APIView
from rest_framework.response import Response
from rest_framework import generics, status
from rest_framework_simplejwt.tokens import RefreshToken
from rest_framework.permissions import IsAuthenticated
from django.contrib.auth.tokens import default_token_generator
```

`from rest_framework import generics` is the one that matters right now. It gives you `generics.ListCreateAPIView` and friends.

NOTE

`from django.shortcuts import render` is left over from the file Django generated with the app. Nothing in `views.py` renders an HTML template, so that import is never used. The two `# Create your views here.` comments sitting in the middle of the file are leftovers too. Unused scraps like these build up in real projects. They are harmless, and it is honest to leave them visible rather than pretend the file was written perfectly in one pass.

The views also need the models, the serializers and the permission classes:

```
# backend/backend/views.py (imports, continued)
from .serializers import (AssignmentSerializer, ChangePasswordSerializer, CourseSerializer,
    EnrollmentSerializer, LessonSerializer, ProfileUpdateSerializer, RegisterSerializer,
    LoginSerializer, ResultSerializer, StudentSerializer, SubmissionSerializer,
    TeacherSerializer)

from .models import (Assignment, Course, Enrollment, Lesson, Profile, Results, Student,
    Submission, Teacher)
from .permissions import AdminWrites, IsAdmin, SubmissionWrites, TeachingStaffWrites,
    role_of
```

The single dot in `from .models` means "from the models module sitting next to this file". It is the same `backend/backend/models.py` you built in Chapter 3.

The password reset views added later brought a second batch of imports with them, which is why the real file has two separate `from .serializers import` lines:

```
# backend/backend/views.py (imports, continued)
from .serializers import (
    PasswordResetRequestSerializer,
    PasswordResetConfirmSerializer,
)

from rest_framework.permissions import AllowAny
from django.core.mail import send_mail
from django.conf import settings
from django.utils.encoding import force_bytes
from django.utils.http import urlsafe_base64_encode
from django.contrib.auth import get_user_model

User = get_user_model()
```

That last line reassigns `User`, overwriting the one imported at the top of the file. In this project both point at exactly the same class, Django's built-in `User`, so nothing changes. `get_user_model()` is the habit to keep, because it still works if a project later swaps in its own user model.

Now the first view. At the bottom of `views.py`, add:

```
backend/backend/views.py

class TeacherListCreateView(generics.ListCreateAPIView):
    queryset = Teacher.objects.all()
    serializer_class = TeacherSerializer
    permission_classes = [AdminWrites]
```

Three lines of settings, and this class answers `GET` and `POST`. Take them one at a time.

`queryset = Teacher.objects.all()` is the set of rows the view works with. `Teacher.objects` is Django's door to the teacher table, and `.all()` means every row, no filter. For a `GET` list, DRF runs this and serializes the result. For a `GET` of one item, DRF searches inside this set for the row you asked for, which is also why an id that is not in the set gives `404` rather than a crash.

`serializer_class = TeacherSerializer` names the translator from Chapter 4. Note there are no brackets after it: you are handing DRF the class itself, not one built copy of it. (Building a copy of a class is called making an *instance*.) DRF makes instances when it needs them, sometimes one, sometimes one per row.

`permission_classes = [AdminWrites]` is the gate. Before any of the above runs, DRF asks every class in this list "may this caller do this?" `AdminWrites` lives in `backend/backend/permissions.py` and says: any signed-in account may read; only an admin may write. A refusal comes back as a `403`, with the class's own `message` wrapped in DRF's usual shape, `{"detail": "Only an admin can change teacher and student records."}`.

Now the detail view. It is the same three lines with a different base class:

backend/backend/views.py

```
class TeacherRetrieveUpdateDestroyAPIView(generics.RetrieveUpdateDestroyAPIView):
    queryset = Teacher.objects.all()
    serializer_class = TeacherSerializer
    permission_classes = [AdminWrites]
```

Same queryset, same serializer, same permission. The only difference is which methods the base class answers, and that this one expects an id in the URL.

NOTE

`permission_classes` is a list because you can stack several, and all of them must agree. Every view in this project uses exactly one, so the list always has a single item.

TIP

If you ever forget `permission_classes` on a new view, it is still not public. `backend/lms/settings.py` sets `DEFAULT_PERMISSION_CLASSES` to `IsAuthenticated`, so a view with no opinion of its own requires a signed-in caller. The settings file spells out why: "Failing closed means a new view is locked until somebody decides otherwise."

GIVING THEM ADDRESSES

A view that no URL points at can never run. Open `backend/backend/urls.py` and add the two patterns:

```
backend/backend/urls.py
```

```
path('teacher/', TeacherListCreateView.as_view(), name='teacher-list'),  
path('teacher/<int:pk>', TeacherRetrieveUpdateDestroyAPIView.as_view(), name='teacher-  
detail'),
```

`path()` takes three things here.

The route string. `'teacher/'` matches exactly that, and remember the `api/` prefix is added by `lms/urls.py`, so the real address is `/api/teacher/`.

The view. `.as_view()` is required and easy to forget. `path()` wants a *function* to call. A class is not a function. `.as_view()` builds a small function that creates the class and runs it. Every class-based view in Django is wired up this way.

`name='teacher-list'` is a label for this pattern. Nothing in the React frontend uses it, because the frontend types the addresses out in `frontend/src/api.js`. It matters inside Python:

`reverse('teacher-list')` is a Django helper that looks a name up and gives you back the address `/api/teacher/`, so you do not have to type the address in two places. It is also what the DRF browsable API uses to build its links.

COMMON MISTAKE

Writing `path('teacher/', TeacherListCreateView, ...)` without `.as_view()`. Django starts and the URL looks fine, then the request explodes with `TypeError: view must be a callable`. If you see that message, count your `.as_view()` calls.

<INT:PK>, AND WHAT PK MEANS

`<int:pk>` is a **path converter**. It has two halves separated by a colon.

`int` is the converter name. It matches one or more digits and, importantly, hands the value to the view as a Python integer, not as the string `"5"`. Django ships several: `str` (anything but a slash), `slug`, `uuid`, `path`. `int` is right here because ids in this database are numbers.

`pk` is the name the captured value is given. It stands for **primary key**, database language for the column that uniquely identifies a row. Every model in Chapter 3 got one for free: an auto-numbering `id` column. `pk` and `id` mean the same thing for these models, and Django lets you write either.

The name is not free choice. DRF's detail views look for a URL keyword called `pk`, because they carry `lookup_field = 'pk'` by default. So `/api/teacher/5/` sets `pk = 5`, DRF runs the equivalent of `Teacher.objects.all().get(pk=5)`, and returns that row.

COMMON MISTAKE

Writing `path('teacher/<int:id>', ...)`. The URL still matches, and then the view raises an error about an unexpected keyword argument `id`, or a missing `pk`. The word has to be `pk`.

Because `int` only matches digits, `/api/teacher/abc/` matches no pattern at all and Django answers `404` before your view is ever reached. You get that check for free.

CHECKPOINT Save both files and start the server from the `backend` folder with `python manage.py runserver 8001`. In another terminal run `curl -i http://127.0.0.1:8001/api/teacher/`. You should see `HTTP/1.1 401 Unauthorized` and the body `{"detail": "Authentication credentials were not provided."}`. A 401 is a success here. It proves the URL matched, the view ran, and the permission gate stopped you. A `404` at this point means the pattern is wrong or the server did not reload.

THE OTHER SEVEN RESOURCES

The remaining fourteen classes in `views.py` are the same three settings with different names. Rather than print them all, here is the whole set as a table. Every one of these has `queryset = <Model>.objects.all()` and `serializer_class = <Model>Serializer`; only the permission class varies.

RESOURCE	PATH	LIST VIEW CLASS	DETAIL VIEW CLASS	PERMISSION CLASS
Teacher	<code>teacher/</code>	<code>TeacherListCreateView</code>	<code>TeacherRetrieveUpdateDestroyAPIView</code>	<code>AdminWrites</code>
Student	<code>student/</code>	<code>StudentListCreateView</code>	<code>StudentRetrieveUpdateDestroyAPIView</code>	<code>AdminWrites</code>
Course	<code>course/</code>	<code>CourseListCreateView</code>	<code>CourseRetrieveUpdateDestroyAPIView</code>	<code>TeachingStaffWrites</code>
Enrollment	<code>enrollment/</code>	<code>EnrollmentListCreateView</code>	<code>EnrollmentRetrieveUpdateDestroyAPIView</code>	<code>TeachingStaffWrites</code>
Lesson	<code>lesson/</code>	<code>LessonListCreateView</code>	<code>LessonRetrieveUpdateDestroyAPIView</code>	<code>TeachingStaffWrites</code>
Assignment	<code>assignment/</code>	<code>AssignmentListCreateView</code>	<code>AssignmentRetrieveUpdateDestroyAPIView</code>	<code>TeachingStaffWrites</code>
Submission	<code>submission/</code>	<code>SubmissionListCreateView</code>	<code>SubmissionRetrieveUpdateDestroyAPIView</code>	<code>SubmissionWrites</code>
Results	<code>results/</code>	<code>ResultsListCreateView</code>	<code>ResultsRetrieveUpdateDestroyAPIView</code>	<code>TeachingStaffWrites</code>

Two of these deserve a second look.

The `Results` row uses the class names `ResultsListCreateView` and `ResultsRetrieveUpdateDestroyAPIView` (plural), but the serializer is `ResultSerializer` (singular) and the model is `Results` (plural). The naming drifted while the project was written. Copy it as it is, or your imports will not resolve.

The path is `results/`, plural, while every other path is singular. `/api/submission/` and `/api/results/`. There is no reason for it beyond how it was typed the first time, and it is now baked into `frontend/src/api.js`, which even carries a comment about it: `// Note the naming: /submission/ is singular but /results/ is plural.`

COMMON MISTAKE

Guessing `/api/result/` or `/api/teachers/`. Both give `404`. The paths are exactly the eight in the table above: seven singular and `results/` plural.

Here is one full pair from the middle of the file so you can see that the pattern really does not change, docstrings and all. A **docstring** is the quoted line just under a `class` or `def`, a note to the next reader that Python keeps attached to the object.

backend/backend/views.py

```
class SubmissionListCreateView(generics.ListCreateAPIView):
    """View to list and create submissions."""
    queryset = Submission.objects.all()
    serializer_class = SubmissionSerializer
    permission_classes = [SubmissionWrites]

class SubmissionRetrieveUpdateDestroyAPIView(generics.RetrieveUpdateDestroyAPIView):
    """View to retrieve, update, or delete a submission."""
    queryset = Submission.objects.all()
    serializer_class = SubmissionSerializer
    permission_classes = [SubmissionWrites]
```

`SubmissionWrites` is the only permission class in the project that treats one method specially. It lets a student `POST` (hand work in) but not `PUT`, `PATCH` or `DELETE`. The reason is in its docstring in `backend/backend/permissions.py`: nothing links a `Student` row to a login account, so the server has no way to tell whose work it is, and "edit your own" cannot be told apart from "edit anyone's".

The finished urls.py

Now build the address file properly, in three sections.

Section 1: the imports. One `path` from Django, then every view class from `backend/backend/views.py`. The real file breaks the import across many lines inside one pair of brackets:

```
# backend/backend/urls.py (section 1 of 3: imports)
from django.urls import path

from backend.views import (ChangePasswordView, PasswordResetConfirmView,
PasswordResetRequestView, RegisterView,

LoginView, ProtectedView, TeacherListCreateView, StudentListCreateView, TeacherRetrieveUpdateDe
stroyAPIView,

StudentRetrieveUpdateDestroyAPIView, CourseListCreateView,
CourseRetrieveUpdateDestroyAPIView, EnrollmentListCreateView,
EnrollmentRetrieveUpdateDestroyAPIView, LessonListCreateView,
LessonRetrieveUpdateDestroyAPIView, AssignmentListCreateView,
AssignmentRetrieveUpdateDestroyAPIView, SubmissionListCreateView,
SubmissionRetrieveUpdateDestroyAPIView, ResultsListCreateView,
ResultsRetrieveUpdateDestroyAPIView)
```

Section 2: the account endpoints. These come first in `urlpatterns`:

```
# backend/backend/urls.py (section 2 of 3: accounts)
urlpatterns = [
    path('login/', LoginView.as_view(), name='login'),
    path('register/', RegisterView.as_view(), name='register'),
    path('profile/', ProtectedView.as_view(), name='protected'),
    path('change-password/', ChangePasswordView.as_view(), name='change-password'),
    path("password-reset/", PasswordResetRequestView.as_view()),
    path("password-reset-confirm/", PasswordResetConfirmView.as_view()),
```

Two details worth noticing. The profile endpoint is at `profile/` but its name is `protected`, because the class is called `ProtectedView`. And the last two patterns have no `name=` at all, unlike every other line in the file. Nothing breaks, because nothing calls `reverse()` on them, but they are the odd ones out.

Section 3: the eight resources. Each gets two lines, a list address and a detail address:

```
# backend/backend/urls.py (section 3 of 3: the resources)
#lms main project
path('teacher/', TeacherListCreateView.as_view(), name='teacher-list'),
path('teacher/<int:pk>/', TeacherRetrieveUpdateDestroyAPIView.as_view(), name='teacher-
detail'),

path('student/', StudentListCreateView.as_view(), name='student-list'),
path('student/<int:pk>/', StudentRetrieveUpdateDestroyAPIView.as_view(), name='student-
detail'),

path('course/', CourseListCreateView.as_view(), name='course-list'),
path('course/<int:pk>/', CourseRetrieveUpdateDestroyAPIView.as_view(), name='course-
detail'),

path('enrollment/', EnrollmentListCreateView.as_view(), name='enrollment-list'),
path('enrollment/<int:pk>/', EnrollmentRetrieveUpdateDestroyAPIView.as_view(),
name='enrollment-detail'),
```

and the second half:

```
# backend/backend/urls.py (section 3 of 3, continued)
path('lesson/', LessonListCreateView.as_view(), name='lesson-list'),
path('lesson/<int:pk>/', LessonRetrieveUpdateDestroyAPIView.as_view(), name='lesson-
detail'),

path('assignment/', AssignmentListCreateView.as_view(), name='assignment-list'),
path('assignment/<int:pk>/', AssignmentRetrieveUpdateDestroyAPIView.as_view(),
name='assignment-detail'),

path('submission/', SubmissionListCreateView.as_view(), name='submission-list'),
path('submission/<int:pk>/', SubmissionRetrieveUpdateDestroyAPIView.as_view(),
name='submission-detail'),

path('results/', ResultsListCreateView.as_view(), name='result-list'),
path('results/<int:pk>/', ResultsRetrieveUpdateDestroyAPIView.as_view(), name='result-
detail'),
]
```

The names follow `<thing>-list` and `<thing>-detail` throughout, and the results pair is named `result-list` and `result-detail` in the singular even though the path and the class names are plural.

That is the finished `backend/backend/urls.py`, in full: the import block, then twenty-two `path()` calls in one list. Nothing else belongs in this file. It contains no logic, only addresses.

TIP

To see every URL Django knows about, including the admin ones, `python manage.py show_urls` works if you have the `django-extensions` package installed. This project does not install it (check `backend/requirements.txt`: Django, djangorestframework, djangorestframework-simplejwt, django-cors-headers, mysqlclient and PyJWT, and that is all). So the fastest check here is to visit a wrong address and read the yellow debug page: with `DEBUG = True`, Django lists every pattern it tried.

The hand-written views

Six endpoints are not simple CRUD over a table, so they get more attention. CRUD is the usual shorthand for create, read, update and delete, the four things a generic view already knows how to do.

Five of the six are written by hand on top of `APIView`. `APIView` is DRF's plainest base class: it hands you the request and expects you to build the response yourself. You add a method named after the HTTP method you want to answer. A method called `post` answers `POST`. A method called `get` answers `GET`. Any method you do not write returns `405 Method Not Allowed`. The sixth, registration, stays a generic view, because creating an account really is a plain "make one row" job.

The real `views.py` grew over time, so the order in the file is not the order that makes sense to learn in. The file goes: imports, `blacklist_user_tokens`, the two password reset views, `RegisterView`, `get_tokens_for_user`, `LoginView`, `ProtectedView`, `ChangePasswordView`, then the sixteen generic views. Below they are grouped by what they do.

GET_TOKENS_FOR_USER

Logging in has to hand back a token. A **JWT** (JSON Web Token) is a long string the server signs. The caller keeps it and sends it with every later request as proof of who they are. The server does not store it or look it up; it just checks the signature, which is what makes JWTs fast and also what makes them hard to cancel.

`djangorestframework-simplejwt`, the library from `requirements.txt`, produces two of them:

`backend/backend/views.py`

```
def get_tokens_for_user(user):
    """Helper to get JWT tokens for a user."""
    refresh = RefreshToken.for_user(user)
    return {
        'refresh': str(refresh),
        'access': str(refresh.access_token),
    }
```

The **access** token is the one you attach to normal requests. The **refresh** token exists to get a fresh access token when the old one expires. This project has no refresh endpoint, so the refresh token is sent and then never used by the frontend. `settings.py` widens the access token's life to eight hours (`"ACCESS_TOKEN_LIFETIME": timedelta(hours=8)`) instead of the library's default five minutes, with the reason written above it: five minutes forces a re-login in the middle of a task while you are learning.

`str(refresh)` and `str(refresh.access_token)` turn the token objects into the plain strings that go into JSON.

LOGINVIEW

This is the endpoint every other part of the app depends on, and it has one surprise in it.

backend/backend/views.py

```
class LoginView(APIView):
    """Login view using phone and password."""

    permission_classes = [AllowAny]

    def post(self, request):
        serializer = LoginSerializer(data=request.data)
        serializer.is_valid(raise_exception=True)
        phone = serializer.validated_data['phone']
        password = serializer.validated_data['password']
```

`permission_classes = [AllowAny]` overrides the project default of `IsAuthenticated`. It has to: nobody is signed in yet when they log in.

`LoginSerializer` is a plain `Serializer` with two fields, `phone` and `password`.

`is_valid(raise_exception=True)` means "check the input and, if it is bad, stop right here and send a 400 listing the bad fields". Without `raise_exception=True` you would have to check the return value and build that response yourself.

Now the surprise:

```
# backend/backend/views.py (LoginView.post, continued)
    try:
        profile = Profile.objects.get(phone=phone)
        user = profile.user
    except Profile.DoesNotExist:
        return Response({'error': 'Invalid phone or password'},
            status=status.HTTP_400_BAD_REQUEST)

    if not user.check_password(password):
        return Response({'error': 'Invalid phone or password'},
            status=status.HTTP_400_BAD_REQUEST)
```

The account is found by `Profile.phone`, not by `User.username`. Django's own `User` model has no phone column. The `Profile` model you built in Chapter 3 does, and it is `unique=True`, so one phone number belongs to exactly one account. `Profile.user` is a `OneToOneField`, which means each profile is tied to exactly one account and each account to at most one profile, so writing `profile.user` hops straight across that link.

This is the single biggest thing to remember about this backend. There is no username login anywhere. A form asking for a username and a password will always fail, no matter how correct the password is.

Then look at the two error messages. They are **identical**: `Invalid phone or password`, for a phone that does not exist and for a phone that exists with the wrong password. That is deliberate, and it is worth understanding why.

If the messages differed, anyone could type phone numbers at the endpoint and read the replies to learn which numbers have accounts. "No such phone" versus "wrong password" is a yes/no answer about whether a person is a member of your school, handed out to strangers. One message for both cases gives an attacker nothing. The cost is a slightly less helpful error for the honest user who mistyped, and that trade is the normal one to make.

`user.check_password(password)` scrambles what was typed through a one-way function (a **hash**) and compares the result to the stored hash. Passwords are never stored in readable form, so there is nothing to compare directly.

See Figure 0.2.

The successful reply is where frontend beginners lose an hour:

```
# backend/backend/views.py (LoginView.post, the reply)
tokens = get_tokens_for_user(user)
return Response({
    'message': 'Login successful',
    'user_id': user.id,
    'username': user.username,
    # The frontend uses this to decide which buttons to show. It is
    # not what enforces anything: the permission classes do that.
    'role': role_of(user),
    'tokens': tokens
})
```

The tokens are **nested** under a `tokens` key. The JSON that comes back looks like this:

```
{
  "message": "Login successful",
  "user_id": 1,
  "username": "admin",
  "role": "admin",
  "tokens": {
    "refresh": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...",
    "access": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9..."
  }
}
```

COMMON MISTAKE

Reading `data.access` in the frontend. It does not exist and comes back as `undefined`. Every later request then sends the header `Authorization: Bearer undefined`, the server rejects it, and you get a stream of 401s that look like a login problem when the login actually worked perfectly. The correct read is `data.tokens.access`. `frontend/src/AuthContext.jsx` carries a comment on that exact line: `// The tokens are nested. data.access does not exist.`

The `role` field comes from `role_of(user)` in `permissions.py`, which returns `"admin"`, `"teacher"` or `"student"`. The comment above it in the code is the important part: the frontend uses the role to decide which buttons to draw, and that is a convenience only. Nothing is enforced by it. If you edited the saved role in your browser's storage, the buttons would appear and every one of them would come back `403`, because the permission classes on the server check the database, not the token.

NOTE

This view does not check `user.is_active`. A deactivated account can still log in through `/api/login/`, unlike the password reset views further down, which filter on `is_active=True`. If you extend this project, that is a real gap to close.

REGISTERVIEW

Creating an account is a plain "make one row" job, so this one goes back to a generic view. It is a `CreateAPIView`, which answers `POST` and nothing else.

backend/backend/views.py

```
class RegisterView(generics.CreateAPIView):
    """Create an account. Admins only.

    There is no public sign-up: a student does not make their own account, an
    admin makes it for them and picks the role. Anonymous callers get a 401,
    signed-in teachers and students a 403.

    If there is no admin left to do this, `python manage.py createsuperuser`
    still works, and a superuser counts as an admin.
    """

    queryset = User.objects.all()
    serializer_class = RegisterSerializer
    permission_classes = [IsAdmin]
```

That docstring is the real one from the file, and it is the design in four sentences. This is not a sign-up form. There is no way for a stranger to create an account in this LMS at all.

`IsAdmin` is stricter than the `AdminWrites` and `TeachingStaffWrites` classes. It has no exception for safe methods, so it blocks everything, for every method, unless the caller's role is admin. Its own `message` is "Only an admin can create accounts."

NOTE

The comment next to `DEFAULT_PERMISSION_CLASSES` in `backend/lms/settings.py` says "the two views that really are public (register, login) say so with `AllowAny`". That comment is out of date. Only `LoginView`, `PasswordResetRequestView` and `PasswordResetConfirmView` use `AllowAny`; registration was locked down to admins later and the comment was never updated. Trust the code, not the comment.

The 401 versus 403 difference is DRF's doing, and it is a useful thing to be able to read:

WHO CALLS	POST /API/REGISTER/	REPLY	WHY
Nobody signed in		401 Unauthorized	No credentials were supplied at all
A signed-in student or teacher		403 Forbidden	Credentials were fine, the role was not
A signed-in admin		201 Created	The new account, with its <code>id</code>

"401 means who are you, 403 means I know who you are and no" is the sentence to keep.

Because a superuser has no `Profile` row, `role_of()` checks `user.is_superuser` first and treats any superuser as an admin. That is the escape hatch mentioned in the docstring: if you somehow lose every admin, `createsuperuser` from the terminal gets you back in.

See Figure 0.5.

PROTECTEDVIEW

One class serves the whole *Your profile* page, answering `GET` to read your details and `PATCH` to change them.

backend/backend/views.py

```
class ProtectedView(APIView):
    """Your own account: GET to read it, PATCH to change it.

    PATCH only ever touches request.user, so there is no way to aim it at
    somebody else's account, and `role` is not a field the serializer accepts.
    """
    permission_classes = [IsAuthenticated]
```

`IsAuthenticated` means any signed-in account, whatever its role. Everyone has a profile to look at.

Both methods need to produce the same shape of user data, so that is pulled out into a helper. A leading underscore on `_user_data` is a Python convention meaning "internal to this class, not part of its public surface". DRF ignores it, because it only routes methods named after HTTP methods.

```
# backend/backend/views.py (ProtectedView, continued)
def _user_data(self, user):
    profile = Profile.objects.filter(user=user).first()
    return {
        'user_id': user.id,
        'username': user.username,
        'email': user.email,
        'first_name': user.first_name,
        'last_name': user.last_name,
        'phone': profile.phone if profile else None,
        'role': role_of(user),
    }
```

Five of these fields come from Django's `User` table. `phone` comes from the `Profile` row alongside it. `.filter(...).first()` is used instead of `.get(...)` on purpose: `.get()` raises an exception when there is no row, `.first()` returns `None`. A superuser made with `createsuperuser` has no profile, so `phone` comes back as `null` for that account rather than crashing the page.

Reading is then two lines:

```
# backend/backend/views.py (ProtectedView, continued)
def get(self, request):
    return Response({
        'message': 'successfully fetched this user',
        'user': self._user_data(request.user)
    })
```

`request.user` is the heart of the whole design. DRF has already read the `Authorization` header, checked the token's signature, and loaded the matching account by the time your method runs. `request.user` is that account. There is no id in the URL and no id in the body, so this view is structurally incapable of showing you somebody else's details.

Updating works the same way:

```
# backend/backend/views.py (ProtectedView, continued)
def patch(self, request):
    serializer = ProfileUpdateSerializer(
        data=request.data,
        context={'request': request},
    )
    serializer.is_valid(raise_exception=True)
    serializer.save()

    request.user.refresh_from_db()
    return Response({
        'message': 'Your details have been saved.',
        'user': self._user_data(request.user)
    })
```

`context={'request': request}` passes the request into the serializer, which reads `self.context["request"].user` to know whose row to write. That is the second half of the guarantee in the docstring.

`refresh_from_db()` re-reads the account from the database. The serializer saved changes through its own copy of the user object, and without this line the response could echo the old name back at you, making a save that worked look like one that failed.

`ProfileUpdateSerializer` accepts four fields only: `first_name`, `last_name`, `email`, `phone`. `role` is not among them, so `{"role": "admin"}` sent to this endpoint is ignored rather than obeyed. Self-promotion is not possible through the API.

See Figure 0.6.

CHANGEPASSWORDVIEW, AND THE TOKEN PROBLEM

backend/backend/views.py

```
class ChangePasswordView(APIView):
    """Change your own password, knowing the old one."""
    permission_classes = [IsAuthenticated]

    def post(self, request):
        serializer = ChangePasswordSerializer(
            data=request.data,
            context={'request': request},
        )
        serializer.is_valid(raise_exception=True)

        user = request.user
        user.set_password(serializer.validated_data['new_password'])
        user.save()
```

`POST`, not `PATCH`, because this is an action rather than a field edit.

The serializer demands three fields: `current_password`, `new_password` and `confirm_password`. Requiring the old one is what makes the change yours to make. A live session on a borrowed laptop is not enough to take an account over.

`set_password()` hashes the new password before storing it. Assigning `user.password = "secret"` directly would store the literal text and break login completely, since `check_password()` expects a hash.

Now the interesting part. When a password changes, old tokens should stop working. Here is the attempt:

```
# backend/backend/views.py (ChangePasswordView, continued)
# Refresh tokens issued earlier are retired. The access token already
# in the caller's hands cannot be revoked – JWTs are not looked up on
# each request – so the frontend signs you out and makes you log in
# again with the new password.
blacklist_user_tokens(user)

return Response(
    {'detail': 'Password changed. Please log in again.'},
    status=status.HTTP_200_OK,
)
```

Read that comment twice. It is the honest limit of JWTs, and it surprises people.

A session cookie is a pointer: the server looks the session up on every request and can delete it at any moment. A JWT is not a pointer. It carries the user id inside itself and is signed. The server checks the signature by doing arithmetic, not by asking the database anything. There is no row to delete, so there is nothing to switch off. An access token stays valid until its expiry, which here is eight hours.

What *can* be cancelled is the refresh token, because `rest_framework_simplejwt.token_blacklist` is listed in `INSTALLED_APPS` and keeps a table of them. That is what the helper does:

backend/backend/views.py

```
def blacklist_user_tokens(user):
    try:
        from rest_framework_simplejwt.token_blacklist.models import (
            OutstandingToken,
            BlacklistedToken,
        )

        tokens = OutstandingToken.objects.filter(user=user)

        for token in tokens:
            BlacklistedToken.objects.get_or_create(token=token)

    except Exception:
        pass
```

`OutstandingToken` is every refresh token ever issued to this account. The loop marks each one blacklisted. `get_or_create` avoids an error if a token is already on the list.

The import sits inside the function, not at the top of the file, and the whole body is wrapped in `try` / `except Exception: pass`. That means: if the blacklist app is not installed, changing a password still succeeds instead of crashing with an import error. It is a deliberate soft failure.

Since the access token cannot be killed, the frontend does the job itself: it clears the saved session and sends you to the login page, which is why the response text reads "Password changed. Please log in again."

NOTE

The same limitation explains why this project has no logout endpoint. Logging out just throws the saved token away in the browser (`clearSession()` in `frontend/src/api.js`). The token stays valid on the server until it expires. For a real deployment you would shorten the access token's life a lot and add a refresh endpoint.

PASSWORDRESETREQUESTVIEW

Forgotten passwords need a route that does not require being signed in, so this pair uses `AllowAny`.

backend/backend/views.py

```
class PasswordResetRequestView(APIView):
    permission_classes = [AllowAny]

    def post(self, request):
        serializer = PasswordResetRequestSerializer(data=request.data)
        serializer.is_valid(raise_exception=True)

        email = serializer.validated_data["email"]
        user = User.objects.filter(email=email, is_active=True).first()
```

The caller sends one field, `email`. Note `is_active=True` in the lookup: a disabled account cannot be resurrected through a reset link.

If an account was found, a link is built and sent:

```
# backend/backend/views.py (PasswordResetRequestView, continued)
if user:
    uid = urlsafe_base64_encode(force_bytes(user.pk))
    token = default_token_generator.make_token(user)

    reset_link = (
        f"{settings.FRONTEND_PASSWORD_RESET_URL}"
        f"?uid={uid}&token={token}"
    )
```

Two pieces go into the link.

`uid` is the account's primary key, encoded. `force_bytes(user.pk)` turns the number `7` into raw bytes, and `urlsafe_base64_encode` turns those into short text with no characters that would upset a URL. This is encoding, not encryption: anybody can decode it. Its job is to travel safely in a query string, nothing more.

`token` is the security. `default_token_generator` is Django's own reset token machinery. The token is built from the user's id, the current time, and, importantly, the existing password hash. That last ingredient means the token stops working the moment the password changes, so a reset link cannot be used twice. `PASSWORD_RESET_TIMEOUT = 60 * 60` in the settings gives it a one hour life.

`settings.FRONTEND_PASSWORD_RESET_URL` defaults to `http://localhost:5173/reset-password`, so a finished link reads roughly:

```
http://localhost:5173/reset-password?uid=Nw&token=cf9k2p-4e1c8e0e5a1c9d3f...
```

Then the email, and the reply:

```
# backend/backend/views.py (PasswordResetRequestView, continued)

    send_mail(
        subject="Reset your password",
        message=f"Click the link below to reset your password:\n\n{reset_link}",
        from_email=settings.DEFAULT_FROM_EMAIL,
        recipient_list=[user.email],
        fail_silently=False,
    )

    return Response(
        {
            "detail": "If an account exists with this email, a password reset link has
been sent."
        },
        status=status.HTTP_200_OK,
    )
```

Look carefully at the indentation. `send_mail` is inside `if user:`. The `return Response(...)` is **outside** it. The same `200` with the same sentence goes back whether the email matched an account or not.

That is the same reasoning as the login error, applied to email addresses. Replying "no account with that email" would let anyone check whether a person is registered here, one address at a time. The wording is chosen to be truthful either way: "If an account exists with this email, a password reset link has been sent."

NOTE

`EMAIL_BACKEND` defaults to `django.core.mail.backends.console.EmailBackend`, so no mail actually leaves your machine. The whole message, link and all, prints in the terminal running `runserver`. That is convenient for learning: you can copy the link straight out of the terminal. It also means the reset flow is not usable end to end in this project, and the README says so under "Known gaps": there is no `/reset-password` page in the frontend yet. `fail_silently=False` means a broken mail setup raises a `500` rather than pretending to have sent something.

Note also that the default link points at port 5173, while this project's Vite dev server runs on **5180** (see `frontend/vite.config.js`). Another reason the link does not land anywhere useful yet.

PASSWORDRESETCONFIRMVIEW

The second half takes the `uid` and `token` back, along with the new password.

backend/backend/views.py

```
class PasswordResetConfirmView(APIView):
    permission_classes = [AllowAny]

    def post(self, request):
        serializer = PasswordResetConfirmSerializer(data=request.data)
        serializer.is_valid(raise_exception=True)

        user = serializer.validated_data["user"]
        new_password = serializer.validated_data["new_password"]

        user.set_password(new_password)
        user.save()

        blacklist_user_tokens(user)

    return Response(
        {
            "detail": "Password has been reset successfully. Please login again."
        },
        status=status.HTTP_200_OK,
    )
```

The view is short because the serializer does the hard part.

`PasswordResetConfirmSerializer.validate()` checks the two new passwords match, decodes the `uid`, loads the user with `User.objects.get(pk=uid, is_active=True)`, checks the token with `default_token_generator.check_token()`, runs Django's password rules, and then puts the user object into `attrs["user"]`. By the time the view runs, `validated_data["user"]` is a verified account. A bad or expired link never reaches this code; it comes back as a `400` saying "Invalid reset link." or "Invalid or expired reset token."

`blacklist_user_tokens(user)` appears again, for the same reason and with the same limitation.

Every endpoint in one table

Base URL: `http://127.0.0.1:8001/api`. This table is read off `backend/backend/urls.py` and `backend/backend/permissions.py`.

PATH	METHODS	WHO MAY CALL IT
/login/	POST	Anyone. No token needed.
/register/	POST	Admin only. 401 anonymous, 403 teacher or student.
/profile/	GET , PATCH	Any signed-in account, always its own.
/change-password/	POST	Any signed-in account, and it must send <code>current_password</code> .
/password-reset/	POST	Anyone. No token needed.
/password-reset-confirm/	POST	Anyone. No token needed, the token in the body is the proof.
/teacher/	GET	Any signed-in account
/teacher/	POST	Admin only
/teacher/<int:pk>/	GET	Any signed-in account
/teacher/<int:pk>/	PUT , PATCH , DELETE	Admin only
/student/ and /student/<int:pk>/	as <code>teacher</code> above	as <code>teacher</code> above (<code>AdminWrites</code>)
/course/	GET	Any signed-in account
/course/	POST	Admin or teacher
/course/<int:pk>/	GET	Any signed-in account
/course/<int:pk>/	PUT , PATCH , DELETE	Admin or teacher
/enrollment/ , /lesson/ , /assignment/ , /results/ and their <int:pk>/ forms	as <code>course</code> above	as <code>course</code> above (<code>TeachingStaffWrites</code>)
/submission/	GET	Any signed-in account
/submission/	POST	Admin, teacher or student
/submission/<int:pk>/	GET	Any signed-in account
/submission/<int:pk>/	PUT , PATCH , DELETE	Admin or teacher only

Reading anything needs only a valid token. The role decides what you may change. A refusal always comes back as a `403` with a readable sentence from the permission class, and the React app prints that sentence straight into its red banner.

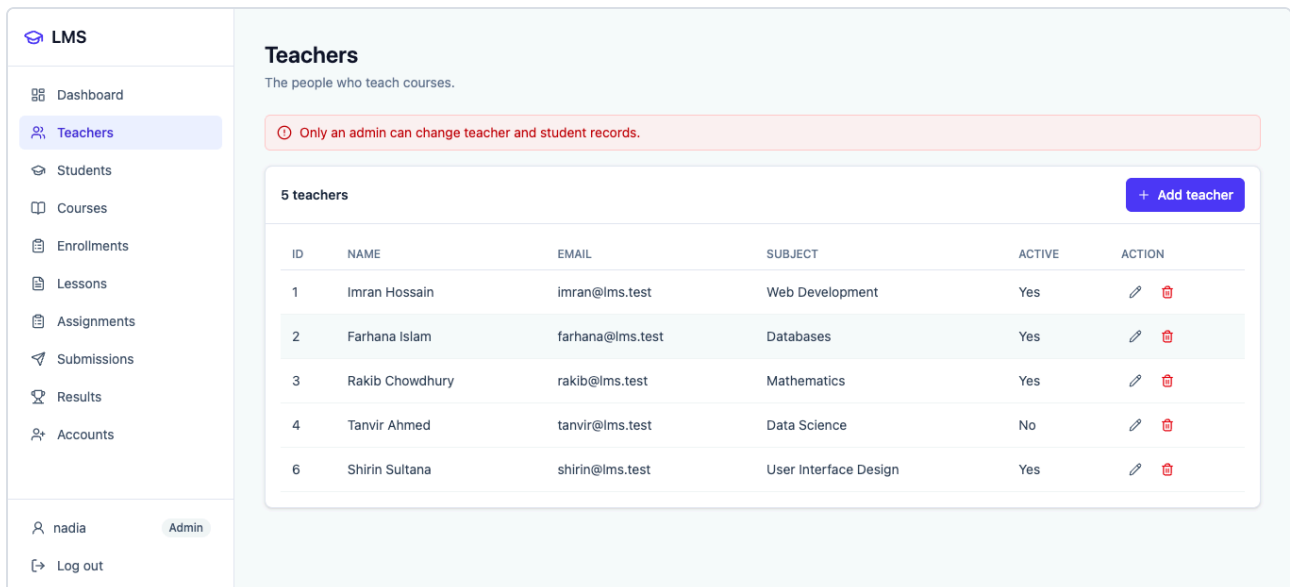


Figure 5.1 A student who forces a write through anyway gets the permission class's own message back from the API, shown here in the red banner.

Trying it without the frontend

You do not need React to use this API. `curl` is a command line program that sends an HTTP request and prints the answer, and it ships with macOS and most Linux systems. Every command below is real and runnable.

Start the server first, from the `backend` folder:

```
python manage.py runserver 8001
```

That serves the API on port 8001. Leave that terminal alone and open a second one.

Step 1: ask for teachers with no token.

```
curl -i http://127.0.0.1:8001/api/teacher/
```

`-i` prints the status line and headers as well as the body. You get something like this (a few headers trimmed):

```
HTTP/1.1 401 Unauthorized
WWW-Authenticate: Bearer realm="api"
Content-Type: application/json

{"detail":"Authentication credentials were not provided."}
```

Step 2: log in. Use the phone number of the admin account you created in the setup chapter.

`01711111111` below is a stand-in; type your own.

```
curl -X POST http://127.0.0.1:8001/api/login/ \
  -H "Content-Type: application/json" \
  -d '{"phone": "0171111111", "password": "your-password"}'
```

`-X POST` sets the method. `-H` adds a **header**, which is an extra line of information travelling alongside the request, and `Content-Type: application/json` tells Django the body is JSON rather than form data. `-d` is the body itself. The single quotes around the JSON keep your shell from mangling the double quotes inside.

TIP

Pipe any of these through `python3 -m json.tool` to get indented, readable JSON instead of one long line: `curl -s http://127.0.0.1:8001/api/teacher/ | python3 -m json.tool`. The `-s` turns off curl's progress meter, which would otherwise mix into the output.

Step 3: capture the access token in a shell variable. A shell variable is a name your terminal remembers for the rest of the session, so you do not have to paste a 300-character string by hand:

```
TOKEN=$(curl -s -X POST http://127.0.0.1:8001/api/login/ \
  -H "Content-Type: application/json" \
  -d '{"phone": "0171111111", "password": "your-password"}' \
  | python3 -c "import sys, json; print(json.load(sys.stdin)['tokens']['access'])")
```

Read the Python at the end: `['tokens']['access']`, two steps down, because of the nesting discussed earlier. `['access']` alone raises a `KeyError`, which is the terminal's version of the `undefined` that catches people in JavaScript.

Check it arrived:

```
echo $TOKEN
```

You should see a long string in three parts separated by dots, beginning `eyJ`.

Step 4: ask for teachers again, this time with the token.

```
curl -s http://127.0.0.1:8001/api/teacher/ \
  -H "Authorization: Bearer $TOKEN" | python3 -m json.tool
```

The header format is exactly `Authorization: Bearer <token>`, one space between the word `Bearer` and the token. `Bearer` comes from `AUTH_HEADER_TYPES` in `settings.py`.

CHECKPOINT This should print a JSON array, `[` on the first line, one object per teacher, `]` at the end. An empty database gives `[]`, which still counts as success. Note there is no `count` or `results` wrapper. Pagination, which is the practice of returning long lists a page at a time, is not configured on this project, so list endpoints return the whole array in one go, and the React code does `data.map(...)` rather than `data.results.map(...)`.

Step 5: create a teacher.

```
curl -s -X POST http://127.0.0.1:8001/api/teacher/ \
  -H "Authorization: Bearer $TOKEN" \
  -H "Content-Type: application/json" \
  -d '{"name": "Ada Lovelace", "email": "ada@example.com", "subject": "Mathematics",
"is_active": true}'
```

The answer is `201` and the created row, including the `id` the database assigned. Those four field names come from `TeacherSerializer`, which lists `['id', 'name', 'email', 'subject', 'is_active']`. `id` is read-only, so sending one is pointless.

Step 6: read one, change it, delete it. Use the `id` you just got back, say `1`:

```
curl -s http://127.0.0.1:8001/api/teacher/1/ -H "Authorization: Bearer $TOKEN"

curl -s -X PATCH http://127.0.0.1:8001/api/teacher/1/ \
  -H "Authorization: Bearer $TOKEN" \
  -H "Content-Type: application/json" \
  -d '{"subject": "Computing"}'

curl -i -X DELETE http://127.0.0.1:8001/api/teacher/1/ -H "Authorization: Bearer $TOKEN"
```

The `PATCH` sends one field and leaves the rest alone. The `DELETE` answers `HTTP/1.1 204 No Content` with an empty body, which is why `frontend/src/api.js` has a special case returning `null` for status 204.

NOTE

`PUT` means "replace the whole thing", so DRF insists on every field the serializer marks as required. On `Teacher` only `subject` is required (in `models.py`, `name` and `email` are `blank=True`, `null=True`, and `is_active` has a default), so a `PUT` with just `{"subject": "Computing"}` happens to be accepted here too. Try the same trick on a student and it fails: `Student.enrollment_date` is required, so `PUT /api/student/1/` without it comes back `400` with `{"enrollment_date": ["This field is required."]}`. The frontend sidesteps the whole question by sending every field on a `PUT`.

CHECKPOINT Log in as a **student** account instead, capture that token the same way, and repeat step 5. You should get `403` and the body `{"detail": "Only an admin can change teacher and student records."}`. That sentence is `AdminWrites.message` in `backend/backend/permissions.py`, arriving in your terminal untouched. Now try the same student token on step 4: the list still works. That is the read-open, write-restricted rule proving itself.

The browsable API in a browser

DRF gives every endpoint a second face: as well as JSON, it can render a web page. When a request arrives from a browser, DRF notices that the browser prefers HTML and renders the response as a styled page instead of raw JSON, with the endpoint's name at the top, the response body in the middle, and a form at the bottom for sending a `POST`.

Visit this in any browser:

```
http://127.0.0.1:8001/api/teacher/
```

See Figure 2.1.

NOTE

`REST_FRAMEWORK` in `settings.py` lists only `JWTAuthentication` under `DEFAULT_AUTHENTICATION_CLASSES`, and does not list `SessionAuthentication`. That means a plain browser visit carries no credentials this API recognises, even if you are logged into Django admin in another tab. You will see the browsable page rendering `{"detail": "Authentication credentials were not provided."}` with a `401`. To see real rows in the browser you need the `Authorization: Bearer ...` header attached, which a header-setting browser extension can do. From the terminal, `curl` remains the simpler route.

Add `?format=json` to any endpoint to skip the HTML page and get the raw body, which is exactly the bytes the React app receives:

```
http://127.0.0.1:8001/api/teacher/?format=json
```

See Figure 4.1.

TIP

The quickest way to fill the database with something to look at is not the API at all. Django admin at `http://127.0.0.1:8001/admin/` has every model registered in `backend/backend/admin.py`, so you can add a few teachers and courses by clicking, then come back and `curl` the list to see them.

What you have now

- A working API. Twenty-two URL patterns under `/api/`, all reachable, all answering real requests.
- `backend/lms/urls.py` sending everything starting with `api/` into `backend/backend/urls.py`, which is why every path in the book begins that way.
- Sixteen generic views covering eight resources, each one carrying three settings: a `queryset`, a `serializer_class` and a `permission_classes`, with the base class supplying the rest.
- `<int:pk>` detail routes, and the knowledge that the keyword must be spelled `pk`.
- Five hand-written `APIView` classes plus one generic `CreateAPIView`: login by phone, admin-only registration, read and patch your own profile, change your password, and the two halves of password reset.
- Login returning its tokens nested under `tokens`, and the memory of why `data.access` will waste your afternoon.
- Two deliberate silences: the same message for a bad phone and a bad password, and the same reply whether or not an email is registered.
- Real `curl` commands that log in, capture an access token, list teachers, create one, patch it and delete it, without a line of React in sight.

Who Can Do What: Roles and permissions.py

Up to now every signed-in account could do everything. Log in, and you could add a teacher, delete a course, change a grade. That is fine while you are the only person using the app. It stops being fine the moment a student gets a login.

In this chapter you build the part of the backend that answers one question, over and over, for every single request that arrives:

Is this person allowed to do this?

You will write one new file, `backend/backend/permissions.py`, add one new field to the `Profile` model, write two migrations, attach the new rules to seventeen views (the register view, plus a list view and a detail view for each of the eight resources), and then mirror the same table in the React app so the buttons match. At the end you will log in as a student and watch the server say no.

6.1 What a permission class actually is

A **request** is one message from the browser to the server. "GET /api/teacher/" means *give me the list of teachers*. "DELETE /api/teacher/3/" means *remove teacher number 3*. The address part, like `/api/teacher/`, is called an **endpoint**: one URL the server knows how to answer.

A **view** is the Python code that handles one of those messages. You already have views such as `TeacherListCreateView` in `backend/backend/views.py`.

A **permission class** sits in front of a view. It is a small Python object with one job. Before Django REST Framework (DRF) lets your view run, it walks through the view's permission classes and calls a method named `has_permission` on each one, handing it the incoming request. Each class returns `True` (let it through) or `False` (stop).

The order is always the same:

1. The browser sends a request with a **token** in the `Authorization` header. A token is the long string of letters the login endpoint handed back; it is proof of who you are, and it travels with every later request.
2. DRF works out who the token belongs to and puts that account on `request.user`.
3. DRF calls `has_permission(request, view)` on every permission class listed on the view.
4. If every one of them returns `True`, the view runs.
5. If any one returns `False`, the view never runs. DRF sends back **403 Forbidden** and never touches the database. (403 is a numeric status code the server puts on its reply. 403 means "I am

refusing this".)

That last point is the important one. A refused request does not partly happen. The delete does not run and then get undone. It never starts.

A permission class also carries a `message` attribute. That is the sentence DRF puts in the 403 response body. You get to write it, and it ends up in front of a real person, so it is worth writing in plain English rather than in error-code.

NOTE

DRF has a second hook called `has_object_permission`, which runs per row and answers "may this person touch *this particular* record?". This project does not use it, and there is a concrete reason why. Nothing in the database links a `Teacher` row or a `Student` row to a login account, so the server genuinely cannot tell whose row is whose. Rules like "you may edit your own submission" are impossible until that link exists. Everything in this chapter is `has_permission`: decisions made from the method and the role alone.

THE DEFAULT THAT IS ALREADY IN PLACE

Open `backend/lms/settings.py` and find the `REST_FRAMEWORK` block. It is already there from an earlier chapter:

`backend/lms/settings.py`

```
REST_FRAMEWORK = {
    "DEFAULT_AUTHENTICATION_CLASSES": (
        "rest_framework_simplejwt.authentication.JWTAuthentication",
    ),
    # Without this, DRF's default is AllowAny, so any view added without its
    # own permission_classes would be public. Failing closed means a new view
    # is locked until somebody decides otherwise; the two views that really
    # are public (register, login) say so with AllowAny.
    "DEFAULT_PERMISSION_CLASSES": (
        "rest_framework.permissions.IsAuthenticated",
    ),
}
```

`DEFAULT_PERMISSION_CLASSES` is the rule for any view that does not name its own. Here it is `IsAuthenticated`, which means "you need a valid token, and that is all I check". So today, a student with a valid token can delete a teacher. That is the hole you are closing.

Setting a default like this is called **failing closed**: a new view you add next month is locked until you decide otherwise, instead of being wide open until you remember to lock it.

NOTE

That comment in `settings.py` was written before this chapter. It says `register` is one of the public views, and by the end of today it will not be: `register` becomes `admins-only`. Comments go stale, code does not. When the two disagree, believe the code.

COMMON MISTAKE

`permission_classes` on a view **replaces** the default, it does not add to it. The moment you write `permission_classes = [AdminWrites]`, the `IsAuthenticated` default is gone from that view. That is why the classes you are about to write check `request.user.is_authenticated` themselves. Leave that check out and an anonymous stranger passes straight through your `SAFE_METHODS` shortcut.

6.2 Step one: give every account a role

A role is just a word stored next to the account: `admin`, `teacher`, or `student`. It lives on the `Profile` model, which is the small table this project already keeps alongside Django's built-in `User` to hold the phone number.

Open `backend/backend/models.py` and look at the top of `Profile`. Add the three constants, the choice list, and the field:

`backend/backend/models.py`

```
class Profile(models.Model):
    ADMIN = "admin"
    TEACHER = "teacher"
    STUDENT = "student"

    ROLE_CHOICES = [
        (ADMIN, "Admin"),
        (TEACHER, "Teacher"),
        (STUDENT, "Student"),
    ]
```

The three capitalised names are constants. `Profile.ADMIN` is just the string `"admin"`, but written as a constant so that a typo becomes an immediate `AttributeError` (Python's "there is no such name" crash) instead of a silent `False` that nobody notices for a month. Compare these two lines:

```
role_of(request.user) == "amdin"      # always False. Silent. Nobody sees it.
role_of(request.user) == Profile.AMDIN # crashes at once with AttributeError
```

`ROLE_CHOICES` is a list of pairs. The first item of each pair is what goes into the database column, the second is the label Django shows in a dropdown in the admin site.

Now the field itself, just under the existing `phone` field:

backend/backend/models.py

```
user = models.OneToOneField(User, on_delete=models.CASCADE)
phone = models.CharField(max_length=20, unique=True)
# Anyone can register, so a new account gets the least privileged role.
# Promote people from /admin/ once you know who they are.
role = models.CharField(max_length=10, choices=ROLE_CHOICES, default=STUDENT)
```

`CharField` means a short piece of text. `max_length=10` is the size of the database column, and `"student"` is seven characters, so ten is enough. `choices=ROLE_CHOICES` makes Django reject anything outside the three words and draw a dropdown in the admin. `default=STUDENT` decides what a brand new row gets if nobody says otherwise, and it is deliberately the weakest of the three.

That comment about "anyone can register" is another one written before registration became admins-only. The behaviour that matters is the `default=STUDENT` on the end of the line.

THE MIGRATION THAT ADDS THE COLUMN

A **migration** is a Python file describing a change to the database structure. Django writes it for you by comparing your models to the migrations already applied. Run this from the `backend` folder:

```
python manage.py makemigrations backend
```

Django writes `backend/backend/migrations/0003_profile_role.py`. This is the whole file, and it is short:

```
backend/backend/migrations/0003_profile_role.py
```

```
# Generated by Django 5.2.7 on 2026-08-03 04:59

from django.db import migrations, models

class Migration(migrations.Migration):

    dependencies = [
        ('backend', '0002_course_enrollment_lesson_assignment_submission_and_more'),
    ]

    operations = [
        migrations.AddField(
            model_name='profile',
            name='role',
            field=models.CharField(choices=[('admin', 'Admin'), ('teacher', 'Teacher'), ('student', 'Student')], default='student', max_length=10),
        ),
    ]
```

Read it once, because it is readable. `dependencies` says "apply `0002` first". `operations` says "add a field called `role` to the `profile` model". That is the entire file. You did not type it and you should not edit it.

THE MIGRATION THAT STOPS YOU LOCKING YOURSELF OUT

Now think about what `default='student'` does to a database that already has accounts in it. Every existing `Profile` row gets the column filled in with `student`. Including yours. Run `migrate` now and the next time you log in you will be a student, unable to add a teacher, unable to create accounts, unable to fix it from inside the app.

The fix is a second migration that changes *data* rather than *structure*. Ask Django for an empty one and name it, again from the `backend` folder:

```
python manage.py makemigrations backend --empty --name existing_accounts_become_admins
```

That creates `0004_existing_accounts_become_admins.py` with an empty `operations` list. Replace its contents with this. First the two functions:

```
backend/backend/migrations/0004_existing_accounts_become_admins.py
```

```
from django.db import migrations

def make_existing_accounts_admins(apps, schema_editor):
    """Everyone who already had an account keeps full access.

    New accounts default to `student`, but the people who registered before
    roles existed were able to do everything, so locking them out on upgrade
    would be a surprise. Promote them once, here.
    """
    Profile = apps.get_model("backend", "Profile")
    Profile.objects.update(role="admin")

def back_to_student(apps, schema_editor):
    Profile = apps.get_model("backend", "Profile")
    Profile.objects.update(role="student")
```

`apps.get_model("backend", "Profile")` fetches the version of the model as it existed at this point in migration history. You use it instead of importing `Profile` directly at the top of the file, because the model in `models.py` keeps changing and a migration has to keep working years later against the shape of the table at the time it ran.

`Profile.objects.update(role="admin")` sets the `role` column to `admin` on every row of the table in one go. It runs once, when this migration is applied, and never again.

The second function is the reverse. If you ever run `migrate backend 0003`, Django undoes `0004` by calling it.

Then the migration class itself:

```
backend/backend/migrations/0004_existing_accounts_become_admins.py
```

```
class Migration(migrations.Migration):
    dependencies = [
        ("backend", "0003_profile_role"),
    ]

    operations = [
        migrations.RunPython(make_existing_accounts_admins, back_to_student),
    ]
```

`RunPython` takes two functions: the one to run going forwards, and the one to run going backwards. Now apply both migrations:

```
python manage.py migrate
```

CHECKPOINT Run `python manage.py showmigrations backend`. You should see `[X] 0003_profile_role` and `[X] 0004_existing_accounts_become_admins`, both with an X meaning applied. Then open `http://127.0.0.1:8001/admin/backend/profile/` and confirm your own account now says **Admin** in the Role column.

NOTE This "promote everyone who already existed" migration is a one-time kindness for accounts created before roles were a thing. Accounts created after it get whatever role the admin picked, and `student` if nobody picked.

WHERE A ROLE GETS SET

There are exactly two places.

When an admin creates the account. `RegisterSerializer` in `backend/backend/serializers.py` accepts a `role`. (A **serializer** is the DRF object that checks incoming JSON and turns it into database rows, and turns rows back into JSON on the way out.)

```
backend/backend/serializers.py
```

```
# Only an admin can reach this endpoint, so letting the caller pick the
# role is safe. Left out, the account is a student.
role = serializers.ChoiceField(
    choices=Profile.ROLE_CHOICES,
    required=False,
    default=Profile.STUDENT,
)
```

The Accounts page in the React app is the form that fills this in. Its dropdown spells out what each role means:

frontend/src/pages/Accounts.jsx

```
<Select
  label="Role"
  required
  value={role}
  onChange={(event) => setRole(event.target.value)}
>
  <option value="student">Student ☐ read only, may hand work in</option>
  <option value="teacher">
    Teacher ☐ course material, enrollments and grading
  </option>
  <option value="admin">Admin ☐ everything, including accounts</option>
</Select>
```

Afterwards, in Django admin, `backend/backend/admin.py` makes `role` editable straight from the list page, so changing somebody's role is two clicks and a Save:

backend/backend/admin.py

```
@admin.register(Profile)
class ProfileAdmin(admin.ModelAdmin):
    # `role` is editable straight from the list, because promoting someone who
    # has just registered is the one admin job this app really needs.
    list_display = ['user', 'phone', 'role']
    list_editable = ['role']
    list_filter = ['role']
    search_fields = ['user__username', 'phone']
```

There is deliberately no third place. In particular, there is no way to change your own role through the API. `ProfileUpdateSerializer`, which powers the Your profile page, accepts only `first_name`, `last_name`, `email` and `phone`. Its docstring says why:

backend/backend/serializers.py

```
"""What you are allowed to change about yourself.

Not on this list, on purpose:
    role      - changing your own would be a promotion. Admins only, in
                Django admin.
    username  - it is how an admin finds you; the phone is what you log in
                with, so there is nothing to gain by renaming yourself.
    password  - has its own endpoint, because it needs the old one first.
"""
```

Because DRF serializers ignore fields they do not declare, posting `{"role": "admin"}` to `/api/profile/` is not rejected with an error. It is simply not read. Nothing happens.

6.3 Step two: create permissions.py

Make the file next to `models.py` and `views.py`. Run this from the top folder of the project:

```
touch backend/backend/permissions.py
```

Start with the module docstring. A **docstring** is a string at the top of a file or a function, written in triple quotes, that explains what the thing is for. Python keeps it in the file and tools can read it. Here it is the map of the whole chapter:

```
backend/backend/permissions.py
```

```
"""Who is allowed to change what.  
  
Reading is open to any signed-in account. Writing depends on the role stored  
on the caller's Profile:  
  
    admin    everything  
    teacher  course content (course, lesson, assignment), enrollments, grading  
    student  may hand in a submission, and nothing else  
  
The API is the thing that enforces this. The frontend hides buttons to match,  
but that is a courtesy, not a control.  
"""
```

That last sentence is the one to remember. Everything else in this chapter is detail.

Then two imports:

```
backend/backend/permissions.py
```

```
from rest_framework.permissions import SAFE_METHODS, BasePermission  
  
from .models import Profile
```

`BasePermission` is the DRF class you build on. It provides a `has_permission` that returns `True`, so subclasses only override what they care about. (A **subclass** is a class that starts from another one and changes a piece of it.) `SAFE_METHODS` is covered in a moment. `Profile` comes in so you can use `Profile.ADMIN` and friends.

6.4 role_of(user), line by line

Every class in this file needs the same answer: what role is the person making this request? That is one function, and it is worth reading slowly because everything else leans on it.

backend/backend/permissions.py

```
def role_of(user):  
    """The caller's role, or None when nobody is signed in.  
  
    Two accounts have no Profile row: a superuser made with `createsuperuser`,  
    and anything created outside the register endpoint. A superuser is treated  
    as an admin; anyone else without a Profile gets the least privilege we  
    have rather than an exception.  
    """
```

Now the body, four decisions in order.

One: is anybody there at all?

```
# backend/backend/permissions.py (inside role_of)  
if not user or not user.is_authenticated:  
    return None
```

`request.user` is not always a `User`. When no token was sent, DRF puts an `AnonymousUser` object there instead. `AnonymousUser` has an `is_authenticated` attribute that is always `False`, while a real `User` always has `True`. The `not user` half of the check guards against `None`, which is what you get if you call `role_of` yourself outside a request.

Returning `None` rather than `"student"` matters. `None` is not one of the three roles, so a later `role` in `(Profile.ADMIN, Profile.TEACHER)` test can never accidentally match a stranger.

Two: is this a superuser?

```
# backend/backend/permissions.py (inside role_of)  
if user.is_superuser:  
    return Profile.ADMIN
```

`is_superuser` is a true-or-false column Django puts on every `User`. It is set by `python manage.py createsuperuser`, the command that makes the account you log into Django admin with.

Here is the awkward truth this line handles. `createsuperuser` asks for a username, an email and a password. It does **not** create a `Profile`, because `Profile` is your model, not Django's. So a superuser has no phone number and no role. Without this line, `role_of` would fall through to the lookup below, find nothing, and treat the most privileged account on the system as a student.

Three: look the Profile up.

```
# backend/backend/permissions.py (inside role_of)  
profile = Profile.objects.filter(user=user).only("role").first()
```


Three pieces, left to right. `.filter(user=user)` describes the query: profiles belonging to this account. `.only("role")` tells Django to fetch just the `role` column instead of every column, because that is all this function reads. `.first()` runs the query and gives back the first row, or `None` if there were none. `.first()` is used rather than `.get()` because `.get()` raises `Profile.DoesNotExist` when there is no row, and a missing profile is a case you want to handle, not an exception you want thrown into the middle of a permission check.

TIP

`.only("role")` is a small thing that runs on every single API request your app ever serves. It is the difference between `SELECT role FROM backend_profile ...` and `SELECT id, user_id, phone, role FROM backend_profile ...` in the database. Not dramatic, but free.

Four: no profile, or a profile.

```
# backend/backend/permissions.py (inside role_of)
if profile is None:
    return Profile.STUDENT

return profile.role
```

WHY THE FALLBACK IS `STUDENT` AND NOT `ADMIN`

This is the design decision worth arguing about, so here is the argument.

An account with no `Profile` row is an account you do not fully understand. It was made by a script, by a test, by a half-finished feature, by somebody poking at the Django shell. You do not know who it is or what it is for.

There are two ways to handle "I do not know":

- **Guess generously.** Give it admin. Now every gap in your data, every half-written script, every forgotten test account is a full administrator of your school.
- **Guess meanly.** Give it the weakest role you have. Now a gap in your data means somebody cannot delete a course and has to come and ask you why.

The second failure is annoying. The first is a breach. When you cannot tell, hand out the **least privilege** and let the human complain. A permission bug that blocks a legitimate user gets reported within the hour. A permission bug that lets the wrong person in gets reported never.

The third option, raising an exception, is worse than both. A crash in a permission check is a 500 error (the status code meaning "the server itself broke"), and a 500 is exactly the sort of thing a developer "fixes" at 2am by wrapping it in a `try/except` that returns `True`.

COMMON MISTAKE

Running `createsuperuser` and then trying to log into the React app with it. It will not work, and the error will say `Invalid phone or password`. Login looks accounts up by `Profile.phone`, and `createsuperuser` makes no `Profile`, so there is no phone number to type. That account can reach `http://127.0.0.1:8001/admin/` and nothing else. The fix is in the README: create the superuser, then open `http://127.0.0.1:8001/admin/backend/profile/add/`, pick that user, give it a phone number, set the role to **Admin**, and save. `role_of()` already treats a superuser as an admin. The missing piece is only the phone number to log in with.

6.5 SAFE_METHODS: reading is open

Every HTTP request carries a **method**, a verb saying what kind of thing it is:

METHOD	WHAT IT MEANS	CHANGES DATA?
GET	give me this	no
HEAD	give me just the headers for this	no
OPTIONS	tell me what I can do here	no
POST	create a new one of these	yes
PUT	replace this one entirely	yes
PATCH	change these fields of this one	yes
DELETE	remove this one	yes

DRF bundles the first three into a constant:

```
SAFE_METHODS = ('GET', 'HEAD', 'OPTIONS')
```

They are called safe because they only read. This project's rule is that all three are open to any signed-in account, whatever their role. A student may look at the teacher list, the course list, the results list. They just cannot change any of it.

TIP

Write `if request.method in SAFE_METHODS` and not `if request.method == "GET"`. `OPTIONS` is what the DRF browsable API sends, and it is also what a browser sends on its own before a cross-site call, to ask the server whether the real call is allowed (that advance question is called a preflight). Check only for `GET` and the browsable API at `/api/teacher/` stops showing a student the form area, and some cross-site requests start failing for reasons that take an afternoon to track down.

6.6 IsAdmin: the strictest class

Registration is different from everything else. There is no public sign-up in this app. A student does not make their own account, an admin makes it for them and picks the role. That means even *reading* the register endpoint is admins-only, so this class does not care about `SAFE_METHODS` at all.

backend/backend/permissions.py

```
class IsAdmin(BasePermission):
    """Admins only, for every method including GET.

    Used on registration: accounts are handed out by an admin, not claimed by
    whoever finds the URL.
    """

    message = "Only an admin can create accounts."

    def has_permission(self, request, view):
        return role_of(request.user) == Profile.ADMIN
```

The whole class is one line of logic. No authentication check is needed here, because `role_of` returns `None` for an anonymous caller and `None` is not equal to `Profile.ADMIN`.

`message` is the sentence a refused caller reads. Notice it says what the rule is, not that something went wrong. "Only an admin can create accounts." tells a teacher exactly what happened and who to go and ask.

It goes on the register view like this:

backend/backend/views.py

```
class RegisterView(generics.CreateAPIView):
    """Create an account. Admins only.

    There is no public sign-up: a student does not make their own account, an
    admin makes it for them and picks the role. Anonymous callers get a 401,
    signed-in teachers and students a 403.

    If there is no admin left to do this, `python manage.py createsuperuser`
    still works, and a superuser counts as an admin.
    """

    queryset = User.objects.all()
    serializer_class = RegisterSerializer
    permission_classes = [IsAdmin]
```

NOTE

401 and 403 are different refusals. **401 Unauthorized** means "I do not know who you are", and it is what an anonymous caller gets, because the JWT authentication class asks the browser to identify itself. **403 Forbidden** means "I know exactly who you are, and the answer is still no". Everything else in this chapter produces 403s.

6.7 RoleWritePermission: the shared base

Three of the four remaining classes share the same shape: reading open, writing limited to a list of roles. Rather than write that three times, write it once and let the three fill in the list.

backend/backend/permissions.py

```
class RoleWritePermission(BasePermission):
    """Signed-in accounts can read. Only `roles_that_may_write` can write."""

    roles_that_may_write = ()
    message = "Your role does not allow this change."

    def has_permission(self, request, view):
        if not request.user or not request.user.is_authenticated:
            return False

        if request.method in SAFE_METHODS:
            return True

        return role_of(request.user) in self.roles_that_may_write
```

Three lines of logic, in order.

Line one turns away anyone without a valid token, whatever the method. Remember that `permission_classes` on a view replaces the `IsAuthenticated` default, so this class has to do that job itself.

Line two is the open door for reading. Any signed-in account, any role, gets `True` for `GET`, `HEAD` and `OPTIONS`.

Line three is reached only for `POST`, `PUT`, `PATCH` and `DELETE`. It asks `role_of` for the caller's role and checks whether it appears in `roles_that_may_write`.

`roles_that_may_write = ()` is an empty tuple. (A **tuple** is a fixed list, written with round brackets, that cannot be added to afterwards.) On this base class an empty one means "nobody may write". That is another deliberate fail-closed default: if you subclass this and forget to set the list, your new resource is read-only rather than open. Better a bug report than a breach.

`self.roles_that_may_write` reads the attribute off the actual class handling the request, so a subclass that sets its own value gets its own value. This is Python inheritance doing the work, and it is why the next two classes are three lines each.

6.8 AdminWrites and TeachingStaffWrites

`Teacher` and `Student` rows are the school's register of who exists. Adding a person, correcting their email, or removing them is an admin job.

backend/backend/permissions.py

```
class AdminWrites(RoleWritePermission):
    """Teacher and Student records: the register of who is in the school."""

    roles_that_may_write = (Profile.ADMIN,)
    message = "Only an admin can change teacher and student records."
```

No `has_permission` here. It inherits the one you just wrote. The class only supplies two values: which roles may write, and what to say when someone may not.

`(Profile.ADMIN,)` needs that trailing comma. In Python, `(x)` is just `x` with brackets around it, while `(x,)` is a tuple of one item. Leave the comma out and `roles_that_may_write` becomes the plain string `"admin"`, and then `role in "admin"` starts asking "do these letters appear inside that word?" instead of "is this one of the allowed roles?". It happens to give the right answer here and the wrong answer somewhere else later. Keep the comma.

Course material and grading are a teacher's daily work, so teachers get those, and admins get them too:

```
backend/backend/permissions.py
```

```
class TeachingStaffWrites(RoleWritePermission):
    """Courses, lessons, assignments, enrollments and results."""

    roles_that_may_write = (Profile.ADMIN, Profile.TEACHER)
    message = "Only a teacher or an admin can change course material and grades."
```

This one class covers five resources: course, lesson, assignment, enrollment and results.

6.9 SubmissionWrites and the one exception

Submissions are the only place a student writes anything. A student needs to hand work in. That is a `POST`. But a student must not be able to edit or delete a submission, and the reason is not squeamishness, it is that the server literally cannot do the check.

Here is the class. Start with the docstring, because the docstring is the argument:

```
backend/backend/permissions.py
```

```
class SubmissionWrites(RoleWritePermission):
    """Same as above, except a student may hand work in.

    A student cannot edit or delete a submission, because nothing links a
    Student row to a login account – so the server has no way to tell whose
    work it is. Handing in is the one write that is safe without that link.
    """

    roles_that_may_write = (Profile.ADMIN, Profile.TEACHER)
    message = "Students can hand work in, but cannot change or remove a submission."
```

So far it is identical to `TeachingStaffWrites` with a different message. The difference is the override:

```
backend/backend/permissions.py
```

```
def has_permission(self, request, view):
    if (
        request.method == "POST"
        and request.user
        and request.user.is_authenticated
        and role_of(request.user) == Profile.STUDENT
    ):
        return True

    return super().has_permission(request, view)
```

Read it as one sentence: *if this is a POST, and somebody is signed in, and that somebody is a student, allow it. Otherwise fall back to the normal rule.*

`super().has_permission(request, view)` calls the parent class's method, the one on `RoleWritePermission`. So every case this override does not explicitly allow is still decided by the shared rule, which is exactly what you want. Add a new method to HTTP tomorrow and it lands in the safe branch by default.

Trace it through for a student:

METHOD	THE IF MATCHES?	RESULT
GET /api/submission/	no, not a POST	falls through, <code>SAFE_METHODS</code> allows it
POST /api/submission/	yes	<code>True</code> , the work is handed in
PUT /api/submission/4/	no, not a POST	falls through, <code>student</code> is not in the write list, refused
PATCH /api/submission/4/	no, not a POST	falls through, refused
DELETE /api/submission/4/	no, not a POST	falls through, refused

Now the reason, properly. Look at the model:

backend/backend/models.py

```
class Submission(models.Model):
    assignment = models.ForeignKey(Assignment, on_delete=models.CASCADE)
    student = models.ForeignKey(Student, on_delete=models.CASCADE)
    submitted_at = models.DateTimeField(auto_now_add=True)
    content = models.TextField()
```

`Submission.student` points at a `Student` row. That pointer is called a **foreign key**: a column holding the id number of a row in another table. A `Student` row is a name, an email, a roll number and an enrollment date. It has no link at all to a `User`, which is what a login account is. Two different tables, no foreign key between them.

So when a request arrives with a token, the server can say "this is the account belonging to username `rina`". It cannot say "this is `Student` row 12". The chain from login to student record does not exist.

That makes "a student may edit their own submission" unimplementable. The server has no way to tell "their own" from "anyone's". Given that, the choice is between letting a student edit *every* submission in the school or letting them edit *none*. None is obviously right.

Creating is different. Creating adds a new row and takes nothing away. The worst a student can do is hand in work under another student's name, which is bad but is a school problem rather than a security hole, and it is written down as a known gap in the README:

`Teacher` and `Student` rows are still not linked to login accounts. So the server cannot tell whose work a submission is, which means:

- a student can hand in work under any student's name, and
- "only show me my own courses / my own grades" is not possible.

The fix is a foreign key from `Teacher` and `Student` to `User`, plus `has_object_permission` on the views. That is a bigger change than adding a role field, and it is out of scope here. What matters is that the limitation is written down in the docstring rather than pretended away.

NOTE

You have now typed the whole of `backend/backend/permissions.py`, just over 100 lines. Top to bottom it is: the module docstring, two imports, `role_of`, then `IsAdmin`, `RoleWritePermission`, `AdminWrites`, `TeachingStaffWrites`, `SubmissionWrites`. If you open the real file and it is in that order with nothing extra, you are done with it.

6.10 Attaching the classes to the views

The classes exist. Nothing uses them yet. Open `backend/backend/views.py` and add the import:

```
backend/backend/views.py
```

```
from .permissions import AdminWrites, IsAdmin, SubmissionWrites, TeachingStaffWrites,
role_of
```

`role_of` comes along too, because two views call it directly: the login view, to tell the frontend what role it just let in, and the profile view, to put the role in the reply it sends about your own account.

Then every one of the sixteen resource views gets a `permission_classes` line. Here is the pattern, using the two teacher views:

```
backend/backend/views.py
```

```
class TeacherListCreateView(generics.ListCreateAPIView):
    queryset = Teacher.objects.all()
    serializer_class = TeacherSerializer
    permission_classes = [AdminWrites]

class TeacherRetrieveUpdateDestroyAPIView(generics.RetrieveUpdateDestroyAPIView):
    queryset = Teacher.objects.all()
    serializer_class = TeacherSerializer
    permission_classes = [AdminWrites]
```


COMMON MISTAKE

Putting the permission class on the list view and forgetting the detail view. They are two separate classes handling two separate URLs. `/api/teacher/` is the list, `/api/teacher/3/` is the detail. Leave `permission_classes` off the detail view and it silently falls back to the `IsAuthenticated` default from settings, which means any signed-in student can `DELETE /api/teacher/3/`. Both views, every time. Count the `permission_classes` lines in `views.py` from `TeacherListCreateView` to the end of the file: there should be sixteen.

Here is the full wiring, so you can check yours against it:

VIEW CLASS	PERMISSION CLASS
<code>RegisterView</code>	<code>IsAdmin</code>
<code>TeacherListCreateView</code> , <code>TeacherRetrieveUpdateDestroyAPIView</code>	<code>AdminWrites</code>
<code>StudentListCreateView</code> , <code>StudentRetrieveUpdateDestroyAPIView</code>	<code>AdminWrites</code>
<code>CourseListCreateView</code> , <code>CourseRetrieveUpdateDestroyAPIView</code>	<code>TeachingStaffWrites</code>
<code>EnrollmentListCreateView</code> , <code>EnrollmentRetrieveUpdateDestroyAPIView</code>	<code>TeachingStaffWrites</code>
<code>LessonListCreateView</code> , <code>LessonRetrieveUpdateDestroyAPIView</code>	<code>TeachingStaffWrites</code>
<code>AssignmentListCreateView</code> , <code>AssignmentRetrieveUpdateDestroyAPIView</code>	<code>TeachingStaffWrites</code>
<code>SubmissionListCreateView</code> , <code>SubmissionRetrieveUpdateDestroyAPIView</code>	<code>SubmissionWrites</code>
<code>ResultsListCreateView</code> , <code>ResultsRetrieveUpdateDestroyAPIView</code>	<code>TeachingStaffWrites</code>
<code>ProtectedView</code> , <code>ChangePasswordView</code>	<code>IsAuthenticated</code> (your own account, any role)
<code>LoginView</code> , <code>PasswordResetRequestView</code> , <code>PasswordResetConfirmView</code>	<code>AllowAny</code> (public on purpose)

TELLING THE FRONTEND WHICH ROLE LOGGED IN

One more line in `views.py`, inside `LoginView.post`. The response the login endpoint sends back gains a `role`:

backend/backend/views.py

```
return Response({
    'message': 'Login successful',
    'user_id': user.id,
    'username': user.username,
    # The frontend uses this to decide which buttons to show. It is
    # not what enforces anything: the permission classes do that.
    'role': role_of(user),
    'tokens': tokens
})
```

Note the comment. That value is decoration for the browser. Nothing on the server reads it back. If someone edited it in transit, all they would change is which buttons their own copy of the page draws.

6.11 The permission table, built from the code

Now put the whole thing on one page. Read across the row for a role, down the column for a resource.

	ACCOUNTS	TEACHER & STUDENT RECORDS	COURSE, LESSON, ASSIGNMENT	ENROLLMENT	SUBMISSION	RESULTS
admin	create	full	full	full	full	full
teacher	no access	read	full	full	full	full
student	no access	read	read	read	read + may hand in	read

"full" means read, create, update and delete. "read" means `GET`, `HEAD` and `OPTIONS` only. "no access" means not even read: `/api/register/` is closed to non-admins entirely, since it is guarded by `IsAdmin` rather than by a `RoleWritePermission`.

Check each cell against the code you wrote:

- accounts** comes from `IsAdmin`. `role_of(request.user) == Profile.ADMIN`, no `SAFE_METHODS` shortcut, so admin only, all methods.
- teacher & student records** come from `AdminWrites`, whose `roles_that_may_write` is `(Profile.ADMIN,)`. Reading is open because it inherits the `SAFE_METHODS` branch.
- course, lesson, assignment, enrollment, results** all come from `TeachingStaffWrites`, `(Profile.ADMIN, Profile.TEACHER)`.
- submission** comes from `SubmissionWrites`. Same two roles for writes, plus the `POST` override for students.

`/Users/tausif/Projects/Ostad/Batch 08/lms/README.md` prints the same rules under its Roles heading (it writes a dash where this table writes "no access"), and it adds one sentence worth memorising:

Reading any list needs only a valid token. Writing is checked by the permission classes in `backend/permissions.py`; a refusal comes back as a 403 with a sentence saying why, and the frontend shows that sentence in the red banner.

If you ever change a `roles_that_may_write` tuple, change the README table in the same commit. A permission table that has drifted out of date is worse than no table, because people trust it.

6.12 What a refusal looks like on the wire

When `has_permission` returns `False`, DRF raises `PermissionDenied` and turns it into a response. The status line is `403 Forbidden` and the body is one small JSON object:

```
{
  "detail": "Only an admin can change teacher and student records."
}
```

That sentence is not DRF's. It is the `message` attribute you wrote on `AdminWrites`. DRF takes the `message` from whichever class refused and puts it in `detail`. Which is why writing the message as a full sentence, aimed at a human, is worth the thirty seconds.

The React app then turns that JSON into something on screen. `readError()` in `frontend/src/api.js` deals with the fact that this backend reports errors in three different shapes:

`frontend/src/api.js`

```
// Django REST Framework reports errors in a few different shapes:
// {"detail": "..."}           a permission or 404 error
// {"error": "..."}           the hand-written login view
// {"phone": ["This field is required"]} a serializer rejecting a field
// This turns all of them into one readable line.
function readError(body, response) {
  if (!body || typeof body !== "object") {
    return `Request failed (${response.status})`;
  }

  if (typeof body.detail === "string") {
    return body.detail;
  }
}
```

The first branch that matters here is `body.detail`. It is returned as-is, thrown as a JavaScript `Error`, caught by the page, put into the page's `error` state, and rendered by `<Alert>`, whose default look is red.

Follow it through `Teachers.jsx`. The delete handler catches the error and stores its message:

frontend/src/pages/Teachers.jsx

```
    } catch (problem) {
      setError(problem.message);
      setTeacherToDelete(null);
    } finally {
      setIsDeleting(false);
    }
  }
```

And the top of the page renders it:

frontend/src/pages/Teachers.jsx

```
<Alert>{error}</Alert>
<Alert variant="success">{notice}</Alert>
```

So the sentence you typed in Python, in a file the browser has never seen, ends up in a red box above a table in React. There is no translation layer and no error code lookup in between.

See Figure 5.1.

6.13 The frontend copy: permissions.js

There is a second copy of this table, in JavaScript, and it is important to understand what it is and is not for. Open `frontend/src/permissions.js`. Its first three lines say it plainly:

frontend/src/permissions.js

```
// A copy of the rules in backend/permissions.py, used only to decide which
// buttons are worth showing. The server is what actually enforces them, so a
// mistake here is a cosmetic bug, not a hole. Keep the two in step.
```

Read that once more. **A mistake here is a cosmetic bug, not a hole.** If this file says a student may delete a teacher, the student sees a delete button, clicks it, and gets a 403 with a red banner. Annoying. Not dangerous. The reason to keep the file correct is so the page does not offer things that are going to fail.

The three role names first, as exported constants so the rest of the app imports them instead of retyping strings:

frontend/src/permissions.js

```
export const ADMIN = "admin";
export const TEACHER = "teacher";
export const STUDENT = "student";
```

Then the table itself, as a plain object mapping a resource name to the list of roles that may write it:

frontend/src/permissions.js

```
const WRITERS = {
  teacher: [ADMIN],
  student: [ADMIN],
  course: [ADMIN, TEACHER],
  enrollment: [ADMIN, TEACHER],
  lesson: [ADMIN, TEACHER],
  assignment: [ADMIN, TEACHER],
  submission: [ADMIN, TEACHER],
  results: [ADMIN, TEACHER],
};
```

Put that side by side with the Python. `teacher: [ADMIN]` is `AdminWrites.roles_that_may_write = (Profile.ADMIN,)`. `course: [ADMIN, TEACHER]` is `TeachingStaffWrites.roles_that_may_write = (Profile.ADMIN, Profile.TEACHER)`. Eight lines here, three tuples there, same information.

Then the two questions a page can ask. First, may this role change or remove existing rows?

frontend/src/permissions.js

```
// May this role change or remove rows of this kind?
export function canWrite(role, resource) {
  return (WRITERS[resource] ?? []).includes(role);
}
```

`WRITERS[resource]` looks the resource up. `?? []` is the nullish coalescing operator: if the left side is `null` or `undefined`, use the right side instead. So a typo like `canWrite(role, "teachers")` with an `s` finds nothing, falls back to an empty list, and returns `false`. Fails closed, same as the Python. `.includes(role)` then checks membership, and `undefined` is never in any list, so an account whose role has not loaded yet gets no buttons rather than all of them.

Second, may this role add a new row? Almost the same, with the submission exception carved out:

frontend/src/permissions.js

```
// May this role add a new row? Same as canWrite everywhere except submissions,
// which a student is allowed to hand in but not to edit afterwards.
export function canCreate(role, resource) {
  if (resource === "submission" && role === STUDENT) {
    return true;
  }
  return canWrite(role, resource);
}
```

That `if` is the mirror of the `has_permission` override in `SubmissionWrites`. Same exception, same reason, expressed twice in two languages. That is the cost of having a copy, and it is why the comment at the top of the file ends with "Keep the two in step."

HOW A PAGE USES THEM

Every resource page asks the same two questions near the top. From `Teachers.jsx`:

frontend/src/pages/Teachers.jsx

```
// The server enforces this too. Hiding the buttons just keeps the page
// honest about what will actually work.
const { user } = useAuth();
const mayCreate = canCreate(user?.role, "teacher");
const mayEdit = canWrite(user?.role, "teacher");
```

`user?.role` uses optional chaining. If `user` is `null`, the whole expression is `undefined` rather than a crash, and `canWrite(undefined, "teacher")` is `false`.

`mayCreate` guards the Add button in the table header:

frontend/src/pages/Teachers.jsx

```
{mayCreate && (
  <Button onClick={openEmptyForm}>
    <Plus size={14} />
    Add teacher
  </Button>
)}
```

`mayEdit` guards the pencil and the bin on each row, and puts a dash there instead:

```

{mayEdit ? (
  <>
    <IconButton
      onClick={() => openFormForEditing(teacher)}
      aria-label="Edit"
    >
      <Pencil size={14} />
    </IconButton>

    <IconButton
      variant="danger"
      onClick={() => askToDelete(teacher)}
      aria-label="Delete"
    >
      <Trash2 size={14} />
    </IconButton>
  </>
) : (
  <span className="px-2 text-slate-400">&mdash;</span>
)}

```

Log in as a student and the Teachers page still loads, still shows every teacher, and offers nothing to click. The Action column is a row of dashes.

LMS

- Dashboard
- Teachers**
- Students
- Courses
- Enrollments
- Lessons
- Assignments
- Submissions
- Results

sadia Student Log out

Teachers

The people who teach courses.

5 teachers

ID	NAME	EMAIL	SUBJECT	ACTIVE	ACTION
1	Imran Hossain	imran@lms.test	Web Development	Yes	—
2	Farhana Islam	farhana@lms.test	Databases	Yes	—
3	Rakib Chowdhury	rakib@lms.test	Mathematics	Yes	—
4	Tanvir Ahmed	tanvir@lms.test	Data Science	No	—
6	Shirin Sultana	shirin@lms.test	User Interface Design	Yes	—

Figure 6.1 The Teachers page as a student sees it. The list is there, because reading is open to any signed-in account. There is no Add button and the Action column is dashes.

THE SIDEBAR AND THE ROUTE

Two more places use the role, both in the same spirit.

The sidebar drops the Accounts link for anyone who is not an admin:

frontend/src/Sidebar.jsx

```
// `onlyFor` keeps a link out of the sidebar for other roles. The route and
// the API check the role again; this is only about not offering it.
{
  address: "/accounts",
  text: "Accounts",
  icon: <UserPlus size={16} />,
  onlyFor: ADMIN,
},
```

frontend/src/Sidebar.jsx

```
const visibleMenu = menu.filter(
  (item) => !item.onlyFor || item.onlyFor === user?.role,
);
```

An item with no `onlyFor` is always shown. An item with one is shown only to that role.

The screenshot shows the LMS interface for a signed-in student. The sidebar on the left lists various resources: Dashboard, Teachers (highlighted), Students, Courses, Enrollments, Lessons, Assignments, Submissions, and Results. At the bottom of the sidebar, the user's name 'sadia' is shown with a 'Student' role tag and a 'Log out' button. The main content area displays the 'Teachers' page, which includes a subtitle 'The people who teach courses.' and a table titled '5 teachers'. The table has columns for ID, NAME, EMAIL, SUBJECT, ACTIVE, and ACTION. The data rows are as follows:

ID	NAME	EMAIL	SUBJECT	ACTIVE	ACTION
1	Imran Hossain	imran@lms.test	Web Development	Yes	—
2	Farhana Islam	farhana@lms.test	Databases	Yes	—
3	Rakib Chowdhury	rakib@lms.test	Mathematics	Yes	—
4	Tanvir Ahmed	tanvir@lms.test	Data Science	No	—
6	Shirin Sultana	shirin@lms.test	User Interface Design	Yes	—

Figure 6.2 The sidebar for a signed-in student. Every resource page is listed, because reading is open, but the Accounts link is gone.

And `ProtectedRoute` checks again at the URL level, because hiding a link never stops anybody typing the address:

frontend/src/components/ProtectedRoute.jsx

```
// `role`, when given, is the one role allowed through. Hiding a link in the
// sidebar is not enough on its own – somebody can always type the URL.
```


frontend/src/components/ProtectedRoute.jsx

```
// The role arrives a moment after the token when a saved session is being
// topped up from /profile/. Waiting avoids bouncing the user by mistake.
if (role && !user?.role) {
  return null;
}

if (role && user.role !== role) {
  return <Navigate to="/" replace />;
}
```

`App.jsx` is where that gets used: the `/accounts` route is wrapped in `<ProtectedRoute role="admin">`. So there are three layers on the Accounts page: the link is hidden, the route redirects, and `/api/register/` returns 403. The first two are convenience. Only the third is security.

TIP

To have two roles signed in at once while you test, open the app normally in one window and in a private or incognito window in the other. Each window has its own `localStorage` (the small store in the browser where this app keeps the token), so each holds its own token. Logging out of one does not touch the other.

6.14 Hands on: get yourself a 403

Time to prove the server, not the buttons, is doing the work. You need three things running: MySQL, `python manage.py runserver 8001` in `backend`, and `npm run dev` in `frontend`.

Step 1. Make a student account. Log in as an admin, open `http://localhost:5180/accounts`, and fill the form in. Pick **Student** in the Role dropdown. Note the phone number you typed, because that is what the new person logs in with, not the username.

Step 2. Log in as that student. Log out, then log in at `http://localhost:5180/login` with the student's phone number. The sidebar has no Accounts link and the badge next to the username reads **Student**.

Step 3. Find a teacher to aim at. Open the Teachers page and note the number in the ID column of any row. Say it is `1`.

Step 4. Open the browser console. Press F12, or right-click the page and choose Inspect, then pick the Console tab. This is a place to type JavaScript that runs inside the page you are looking at. It has the page's `localStorage`, so it has the student's token.

Step 5. Write the delete by hand. No button is involved. This is a raw request, exactly the kind a determined person would send:

```
const response = await fetch("http://127.0.0.1:8001/api/teacher/1/", {
  method: "DELETE",
  headers: {
    Authorization: "Bearer " + localStorage.getItem("lms_access_token"),
  },
});

console.log(response.status);
console.log(await response.json());
```

"lms_access_token" is the key `api.js` saves the token under, near the top of the file:

frontend/src/api.js

```
const TOKEN_KEY = "lms_access_token";
```

Press Enter. The console prints:

```
403
{detail: 'Only an admin can change teacher and student records.'}
```

Refresh the Teachers page. Teacher 1 is still there.

CHECKPOINT You just saw the entire chapter work end to end. The request was well-formed. The token was real and valid. The account was genuinely signed in. `AdminWrites.has_permission` ran, `role_of` returned `"student"`, `"student"` was not in `(Profile.ADMIN,)`, the method was not in `SAFE_METHODS`, and the function returned `False`. `TeacherRetrieveUpdateDestroyAPIView` never ran. No SQL `DELETE` was ever sent. And the sentence in the console is the `message` attribute you typed into `permissions.py`.

Now try the same call against something a student *may* do:

```
const response = await fetch("http://127.0.0.1:8001/api/teacher/", {
  headers: {
    Authorization: "Bearer " + localStorage.getItem("lms_access_token"),
  },
});

console.log(response.status);
```

That prints `200`. Reading is open to any signed-in account, exactly as the module docstring promised. The difference between the two calls is one word: `DELETE` against `GET`.

COMMON MISTAKE

Deleting the `canWrite` check from a React page to "fix" a missing button, then wondering why the server still refuses. The React file only decides what to draw. Change `frontend/src/permissions.js` and you change which buttons appear. Change `backend/backend/permissions.py` and you change what is allowed. If you actually mean to give teachers the right to add teachers, the edit is `roles_that_may_write = (Profile.ADMIN, Profile.TEACHER)` on `AdminWrites` in Python, then the matching `teacher: [ADMIN, TEACHER]` in `WRITERS`, then the README table. Three files, one commit.

6.15 Keeping the two files in step

Because the table lives in two languages, here is the checklist for changing it:

1. Edit `roles_that_may_write` on the right class in `backend/backend/permissions.py`.
2. Edit the matching line of `WRITERS` in `frontend/src/permissions.js`.
3. Edit the roles table in `README.md`.
4. Log in as each of the three roles and click the thing you changed.

Step four is the one people skip. It is also the only one that catches the case where you edited the list view and forgot the detail view.

NOTE

A single shared source of truth would be nicer than two copies, and you could serve the table from an endpoint and have React fetch it. This project deliberately does not, because the second copy is small, obvious, and cannot cause a breach when it is wrong. An abstraction that has to be traced through four files to answer "may a teacher delete a course?" would cost more than it saves here.

What you have now

- A `role` field on `Profile` holding one of `admin`, `teacher` or `student`, with `student` as the default, plus migrations `0003_profile_role` and `0004_existing_accounts_become_admins` so that upgrading did not lock anybody out.
- `backend/backend/permissions.py` with `role_of()` and five permission classes: `IsAdmin`, `RoleWritePermission`, `AdminWrites`, `TeachingStaffWrites`, `SubmissionWrites`.
- A `role_of()` that treats a superuser as an admin and any other account without a `Profile` as a student, so an unknown caller gets the least privilege rather than the most.
- Reading open to every signed-in account through `SAFE_METHODS`, writing decided by role.
- One `POST`-only exception letting a student hand work in, with the reason written into the docstring: nothing links a `Student` row to a login account.

- All sixteen resource views wired to a permission class, and `/api/register/` closed to everyone but admins.
- Refusals that arrive as a 403 carrying the sentence you wrote, surfaced in the app's red banner.
- `frontend/src/permissions.js` with `canWrite` and `canCreate`, hiding buttons a role cannot use, and a clear understanding that it is the server, not the buttons, that enforces anything.

Starting the React App

Up to now everything you built lived on the server. Django holds the data, and Django REST Framework hands that data out as JSON at addresses like `/api/teacher/`. That is half a web app. Nobody wants to read raw JSON.

This chapter builds the other half: the part that runs inside the browser and draws the screens. It is written in React, and it lives in a completely separate folder from the backend, at `lms/frontend`.

By the end of this chapter you will have a running React app with a blank-but-working page, and you will understand every file that made it happen. The actual screens (login, sidebar, tables, forms) come in the chapters after this one.

What React is

Start with the problem React solves.

A web page is made of HTML elements: a heading, a table, a row inside that table, a button. The browser keeps all of those in memory as a tree of objects. That tree is called the **DOM**, short for Document Object Model. When you change the DOM, the screen changes.

Without React, you change the screen by giving the browser instructions, one at a time:

- find the table body
- make a new `<tr>` element
- make three `<td>` elements and put text in each one
- put the `<td>`s inside the `<tr>`
- put the `<tr>` inside the table body
- and if there was a "no teachers yet" message on screen, remove that too

That is a lot of steps for "a teacher was added". Every one of those steps is a place to make a mistake. Miss the last step and you get a table with rows in it *and* a message saying there are no rows.

React flips it around. In React you never write those steps. Instead you write a **description**: given the data I have right now, here is what the screen should look like. React compares your new description with what is currently on the screen, works out the smallest set of DOM changes needed, and applies them.

You describe. React makes reality match the description.

So when a teacher is added, you do not add a table row. You add the teacher to a list of data, and you tell React the data changed. React re-runs your description, notices there is one more row in it than on screen, and inserts exactly one `<tr>`. The "no teachers yet" message disappears on its own, because

your description says that message only appears when the list is empty, and the list is no longer empty.

A **component** is one of these descriptions, packaged as a function. `Sidebar` is a component.

`Teachers` is a component. A component is a plain JavaScript function that returns a description of some screen. Components can use other components inside them, which is how a whole app gets built out of small pieces.

NOTE

React does not talk to your database and does not know Django exists. It draws screens. When it needs data, it asks the API over HTTP, exactly the way the browser address bar did in the last chapter. The two halves of this project only ever meet at `http://127.0.0.1:8001/api/...`, which is the address written into `frontend/src/api.js`.

What Vite is, and why it is not part of React

React is a library. It is JavaScript code you import. It does not run anything, it does not open a browser, and it does not serve files over a network.

You still need something to:

- serve your files at a local web address so the browser can load them
- take the modern JavaScript you write and turn it into something browsers actually accept
- reload the page (or just the changed part of it) the instant you save a file
- bundle everything into a small set of files when you are ready to publish

That job belongs to a **build tool**, meaning a program that prepares your source files for a browser. This project uses **Vite** (pronounced "veet", French for "quick").

Keep the two apart in your head:

THING	WHAT IT IS	WHEN IT MATTERS
React	A library your code imports to describe screens	While your app is running, in the browser
Vite	A program that serves and builds your files	While you are working, and when you publish

You could use React without Vite. You could use Vite without React. They are packaged together here because that combination is the common one, and because Vite ships a ready-made React starter.

Vite has two modes:

- **dev mode** (`npm run dev`), a small web server on your own machine that watches your files. Save a file, and the browser updates almost instantly.
- **build mode** (`npm run build`), which reads everything, squeezes it into a handful of files in a folder called `dist`, and stops. Those files are what you would upload to a real web host.

While you follow this book you will live in dev mode.

Creating the app for real

Vite needs **Node.js**, which is a program that runs JavaScript outside a browser. Installing Node also installs **npm**, the tool that downloads JavaScript packages. Check both are there:

```
node --version
npm --version
```

The project was built on Node `v25.9.0` and npm `11.12.1`. Anything reasonably current will do. If either command says "command not found", install Node from nodejs.org first, then close and reopen the terminal.

COMMON MISTAKE

Installing Node while a terminal window is open, then wondering why `node --version` still fails in that window. A terminal reads the list of available commands when it starts. Open a new terminal after installing anything.

Now create the app. Open a terminal **in the `lms` folder**, the one that already contains `backend`:

```
cd "/Users/tausif/Projects/Ostad/Batch 08/lms"
npm create vite@latest frontend -- --template react
```

Read that command piece by piece, because it looks strange:

- `npm create vite@latest` means "download the newest Vite project creator and run it". You are not installing anything permanently, you are running a one-off generator.
- `frontend` is the name of the folder to create. It sits next to `backend`.
- `--` is a separator. Everything after it is passed to the Vite generator instead of being read by npm itself.
- `--template react` picks the plain React starter. There is also a `react-ts` template for TypeScript. This project uses plain JavaScript, so use `react`.

The generator writes a folder full of files and then prints three suggested commands. Follow the first two:

```
cd frontend
npm install
```

`npm install` reads `package.json`, sees which packages the project needs, downloads them, and puts them in a folder called `node_modules`. It takes a few seconds and finishes with a summary line such as `added 190 packages in 8s`. The exact number will differ on your machine, and that is fine.

NODE_MODULES AND PACKAGE-LOCK.JSON

Two things appeared that beginners find alarming. Both are normal.

`node_modules/` is where every downloaded package lives. It is enormous, often tens of thousands of files, because your packages have their own packages, which have their own packages. You never edit anything in it, never read it, and never commit it to Git. If it ever gets into a strange state, the fix is to delete the whole folder and run `npm install` again. Nothing is lost, because everything in it came from the internet and can be downloaded again.

`package-lock.json` is a record of the *exact* version of every single package that got installed, including the packages-of-packages. In this project it is a little over 3,000 lines long. You do not edit it by hand. npm rewrites it whenever you install or remove something.

Why does it exist? Look at `package.json`:

```
"react": "^19.2.7"
```

That `^` means "19.2.7 or any newer 19.x". So on your machine today `npm install` might fetch 19.2.7, and on a colleague's machine next month it might fetch 19.4.0. That is usually fine and occasionally the cause of a bug that only happens on one machine. `package-lock.json` removes the guesswork: it pins the exact versions, so everyone who clones the repo gets the same `node_modules`.

TIP

`node_modules` is ignored by Git, `package-lock.json` is not. Commit `package-lock.json`. The frontend `.gitignore` that Vite generated already has the rules for this. It ignores `node_modules`, `dist`, `dist-ssr`, `*.local`, log files, `.DS_Store`, and editor folders such as `.vscode/*` and `.idea`.

Installing the rest

The starter gives you React and Vite. This project needs four more packages. Install them in two commands, still inside the `frontend` folder:

```
npm install react-router-dom lucide-react
npm install tailwindcss @tailwindcss/vite
```

What each one is for:

- **react-router-dom** gives the app more than one screen. Without it, a React app is a single page with a single view. With it, `/teachers` shows the Teachers page and `/courses` shows the Courses page, and the browser Back button works.
- **lucide-react** is a set of icons, delivered as React components. The sidebar in this app uses `LayoutDashboard`, `Users`, `GraduationCap`, `BookOpen`, `ClipboardList`, `FileText`, `Send`, `Trophy`, `UserPlus`, `UserRound` and `Logout`.

- **tailwindcss** is the styling system. You style elements by putting short class names on them rather than by writing a separate CSS file.
- **@tailwindcss/vite** is the piece that plugs Tailwind into Vite. Tailwind 4 works as a Vite plugin, which is why there is no `tailwind.config.js` and no `postcss.config.js` in this project. Older tutorials will tell you to create both. Do not.

NOTE

All four went in with a plain `npm install`, with no `-D` flag, which is why they land in `dependencies` and not `devDependencies` in the real `package.json`. Tailwind is arguably a build-time tool and could have gone in `devDependencies`. For an app you run with `npm run dev` and build yourself, the distinction changes nothing.

EVERY DEPENDENCY IN THE REAL PACKAGE.JSON

This is the complete list from `frontend/package.json`, with a plain-English line for each.

dependencies (things the app itself uses):

PACKAGE	VERSION	WHAT IT DOES
@tailwindcss/vite	^4.3.3	Connects Tailwind to Vite so it runs automatically on every save.
lucide-react	^1.27.0	Icon set. Each icon is a component, for example <code><Users size={16} /></code> .
react	^19.2.7	The library itself. Lets you write components and describe screens.
react-dom	^19.2.7	The part of React that talks to the browser DOM. <code>createRoot</code> comes from its <code>react-dom/client</code> entry point.
react-router-dom	^7.11.0	Multiple screens at different URLs, plus <code>Link</code> for navigating without a full page reload.
tailwindcss	^4.3.3	The styling system. Reads your class names and generates the CSS for them.

devDependencies (things only you, the developer, use):

PACKAGE	VERSION	WHAT IT DOES
<code>@eslint/js</code>	<code>^10.0.1</code>	ESLint's own recommended set of rules.
<code>@types/react</code>	<code>^19.2.17</code>	Type descriptions for React. Not required for plain JavaScript, but editors use them for autocomplete.
<code>@types/react-dom</code>	<code>^19.2.3</code>	The same, for <code>react-dom</code> .
<code>@vitejs/plugin-react</code>	<code>^6.0.3</code>	Teaches Vite to read JSX, and enables Fast Refresh so edits appear without losing what is on screen.
<code>eslint</code>	<code>^10.6.0</code>	Reads your code and reports likely mistakes, such as a variable you never use.
<code>eslint-plugin-react-hooks</code>	<code>^7.1.1</code>	Extra rules for React hooks, the <code>useState</code> and <code>useEffect</code> functions you meet later in the book.
<code>eslint-plugin-react-refresh</code>	<code>^0.5.3</code>	Warns when a file is written in a way that breaks Fast Refresh.
<code>globals</code>	<code>^17.7.0</code>	A list of names the browser provides for free, like <code>window</code> and <code>document</code> , so ESLint does not flag them as undefined.
<code>vite</code>	<code>^8.1.1</code>	The dev server and the builder.

The difference between the two lists is simple. If you published this app to a real web host, the built files would contain React and your components. They would not contain ESLint. Tools that only help while writing code belong in `devDependencies`.

The npm scripts

The `scripts` block in `package.json` is a set of shortcuts. Type `npm run <name>` and npm runs the command on the right.

```
// frontend/package.json (scripts section)
"scripts": {
  "dev": "vite",
  "build": "vite build",
  "lint": "eslint .",
  "preview": "vite preview"
}
```

COMMAND	WHAT HAPPENS
<code>npm run dev</code>	Starts the Vite dev server and leaves it running. This is the one you use all day. Stop it with <code>Ctrl + C</code> .
<code>npm run build</code>	Builds the files for publishing into <code>dist/</code> . Runs once and exits. You do not need this until you publish.
<code>npm run lint</code>	Runs ESLint over the whole project and lists problems. Prints nothing at all when everything is clean.
<code>npm run preview</code>	Serves the already-built <code>dist/</code> folder so you can check the built version behaves like the dev version. It has nothing to serve until you have run <code>npm run build</code> .

COMMON MISTAKE

Running `npm dev` instead of `npm run dev`. npm has a handful of built-in commands (`install`, `start`, `test`) that work without `run`. Your own scripts need `run`. When in doubt, type `npm run` with nothing after it and npm lists every available script.

`npm run dev` is a foreground process, which means it keeps hold of the terminal instead of giving your prompt back. That is correct behaviour. It has to keep running so it can watch your files. You need **two terminals open** while working on this project: one running Django on port 8001, one running Vite on port 5180.

vite.config.js

The generator wrote a small `vite.config.js`. This project's version has one thing added and comments explaining it. Here it is exactly as it appears in the repo:

frontend/vite.config.js

```
// Vite is the tool that runs the app while you work on it.
// This file tells it two things: use React, and use Tailwind.

import { defineConfig } from "vite";
import react from "@vitejs/plugin-react";
import tailwindcss from "@tailwindcss/vite";

export default defineConfig({
  plugins: [react(), tailwindcss()],

  // Which port to open on. Other projects on this machine already use
  // 5173 to 5175, so this one uses 5180 to stay out of their way.
  // If 5180 is busy too, Vite quietly picks the next free number and tells
  // you which one in the terminal.
  server: {
    port: 5180,
  },
});
```

Now the parts.

export default defineConfig({ ... }). Vite reads this file and expects the default export to be a settings object. **defineConfig** is a wrapper that does nothing when the program runs, it just returns what you pass it. Its only purpose is to let your editor know the shape of the object, so you get autocomplete and a red squiggle when you misspell **server**. You could write **export default { ... }** and it would work identically, with worse editor support.

plugins: [react(), tailwindcss()]. A plugin is a piece of code that hooks into the build. Note the brackets after each name: you are calling each function and putting the *result* in the list, not the function itself.

- react()** is what makes JSX work. JSX (the **<div>** syntax you write inside JavaScript) is not valid JavaScript, and no browser understands it. This plugin converts every JSX tag into a normal function call before the browser sees it. It also wires up Fast Refresh, so saving a component swaps in the new version while keeping whatever you had typed into a form.
- tailwindcss()** scans your source files for class names such as **px-3** and **bg-white**, generates the CSS for exactly those, and injects it. It re-runs on every save. This is why the project has no Tailwind config file: in Tailwind 4 the plugin handles it.

server: { port: 5180 }. Vite's default port is 5173. This project asks for 5180 instead, and the comment says why: other projects on the same machine already sit on 5173 to 5175. Pinning a port you know is free means the address in your browser is the same every day and your bookmarks keep working.

That number is not just a convenience. It has to line up with the backend. A browser labels every page with its **origin**, which is the protocol, host and port taken together, for example

`http://localhost:5180`. Django refuses requests from origins it does not recognise, and the list of allowed origins is in `lms/settings.py`:

`backend/lms/settings.py`

```
# The Vite dev server runs on 5180 (see frontend/vite.config.js). 5173 is kept
# here because that is Vite's own default and the port the README used to name.
CORS_ALLOWED_ORIGINS = [
    'http://localhost:5180',
    'http://127.0.0.1:5180',
    'http://localhost:5173',
    'http://127.0.0.1:5173',
]
```

CORS is short for Cross-Origin Resource Sharing. It is the browser rule that stops a page on one origin from reading data from another origin unless that other server says it is allowed. Your React page sits on port 5180 and your Django API sits on port 8001, so they are different origins, and Django has to say yes.

Both ports are listed, and both spellings of "this machine" are listed, because `localhost` and `127.0.0.1` count as different origins to a browser even though they point at the same place.

COMMON MISTAKE

Changing the port in `vite.config.js` and forgetting `CORS_ALLOWED_ORIGINS`. The app loads fine, the login page appears, and then every API call fails with a CORS error in the browser console while Django's terminal shows nothing wrong. If you move off 5180, add the new origin to that list in `settings.py` and restart the Django server.

NOTE

If port 5180 happens to be in use, Vite does not crash. It prints `Port 5180 is in use, trying another one...`, takes the next free port, and shows you the real address. Always read the address Vite prints rather than assuming. If it did pick a different port, your API calls will hit the CORS wall described above.

index.html and the idea of a single page app

Open `frontend/index.html`. It is the whole HTML of the whole application:

```

<!-- frontend/index.html -->
<!doctype html>
<html lang="en">
  <head>
    <meta charset="UTF-8" />
    <link rel="icon" type="image/svg+xml" href="/favicon.svg" />
    <meta name="viewport" content="width=device-width, initial-scale=1.0" />
    <title>LMS</title>
  </head>
  <body>
    <div id="root"></div>
    <script type="module" src="/src/main.jsx"></script>
  </body>
</html>

```

Thirteen lines, and no visible content at all. The `<body>` holds one empty `<div>` and one `<script>` tag.

`<div id="root"></div>` is the empty box React fills. Everything you will ever see in this app (sidebar, tables, forms, dialogs) gets created by React and placed inside that div. It is empty on purpose. The `id="root"` is the label React uses to find it.

`<script type="module" src="/src/main.jsx">` loads the first file of your JavaScript. `type="module"` tells the browser this file uses `import` statements to pull in other files, so it should go and fetch those too. That is how one script tag ends up loading dozens of files.

NOTE

A browser cannot run `.jsx` directly. In dev mode Vite intercepts the request for `/src/main.jsx`, converts the JSX, and sends back plain JavaScript. That is one of the jobs you installed Vite for.

The other two `<head>` lines are ordinary web page things: the `viewport` meta tag makes the page size itself sensibly on a phone, and the `icon` link points at `public/favicon.svg`, the small picture in the browser tab. Anything you put in `frontend/public` is served from the root of the site, so `public/favicon.svg` becomes `/favicon.svg`.

This structure is what people mean by a **single page app**, usually shortened to SPA. There is one real HTML page. When you click "Students" in the sidebar, the browser does *not* request a new page from a server. React Router notices the click, changes the address in the bar, and swaps out the part of the screen that should change. Nothing flashes white, nothing reloads, and no trip to Django happens for the page itself. Only the data is fetched, as JSON, in the background.

The traditional alternative is a server-rendered app, where every click asks the server for a complete new HTML page. Django can do that perfectly well on its own. This project splits the work instead: Django serves data only, React serves screens only.

TIP

The Vite starter also generates `src/App.css`, `src/assets/react.svg` and `public/vite.svg` for its demo counter page. None of them exist in the finished project. Delete them once you start replacing `App.jsx` in the next chapter, and delete the demo markup inside `App.jsx` too. This project keeps its own `public/favicon.svg` instead of `vite.svg`, which is why `index.html` points at `/favicon.svg`.

src/main.jsx, line by line

This is the first file of yours that runs. It is thirteen lines and every one matters.

frontend/src/main.jsx

```
import { createRoot } from "react-dom/client";
import { BrowserRouter } from "react-router-dom";
import App from "./App.jsx";
import { AuthProvider } from "./AuthContext.jsx";
import "./index.css";
```

The imports, in order:

- `createRoot` comes from `react-dom/client`. It is the function that connects React to a real DOM element.
- `BrowserRouter` comes from `react-router-dom`. It is the component that watches the browser address bar.
- `App` is your own file, `src/App.jsx`. Note the curly braces are absent here. `App` is a **default export**, meaning the file's one main thing, so you import it without braces and may name it anything. `createRoot` and `BrowserRouter` are **named exports**, meaning one of several things a file offers by name, so they need braces and the exact name.
- `AuthProvider` is your own file too, `src/AuthContext.jsx`. It holds the answer to "who is signed in".
- `import "./index.css"` imports a stylesheet with no name attached. You are not using anything from it; you are telling Vite "this CSS file belongs to the app, include it". That single line is what pulls Tailwind into the page.

COMMON MISTAKE

Writing `import App from "./App"` without the `.jsx`. Vite can usually work the extension out for you, so this often appears to work and then confuses you later when a tool that is stricter refuses it. Every import of a local file in this project's source spells out the extension. Match what the repo does and the question never comes up.

Now the second half:

```
// frontend/src/main.jsx (continued)
createRoot(document.getElementById("root")).render(
  <BrowserRouter>
    <AuthProvider>
      <App />
    </AuthProvider>
  </BrowserRouter>,
);
```

`document.getElementById("root")` is plain browser JavaScript. It finds the empty `<div id="root">` from `index.html` and hands you that element.

`createRoot(...)` tells React "this element is yours now". It gives back a root object with a `render` method.

`.render(...)` takes a description of a screen and puts it inside that element. From this moment on, React owns everything inside `#root`.

The thing being rendered is three components nested inside each other. That nesting is not decoration. Each wrapper makes something available to everything inside it, and the order decides what is available to whom.

Outermost: `BrowserRouter`. It reads the current URL, keeps an eye on the Back and Forward buttons, and provides the machinery that `Link`, `Navigate`, `useNavigate` and `useLocation` need. Anything that wants to know the current address, or wants to send the user somewhere else, must be inside it.

Middle: `AuthProvider`. It holds the signed-in user, whether anyone is signed in at all, and the `login` and `logout` functions. Anything that wants to know who is signed in must be inside it.

Innermost: `App`. Your route list. Every actual page of the LMS is reached through it.

Why this exact order, and not `AuthProvider` outside `BrowserRouter`?

Because auth needs routing, and routing does not need auth. Look at what `AuthProvider` and the pages under it actually do. `ProtectedRoute` sends a signed-out visitor to `/login` with `<Navigate>`. `Sidebar` calls `useNavigate()` so that logging out lands you on the login page:

```
// frontend/src/Sidebar.jsx (the log out handler)
function handleLogout() {
  logout();
  navigate("/login", { replace: true });
}
```

`useNavigate` only works when there is a router above it in the tree. Put `AuthProvider` on the outside and every redirect inside it breaks, because the router would no longer be an ancestor.

The second half of the rule is the same idea one level down. `App` must be inside `AuthProvider`, because the routes it defines include `ProtectedRoute`, and `ProtectedRoute` asks who is signed in through the `useAuth()` hook. A component can only read from a provider that sits above it.

Read the order as a sentence: **routing has to exist before anything can redirect, and auth has to exist before any page can ask who is signed in.**

NOTE

The Vite starter wraps everything in `<StrictMode>`. This project does not. `StrictMode` deliberately runs some code twice in development to expose bugs, which makes API calls appear twice in the network tab and confuses beginners badly. Its absence here is a teaching choice, not an oversight.

The trailing comma after `</BrowserRouter>` is just the code formatter's house style. It changes nothing.

src/index.css and utility classes

The entire stylesheet of this application is one line:

```
/* frontend/src/index.css */
@import "tailwindcss";
```

That is the Tailwind 4 way in. The `@tailwindcss/vite` plugin sees this import, scans your `.jsx` files for class names, generates CSS for the ones you actually used, and substitutes it here. If you write `px-3` in a component, the rule for `px-3` appears. If you never write `px-96`, its rule is never generated. That is why the finished CSS stays small no matter how big Tailwind gets.

WHAT A UTILITY CLASS IS

Normally you write CSS like this: invent a name such as `sidebar-link`, write a rule for it in a stylesheet, and put `class="sidebar-link"` on the element. Two files, one concept, and you have to keep them in step.

Tailwind takes a different route. It pre-defines hundreds of tiny classes that each do exactly one thing, and you combine them on the element. These are **utility classes**. `p-3` sets padding. `flex` sets `display: flex`. `text-sm` sets a small font size. Nothing is invented; you assemble.

Here are three real examples from `src/Sidebar.jsx`.

The outermost element of the sidebar layout:

```
// frontend/src/Sidebar.jsx (outer wrapper)
<div className="flex min-h-screen bg-slate-50">
```

Three classes, three decisions:

CLASS	MEANING
<code>flex</code>	Lay the children out in a row, side by side
<code>min-h-screen</code>	Be at least as tall as the browser window
<code>bg-slate-50</code>	Background colour: the palest grey in the "slate" set

The sidebar column itself:

```
// frontend/src/Sidebar.jsx (the sidebar column)
<div className="flex w-60 flex-col border-r border-slate-200 bg-white">
```

CLASS	MEANING
<code>flex</code> + <code>flex-col</code>	Lay the children out, stacked top to bottom
<code>w-60</code>	A fixed width, 15rem, which is 240 pixels at default settings
<code>border-r</code>	Draw a border on the right edge only
<code>border-slate-200</code>	Make that border a light grey
<code>bg-white</code>	White background, so the column stands out from the <code>bg-slate-50</code> page

And one menu link:

```
// frontend/src/Sidebar.jsx (a menu link)
className={
  "mb-1 flex items-center gap-3 rounded-md px-3 py-2 text-sm " +
  colours
}
```

CLASS	MEANING
<code>mb-1</code>	A small margin below, so links do not touch
<code>flex items-center</code>	Icon and text on one line, vertically centred against each other
<code>gap-3</code>	A gap between the icon and the text
<code>rounded-md</code>	Medium rounded corners
<code>px-3 py-2</code>	Padding: 3 units left and right, 2 units top and bottom
<code>text-sm</code>	Small text

The numbers are steps on a scale, not pixels. `px-3` and `py-2` come from the same spacing scale as `gap-3` and `mb-1`, which is why unrelated components in this app end up consistent with each other without anyone measuring anything.

Notice too that `colours` is a variable holding more classes, chosen a few lines earlier:

```
// frontend/src/Sidebar.jsx (choosing the highlight)
let colours = "text-slate-600 hover:bg-slate-100";
if (isTheCurrentPage) {
  colours = "bg-indigo-50 font-medium text-indigo-700";
}
```

`hover:bg-slate-100` is a utility with a prefix. The `hover:` part means "only when the mouse is over this element". That single string replaces what would be a separate `:hover` block in a stylesheet.

The class list is a plain string, so building it up with `+` is ordinary JavaScript. This is the everyday way conditional styling is done in the project.

TIP

You do not need to memorise Tailwind class names. Search the Tailwind docs for the CSS property you want, for example "padding" or "border radius", and it shows the classes. After a week the common ones (`flex`, `gap-2`, `px-4`, `text-sm`, `rounded-md`, `bg-white`) will be muscle memory.

JSX, and the three rules beginners trip on

Look again at what the sidebar returns. It looks like HTML sitting in the middle of a JavaScript file:

frontend/src/Sidebar.jsx

```
<span className="text-lg font-semibold text-slate-900">LMS</span>
```

That is **JSX**. It is a syntax extension: HTML-looking tags written directly inside JavaScript. Browsers do not understand it. `@vitejs/plugin-react` converts every tag into a function call before the browser gets the file. You never see the converted version, and you do not need to.

JSX is very close to HTML. Three differences catch everyone.

RULE 1: `CLASSNAME`, NOT `CLASS`

`class` is a reserved word in JavaScript, so JSX uses `className` for the HTML class attribute. Every styled element in this project spells it that way:

frontend/src/Sidebar.jsx

```
<GraduationCap size={22} className="text-indigo-600" />
```

Write `class="..."` and React warns you in the browser console and your styles do not apply. When copying HTML in from anywhere else, this is the first thing to fix. (`for` on a `<label>` becomes `htmlFor` for the same reason.)

RULE 2: EXACTLY ONE PARENT ELEMENT

A component returns one thing. This is fine, because everything is inside a single outer `<div>`:

```
// frontend/src/Sidebar.jsx (the shape of the return, with the middle left out)
return (
  <div className="flex min-h-screen bg-slate-50">
    <div className="flex w-60 flex-col border-r border-slate-200 bg-white">
      ...
    </div>
    <div className="flex-1 overflow-x-auto p-8">
      <Outlet />
    </div>
  </div>
);
```

Two elements side by side at the top level is a syntax error. If you genuinely do not want a wrapper `<div>` in the output, React offers an empty tag `<>...</>`, called a fragment, which groups children without producing any element.

Note also the parentheses after `return`. JavaScript will happily insert a semicolon after a bare `return` on its own line and hand back nothing at all. Opening a parenthesis on the same line as `return` prevents that.

RULE 3: CURLY BRACES MEAN "JAVASCRIPT GOES HERE"

Inside JSX, everything is treated as text until you open a curly brace. Inside braces, you are back in JavaScript and any expression works.

Three real uses from the sidebar. A value inserted as content:

frontend/src/Sidebar.jsx

```
<span className="truncate">{user?.username ?? "Signed in"}</span>
```

A value passed as an attribute. Note there are no quotes around `{16}`, because it is the number 16, not the string "16":

frontend/src/Sidebar.jsx

```
<UserRound size={16} />
```

A whole expression building the class string, which is why the braces wrap the entire `"..." + colours`:

frontend/src/Sidebar.jsx

```
className={
  "mb-1 flex items-center gap-3 rounded-md px-3 py-2 text-sm " +
  colours
}
```

The word "expression" is doing work there. An expression is any piece of code that produces a value: a variable, a function call, a piece of arithmetic, or a ternary. A ternary is the short `condition ? valueIfTrue : valueIfFalse` form of an `if`, and because it produces a value it is allowed inside braces. A real `if` statement or a `for` loop is not, because those produce nothing. When a page needs a condition, the project either uses a ternary inside the braces (as the profile link does) or works the answer out on a normal line above the `return` (as `colours` does).

One more thing braces enable: comments. `// like this` inside JSX would be printed on screen as literal text. A JSX comment is a brace containing a block comment, exactly as the project writes them:

frontend/src/Sidebar.jsx

```
{/* The name doubles as the way into your own profile. */}
```

eslint.config.js

The Vite starter also generated `eslint.config.js`. ESLint reads your code and flags likely mistakes: a variable you declared and never used, a hook called in the wrong place, a mistyped name. You do not have to run it, but `npm run lint` is a cheap way to catch a whole class of bug before the browser does.

The generated file is left as it came apart from one added rule, which is worth understanding because you will otherwise hit the problem it fixes:

```
// frontend/eslint.config.js (rules section)
rules: {
  // ESLint's own no-unused-vars does not count a name being used inside
  // JSX. Without this, every imported component and icon is reported as
  // unused. eslint-plugin-react would normally fix that, but it does not
  // support ESLint 10 yet, so capitalised names are ignored instead.
  'no-unused-vars': ['error', { varsIgnorePattern: '^[A-Z]' }],
},
```

The situation: you `import { Users } from "lucide-react"` and then use it only as `<Users size={16} />`. Plain ESLint scans for `Users` used as a JavaScript name, does not look inside JSX tags, finds nothing, and reports the import as unused. Every component and icon in the project would be flagged.

`varsIgnorePattern: '^[A-Z]'` says "do not report unused variables whose name starts with a capital letter". Component and icon names are capitalised by convention (React requires it, so it can tell your components apart from real HTML tags), so the pattern maps neatly onto exactly the names that JSX consumes. Lowercase variables are still checked normally.

The rest of the file is standard: it ignores the `dist` folder with `globalIgnores(['dist'])`, applies to all `.js` and `.jsx` files, turns on the recommended JavaScript rules plus the two React plugins, and tells ESLint that browser globals such as `window` and `document` exist.

CHECKPOINT Run `npm run lint` in the `frontend` folder. On a clean project it prints the two npm header lines and then nothing at all, and returns you to the prompt. Silence is success. If it prints errors about files inside `dist`, something is off with the `globalIgnores(['dist'])` line.

The finished folder layout

This is what the `frontend` folder looks like when the whole app is done, so you know where the chapters ahead are heading. You will not have most of these files yet.

```
frontend/
  index.html
  package.json
  package-lock.json
  vite.config.js
  eslint.config.js
  .gitignore
  README.md
  public/
    favicon.svg
  src/
    main.jsx          first file to run; mounts the app
    App.jsx           the route list: which URL shows which page
    Sidebar.jsx       the left menu and the frame around every page
    AuthContext.jsx   holds the signed-in user; provides logIn and logOut
    auth.js           the context object and the useAuth hook
    api.js            every call to the Django API lives here
    permissions.js    role names and who is allowed to do what
    flash.js          the short-lived success messages that clear themselves
    index.css         one line, which turns Tailwind on
    components/
      index.js        re-exports the components below, for tidy imports
      styles.js       shared class strings
      Alert.jsx
      Button.jsx
      Checkbox.jsx
      ConfirmDialog.jsx
      Div.jsx
      IconButton.jsx
      Input.jsx
      PageHeader.jsx
      ProtectedRoute.jsx
      Select.jsx
      Table.jsx
      Textarea.jsx
    pages/
      Dashboard.jsx
      Teachers.jsx
      Students.jsx
      Courses.jsx
      Enrollments.jsx
      Lessons.jsx
      Assignments.jsx
      Submissions.jsx
      Results.jsx
      Accounts.jsx
      Profile.jsx
```

```
Login.jsx
NotFound.jsx
```

A `dist/` folder will also appear the first time you run `npm run build`. It is generated, and `.gitignore` keeps it out of Git.

The split is deliberate and worth naming now. `pages/` holds one file per screen, each matching one route in `App.jsx`. `components/` holds small pieces used by many pages, such as a button or an input. **The loose files in `src/`** are the app-wide plumbing: routing, auth, API, permissions.

Checkpoint: get it running

Everything above is in place. Start the dev server. From the `frontend` folder:

```
npm run dev
```

CHECKPOINT Vite prints a banner. It looks like this:

```
`` VITE v8.1.5 ready in 481 ms
```

→ Local: `http://localhost:5180/` → Network: use `--host` to expose → press `h` + enter to show help ``

Your version number and timing will differ. What matters is the port: it must say **5180**. Open that address in a browser. At this stage you will see whatever the Vite starter page shows, or a blank white page if you have already emptied `App.jsx`. Either is correct. A blank page is not a failure, it means React mounted and had nothing to draw.

To be sure React really is running rather than showing a cached file, open the browser developer tools with `F12`, go to the Elements tab, and expand `<div id="root">`. If there is anything inside it, React put it there, because `index.html` ships that div empty.

TIP

Leave `npm run dev` running in its own terminal for the rest of the book, and leave `python manage.py runserver 8001` running in another. Two terminals, two servers, all the time. Every "why is nothing loading" moment in the coming chapters is one of those two having been stopped.

COMMON MISTAKE

Running `npm run dev` from the wrong folder. If you are in `lms` rather than `lms/frontend`, npm stops with `npm error enoent Could not read package.json`. If you are in `lms/backend`, the same thing happens. Run `cd frontend` first, and check where you are with `pwd`.

Stop the server with `Ctrl + C` when you are done. The terminal returns to a normal prompt.

What you have now

- A real React 19 project at `lms/frontend`, created with `npm create vite@latest frontend -- --template react`
- The four extra packages this app needs: `react-router-dom`, `lucide-react`, `tailwindcss` and `@tailwindcss/vite`
- A `vite.config.js` with the React and Tailwind plugins and the dev server pinned to port 5180, matching `CORS_ALLOWED_ORIGINS` in `backend/lms/settings.py`
- An understanding of `index.html`: one empty `<div id="root">` and one module script, which is what makes this a single page app
- A `src/main.jsx` that mounts the app with `createRoot`, wrapped in `BrowserRouter` then `AuthProvider` then `App`, in that order and for that reason
- Tailwind switched on by the single `@import "tailwindcss";` line in `src/index.css`, and a working idea of what a utility class is
- The three JSX rules: `className` not `class`, one parent element, curly braces for JavaScript
- A dev server you can start with `npm run dev` and reach at `http://localhost:5180`

Talking to the Backend: api.js

In Chapter 7 you built the React shell: the sidebar, the router, the small UI pieces. Every page in it is going to need data, and that data lives in the Django server you built in the first half of this book. This chapter builds the piece in the middle: one file that knows how to talk to Django, and one small file that handles the little green "Saved" messages.

By the end of this chapter you will have written `frontend/src/api.js` and `frontend/src/flash.js` from an empty file to the finished thing.

NOTE

In this book, when a code block starts with a comment like `// frontend/src/api.js`, that line is a signpost telling you which file the code belongs to. It is not a line you type, and it is not in the real file. Everything below that first comment line is the real file's content, character for character.

Why every network call lives in one file

A **network call** is when your browser asks another computer for something. The browser sends a request over the internet (or over your own machine, in our case) and gets an answer back. The React app runs at `http://localhost:5180`. Django runs at `http://127.0.0.1:8001`. They are two separate programs. The only way the React app can see a teacher row is to ask Django for it.

An **endpoint** is one address on the server that does one job. `/api/teacher/` is an endpoint that lists teachers or adds one. `/api/login/` is an endpoint that checks a password. Django's `backend/backend/urls.py` is the list of every endpoint this project has.

The browser has a built-in function for asking, called `fetch`. You could call `fetch` directly inside every page. Eight resource pages, four operations each, would be thirty-two places where you write out the server address, attach the login token, check whether the answer was an error, and turn the error into something a human can read.

Thirty-two copies of the same four steps is thirty-two chances to get one of them slightly wrong. So all of it goes in one file. Pages then call things like `teachersApi.list()` and never think about tokens or headers at all.

The real file opens with this comment, and it is worth reading before you write a line:

frontend/src/api.js

```
// Every call to the Django backend lives in this one file.
//
// Two things about the backend shape the code here:
// - you log in with a phone number, and the tokens come back nested
//   under `tokens`, so it is data.tokens.access and not data.access
// - list endpoints are not paginated, so they return a plain array
```

Both of those quirks come from decisions you made in the Django half of the book. Writing them down at the top of the file that has to live with them saves the next person a confusing afternoon.

Create the file now. From the `frontend` folder:

```
touch src/api.js
```

`touch` makes an empty file. If you would rather make it in your editor's file tree, that is exactly the same thing.

BASE_URL and the frontend environment variable

Every request has to start with the address of the Django API. That address is not the same on your laptop as it would be on a real server, so it should not be typed into thirty-two places, and ideally it should not be typed into the source code at all.

Add the first real line of the file:

frontend/src/api.js

```
const BASE_URL = import.meta.env.VITE_API_URL ?? "http://127.0.0.1:8001/api";
```

Three new things there.

`import.meta.env` is Vite's box of **environment variables**. An environment variable is a setting that comes from outside the code: from a file on disk or from the shell that started the program. The point is that you can change the setting without editing and re-committing the source.

`VITE_API_URL` is the name of one such setting. Vite has a safety rule: only variables whose names begin with `VITE_` are handed to the browser. Everything else stays on your machine. That rule exists because anything the browser can see, your users can see. A database password named `DB_PASSWORD` would never reach the browser. A public API address named `VITE_API_URL` is fine.

`??` is the **nullish coalescing operator**. `a ?? b` means "use `a`, unless `a` is missing, in which case use `b`." So: use `VITE_API_URL` if somebody set one, otherwise fall back to `http://127.0.0.1:8001/api`. Since you are running Django locally, nobody has to set anything and the fallback just works.

That fallback ends in `/api` because `backend/lms/urls.py` mounts every endpoint under that prefix:

```
# backend/lms/urls.py (two lines, for reference)
path('admin/', admin.site.urls),
path('api/', include('backend.urls')),
```

To point the app at a different backend, make a file called `.env` inside the `frontend` folder:

frontend/.env

```
VITE_API_URL=https://lms-api.example.com/api
```

COMMON MISTAKE

Vite reads `.env` once, when the dev server starts. Editing `.env` while `npm run dev` is running changes nothing. Stop the server with `Ctrl+C` and run `npm run dev` again.

NOTE

The value must not end with a slash. Every call in this file adds its own leading slash, for example `request("/login/")`. If you set `VITE_API_URL=http://127.0.0.1:8001/api/` you get `http://127.0.0.1:8001/api//login/`, and Django will answer 404.

COMMON MISTAKE

Changing the backend address is only half the job. Django refuses requests from **origins** it does not know about, where an origin is the scheme, host and port together, like `http://localhost:5180`. The `CORS_ALLOWED_ORIGINS` list in `backend/lms/settings.py` names four of them: `http://localhost:5180`, `http://127.0.0.1:5180`, `http://localhost:5173` and `http://127.0.0.1:5173`. `frontend/vite.config.js` sets the dev server to port 5180, but if 5180 is busy Vite quietly picks the next free number and prints it in the terminal. If that happens, add the new port to the list, or every single request will be blocked by the browser before Django even sees it.

Remembering who is logged in: localStorage

When a user logs in, Django hands back a **token**. A token is a long random-looking string that proves "I am the person who typed the right password." Every later request carries that token so Django knows who is asking.

The token has to survive a page refresh. If it only lived in a React variable, hitting F5 would log the user out, because refreshing throws away all JavaScript memory and starts over.

`localStorage` is a small store built into the browser. It holds text under names you choose, and it keeps that text after the tab is closed and even after the computer is restarted. It is not shared between different websites.

Two names are enough for this app:

frontend/src/api.js

```
const TOKEN_KEY = "lms_access_token";
const USER_KEY = "lms_user";
```

Naming the keys as constants rather than typing the strings in four places means a typo becomes an error you can see, instead of a silent "no token found".

Now the four helpers. The first one is a one-liner:

frontend/src/api.js

```
export function getToken() {
  return localStorage.getItem(TOKEN_KEY);
}
```

`export` means other files may import this function. `localStorage.getItem` returns the stored text, or `null` if nothing was ever stored under that name.

Saving is two writes:

frontend/src/api.js

```
export function saveSession(token, user) {
  localStorage.setItem(TOKEN_KEY, token);
  localStorage.setItem(USER_KEY, JSON.stringify(user));
}
```

`localStorage` only stores text. The `user` here is a JavaScript object. `AuthContext.jsx` builds it out of three fields, so it looks like `{ id: 1, username: "tausif", role: "admin" }`. `JSON.stringify` turns an object into a line of text so it can be stored. It is the same JSON format Django speaks, which is convenient but not a coincidence.

Reading the user back is the interesting one:

frontend/src/api.js

```
export function getSavedUser() {
  const raw = localStorage.getItem(USER_KEY);
  if (!raw) {
    return null;
  }
  try {
    return JSON.parse(raw);
  } catch {
    return null;
  }
}
```

`JSON.parse` is the opposite of `JSON.stringify`: text in, object out. And it is fussy. If the text is not valid JSON, `JSON.parse` does not return `null` or `undefined`, it **throws**. Throwing means the function stops dead and the error travels up until somebody catches it. Nobody catching it means a white screen: the whole React app crashes.

So why would the stored text ever be broken? A few real ways:

- You opened the browser console during development and typed `localStorage.setItem("lms_user", "hello")` while poking around.
- A browser extension wrote to storage.
- An older version of the app stored something in a different shape, or storage ran out mid-write and saved half a line.

None of those are the user's fault, and none of them are worth a crash. `try { ... } catch { ... }` says: attempt the risky thing, and if it throws, run the `catch` block instead of dying. Here the `catch` returns `null`, which is the same thing the function returns when there is no saved user at all. The app then treats it as "nobody is logged in" and shows the login page. A broken value and a missing value should lead to the same harmless place.

NOTE

`catch {` with no parentheses is modern JavaScript for "I do not care what the error object was, just catch it." You will also see `catch (error) {` when the error is actually used. Both appear in this project.

Clearing is the pair to saving:

frontend/src/api.js

```
export function clearSession() {
  localStorage.removeItem(TOKEN_KEY);
  localStorage.removeItem(USER_KEY);
}
```

This is the whole of "log out" on the frontend. There is no logout endpoint in `backend/backend/urls.py`, so discarding the token is all the browser can do. The token stays valid on the server until it expires, which `SIMPLE_JWT` in `backend/lms/settings.py` puts at eight hours.

CHECKPOINT Once the app is running and you have logged in, open the browser's developer tools (F12 or right-click, Inspect), go to the Console tab, and type `localStorage.getItem("lms_access_token")`. You should see a long string in three parts separated by dots. Type `localStorage.getItem("lms_user")` and you should see something like `{"id":1,"username":"tausif","role":"admin"}`. Now type `localStorage.clear()` and refresh the page: you are back at the login screen. That is exactly what `clearSession()` does.

Making Django's errors readable: readError()

When something goes wrong, Django REST Framework (DRF for short, the library that turns your Django models into an API) sends back JSON describing the problem. The trouble is that it does not always use the same shape. Three different shapes turn up in this project.

Shape one, `detail`. DRF's own errors, for permissions and missing pages, look like this. A teacher or student calling `DELETE /api/teacher/5/` gets a 403 carrying the message from the `AdminWrites` class in `backend/backend/permissions.py`:

```
{ "detail": "Only an admin can change teacher and student records." }
```

Shape two, `error`. The `LoginView` in `backend/backend/views.py` was written by hand and chose its own key:

```
{ "error": "Invalid phone or password" }
```

See Figure 0.2.

Shape three, a field map. When a **serializer** (the Django class that checks incoming data) rejects a field, the answer is an object where each key is a field name and each value is a **list** of complaints about that field. `RegisterSerializer` produces exactly this:

```
{
  "phone": ["This phone number is already registered."],
  "username": ["That username is already taken."]
}
```

Here is why this matters so much. Suppose a page does this:

```
setError(body); // do NOT do this
```

React would try to put an object where a string belongs, and the screen would read:

```
[object Object]
```

That is what JavaScript prints when it is forced to turn an object into text with no instructions. It tells the user nothing, and worse, it tells you nothing when you are debugging.

`readError` exists to turn all three shapes into one plain line of text. Here is its comment and its opening guard:

frontend/src/api.js

```
// Django REST Framework reports errors in a few different shapes:
// {"detail": "..."} a permission or 404 error
// {"error": "..."} the hand-written login view
// {"phone": ["This field is required"]} a serializer rejecting a field
// This turns all of them into one readable line.
function readError(body, response) {
  if (!body || typeof body !== "object") {
    return `Request failed (${response.status})`;
  }
}
```

Note there is no `export` on this one. It is only used inside this file, so it stays private.

The guard handles the case where there is no usable body at all. Some errors come back as an HTML page (Django's yellow debug page) or as nothing. `typeof body !== "object"` checks the kind of value. If it is not an object, there is nothing to dig into, so the function falls back to the **status code**, the number the server sends with every answer: 200 means fine, 400 means bad request, 401 means not signed in, 403 forbidden, 404 not found, 500 server crash. The user at least sees `Request failed (500)` and you know where to look.

The backticks with `${...}` inside are a **template literal**, JavaScript's way of dropping a value into the middle of a string.

The two easy shapes come next:


```
// frontend/src/api.js (readError, continued)
if (typeof body.detail === "string") {
  return body.detail;
}

if (typeof body.error === "string") {
  return body.error;
}
```

If the object has a `detail` that really is a string, that string is the message. Same for `error`. The `typeof` check matters: a serializer could in theory produce a field literally called `detail` holding a list, and returning a list where a string belongs would put us right back at `[object Object]`.

The third shape needs a loop:

```
// frontend/src/api.js (readError, continued)
const parts = [];
for (const [field, value] of Object.entries(body)) {
  const text = Array.isArray(value) ? value.join(" ") : String(value);
  parts.push(field === "non_field_errors" ? text : `${field}: ${text}`);
}
```

Reading it slowly:

- `Object.entries(body)` turns `{ phone: [...], username: [...] }` into a list of pairs: `[["phone", [...]], ["username", [...]]]`.
- `for (const [field, value] of ...)` walks that list, pulling each pair apart into a name and a value in one step. This is called **destructuring**.
- `Array.isArray(value) ? value.join(" ") : String(value)` says: if the value is a list, glue its items together with spaces; if it is anything else, force it to text. The `? :` form is a **ternary**, a compact `if/else` that produces a value.
- `parts.push(...)` adds the finished line to the collection.
- `non_field_errors` is DRF's key for a complaint about the record as a whole rather than about one field. Prefixing that with `non_field_errors:` would be noise, so it goes in bare. Every other field gets its name in front, giving `phone: This phone number is already registered.` The serializers in this project aim their complaints at named fields, so you are unlikely to see `non_field_errors` here, but the branch costs one line and keeps the message clean if one ever turns up.

Then the ending:

```
// frontend/src/api.js (readError, continued)
if (parts.length === 0) {
  return `Request failed (${response.status})`;
}

return parts.join("\n");
}
```

An empty object `{}` produces no parts, so the status-code fallback is used again. Otherwise the lines are joined with `\n`, the newline character, so two rejected fields become two lines.

TIP

`frontend/src/components/Alert.jsx` renders that message with the Tailwind class `whitespace-pre-line`, which tells the browser to honour those `\n` breaks. Without it, two field errors would run together on one line.

CHECKPOINT

Once the Accounts page is wired up, log in as an admin, open it, and create an account using a phone number that already belongs to somebody. The red banner should read `phone: This phone number is already registered.` If it says `[object Object]`, the page is showing the raw body instead of `problem.message`.

handleExpiredToken() and a custom browser event

Tokens expire. This project's `ACCESS_TOKEN_LIFETIME` is eight hours, and `backend/backend/urls.py` has no refresh endpoint, so when Django says "this token is no longer good" there is nothing clever to do. The session has to be dropped and the user sent back to the login page.

Dropping the session is easy, `clearSession()` does it. Sending the user back to `/login` is the awkward part, and it is worth understanding why.

Navigation in this app is done by react-router-dom, through the `useNavigate` hook or the `<Navigate>` component. A **hook** is a function whose name begins with `use` and which plugs into React's own machinery. **Hooks can only be called inside a React component.** `api.js` is not a component. It is a plain file of functions that gets imported by components. It has no idea that a router exists, and giving it one would tie the network layer to the UI layer, which is exactly the tangle we are avoiding.

So `api.js` does not navigate. It shouts, and something that *can* navigate listens.

frontend/src/api.js

```
// There is no refresh endpoint on this backend. When the access token is
// rejected the only thing left to do is drop the session and let the router
// send the user back to /login.
function handleExpiredToken() {
  clearSession();
  window.dispatchEvent(new Event("lms:unauthorised"));
}
```

`window` is the browser tab itself, available to any JavaScript on the page. It has a built-in messaging system. You already know half of it: `onClick` is listening for a `click` event. What is less obvious is that you are allowed to invent your own event names and fire them yourself. That is a **custom event**.

`new Event("lms:unauthorised")` builds an event with a name of our choosing. The `lms:` prefix is a convention to keep our name from colliding with a real browser event or one from a library.

`window.dispatchEvent(...)` fires it. If nobody is listening, nothing happens and no error occurs.

Somebody is listening. In `AuthContext.jsx`, which you will meet properly in the next chapter, this runs when the app starts:

```
// frontend/src/AuthContext.jsx (an excerpt, for context)
// api.js fires this event when the backend rejects the token. There is no
// refresh endpoint, so the session is simply dropped.
useEffect(() => {
  function handleUnauthorised() {
    setUser(null);
    setIsLoggedIn(false);
  }

  window.addEventListener("lms:unauthorised", handleUnauthorised);
  return () =>
    window.removeEventListener("lms:unauthorised", handleUnauthorised);
}, []);
```

`addEventListener` registers the function. When the event fires, React state changes, `ProtectedRoute` notices that `isLoggedIn` is now false, and it renders `<Navigate to="/login" replace state={{ from: location.pathname }} />`. The router does the navigating, as it should, and the page you were on is remembered so the login screen can send you back there.

The line that returns `removeEventListener` is **cleanup**. If the provider is ever removed from the screen (React calls that unmounting), the listener is taken off `window` so it does not linger and fire into a component that is gone.

NOTE

This is one-way. `api.js` depends on nothing from React, and React depends on `api.js` only for the functions it imports. You could take `api.js` and use it in a plain HTML page with no React at all, and it would still work.

A short detour: Promises and `async/await`

The next function uses `async` and `await` on every other line, so here is the minimum you need to read it.

Network requests are **asynchronous**. Your code asks for something and the answer arrives later, maybe in five milliseconds, maybe in two seconds. JavaScript refuses to sit and wait, because sitting and waiting would freeze the whole page: no scrolling, no clicking, nothing.

A **Promise** is the placeholder JavaScript hands you in the meantime. It is an object that means "an answer will go here eventually, or an error will." `fetch(...)` returns a Promise immediately, before the server has said a word.

`await` unwraps a Promise. `const response = await fetch(url)` means "pause this function here until the answer arrives, then put the real answer in `response`." Only this function pauses. The rest of the page keeps running.

`async` is the permission slip. You may only use `await` inside a function marked `async`. Marking a function `async` also means it returns a Promise itself, so whoever calls it must `await` it in turn. That is why the pages have `await teachersApi.list()` inside `async function load()`.

If the awaited Promise fails, the `await` line throws, exactly like `JSON.parse` throwing on bad text. Which means the same `try / catch` you already met handles both. That is the whole reason `async / await` was added to the language: it lets asynchronous code be read top to bottom, with ordinary error handling.

COMMON MISTAKE

Forgetting `await`. `const rows = teachersApi.list()` puts the Promise object itself into `rows`, not the array of teachers. Then `rows.map(...)` fails, or the table silently renders nothing. If a list is mysteriously empty, check for a missing `await` first.

`request()`, line by line

This is the engine. Every other function in the file is a thin wrapper around it.

Start with the signature:

`frontend/src/api.js`

```
async function request(path, options = {}) {  
  const { auth = true, ...rest } = options;
```

`path` is the bit after the base address, for example `/teacher/`. `options` is everything else the caller wants to say, and `= {}` gives it an empty object as a default so `request("/profile/")` with one argument works.

The second line is destructuring again, with two extras. `auth = true` pulls an `auth` property out of `options`, and if there is not one, it defaults to `true`. So every call sends the token unless the caller explicitly says `auth: false`. Defaulting to secure is the right way round. `...rest` is the **rest operator**: it gathers every remaining property (`method`, `body`, `headers`) into a new object called `rest`. The effect is that `auth` is peeled off and kept for us, and `rest` holds only things `fetch` understands. Passing an unknown `auth` property to `fetch` would not crash, but keeping the object clean makes the intent obvious.

Now the headers. **Headers** are the labels on the envelope: small name-and-value pairs sent alongside the request that describe it.

```
// frontend/src/api.js (request, continued)
const headers = { ...(rest.headers ?? {}) };
if (rest.body !== undefined) {
  headers["Content-Type"] = "application/json";
}
```

The first line copies any headers the caller supplied, or starts from an empty object if there were none.

`{ ...x }` is the **spread operator**: it makes a new object holding a copy of `x`'s properties. Copying rather than using the caller's object directly means we never modify something that belongs to somebody else.

`Content-Type: application/json` tells Django "the data I am sending is JSON, please parse it as JSON." It is set only when there actually is a body. A `GET` or a `DELETE` sends no data, and announcing a content type for content that does not exist is meaningless. The check is `!== undefined` rather than a plain truthiness check, so a body that happens to be an empty string still gets the header.

The token comes next:

```
// frontend/src/api.js (request, continued)
const token = getToken();
if (auth && token) {
  headers.Authorization = `Bearer ${token}`;
}
```

`Authorization` is the standard header for proving who you are. The value has to read `Bearer` and then the token, with one space between. "Bearer" means "whoever bears this token is treated as its owner", and it is the word the token library is configured to expect: `"AUTH_HEADER_TYPES": ("Bearer",)` inside the `SIMPLE_JWT` block of `backend/lms/settings.py`.

Both conditions must hold. `auth` false means the caller does not want the header, for example on login. No token means there is nothing to send, which is the normal state before anyone has logged in.

**COMMON
MISTAKE**

Writing `Bearer${token}` with no space, or `JWT ${token}`, or just the raw token. All three produce a 401 with a `detail` message about the Authorization header, and the app will look like the password is wrong when it is not.

Now the call itself:

```
// frontend/src/api.js (request, continued)
let response;
try {
  response = await fetch(BASE_URL + path, { ...rest, headers });
} catch {
  throw new Error(
    "Could not reach the server. Is Django running on http://127.0.0.1:8001?",
  );
}
```

`fetch` throws only when the request never got an answer at all: Django is not running, the machine is offline, the address does not resolve, the browser blocked it. It does **not** throw for 404 or 500. Those are perfectly successful conversations in which the server said something unwelcome, and they are handled further down.

The raw error from a failed `fetch` reads `TypeError: Failed to fetch`, which means nothing to a beginner staring at a red banner. So it is replaced with a sentence that names the most likely cause and tells the reader exactly what to check. `throw new Error("...")` creates an error carrying that text, which the calling page will catch as `problem.message`.

Note `let response` is declared outside the `try`. A variable declared with `const` or `let` inside a block only exists inside that block, and we need `response` on the lines below.

CHECKPOINT

Stop the Django server (`Ctrl+C` in the terminal running `runserver`) and reload the Teachers page. The red banner should read `Could not reach the server. Is Django running on http://127.0.0.1:8001?` Start Django again and reload: the table comes back. That one message will save you more time than any other line in this file.

The rejected-token branch:

```
// frontend/src/api.js (request, continued)
if (response.status === 401 && auth) {
  handleExpiredToken();
  throw new Error("Your session has expired. Please log in again.");
}
```

401 means "not authenticated": Django did not accept the token. The `&& auth` half is the important part. The three calls that pass `auth: false` (login and the two password-reset calls) never sent a token in the first place, so a refusal from them cannot mean "your session ran out". Without that check, a failed call on a page where nobody is logged in could fire `lms:unauthorised` and clear a session that does not exist.

On this backend a wrong password does not even come back as 401: `LoginView` answers 400 with `{"error": "Invalid phone or password"}`, which `readError` turns into that exact sentence for the red banner. So both paths end up saying something true.

When the branch does run, two things happen: the session is dropped and the event fires (so the router moves the user), and an error is thrown so the page's `catch` block stops rather than carrying on with no data.

The empty-answer branch:

```
// frontend/src/api.js (request, continued)
// DELETE returns 204 with an empty body, so there is nothing to parse.
if (response.status === 204) {
  return null;
}
```

204 is "No Content." It is what DRF returns after a successful delete: the operation worked, and there is nothing to say about it. Trying to read a body here would give an empty string, and `JSON.parse("")` throws. Returning `null` early sidesteps all of that. Delete callers ignore the return value anyway.

Reading the body, carefully:

```
// frontend/src/api.js (request, continued)
const text = await response.text();
let body = null;
if (text) {
  try {
    body = JSON.parse(text);
  } catch {
    body = null;
  }
}
```

The obvious way to do this would be `const body = await response.json()`. That is avoided on purpose. `response.json()` throws if the answer is not JSON, and there are real situations where it is not: Django's HTML debug page on a 500 error, or an empty body on some other status.

So the answer is read as raw text first, which never throws. If the text is empty, `body` stays `null`. If there is text, parsing is attempted inside a `try`, and a failure leaves `body` as `null` rather than crashing. `readError` already knows what to do with a `null` body: it falls back to `Request failed (500)`. The user gets a status code instead of a white screen.

Finally:

```
// frontend/src/api.js (request, continued)
if (!response.ok) {
  throw new Error(readError(body, response));
}

return body;
}
```

`response.ok` is a shortcut for "the status is in the 200s." Anything else means the server refused, and the refusal is turned into a readable line by `readError` and thrown. Otherwise the parsed data is returned, and the page gets its array of teachers.

Look at what this design gives every page: a successful call returns data directly, and *any* failure arrives as a thrown error with a sensible `.message`. That is why every page can be written as one shape:

```
// frontend/src/pages/Teachers.jsx (an excerpt, for context)
try {
  // The list endpoint is not paginated, so this is a plain array.
  setTeachers(await teachersApi.list());
  setError('');
} catch (problem) {
  setError(problem.message);
} finally {
  setIsLoading(false);
}
```

Server down, token rejected, 403 from permissions, a rejected phone field: all of them land in the same `catch`, all of them are already a readable sentence.

The auth calls

With `request` written, the login and account functions are short. Each one names an endpoint, a method, and a body.

An **HTTP method** is the verb of the request. `GET` reads. `POST` creates or performs an action. `PUT` replaces a record. `PATCH` changes part of one. `DELETE` removes one.

Login first:

frontend/src/api.js

```
// --- auth -----  
  
export function login(phone, password) {  
  return request("/login/", {  
    method: "POST",  
    body: JSON.stringify({ phone, password }),  
    auth: false,  
  });  
}
```

Three things to notice. The credentials go in the body, never in the URL, because URLs end up in browser history and server logs. `{ phone, password }` is shorthand for `{ phone: phone, password: password }`. And `auth: false`, because there is no token yet. Sending an `Authorization` header here would be sending nothing useful, and if a stale token were in storage it would be sending something actively wrong.

Also notice: **you log in with a phone number**, not a username and not an email. That is how the Django `LoginView` looks accounts up, with `Profile.objects.get(phone=phone)`.

Registration is the one that surprises people:

frontend/src/api.js

```
// Creating an account is an admin job, so this call carries the admin's own  
// token. There is no public sign-up.  
export function register(details) {  
  return request("/register/", {  
    method: "POST",  
    body: JSON.stringify(details),  
  });  
}
```

No `auth: false` here. In most apps registration is public, so this looks like a bug. It is not. In this LMS there is no public sign-up: an admin creates accounts on the Accounts page, and `/api/register/` is guarded by the `IsAdmin` permission class you wrote in `backend/backend/permissions.py`. The token has to go along, or Django would answer 401 to its own admin.

The `details` object is built by `Accounts.jsx` and carries `username`, `email`, `password`, `phone`, `role`, `first_name` and `last_name`, matching `RegisterSerializer`.

Reading your own account:

frontend/src/api.js

```
export function fetchProfile() {  
  return request("/profile/");  
}
```

One argument, no options. `method` defaults to `GET` inside `fetch`, `auth` defaults to `true` inside `request`, and there is no body so no `Content-Type` is set. The whole request is described by its path.

Editing your own account:

frontend/src/api.js

```
// Your own name, email and phone. The server only ever edits the account the  
// token belongs to, and it will not accept `role` from here.  
export function updateProfile(values) {  
  return request("/profile/", {  
    method: "PATCH",  
    body: JSON.stringify(values),  
  });  
}
```

`PATCH` and not `PUT`, because this sends a few fields rather than a whole replacement record. There is no user id in the URL: `ProtectedView.patch` works on `request.user`, so the server decides whose account to edit from the token. That is deliberate, and it means no one can edit somebody else's profile by changing a number in the URL. `ProfileUpdateSerializer` accepts only `first_name`, `last_name`, `email` and `phone`, so sending `{"role": "admin"}` is ignored rather than obeyed.

Changing a password:

frontend/src/api.js

```
// Needs the current password, not just a live session.  
export function changePassword(values) {  
  return request("/change-password/", {  
    method: "POST",  
    body: JSON.stringify(values),  
  });  
}
```

`values` carries `current_password`, `new_password` and `confirm_password`, the three fields `ChangePasswordSerializer` declares. Requiring the old password means a browser tab left logged in on a shared machine is not enough to take the account over.

And the two reset calls, for a user who cannot log in at all:

frontend/src/api.js

```
export function requestPasswordReset(email) {
  return request("/password-reset/", {
    method: "POST",
    body: JSON.stringify({ email }),
    auth: false,
  });
}

export function confirmPasswordReset(details) {
  return request("/password-reset-confirm/", {
    method: "POST",
    body: JSON.stringify(details),
    auth: false,
  });
}
```

Both are `auth: false`, because somebody who has forgotten their password has no token by definition. The first takes an email and asks Django to send a link. The second takes the `uid`, `token`, `new_password` and `confirm_password` that `PasswordResetConfirmSerializer` expects, where `uid` and `token` come out of the link.

Here is the whole set at a glance:

FUNCTION	METHOD	PATH	SENDS TOKEN?
<code>login(phone, password)</code>	POST	<code>/login/</code>	No (<code>auth: false</code>)
<code>register(details)</code>	POST	<code>/register/</code>	Yes (admin only)
<code>fetchProfile()</code>	GET	<code>/profile/</code>	Yes
<code>updateProfile(values)</code>	PATCH	<code>/profile/</code>	Yes
<code>changePassword(values)</code>	POST	<code>/change-password/</code>	Yes
<code>requestPasswordReset(email)</code>	POST	<code>/password-reset/</code>	No (<code>auth: false</code>)
<code>confirmPasswordReset(details)</code>	POST	<code>/password-reset-confirm/</code>	No (<code>auth: false</code>)

NOTE

`requestPasswordReset` and `confirmPasswordReset` are exported but no page in the app calls them yet. The backend endpoints exist in `backend/backend/urls.py` and work, and the two functions are ready for the reset screens. Leaving them in the API file is fine, since the file's job is to describe what the backend can do.

The resource() factory

Eight resources are left: teachers, students, courses, enrollments, lessons, assignments, submissions and results. Every one of them supports the same four operations, and every one follows the same URL pattern. Written out longhand that would be thirty-two nearly identical functions, differing only in one word.

Instead, one function builds them:

frontend/src/api.js

```
// --- resources -----

// Every resource has the same four calls, so they are built from the path
// once instead of being written out eight times over.
function resource(path) {
  return {
    list: () => request(`/${path}/`),
    create: (values) =>
      request(`/${path}/`, { method: "POST", body: JSON.stringify(values) }),
    update: (id, values) =>
      request(`/${path}/${id}/`, {
        method: "PUT",
        body: JSON.stringify(values),
      }),
    remove: (id) => request(`/${path}/${id}/`, { method: "DELETE" }),
  };
}
```

`resource` takes a path segment such as `"teacher"` and returns an object holding four functions. Those functions are written as **arrow functions**: `(values) => request(...)` is a shorter way of writing `function (values) { return request(...); }`. With no braces, the value of the expression is returned automatically.

The four functions can still see the `path` variable that was passed in, even after `resource` has finished running. A function that remembers the variables from where it was created is called a **closure**. It is what makes this pattern work: each returned object carries its own `path` around with it forever.

So `resource("teacher")` produces functions that hit `/teacher/` and `/teacher/5/`, while `resource("course")` produces functions that hit `/course/` and `/course/5/`, and the code was written once.

`update` uses `PUT` rather than `PATCH`. `PUT` replaces the whole record, so the pages send every field of the form each time, which is exactly what the eight resource pages do.

`remove` is called `remove` and not `delete` because `delete` is a reserved word in JavaScript and cannot be used as a bare property name in this position.

Now the eight exports:

frontend/src/api.js

```
export const teachersApi = resource("teacher");
export const studentsApi = resource("student");
export const coursesApi = resource("course");
export const enrollmentsApi = resource("enrollment");
export const lessonsApi = resource("lesson");
export const assignmentsApi = resource("assignment");
export const submissionsApi = resource("submission");

// Note the naming: /submission/ is singular but /results/ is plural.
export const resultsApi = resource("results");
```

That last comment is doing real work. Seven of the eight backend paths are singular: `/teacher/`, `/student/`, `/course/`, `/enrollment/`, `/lesson/`, `/assignment/`, `/submission/`. The eighth is plural: `/results/`. That is not a typo in the frontend, it is what `backend/backend/urls.py` actually registers:

```
# backend/backend/urls.py (two lines, for reference)
path('submission/', SubmissionListCreateView.as_view(), name='submission-list'),
path('results/', ResultsListCreateView.as_view(), name='result-list'),
```

Writing `resource("result")` looks tidier and gives you a 404 on every call to the Results page. The comment is there so the next person does not "fix" it.

EXPORTED OBJECT	REQUESTS IT MAKES
teachersApi	/teacher/ , /teacher/{id}/
studentsApi	/student/ , /student/{id}/
coursesApi	/course/ , /course/{id}/
enrollmentsApi	/enrollment/ , /enrollment/{id}/
lessonsApi	/lesson/ , /lesson/{id}/
assignmentsApi	/assignment/ , /assignment/{id}/
submissionsApi	/submission/ , /submission/{id}/
resultsApi	/results/ , /results/{id}/ (plural)

And every one of them offers the same four calls:

CALL	METHOD	URL	RETURNS
<code>.list()</code>	GET	<code>/teacher/</code>	array of rows
<code>.create(values)</code>	POST	<code>/teacher/</code>	the new row
<code>.update(id, values)</code>	PUT	<code>/teacher/5/</code>	the updated row
<code>.remove(id)</code>	DELETE	<code>/teacher/5/</code>	<code>null</code> (204)

`.list()` returns a plain array, not an object with a `results` key inside it, because `REST_FRAMEWORK` in `backend/lms/settings.py` sets no pagination class. Pages write `teachers.map(...)`, never `teachers.results.map(...)`.

From a page it reads like this:

```
// frontend/src/pages/Teachers.jsx (an excerpt, for context)
if (editingId === 0) {
  await teachersApi.create(values);
  setNotice("Teacher added.");
} else {
  await teachersApi.update(editingId, values);
  setNotice("Teacher updated.");
}
```

No URL, no token, no header, no status check. All of that is behind `request`.

See Figure 4.1.

CHECKPOINT With Django running, the Vite dev server running, and yourself logged in, open the browser console on any page of the app and run: `javascript const { teachersApi } = await import("/src/api.js"); console.log(await teachersApi.list());` You should see an array of teacher objects, each with `id`, `name`, `email`, `subject` and `is_active`, which are exactly the fields `TeacherSerializer` lists. That proves `BASE_URL`, the token header and the JSON parsing are all working, before a single page has been written.

TIP Keep the Network tab of your developer tools open while clicking around. Every call from this file shows up there. Click one and look at the Headers panel: you can see the `Authorization: Bearer ...` line that `request()` added, and the Response panel shows the exact JSON that `readError()` would be reading.

flash.js: short messages that clean up after themselves

After a save, the user should see something. A green "Teacher added." banner is enough. But a banner that never disappears is worse than none at all: add three teachers and you are looking at a stack of stale congratulations.

Every page could set a timer and clear its own message, but that is the same eight lines eight times over. So it goes into one small hook.

```
touch src/flash.js
```

A hook, as mentioned earlier, is a function whose name begins with `use` and which is allowed to use React features like state and effects. `useState` and `useEffect` are React's own. `useFlash` is ours, built out of theirs.

```
frontend/src/flash.js
```

```
import { useEffect, useState } from "react";

// "Saved", "Updated", "Deleted" – a short message that clears itself, so no
// page has to remember to take it back down.
export function useFlash(millisecondsOnScreen = 4000) {
  const [message, setMessage] = useState("");
```

`useState("")` gives back two things: the current message, and a function to change it. Starting at an empty string means no banner, because `Alert.jsx` returns `null` when `children` is empty.

The default of `4000` is four seconds. Callers can ask for longer.

The effect is where the disappearing happens:

```
// frontend/src/flash.js (useFlash, continued)
useEffect(() => {
  if (!message) {
    return;
  }

  const timer = setTimeout(() => setMessage(""), millisecondsOnScreen);
  return () => clearTimeout(timer);
}, [message, millisecondsOnScreen]);

return [message, setMessage];
}
```

`useEffect` runs code after React has drawn the screen. The array at the end, `[message, millisecondsOnScreen]`, is the **dependency list**: React re-runs the effect whenever one of those values changes. So every new message starts a fresh countdown.

`if (!message) return;` skips everything when there is nothing on screen. No message, no timer.

`setTimeout(fn, ms)` is the browser's "run this once, later." Here it sets the message back to empty after four seconds, and React re-renders without the banner.

The returned function is the cleanup, and it matters more than it looks. React runs it before the *next* run of the effect, and again when the component is removed from the screen. Two real problems it prevents:

- The user saves twice quickly. Without cleanup, the first timer is still counting and would blank the second message early. `clearTimeout` cancels it, so the second message gets its full four seconds.
- The user navigates away one second after saving. Without cleanup, the timer still fires three seconds later and calls `setMessage` on a component that no longer exists.

The last line returns `[message, setMessage]` as an array, deliberately matching the shape of `useState`. That is why using it looks so ordinary:

```
// frontend/src/pages/Teachers.jsx (an excerpt, for context)
const [notice, setNotice] = useFlash();
```

Swap `useState("")` for `useFlash()` and the message now cleans itself up. Nothing else in the page changes. The Accounts page asks for longer, because its message contains a phone number the admin may want to write down:

```
// frontend/src/pages/Accounts.jsx (an excerpt, for context)
const [notice, setNotice] = useFlash(8000);
```

The screenshot shows a web application interface for an LMS. On the left is a sidebar with a menu: LMS, Dashboard, Teachers (selected), Students, Courses, Enrollments, Lessons, Assignments, Submissions, Results, and Accounts. At the bottom of the sidebar, it shows a user profile for 'nadia' with an 'Admin' role and a 'Log out' button. The main content area is titled 'Teachers' with the subtitle 'The people who teach courses.' A green banner at the top of the main area says 'Teacher added.' Below this is a section titled '5 teachers' with a '+ Add teacher' button. It contains a table with 5 rows of teacher data. Each row has columns for ID, NAME, EMAIL, SUBJECT, ACTIVE, and ACTION. The ACTION column contains edit and delete icons. The table data is as follows:

ID	NAME	EMAIL	SUBJECT	ACTIVE	ACTION
1	Imran Hossain	imran@lms.test	Web Development	Yes	
2	Farhana Islam	farhana@lms.test	Databases	Yes	
3	Rakib Chowdhury	rakib@lms.test	Mathematics	Yes	
4	Tanvir Ahmed	tanvir@lms.test	Data Science	No	
6	Shirin Sultana	shirin@lms.test	User Interface Design	Yes	

Figure 8.1 The green "Teacher added." banner that useFlash clears on its own after four seconds.

CHECKPOINT Add a teacher. The green "Teacher added." banner appears, and disappears on its own after about four seconds. Now add two teachers in quick succession: the second banner should stay up for its own full four seconds, not vanish early on the first one's timer. That is `clearTimeout` doing its job.

The one message that has to survive a redirect

There is a second problem in `flash.js`, and it is a different shape.

When a user changes their password on the Profile page, the app signs them out on purpose. The old token stays technically valid on the server until it expires, so ending the session is the honest thing to do. Being dumped on the login screen with no explanation, however, reads like a failure, when in fact the change worked.

So a message needs to travel from the Profile page to the login page, across a sign-out.

The obvious tool is router state. `react-router-dom` lets you attach data to a navigation, and `Login.jsx` already reads `location.state?.from` that way. It does not work here, and the comment in the real file explains exactly why:

frontend/src/flash.js

```
const HANDOFF_KEY = "lms_handoff_notice";

// A message that has to survive both a page change and a sign-out.
//
// Router state cannot carry this. Signing out re-renders whatever page you
// were on inside ProtectedRoute, which fires its own redirect to /login and
// replaces the history entry – throwing away any state that came with it. So
// the message goes somewhere the redirect cannot touch, and the login page
// takes it exactly once.
```

Read that carefully, because it is a genuinely confusing sequence. `logout()` sets `isLoggedIn` to false. That re-renders the Profile page, which is wrapped in `ProtectedRoute`. `ProtectedRoute` sees a logged-out user and returns a `<Navigate to="/login" replace ... />` all by itself. That redirect happens first, and it carries only the `from` path, not our message. Our own `navigate("/login")` call, state or no state, is not the navigation that wins.

The fix is to put the message somewhere no redirect can reach:

frontend/src/flash.js

```
export function leaveNoticeForLogin(message) {
  sessionStorage.setItem(HANDOFF_KEY, message);
}

export function takeNoticeLeftForLogin() {
  const message = sessionStorage.getItem(HANDOFF_KEY);
  if (message) {
    sessionStorage.removeItem(HANDOFF_KEY);
  }
  return message ?? "";
}
```

`sessionStorage` is `localStorage`'s short-lived twin. Same two functions, `setItem` and `getItem`, same text-only storage. The difference is lifetime: `sessionStorage` is emptied when the tab is closed. A one-time note between two screens should not still be sitting there tomorrow, so `sessionStorage` is the right shelf.

`takeNoticeLeftForLogin` is named "take" rather than "get" for a reason: reading removes. Without the `removeItem`, the login page would show "Password changed. Please log in again." every time it was visited for the rest of the session, long after it stopped being true. The `?? ""` at the end turns a missing value into an empty string, so `Alert` gets text it can handle rather than `null`.

The two halves in the real pages. Profile leaves the note, then signs out:

```
// frontend/src/pages/Profile.jsx (an excerpt, for context)
leaveNoticeForLogin("Password changed. Please log in again.");
logout();
navigate("/login", { replace: true });
```

And Login picks it up:

```
// frontend/src/pages/Login.jsx (an excerpt, for context)
// Left by the profile page after a password change, so the sign-out that
// follows does not look like something went wrong. Read once, on the first
// render, and cleared as it is read.
const [notice] = useState(() => takeNoticeLeftForLogin());
```

Passing a function to `useState` rather than a value is called **lazy initial state**. React calls that function once, on the very first render, and never again. If it were written `useState(takeNoticeLeftForLogin())`, the function would run on every render, and since it deletes as it reads, the note would be gone before it ever made it to the screen. Only `notice` is destructured out, with no setter, because nothing on the login page ever changes this message.

CHECKPOINT Log in, go to Your profile, change your password. You should be sent to the login screen with a green banner reading "Password changed. Please log in again." Log in with the new password. Now navigate to the login page again by any other route: the green banner is gone. That is `removeItem` inside `takeNoticeLeftForLogin`.

`flash.js` is 39 lines, so here is the finished file whole:

```

import { useEffect, useState } from "react";

const HANDOFF_KEY = "lms_handoff_notice";

// A message that has to survive both a page change and a sign-out.
//
// Router state cannot carry this. Signing out re-renders whatever page you
// were on inside ProtectedRoute, which fires its own redirect to /login and
// replaces the history entry – throwing away any state that came with it. So
// the message goes somewhere the redirect cannot touch, and the login page
// takes it exactly once.
export function leaveNoticeForLogin(message) {
  sessionStorage.setItem(HANDOFF_KEY, message);
}

export function takeNoticeLeftForLogin() {
  const message = sessionStorage.getItem(HANDOFF_KEY);
  if (message) {
    sessionStorage.removeItem(HANDOFF_KEY);
  }
  return message ?? "";
}

// "Saved", "Updated", "Deleted" – a short message that clears itself, so no
// page has to remember to take it back down.
export function useFlash(millisecondsOnScreen = 4000) {
  const [message, setMessage] = useState("");

  useEffect(() => {
    if (!message) {
      return;
    }

    const timer = setTimeout(() => setMessage(""), millisecondsOnScreen);
    return () => clearTimeout(timer);
  }, [message, millisecondsOnScreen]);

  return [message, setMessage];
}

```

`api.js` is 210 lines, too long to reprint, but you have now built every part of it in order: the base address, the four session helpers, `readError`, `handleExpiredToken`, `request`, the seven auth calls, `resource`, and the eight exported objects. Read it end to end in your editor once before moving on. It should look familiar all the way down.

Two things worth knowing about localStorage

NOTE

A token in `localStorage` can be read by any JavaScript running on the page. That includes a badly chosen npm package. The safer alternative is an `HttpOnly` cookie, which is a cookie the browser will send to the server but will not let JavaScript read. Using one needs extra work on the Django side, including protection against cross-site request forgery, where another website tricks your browser into sending a request with your cookie attached. This project uses `localStorage` because it is the simplest thing that works while you are learning. Know the trade-off before you put something like this in front of real users.

COMMON MISTAKE

`localStorage` is stored per origin, and `http://localhost:5180` and `http://127.0.0.1:5180` count as two different origins even though they reach the same server. Log in at one address and you will appear logged out at the other, with two separate tokens sitting in two separate stores. Pick one address and stay on it.

What you have now

- `frontend/src/api.js`, the single file that every network call in the app goes through
- A `BASE_URL` that reads `VITE_API_URL` from a `.env` file, with `http://127.0.0.1:8001/api` as the fallback
- `getToken`, `saveSession`, `getSavedUser` and `clearSession`, keeping the session in `localStorage` so a refresh does not log the user out, with a `try/catch` so corrupt stored text cannot crash the app
- `readError()`, turning DRF's three error shapes into one readable line, so users see `phone: This phone number is already registered.` instead of `[object Object]`
- `handleExpiredToken()` firing a custom `lms:unauthorised` window event, letting `api.js` end a session without knowing that a router exists
- `request()`, which attaches the `Bearer` token, sets `Content-Type` only when there is a body, handles 401 and 204, parses the body defensively, and throws a readable `Error` for anything that failed
- Seven auth functions, three of them passing `auth: false` because there is no token yet, and `register` deliberately passing the admin's token because there is no public sign-up
- `resource()` producing `list`, `create`, `update` and `remove` for a path, and the eight exported objects built from it, including the plural `resource("results")`
- `frontend/src/flash.js` with `useFlash()`, a self-clearing green banner with a cancelled timer on cleanup
- `leaveNoticeForLogin()` and `takeNoticeLeftForLogin()`, carrying one message through a sign-out in `sessionStorage`, where the router's own redirect cannot lose it

Signing In: Auth Context, Protected Routes and the Login Page

In the last chapters you built a backend that hands out a token when you post a phone number and a password to `/api/login/`, and a frontend that can talk to it through `src/api.js`. What the frontend still cannot do is *remember* who you are. Every page starts from nothing. There is no login screen, no way to keep people out of pages they should not see, and no place that says "the person using this browser is Tausif, and he is an admin".

A **token** here is just a long string of characters the server gives you after you prove who you are. You send it back with every later request, and the server reads it instead of asking for your password again.

This chapter builds that missing piece. By the end you will have six files working together:

FILE	JOB
<code>src/auth.js</code>	Creates the shared box that holds the signed-in user, plus the <code>useAuth()</code> hook
<code>src/AuthContext.jsx</code>	Fills that box: reads localStorage, logs in, logs out
<code>src/components/ProtectedRoute.jsx</code>	Blocks a page if you have no token, or the wrong role
<code>src/App.jsx</code>	The route table: which URL shows which page
<code>src/Sidebar.jsx</code>	The left menu and the layout every signed-in page sits inside
<code>src/pages/Login.jsx</code>	The login form itself

9.1 The problem: everybody needs to know who is signed in

Think about what the app has to do once a person is signed in.

The sidebar must print their username and their role badge. The Teachers page must hide the Add and Delete buttons from a student. The Accounts page must refuse to open at all for anyone who is not an admin. `api.js` must attach the token to every request. The Log out button must throw the token away.

That is five separate jobs, scattered across the app, all needing the same two facts: *is anyone signed in*, and *who*.

The obvious first idea is to keep the user in `App.jsx` and pass it down as a **prop** (a value handed from a parent component to a child, written like an HTML attribute):

```
// this is NOT what we build - it is the problem
<Sidebar user={user}>
  <Teachers user={user} />
</Sidebar>
```

That works for one level. It falls apart at three. `App` gives the user to `Sidebar`, `Sidebar` gives it to whatever page is showing, that page gives it to a table component, the table gives it to a row component, and the row is the only one that actually wanted it. Every component in the middle has to accept a prop it does not care about and pass it along. This is called **prop drilling**, and the pain is real: add one new field to the user object and you edit ten files.

React's answer is **Context**. A context is a box you put a value into once, high up in the tree, and any component underneath can reach in and take it out directly. Nothing in the middle has to know it exists. That is what `auth.js` and `AuthContext.jsx` are.

NOTE

Context solves *sharing*, not *storing*. The value inside the box still lives in normal `useState` state, and it still disappears when the browser tab reloads. Keeping it across a reload is `localStorage`'s job, and you will see exactly how in section 9.3.

9.2 auth.js: the box and the hook

Make the file first. From the `frontend` folder:

```
touch src/auth.js
```

Two imports. `createContext` makes a new context object. `useContext` is the hook that reads one. (A **hook** is a React function whose name starts with `use`, called from inside a component to give that component extra abilities.)

```
frontend/src/auth.js
```

```
import { createContext, useContext } from "react";
```

Now create the context itself. The argument to `createContext` is the **default value**, used only when a component reads the context without any provider above it. Passing `null` is deliberate: it gives us a way to catch that mistake in a moment.

The real file puts a comment above that line, explaining why this tiny file exists at all instead of living inside `AuthContext.jsx`:

frontend/src/auth.js

```
// The context and its hook live apart from AuthContext.jsx so that file only
// exports a component. Vite's fast refresh gives up on any file that mixes
// components with other exports.
export const AuthContext = createContext(null);
```

Fast refresh is the Vite feature that swaps a changed component into the running page without reloading it, so text you have typed into a form survives the edit. It can only do that when it is sure a file exports components and nothing else. Put `export const AuthContext = ...` next to `export function AuthProvider(...)` and Vite loses that certainty, so every save does a full page reload, and a full page reload throws away whatever you had typed. Splitting the file costs eight lines and keeps the better behaviour.

COMMON MISTAKE

Moving `createContext` back into `AuthContext.jsx` "to keep things together". Nothing breaks visibly, so people do it. Then they spend a week wondering why the dev server reloads the whole page every time they touch auth code. The `eslint-plugin-react-refresh` package listed in `frontend/package.json` is there to warn you about exactly this.

Last, the hook. Any component that wants the user calls `useAuth()`.

frontend/src/auth.js

```
export function useAuth() {
  const value = useContext(AuthContext);
  if (!value) {
    throw new Error("useAuth must be used inside an AuthProvider");
  }
  return value;
}
```

The `if (!value)` check is the payoff of passing `null` to `createContext`. If a component calls `useAuth()` and there is no `AuthProvider` anywhere above it, `useContext` returns that `null` default. Without the check, the component would quietly get `undefined` and crash later on something like `user.role`. With it, you get a clear error message naming the actual problem. Crashing loudly at the source beats a confusing `Cannot read properties of undefined` twenty lines away.

That is the whole file, all fourteen lines of it:

frontend/src/auth.js

```
import { createContext, useContext } from "react";

// The context and its hook live apart from AuthContext.jsx so that file only
// exports a component. Vite's fast refresh gives up on any file that mixes
// components with other exports.
export const AuthContext = createContext(null);

export function useAuth() {
  const value = useContext(AuthContext);
  if (!value) {
    throw new Error("useAuth must be used inside an AuthProvider");
  }
  return value;
}
```

9.3 AuthContext.jsx: filling the box

```
touch src/AuthContext.jsx
```

This file exports one component, `AuthProvider`. It wraps the whole app, holds the state, and puts that state into the context. We will build it in six small pieces.

THE IMPORTS

frontend/src/AuthContext.jsx

```
import { useEffect, useState } from "react";
import { AuthContext } from "../auth.js";
import {
  clearSession,
  fetchProfile,
  getSavedUser,
  getToken,
  login as loginRequest,
  saveSession,
} from "../api.js";
```

Everything that touches the network or `localStorage` comes from `api.js`, the file you wrote earlier. `login as loginRequest` renames the import while bringing it in, because this file is about to define its own function called `login`, and two similar names in one file is asking for trouble.

THE STATE, AND WHY A REFRESH DOES NOT SIGN YOU OUT

frontend/src/AuthContext.jsx

```
export function AuthProvider({ children }) {  
  // The token is read back out of localStorage on the first render so a page  
  // refresh does not log you out.  
  const [user, setUser] = useState(() => getSavedUser());  
  const [isLoggedIn, setIsLoggedIn] = useState(() => Boolean(getToken()));
```

`children` is the special prop React fills with whatever you put between the opening and closing tags of a component. `<AuthProvider><App /></AuthProvider>` means `children` is `<App />`.

Look closely at the two `useState` calls. They are given a **function**, not a value. `useState(() => getSavedUser())` is a **lazy initialiser**: React calls that function once, on the very first render, and never again. Compare it with `useState(getSavedUser())`, which would run `getSavedUser()` on *every* render and throw the result away every time except the first. Reading `localStorage` is not expensive, but the habit matters and the intent is clearer.

This is the code that makes refreshing the page survivable. `getSavedUser()` reads the `lms_user` key out of `localStorage` and turns the stored text back into an object. `getToken()` reads `lms_access_token`. `localStorage` is a small store the browser keeps on disk, one per website, and it outlives tab closes and reboots. So when the browser reloads and React starts again from zero, the very first render already has the user back, and `isLoggedIn` is already `true`. The user never sees a flash of the login page.

`Boolean(getToken())` turns the token string into a plain true or false. `localStorage.getItem` returns `null` when the key is missing, and `Boolean(null)` is `false`.

TIP

Open the browser DevTools (F12), go to the **Application** tab, and click **Local Storage** in the left column. After you log in you will see two rows:

`lms_access_token` holding a long string with two dots in it, and `lms_user` holding something like `{"id":1,"username":"tausif","role":"admin"}`. Delete both rows, refresh, and you are signed out. That is the entire session, visible and editable.

LISTENING FOR A DEAD TOKEN

The backend does not tell the frontend when a token expires. It just starts answering `401 Unauthorized` (the HTTP status code meaning "I do not accept who you say you are"). Your `api.js` already handles that in one place: on a 401 it calls `handleExpiredToken()`, which clears the session and fires a browser event named `lms:unauthorised`. Something has to listen for that event and flip the React state, otherwise the app still *thinks* you are signed in while every request fails.

frontend/src/AuthContext.jsx

```
// api.js fires this event when the backend rejects the token. There is no
// refresh endpoint, so the session is simply dropped.
useEffect(() => {
  function handleUnauthorised() {
    setUser(null);
    setIsLoggedIn(false);
  }

  window.addEventListener("lms:unauthorised", handleUnauthorised);
  return () =>
    window.removeEventListener("lms:unauthorised", handleUnauthorised);
}, []);
```

`useEffect` runs code after the component has rendered. The `[]` at the end is the **dependency array**: an empty one means "run this once when the component first appears, and never again". The function you `return` from an effect is the **cleanup**, and React runs it when the component goes away. Adding a listener without removing it later leaves dead listeners piling up in memory, so every `addEventListener` in your code should have a matching `removeEventListener` in the cleanup.

Why an event at all, instead of `api.js` calling `setIsLoggedIn` directly? Because `api.js` is a plain JavaScript file with no React in it. It cannot reach React state. A browser event is a message any code can shout and any code can listen for, which lets the two files cooperate without importing each other.

BACK-FILLING A ROLE THAT IS NOT THERE

Roles were added to this project after the first version shipped. A browser that logged in before that change has an `lms_user` object in `localStorage` with `id` and `username` but no `role`. Treating that person as having no permissions would hide every button from them for no reason.

frontend/src/AuthContext.jsx

```
// A session saved before roles existed has no role on it. Rather than treat
// that account as having no permissions, ask /profile/ what it is.
useEffect(() => {
  if (!isLoggedIn || user?.role) {
    return;
  }

  let stillMounted = true;
```

The guard reads: if nobody is signed in, do nothing; if we already know the role, do nothing.

`user?.role` uses **optional chaining**, the `?.` operator, which gives `undefined` instead of crashing when `user` is `null`.

`stillMounted` handles a race. The request below takes time. If the user clicks Log out while it is still travelling, this component's state should not be written to afterwards. The cleanup sets the flag to `false` and the response is ignored.

frontend/src/AuthContext.jsx

```
async function fillInTheRole() {
  try {
    const data = await fetchProfile();
    if (!stillMounted) {
      return;
    }
    const nextUser = {
      id: data.user.user_id,
      username: data.user.username,
      role: data.user.role,
    };
    saveSession(getToken(), nextUser);
    setUser(nextUser);
  } catch {
    // A dead token is already handled by api.js; nothing to add here.
  }
}
```

`fetchProfile()` hits `GET /api/profile/`, which the Django `ProtectedView` answers with `{"message": "successfully fetched this user", "user": {...}}`. The fields you want are one level down, inside `data.user`, and the id field there is called `user_id`, not `id`. `saveSession` writes the repaired object back to `localStorage` so the next page load does not have to ask again.

The empty `catch` is not laziness. If the token is dead, `api.js` has already fired `lms:unauthorised` and the effect above has already signed the user out. There is nothing useful left for this code to do.

frontend/src/AuthContext.jsx

```
fillInTheRole();
return () => {
  stillMounted = false;
};
}, [isLoggedIn, user?.role]);
```

The dependency array `[isLoggedIn, user?.role]` means the effect re-runs whenever either value changes. Once the role is filled in, `user?.role` holds a real string, the effect runs one more time, hits the guard on the first line, and stops. No loop.

LOGIN: THE NESTED TOKENS

frontend/src/AuthContext.jsx

```
async function login(phone, password) {
  const data = await loginRequest(phone, password);

  // The tokens are nested. data.access does not exist.
  const nextUser = {
    id: data.user_id,
    username: data.username,
    role: data.role,
  };
  saveSession(data.tokens.access, nextUser);

  setUser(nextUser);
  setIsLoggedIn(true);
}
```

`loginRequest` posts to `/api/login/` with `auth: false`, meaning no Authorization header, which is correct because you do not have a token yet. Django's `LoginView` replies with this shape:

```
{
  "message": "Login successful",
  "user_id": 1,
  "username": "tausif",
  "role": "admin",
  "tokens": { "refresh": "...", "access": "..." }
}
```

Note where things sit. `user_id`, `username` and `role` are at the top level. The tokens are one level deeper, under `tokens`. That is why the code says `data.tokens.access`.

COMMON MISTAKE

Writing `saveSession(data.access, nextUser)`. Many tutorials use a backend that returns `{"access": "...", "refresh": "..."}` at the top level, and the muscle memory is strong. Here `data.access` is `undefined`, so `localStorage` stores the literal text `"undefined"` as your token, login *appears* to work, and then every single page fails with a 401 that instantly signs you back out. If logging in bounces you straight back to the login screen, check this line first.

Then three things happen in order: the token and user go into `localStorage` (survives refresh), `setUser` puts the user into React state, `setIsLoggedIn(true)` flips the gate. Any component calling `useAuth()` re-renders with the new values.

The `refresh` token is thrown away. This project never uses it, because there is no refresh endpoint in `backend/backend/urls.py`.

LOGOUT: THERE IS NOTHING TO CALL

frontend/src/AuthContext.jsx

```
// There is no logout endpoint. Discarding the token is all the frontend
// can do; it stays valid on the server until it expires.
function logout() {
  clearSession();
  setUser(null);
  setIsLoggedIn(false);
}
```

This surprises beginners, so it is worth being blunt about it. A **JWT** (JSON Web Token, the kind of token this backend hands out) is **stateless**: the server keeps no list of who is signed in. It just checks the signature on the token you send. Nothing in the backend can be told "forget this token". So logging out means forgetting it on *your* side: delete the two `localStorage` keys, clear the state. If somebody had copied that token string out of your browser before you logged out, it would keep working until it expired. That is the trade you accept with JWTs, and pretending otherwise helps nobody.

THE PROVIDER VALUE

frontend/src/AuthContext.jsx

```
return (
  <AuthContext.Provider value={{ user, isLoggedIn, login, logout }}>
    {children}
  </AuthContext.Provider>
);
}
```

`AuthContext.Provider` is the component that puts a value into the box. Everything rendered inside it, at any depth, can pull that value out with `useAuth()`. The double braces are not special syntax: the outer `{}` means "a JavaScript expression goes here", the inner `{}` is an object with four keys. Two pieces of data and two functions, which is exactly what the rest of the app needs.

The finished file is 92 lines and is built from the six sections above, in this order: imports, the two `useState` lines, the `lms:unauthorised` effect, the role back-fill effect, `login`, `logout`, and the returned provider.

PLUGGING IT IN

The provider only helps components that sit inside it, so it goes near the very top, in `main.jsx`:

frontend/src/main.jsx

```
createRoot(document.getElementById("root")).render(  
  <BrowserRouter>  
    <AuthProvider>  
      <App />  
    </AuthProvider>  
  </BrowserRouter>,  
);
```

The nesting order matters. `BrowserRouter` is outermost because `AuthProvider` and everything below it use router hooks like `useNavigate` and `useLocation`, and those hooks only work inside a router. `AuthProvider` comes next so that every route in `App` can call `useAuth()`.

COMMON MISTAKE

Putting `<AuthProvider>` inside `App.jsx` around only some routes, then calling `useAuth()` from `Sidebar`. You get the thrown error from section 9.2: `useAuth` must be used inside an `AuthProvider`. The message is telling the exact truth. Move the provider up.

9.4 ProtectedRoute.jsx: the bouncer

```
mkdir -p src/components  
touch src/components/ProtectedRoute.jsx
```

This component renders nothing of its own. It looks at the auth state and decides whether to show what it was given, or to send the browser somewhere else.

frontend/src/components/ProtectedRoute.jsx

```
import { Navigate, useLocation } from "react-router-dom";  
import { useAuth } from "../auth.js";  
  
// `role`, when given, is the one role allowed through. Hiding a link in the  
// sidebar is not enough on its own – somebody can always type the URL.  
export default function ProtectedRoute({ role, children }) {  
  const { isLoggedIn, user } = useAuth();  
  const location = useLocation();
```

Two props. `children` is the thing to show if the checks pass. `role` is optional: leave it off and any signed-in person gets through; set it to `"admin"` and only admins do.

`useLocation()` returns an object describing the current URL, and `location.pathname` is the path part of it, like `/teachers`.

```
frontend/src/components/ProtectedRoute.jsx
```

```
if (!isLoggedIn) {  
  // `state` remembers where you were headed so the login page can send you  
  // back there instead of always dropping you on the dashboard.  
  return <Navigate to="/login" replace state={{ from: location.pathname }} />;  
}
```

`Navigate` is a component that performs a redirect when it renders. Returning it instead of a page is how a React Router app says "go somewhere else".

The `replace` prop is small and easy to skip, and skipping it produces a genuinely broken app. Browser history is a stack of pages you can walk back through with the Back button. By default a navigation **pushes** a new entry on top. So a signed-out visitor typing `/teachers` would get: `/teachers` pushed, then `/login` pushed on top. Press Back and the browser returns to `/teachers`, where `ProtectedRoute` immediately redirects to `/login` again. The Back button is now a trap. `replace` **overwrites** the current entry instead of pushing, so `/teachers` never stays in history and Back does what the user expects.

`state` carries a hidden piece of data along with the navigation. It does not appear in the URL. Here it remembers where the person was trying to go, and `Login.jsx` reads it back to finish the journey after a successful sign-in.

```
frontend/src/components/ProtectedRoute.jsx
```

```
// The role arrives a moment after the token when a saved session is being  
// topped up from /profile/. Waiting avoids bouncing the user by mistake.  
if (role && !user?.role) {  
  return null;  
}  
  
if (role && user.role !== role) {  
  return <Navigate to="/" replace />;  
}  
  
return children;  
}
```

The first of these three blocks is the back-fill effect from section 9.3 showing up again. Picture an admin who refreshes the page while sitting on `/accounts`, with an old saved session that has no role on it. On that first render `isLoggedIn` is `true` but `user.role` is `undefined`. Without this guard the next check would fire, decide `undefined !== "admin"`, and throw a real admin back to the dashboard a fraction of a second before `/profile/` answers with the truth. Returning `null` renders nothing for that brief moment, and when the role arrives the component re-renders and lets them through.

The second block is the actual role gate: wrong role, back to the dashboard with `replace` for the same Back-button reason.

Falling past all three checks returns `children`, and the page renders normally.

OUTLET OR CHILDREN?

You will see other tutorials write this component so it ends with `return <Outlet />` instead of `return children`. `Outlet` is React Router's placeholder for "whichever child route matched". Both styles work, but they are used differently:

- `<Outlet />` only works when the component is a **layout route** with routes nested inside it.
- `children` works anywhere: around a layout, around one single page, or around a fragment of a page.

This project needs the second, because `ProtectedRoute` is used in both shapes: once around `<Sidebar />` to guard the whole app, and once around `<Accounts />` to guard a single page. The `Outlet` still exists in this app, but it lives in `Sidebar.jsx`, which is the actual layout.

CHECKPOINT Start the frontend with `npm run dev`. This project's `vite.config.js` sets the port to 5180, so the app opens at `http://localhost:5180`. Open DevTools, delete `lms_access_token` from Local Storage, and type `http://localhost:5180/teachers` into the address bar. The URL should snap to `/login` immediately. Now press the browser's Back button. You should stay on `/login`. If Back throws you into a redirect loop, the `replace` prop is missing.

9.5 App.jsx: the route table

`App.jsx` maps URLs to pages. It has no markup of its own beyond routes.

frontend/src/App.jsx

```
import { Navigate, Route, Routes } from "react-router-dom";
import Sidebar from "../Sidebar.jsx";
import { ProtectedRoute } from "../components/index.js";
```

The page imports follow: `Dashboard`, `Teachers`, `Students`, `Courses`, `Enrollments`, `Lessons`, `Assignments`, `Submissions`, `Results`, `Login`, `Accounts`, `Profile` and `NotFound`, each from `./pages/`.

The first two routes are public, which here means "reachable with no token".

frontend/src/App.jsx

```
<Route path="/login" element={<Login />} />

{/* /register used to be a public page. There is no public sign-up any
   more: an admin creates accounts at /accounts, inside the app. */}
<Route path="/register" element={<Navigate to="/login" replace />} />
```

`/register` is a piece of project history left in on purpose. An earlier version of this app had a public sign-up form. In a learning management system that is wrong: you do not want strangers creating their own teacher accounts. So sign-up moved inside the app, behind an admin-only page, and `/register` became a redirect. Old bookmarks and old links still land somewhere sensible instead of on a 404.

Next comes the interesting part, a `Route` with an `element` but **no** `path`. That is a **layout route** (also called a pathless route). It matches no URL by itself; it exists to wrap the routes nested inside it.

frontend/src/App.jsx

```
{/* Every page inside the sidebar needs a token. Without one,
   ProtectedRoute sends you to /login. */}
<Route
  element={
    <ProtectedRoute>
      <Sidebar />
    </ProtectedRoute>
  }
/>
```

Read what this does. Every route nested inside renders `ProtectedRoute` first, then `Sidebar`, then the page itself in `Sidebar`'s `<Outlet />`. One wrapper buys two things for all eleven pages: the login check, and the sidebar layout. Nobody can forget to add either one to a new page, because adding a page here means putting it inside this block.

frontend/src/App.jsx

```
<Route path="/" element={<Dashboard />} />
<Route path="/teachers" element={<Teachers />} />
<Route path="/students" element={<Students />} />
<Route path="/courses" element={<Courses />} />
<Route path="/enrollments" element={<Enrollments />} />
<Route path="/lessons" element={<Lessons />} />
<Route path="/assignments" element={<Assignments />} />
<Route path="/submissions" element={<Submissions />} />
<Route path="/results" element={<Results />} />

{/* Your own account. Every role gets this one. */}
<Route path="/profile" element={<Profile />} />
```

Then one page needs more than a token. It needs the admin role, so it gets a second `ProtectedRoute` inside the first.

frontend/src/App.jsx

```
    { /* Admins only, and the API refuses everyone else regardless. */ }  
    <Route  
      path="/accounts"  
      element={  
        <ProtectedRoute role="admin">  
          <Accounts />  
        </ProtectedRoute>  
      }  
    />  
  </Route>
```

Two `ProtectedRoute`s stack cleanly: the outer one asks "signed in?", the inner one asks "admin?". A student who types `/accounts` fails the second check and lands on the dashboard.

Read that comment carefully, because it is the whole security model in one line: the frontend check is about not offering a page that would fail. The Django `IsAdmin` permission class on `/api/register/`, in `backend/backend/permissions.py`, is what actually stops a non-admin creating accounts. If you deleted `ProtectedRoute` entirely, a student could open the Accounts page, but the form on it would come back `403 Forbidden` from the server (the status code meaning "I know who you are, and you are not allowed"). Frontend checks are courtesy. Backend checks are security.

Finally:

frontend/src/App.jsx

```
<Route path="*" element={<NotFound />} />
```

`*` is the catch-all. React Router picks the most specific match, so this only wins when nothing else matched, and it renders the 404 page. It sits outside the layout route on purpose: someone who is not signed in and types a nonsense URL should see "page not found", not get redirected to login.

The finished file, with the middle routes shortened so the shape is easy to see:

frontend/src/App.jsx

```
export default function App() {
  return (
    <Routes>
      <Route path="/login" element={<Login />} />
      <Route path="/register" element={<Navigate to="/login" replace />} />

      <Route
        element={
          <ProtectedRoute>
            <Sidebar />
          </ProtectedRoute>
        }
      />

      <Route path="/" element={<Dashboard />} />
      { /* ...teachers, students, courses, enrollments, lessons,
         assignments, submissions, results, profile... */ }
      <Route
        path="/accounts"
        element={
          <ProtectedRoute role="admin">
            <Accounts />
          </ProtectedRoute>
        }
      />
    </Routes>

    <Route path="*" element={<NotFound />} />
  </Routes>
);
}
```

9.6 Sidebar.jsx: the menu and the hole the pages fall into

The menu is data, not markup. One array at the top of the file, outside the component so it is built once instead of on every render.

frontend/src/Sidebar.jsx

```
const menu = [
  { address: "/", text: "Dashboard", icon: <LayoutDashboard size={16} /> },
  { address: "/teachers", text: "Teachers", icon: <Users size={16} /> },
  { address: "/students", text: "Students", icon: <GraduationCap size={16} /> },
  { address: "/courses", text: "Courses", icon: <BookOpen size={16} /> },
];
```

Each entry has three keys: where the link goes, what it says, and an icon component from `lucide-react` (the icon library listed in `frontend/package.json`). Adding a page to the menu means adding one line here, not writing another `<Link>` by hand. The array carries on with Enrollments, Lessons, Assignments, Submissions and Results in the same shape.

The last entry has a fourth key:

frontend/src/Sidebar.jsx

```
// `onlyFor` keeps a link out of the sidebar for other roles. The route and
// the API check the role again; this is only about not offering it.
{
  address: "/accounts",
  text: "Accounts",
  icon: <UserPlus size={16} />,
  onlyFor: ADMIN,
},
];
```

`ADMIN` is imported from `./permissions.js`, where it is the string `"admin"`. Using the constant instead of typing `"admin"` by hand means a typo shows up as a broken import you notice at once, rather than a link that silently never appears.

Inside the component: three hooks from React Router and the auth context, one remembered preference, and one filter.

frontend/src/Sidebar.jsx

```
export default function Sidebar() {
  const location = useLocation();
  const navigate = useNavigate();
  const { user, logOut } = useAuth();

  const [isCollapsed, setIsCollapsed] = useState(
    () => localStorage.getItem(COLLAPSED_KEY) === "true",
  );

  useEffect(() => {
    localStorage.setItem(COLLAPSED_KEY, String(isCollapsed));
  }, [isCollapsed]);

  const visibleMenu = menu.filter(
    (item) => !item.onlyFor || item.onlyFor === user?.role,
  );
}
```

The one piece of state the sidebar owns is whether it is collapsed. It is read back out of `localStorage` on the first render and written back whenever it changes, so the choice survives a reload the same way the session does. The key it is stored under sits at the top of the file, next to the menu:

frontend/src/Sidebar.jsx

```
// Whether the sidebar is collapsed is remembered across reloads, the same way
// the session is. It is only a preference, so a broken value is not worth
// guarding against beyond the === "true" check.
const COLLAPSED_KEY = "lms:sidebarCollapsed";
```

`localStorage` only holds strings, which is why the value goes in through `String(isCollapsed)` and comes back out through a comparison with `"true"`. Anything else in there — a stale value, junk somebody typed into the console — reads as `false`, and an expanded sidebar is the safe way to be wrong. Compare that with the token in `api.js`, where a bad value has to be caught and turned into "not logged in"; here it costs nothing, so nothing is spent on it.

Read the filter as: keep this item if it has no `onlyFor` at all, or if its `onlyFor` matches my role. Nine items have no `onlyFor` and always survive. Accounts survives only for an admin.

frontend/src/Sidebar.jsx

```
function handleLogout() {
  logout();
  navigate("/login", { replace: true });
}
```

`logout()` clears the token and the state. `navigate` is the code equivalent of clicking a link, and `{ replace: true }` is the same idea as the `replace` prop on `Navigate`: it overwrites the current history entry so pressing Back after signing out does not walk you into a page you no longer have a token for.

The links are a `.map` over the filtered array. React needs a `key` prop on each item in a rendered list so it can tell rows apart between renders, and the address is unique, so it does the job.

frontend/src/Sidebar.jsx

```
{visibleMenu.map((item) => {
  const isTheCurrentPage = location.pathname === item.address;

  let colours = "text-slate-600 hover:bg-slate-100";
  if (isTheCurrentPage) {
    colours = "bg-indigo-50 font-medium text-indigo-700";
  }
  ...
})}
```

That is the whole active-link highlight: compare the current path from `useLocation()` with the link's address, and swap the Tailwind classes. When the path changes, `useLocation()` returns a new value, the sidebar re-renders, and the highlight moves. No manual bookkeeping.

frontend/src/Sidebar.jsx

```
return (
  <Link
    key={item.address}
    to={item.address}
    title={isCollapsed ? item.text : undefined}
    className={
      "mb-1 flex items-center rounded-md py-2 text-sm " +
      rowLayout +
      " " +
      colours
    }
  >
    {item.icon}
    {!isCollapsed && item.text}
  </Link>
);
}}}
```

NOTE

`<Link to="/teachers">` and `<a href="/teachers">` look identical on screen and behave completely differently. The plain `<a>` makes the browser throw away the page and download everything again, which resets all React state. `Link` changes the URL and swaps the component in place, keeping state and skipping the reload. Inside a React Router app, always use `Link`.

COLLAPSING THE SIDEBAR

Two things change when `isCollapsed` is true: the strip gets narrower, and every row loses its text. The width is one class swap on the outer `<div>`, `w-16` instead of `w-60`, with `transition-[width]` so it slides rather than jumps. The rows share one small variable, because the same layout is needed by the links, the profile row and the log out button:

frontend/src/Sidebar.jsx

```
// Collapsed rows are icon-only, so the label has to survive as a tooltip.
const rowLayout = isCollapsed ? "justify-center px-0" : "gap-3 px-3";
```

Expanded, a row is `gap-3 px-3`: icon, a gap, then the text. Collapsed, it is `justify-center px-0`: the icon alone, centred in the narrow strip. That is why the `className` on the `<Link>` above is built in three pieces — the part that never changes, then `rowLayout`, then `colours`.

The text itself is dropped by `{!isCollapsed && item.text}`. In JSX, `false` renders as nothing at all, so that line means "draw the text only when there is room for it". The label is not lost, though — it moves to `title={isCollapsed ? item.text : undefined}`, which is the tooltip the browser shows on hover. Passing `undefined` when the sidebar is open is deliberate: it leaves the attribute off entirely, rather than adding a tooltip that repeats a label already on screen.

The toggle itself lives in the header, and it swaps places with the logo:

frontend/src/Sidebar.jsx

```
    /* Collapsed, the toggle takes the logo's place: at this width there
       is only room for one thing, and the way back out matters more. */
    {isCollapsed ? (
      <button
        onClick={() => setIsCollapsed(false)}
        title="Expand sidebar"
        aria-label="Expand sidebar"
        aria-expanded={false}
        className="rounded-md p-1.5 text-slate-500 hover:bg-slate-100 hover:text-slate-
700"
      >
        <PanelLeftOpen size={20} />
      </button>
    ) : (
      <>
        <GraduationCap size={22} className="text-indigo-600" />
        <span className="text-lg font-semibold text-slate-900">LMS</span>
        <button
          onClick={() => setIsCollapsed(true)}
          title="Collapse sidebar"
          aria-label="Collapse sidebar"
          aria-expanded={true}
          className="ml-auto rounded-md p-1.5 text-slate-500 hover:bg-slate-100
hover:text-slate-700"
        >
          <PanelLeftClose size={20} />
        </button>
      </>
    )}
```

`PanelLeftOpen` and `PanelLeftClose` are two more `lucide-react` icons, imported at the top of the file alongside the menu ones. `<>...</>` is a **fragment**: a wrapper that groups several elements without adding a real `<div>` to the page, which is what lets both branches of the `?:` return "one thing" while one of them is really three.

NOTE

Collapsing is a display preference and nothing more. It hides link text, not links: the Accounts entry is already gone for a teacher or a student, and that is the `onlyFor` filter, not this. Nothing here decides what anybody may do — `permissions.js` chooses what to draw, and `backend/backend/permissions.py` is what actually refuses.

At the bottom of the sidebar sits the profile link, doubling as the "who am I" display:

frontend/src/Sidebar.jsx

```
<UserRound size={16} />
{!isCollapsed && (
  <>
    <span className="truncate">{user?.username ?? "Signed in"}</span>
    <span className="ml-auto rounded-full bg-slate-100 px-2 py-0.5 text-xs
font-medium text-slate-600">
      {roleLabel(user?.role)}
    </span>
  </>
)}
```

`??` is the **nullish coalescing operator**: use the left side unless it is `null` or `undefined`, in which case use the right. So a user object that has not loaded yet shows "Signed in" rather than an empty gap. `roleLabel` comes from `permissions.js` and turns `"admin"` into `"Admin"`, `"teacher"` into `"Teacher"`, `"student"` into `"Student"`, and anything unrecognised into `"No role"`, which is why the badge never renders blank.

Then the log out button, and the piece that makes this a layout:

frontend/src/Sidebar.jsx

```
<div className="flex-1 overflow-x-auto p-8">
  <Outlet />
</div>
```

`<Outlet />` is the hole. Whichever nested route in `App.jsx` matched the current URL gets rendered right there. On `/teachers` the `Outlet` is `<Teachers />`. On `/` it is `<Dashboard />`. The sidebar itself never re-renders from scratch when you move between pages, which is why the menu does not flicker while navigating.

`overflow-x-auto` on that wrapper is the reason wide data tables scroll sideways inside the content area instead of pushing the whole page off screen.

9.7 Login.jsx: the form

```
touch src/pages/Login.jsx
```

```
frontend/src/pages/Login.jsx
```

```
import { useState } from "react";
import { useLocation, useNavigate } from "react-router-dom";
import { GraduationCap } from "lucide-react";
import { useAuth } from "../auth.js";
import { takeNoticeLeftForLogin } from "../flash.js";
import { Alert, Button, Input } from "../components/index.js";
```

Four pieces of state, one per thing that can change on this screen:

```
frontend/src/pages/Login.jsx
```

```
export default function Login() {
  const [phone, setPhone] = useState("");
  const [password, setPassword] = useState("");
  const [error, setError] = useState("");
  const [isSaving, setIsSaving] = useState(false);
```

`phone` and `password` make these **controlled inputs**. In a controlled input, React state is the single source of truth: the `value` shown in the box comes from state, and every keystroke calls `onChange`, which writes to state, which re-renders the box with the new value. It feels like a loop, and it is, but it means your code can always read exactly what is in the form without reaching into the page elements by hand.

`error` holds the message from a failed attempt. `isSaving` is the **busy flag** that stops someone hammering the button while a request is still travelling.

```
frontend/src/pages/Login.jsx
```

```
const navigate = useNavigate();
const location = useLocation();
const { logIn } = useAuth();

// ProtectedRoute stores the page you were trying to reach here.
const goBackTo = location.state?.from ?? "/";
```

Here is the other end of the `state={{ from: location.pathname }}` you wrote in `ProtectedRoute`. Someone who typed `/results` while signed out was redirected here carrying `from: "/results"`, so after logging in they land on Results. Somebody who came to `/login` directly has no state, `location.state?.from` is `undefined`, and `??` falls back to `/`, the dashboard.

frontend/src/pages/Login.jsx

```
// Left by the profile page after a password change, so the sign-out that
// follows does not look like something went wrong. Read once, on the first
// render, and cleared as it is read.
const [notice] = useState(() => takeNoticeLeftForLogin());
```

This handles one specific moment. When you change your password on the profile page, `Profile.jsx` calls `leaveNoticeForLogin("Password changed. Please log in again.")` and then `logout()`, so you can prove the new password works. Without a message, being dumped on the login screen looks like a failure. `flash.js` writes that short line into `sessionStorage` (the same idea as `localStorage`, except the browser wipes it when the tab closes), and `takeNoticeLeftForLogin()` reads it and deletes it in the same call.

Note the shape: `const [notice] = useState(...)` takes only the first item of the pair and never the setter, because nothing ever changes this value. The lazy initialiser matters more than usual here, since the function has a side effect (it deletes the stored message). Running it on every render would be wrong; running it once on the first render is exactly right. And because the message is consumed as it is read, refreshing the login page makes it disappear, which is what you want.

THE SUBMIT HANDLER

frontend/src/pages/Login.jsx

```
async function handleSubmit(event) {
  event.preventDefault();
  setError('');
  setIsSaving(true);

  try {
    // The backend looks you up by phone number, not by username.
    await login(phone, password);
    navigate(goBackTo, { replace: true });
  } catch (problem) {
    setError(problem.message);
  } finally {
    setIsSaving(false);
  }
}
```

Line by line:

`event.preventDefault()` stops the browser's built-in form behaviour, which is to pack the fields into the URL and reload the page. Forget this one line and the page blinks, the URL grows a `?` on the end, and none of your JavaScript runs.

`setError("")` clears any message from the previous attempt, so a second try does not show a stale error while it is still loading.

`setIsSaving(true)` turns the busy flag on before the request starts.

`await login(phone, password)` is the `login` from `AuthContext.jsx`, reached through `useAuth()`. `await` pauses here until the network round trip finishes.

Say it once more, because it is the single most common stumble in this project: **you log in with a phone number, not a username.** The Django `LoginView` does `Profile.objects.get(phone=phone)`. Your username is what the sidebar prints; it is not what signs you in. That is also why the input is labelled "Phone number" with a placeholder of `01711110001`.

`navigate(goBackTo, { replace: true })` runs only if `login` did not throw. `replace` again: the login page should not sit in history behind the dashboard, or Back would take a signed-in user to a login form.

`catch (problem) { setError(problem.message) }` catches whatever `api.js` threw. Because `readError` in `api.js` already flattened Django's several error shapes into one readable line, this page does not need to know anything about how Django REST Framework formats its responses. A wrong password produces `{"error": "Invalid phone or password"}` from the backend, `readError` pulls out the `error` key, and that exact sentence lands in the red box. A Django server that is not running produces `Could not reach the server. Is Django running on http://127.0.0.1:8001?` from the `fetch` catch in `api.js`, in the same red box.

`finally` runs whether or not there was an error, which is the point of using it. Put `setIsSaving(false)` only in the `try` and a failed login leaves the button stuck on "Logging in..." forever.

THE MARKUP

frontend/src/pages/Login.jsx

```
<Alert variant="success">{notice}</Alert>
<Alert>{error}</Alert>
```

Both alerts are always in the tree, and both usually render nothing. The `Alert` component's first line is `if (!children) { return null; }`, so an empty string produces no element at all. That is why there is no `{error && <Alert ...>}` dance here. The default `variant` is `"error"`, so the second one is red without being told.

frontend/src/pages/Login.jsx

```
<Input
  label="Phone number"
  placeholder="01711110001"
  required
  value={phone}
  onChange={(event) => setPhone(event.target.value)}
/>
```

`value` plus `onChange` is the controlled-input pair. `event.target` is the `<input>` element, `event.target.value` is the text now in it. `required` is plain HTML: the browser refuses to submit an empty field and shows its own bubble, no JavaScript needed.

The password field is the same with `type="password"` added so the characters render as dots.

frontend/src/pages/Login.jsx

```
<Button
  type="submit"
  disabled={isSaving}
  className="mt-4 w-full justify-center disabled:opacity-60"
>
  {isSaving ? "Logging in..." : "Log in"}
</Button>
```

`type="submit"` is what connects the button to the form's `onSubmit`, which is what runs `handleSubmit`. `Button.jsx` defaults `type` to `"button"`, so leaving this off gives you a button that looks right and does absolutely nothing when clicked.

`disabled={isSaving}` and the label swap are the busy flag showing up on screen: during the request the button is greyed to 60% opacity, unclickable, and reads "Logging in..."

frontend/src/pages/Login.jsx

```
{/* There is no sign-up link. Accounts are handed out by an admin. */}
<p className="mt-4 text-center text-sm text-slate-500">
  No account? Ask an admin to make you one.
</p>
```

Most login pages have a "Create an account" link here. This one says the opposite on purpose, and it matches the `/register` redirect in `App.jsx` and the `IsAdmin` permission on the backend register endpoint. All three tell the same story.

See Figure 0.1.

Type a wrong password and the `catch` fills the red `Alert` with the sentence Django sent back.

See Figure 0.2.

COMMON MISTAKE

Testing with your Django superuser's username. It will always fail with "Invalid phone or password", and you will start suspecting your token code. The login view searches the `Profile` table by phone, so an account with no `Profile` row cannot log in at all through the API, no matter how correct the password is. If your admin cannot get in, open the Django admin at `http://127.0.0.1:8001/admin/` and check that a `Profile` with a phone number exists for that user.

CHECKPOINT

Log in with a real phone and password. You should land on the dashboard, the sidebar should show your username with a role badge next to it, and Local Storage should now hold `lms_access_token` and `lms_user`. Press F5. You should stay on the dashboard, because of the lazy initialisers in `AuthProvider`.

9.8 What happens when you press Log in

Here is the whole journey, one step at a time, naming the real file doing the work.

1. You type into the two boxes. Each keystroke fires `onChange` in `frontend/src/pages/Login.jsx`, which calls `setPhone` or `setPassword` and re-renders the field with the new text.
2. You click the button. It is `type="submit"`, so the browser fires the form's `onSubmit`, which is `handleSubmit` in `Login.jsx`.
3. `handleSubmit` calls `event.preventDefault()`, so the browser does not reload the page. It clears `error` and sets `isSaving` to `true`. The button greys out and reads "Logging in..."
4. It calls `login(phone, password)`. That function came out of the context via `useAuth()` in `frontend/src/auth.js`, and it was defined in `frontend/src/AuthContext.jsx`.
5. `login` calls `loginRequest`, which is `login` from `frontend/src/api.js`. That runs `request("/login/", { method: "POST", body: JSON.stringify({ phone, password }), auth: false })`.
6. `request` builds the URL from `BASE_URL` (which is `http://127.0.0.1:8001/api` unless a `VITE_API_URL` setting overrides it), giving `http://127.0.0.1:8001/api/login/`. It sets `Content-Type: application/json` because there is a body, skips the Authorization header because `auth` is `false`, and calls `fetch`.
7. Django routes that URL through `backend/lms/urls.py` and `backend/backend/urls.py` to `LoginView` in `backend/backend/views.py`. It validates the two fields, looks up `Profile.objects.get(phone=phone)`, checks `user.check_password(password)`, builds a JWT pair with `get_tokens_for_user`, and responds with `message`, `user_id`, `username`, `role` and a nested `tokens` object.
8. Back in `api.js`, `request` sees a 200, parses the JSON body and returns it. (Had it been a 400, `readError` would have pulled out the `error` key and `request` would have thrown `new`

`Error("Invalid phone or password")`), which `handleSubmit`'s `catch` would put in the red `Alert`, and the journey would stop here.)

9. In `AuthContext.jsx`, `login` builds `nextUser` from `data.user_id`, `data.username` and `data.role`, then calls `saveSession(data.tokens.access, nextUser)`. `api.js` writes `lms_access_token` and `lms_user` into `localStorage`.
10. `login` calls `setUser(nextUser)` and `setIsLoggedIn(true)`. The value object passed to `AuthContext.Provider` is now a new object, so every component using `useAuth()` is scheduled to re-render.
11. `await login(...)` resolves back in `Login.jsx`, and the next line runs: `navigate(goBackTo, { replace: true })`. The URL becomes `/` (or whatever page you had originally aimed at), replacing `/login` in history.
12. `frontend/src/App.jsx` matches the new URL. `/` sits inside the layout route, so React renders `ProtectedRoute` first.
13. `frontend/src/components/ProtectedRoute.jsx` calls `useAuth()`, sees `isLoggedIn` is `true`, is not given a `role` prop, and so returns its `children`: `<Sidebar />`. (This is why the order in step 10 matters. If the state flip had not happened before the navigation, this check would fail and bounce you straight back to `/login`.)
14. `frontend/src/Sidebar.jsx` renders. It filters `menu` by `user?.role`, so an admin sees ten links including Accounts and a student sees nine. It prints `user?.username ?? "Signed in"` and `roleLabel(user?.role)` in the profile link at the bottom.
15. `<Outlet />` inside `Sidebar` renders the matched child route, `<Dashboard />` from `frontend/src/pages/Dashboard.jsx`.
16. `Dashboard` appears for the first time and fetches all eight lists through `api.js` to count them. This time `getToken()` returns the token you just saved, so `request` attaches `Authorization: Bearer <token>`, and Django's permission classes let the data through.

See Figure 0.3.

CHECKPOINT Log in as a student rather than an admin. The sidebar should have no Accounts link, because of the `onlyFor` filter in `Sidebar.jsx`. Now type `http://localhost:5180/accounts` into the address bar by hand. You should be bounced to the dashboard by the `role="admin"` check in `ProtectedRoute`. Two separate defences, and the Django `IsAdmin` class behind both of them makes three.

What you have now

- `src/auth.js`: one context and one `useAuth()` hook, kept in their own file so Vite fast refresh keeps working, with a clear thrown error when the provider is missing.
- `src/AuthContext.jsx`: state that reloads itself from `localStorage` on first render, a listener for the `lms:unauthorised` event fired by `api.js`, a back-fill that asks `/profile/` for a missing role,

and `login` / `logout` functions shared through the provider.

- `src/components/ProtectedRoute.jsx`: a signed-in check that redirects with `replace` and remembers where you were headed, plus an optional `role` prop for admin-only pages.
- `src/App.jsx`: the full route table, with a layout route that wraps eleven pages in one auth check and one sidebar, a `/register` redirect where the old public sign-up used to be, an admin-only `/accounts` route, and a catch-all 404.
- `src/Sidebar.jsx`: a menu built from an array, role filtering through `onlyFor`, an active-link highlight from `useLocation`, a profile link with a role badge, a Log out button, and the `Outlet` every page renders into.
- `src/pages/Login.jsx`: controlled inputs, phone-number sign-in, a busy flag on the button, backend error messages shown in a red alert, and a handoff notice from the profile page shown in a green one.
- A working sign-in loop: log in, refresh the page and stay in, close the tab and come back still signed in, log out and be locked out again.

The Reusable Pieces: the components Folder

Open any two pages of this app side by side. Open `Teachers.jsx` and `Courses.jsx`. You will see the same shapes twice: a title with a grey subtitle under it, a red banner for errors, a white card with a form in it, text boxes with a small grey label above each one, a purple Save button, a table with an uppercase header row, and a pop-up asking "are you sure?" before anything is deleted.

There are thirteen pages in `frontend/src/pages/`. Eight of them (Teachers, Students, Courses, Enrollments, Lessons, Assignments, Submissions, Results) need nearly everything on that list, and four more need part of it. If every page wrote out its own text box, its own button and its own table header, you would be copying the same forty characters of styling into forty different places. Change your mind about the shade of purple and you would have to find all forty.

That is what this chapter fixes. The folder `frontend/src/components/` holds fourteen files: twelve components, one file of shared style strings, and one file whose only job is to hand the other twelve out. Each one is a piece the pages borrow instead of rebuilding.

What a component actually is

A **component** in React is a JavaScript function that returns some HTML-looking markup. That markup is called **JSX**: it is HTML written inside a JavaScript file, and the build tool turns it into real browser instructions before it ships.

The rule that makes React work is this: a function whose name starts with a **capital letter** can be used as if it were an HTML tag.

So if you write a function called `PageHeader`, you can then write `<PageHeader />` anywhere, and React will call your function and drop whatever it returns in that spot. That is the whole idea. A component is a tag you invented.

Here is the smallest one in the folder, and it is a good place to start because it has nothing clever in it at all.

```
export default function PageHeader({ title, subtitle }) {
  return (
    <>
      <h1 className="text-2xl font-semibold text-slate-900">{title}</h1>
      <p className="mt-1 mb-6 text-sm text-slate-500">{subtitle}</p>
    </>
  );
}
```

Four things in there need explaining, and every other file in this folder reuses all four.

`export default` means "this file hands out one main thing, and here it is." Another file can then pull it in with `import PageHeader from "../PageHeader.jsx"`.

`className` is React's spelling of the HTML `class` attribute. `class` is already a reserved word in JavaScript, so React renamed it. The long string of short words inside it (`text-2xl`, `font-semibold`, `text-slate-900`) is **Tailwind**. Tailwind is a CSS library where each little word is one styling rule: `text-2xl` sets the font size, `mb-6` puts a gap under the element, `text-slate-500` makes the text mid-grey. You stack them up instead of writing a separate `.css` file. This project turns Tailwind on in exactly one line, and that line is the entire contents of `frontend/src/index.css`:

```
/* frontend/src/index.css */
@import "tailwindcss";
```

`{title}` with curly braces means "stop being markup for a second and evaluate this JavaScript". Whatever `title` holds gets printed there.

`<>` and `</>` are a **fragment**. A component function has to return one single thing. `PageHeader` wants to return two things, an `<h1>` and a `<p>`. Wrapping them in an empty tag pair satisfies the rule without adding a pointless extra `<div>` to the page.

Props: how a page tells a component what to say

Look again at the first line: `function PageHeader({ title, subtitle })`.

The values a component receives are called **props**, short for properties. You pass them the same way you pass HTML attributes. This is the real line from the Teachers page:

```
// frontend/src/pages/Teachers.jsx (inside the returned JSX)
<PageHeader title="Teachers" subtitle="The people who teach courses." />
```

React collects those into a single object, `{ title: "Teachers", subtitle: "The people who teach courses." }`, and hands that object to your function as its one argument.

The curly braces in `function PageHeader({ title, subtitle })` are **destructuring**: a JavaScript shorthand for pulling named keys out of an object. Instead of receiving the whole object and writing `props.title` everywhere, you name the keys you want up front and get two plain variables. It saves typing and it documents, right in the first line, exactly which props this component understands.

Props can hold anything JavaScript can hold, not just text. A number, an array, an object, even a function. On the Dashboard the title is built at the moment of rendering, so the braces come out again:

frontend/src/pages/Dashboard.jsx

```
<PageHeader
  title={`Welcome back, ${user?.username ?? "there"}`}
  subtitle="A quick count of everything in the system."
/>
```

NOTE

Quotes mean "this exact text". Curly braces mean "run this JavaScript and use the result". `title="Teachers"` and `title={"Teachers"}` do the same thing; `title={someVariable}` is the only way to pass a variable.

children: the stuff between the opening and closing tags

Some components take a value. Others need to wrap around something. When you write a component with an opening and a closing tag and put things in the middle, React gives you that middle as a prop named `children`.

frontend/src/components/Div.jsx

```
export default function Div({ className = "", children, ...rest }) {
  return (
    <div className={className} {...rest}>
      {children}
    </div>
  );
}
```

`Div` is the plainest possible example: it takes whatever you put inside it and puts it inside a real `<div>`. Elsewhere in the folder, `Table` uses `children` for its table rows, `Button` uses it for the label text, and `Alert` uses it for the message.

NOTE

`Div.jsx` is exported from `index.js` but no page in this project imports it. It is a spare part. Do not go hunting for the page that uses it; there isn't one. It costs nothing to leave there, and it is the clearest illustration of `children` in the whole folder, which is why we started with it.

The rest spread pattern

Now the third piece in that `Div` signature: `...rest`.

Look at the Login page. It passes `type="password"` and `required` and `value` and `onChange` to `Input`. The Students page passes `type="date"`. The Results page passes `type="number"` and `step="0.01"`. A text box in a browser accepts dozens of attributes, and `Input` cannot possibly list them all as named props.

`...rest` is JavaScript's **rest** syntax. It means "every prop I did not name above, gather them into one object called `rest`". Then `{...rest}` inside the JSX is **spread** syntax: "unpack that object back out as individual attributes here".

Together they mean: name the props you actually care about, and pass everything else straight through to the real HTML element underneath. `Input` is the shortest place to see it work.

frontend/src/components/Input.jsx

```
import { useId, useState } from "react";
import { Eye, EyeOff } from "lucide-react";
import { inputBox, labelText } from "./styles.js";

export default function Input({ label, className = "", type = "text", ...rest }) {
  // A password box gets an eye button that turns it back into a plain text
  // box. Every password field in the app is an Input, so the pages have
  // nothing to do.
  const isPassword = type === "password";
  const [isShowing, setIsShowing] = useState(false);

  // The eye button has to sit outside the <label>, otherwise its own text
  // becomes part of the box's name and a screen reader reads out
  // "Password Show password". So the password version ties the two together
  // with an id instead of by nesting.
  const id = useId();

  if (!isPassword) {
    return (
      <label className={className}>
        <span className={labelText}>{label}</span>
        <input className={inputBox} type={type} {...rest} />
      </label>
    );
  }
}
```

`label` and `className` are handled by hand. `required`, `value`, `onChange`, `placeholder`, `step` and anything else you invent tomorrow all land in `rest` and get forwarded to the real `<input>`. Neither you nor this file has to know the list in advance.

The other thing on that line is `className = ""`. That equals sign is a **default value**. If nobody passes `className`, it is the empty string instead of `undefined`. That matters because the code later glues strings together, and gluing `undefined` onto a string gives you the literal word "undefined" in your HTML.

`type` is the one attribute that is named rather than left in `rest`, and it is named for a reason: the component has to look at it. Everything above the `if` is the same for every kind of box; the `if` then answers the only question that changes the markup, which is whether this is a password field. An **early return** like that keeps the ordinary case — the date box, the number box, the plain text box — exactly as short as it was before the password feature existed. Note that `type` is still passed on to the real `<input>` by hand, because naming a prop takes it out of `rest`.

COMMON MISTAKE

Forgetting `...rest` when you write your own component. You then wonder why `<Input required />` does not make the field required. It is because your component quietly swallowed the prop and never handed it to the real `<input>` underneath.

The password box, and the eye button

The other half of the file is the password version: the same box, plus a small eye button that turns the dots back into readable text while you hold the password in your head.

```

const action = isShowing ? "Hide password" : "Show password";

return (
  <div className={className}>
    <label htmlFor={id} className={labelText}>
      {label}
    </label>

    /* This wrapper is what the button is positioned against, and the extra
       right padding keeps the typed text from running underneath it. */
    <div className="relative">
      <input
        id={id}
        className={inputBox + " pr-10"}
        type={isShowing ? "text" : "password"}
        {...rest}
      />

      <button
        type="button"
        onClick={() => setIsShowing(!isShowing)}
        aria-label={action}
        aria-pressed={isShowing}
        title={action}
        className="absolute inset-y-0 right-0 flex items-center rounded-r-md px-3 text-
        slate-500 hover:text-slate-700"
      >
        {isShowing ? <EyeOff size={16} /> : <Eye size={16} />}
      </button>
    </div>
  </div>
);
}

```

`isShowing` is a piece of state holding one true-or-false value, and the whole feature hangs off it. The `type` of the real `<input>` is `{isShowing ? "text" : "password"}: password` draws dots, `text` draws the characters. Nothing is fetched, nothing is decrypted, nothing is sent anywhere — the browser was always holding the typed characters, and the box was only choosing not to draw them.

Three details in that markup are worth taking away, because they come up whenever you put a control inside a form field.

1. **The button cannot live inside the `<label>`.** A label's text is the accessible name of the field it wraps, so a button nested inside one gets read out as part of the name: a screen reader would announce the box as "Password Show password". The password branch therefore stops nesting and

joins the two by id instead — `htmlFor={id}` on the label, `id={id}` on the input. `useId()` is React's hook for exactly this: it hands out an id that is unique on the page, so two password boxes on one screen never collide.

2. **relative and absolute are a pair.** `absolute` positions an element against its nearest positioned ancestor, so the wrapper `<div className="relative">` exists purely to be that ancestor. Without it the button would fly off to a corner of the page. `inset-y-0 right-0` pins it to the full height of the box at the right-hand edge, and the extra `pr-10` on the input reserves the space so long passwords do not run underneath it.
3. **The name says what the button will do next.** `aria-label` and `title` both read "Show password" while the password is hidden, not "Password is hidden". `aria-pressed` carries the state separately, which is what a toggle button is supposed to do. And `type="button"` is not decoration: a `<button>` inside a form defaults to `type="submit"`, so leaving it off would mean clicking the eye submits the login form.

NOTE

Every password field in this app is an `<Input type="password">` — the Login page, the Accounts form, and both password boxes on the Profile page. All of them got the eye button the moment this file grew one. Not a single page was edited. That is the argument for having a shared component at all, and it is much easier to see here than in a paragraph about reuse.

styles.js: two strings that live in one place

`Input`, `Select` and `Textarea` are three different components, but on screen the boxes look identical. Their labels look identical too. If the Tailwind classes were typed into each of the three files, then changing the focus colour would mean three edits, and the day you missed one, one form field would look wrong and you would not notice for a month.

So the shared strings were pulled out into a file of their own. This is the entire file.

frontend/src/components/styles.js

```
export const inputBox =
  "w-full rounded-md border border-slate-300 px-3 py-2 text-sm outline-none focus:border-indigo-500";

export const labelText = "mb-1 block text-sm font-medium text-slate-700";
```

Reading the first one word by word: `w-full` makes the box as wide as the space it sits in, `rounded-md` gives it soft corners, `border border-slate-300` puts a light grey outline on it, `px-3 py-2` is the padding inside, `text-sm` is the font size, `outline-none` turns off the browser's own blue focus ring, and `focus:border-indigo-500` turns the border purple while the box has the cursor in it. That `focus:` part is a Tailwind **variant**: the rule after the colon applies only in that state.

`export const` (no `default`) means this file hands out several named things rather than one main thing. That is why the import at the top of `Input.jsx` has braces around the names:

```
// frontend/src/components/Input.jsx (first line)
import { inputBox, labelText } from "../styles.js";
```

TIP

Change `focus:border-indigo-500` to `focus:border-red-500` in `styles.js`, save, and click into the Subject box on the Teachers page. Every text box, dropdown and text area in the whole app turns red on focus at once. That single edit is the entire argument for this file. Put it back afterwards.

The `looks` object: one component, several appearances

Three components in this folder (`Button`, `IconButton`, `Alert`) come in more than one flavour. A Save button is solid purple, a Cancel button is white with a grey outline, a Delete button is solid red. They behave identically. Only the colours differ.

The pattern used every time is a plain JavaScript object named `looks`, sitting above the component, mapping a short name to a class string. The component then takes a `variant` prop and looks the string up.

Here is `Button`, which is the one you will meet most often.

frontend/src/components/Button.jsx

```
const looks = {
  primary:
    "inline-flex items-center gap-2 rounded-md bg-indigo-600 px-3 py-2 text-sm font-medium text-white hover:bg-indigo-700",
  secondary:
    "inline-flex items-center gap-2 rounded-md border border-slate-300 bg-white px-3 py-2 text-sm font-medium text-slate-700 hover:bg-slate-50",
  danger:
    "inline-flex items-center gap-2 rounded-md bg-red-600 px-3 py-2 text-sm font-medium text-white hover:bg-red-700",
};
```

All three share `inline-flex items-center gap-2`. That trio is what lets a button hold an icon and a word side by side, lined up in the middle, with a small gap between them. `hover:` is another Tailwind variant: the colour after it applies only while the mouse is over the button.

And the function itself:


```
// frontend/src/components/Button.jsx (continued)
export default function Button({
  variant = "primary",
  type = "button",
  className = "",
  children,
  ...rest
}) {
  return (
    <button type={type} className={looks[variant] + " " + className} {...rest}>
      {children}
    </button>
  );
}
```

`looks[variant]` is a lookup by name. If `variant` is `"danger"`, `looks["danger"]` is the red string.

Then `+ " " + className` glues on whatever extra classes the page passed. The shared look comes from `looks`, and a page can still add a one-off tweak without editing this file. The Login page does exactly that to make its button stretch across the card:

frontend/src/pages/Login.jsx

```
<Button
  type="submit"
  disabled={isSaving}
  className="mt-4 w-full justify-center disabled:opacity-60"
>
  {isSaving ? "Logging in..." : "Log in"}
</Button>
```

See Figure 0.1.

`type = "button"` deserves a moment. In plain HTML, a `<button>` inside a `<form>` defaults to `type="submit"`, which means clicking it sends the form. That is almost never what a Cancel button should do. This component flips the default to `"button"`, so buttons are inert unless you explicitly ask for `type="submit"`. Look at the Teachers form: Save says `type="submit"`, Cancel says nothing about `type` and therefore does nothing but run its `onClick`.

PROP	TYPE	DEFAULT	WHAT IT DOES
variant	"primary" "secondary" "danger"	"primary"	Picks the colour set from looks
type	string	"button"	Pass "submit" to make it submit the form
className	string	""	Extra Tailwind classes glued on the end
children	JSX	none	The label, and optionally an icon before it
...rest	anything	none	onClick, disabled and friends go to the real <button>

Here is the pair from the Teachers form, showing primary and secondary together:

frontend/src/pages/Teachers.jsx

```
<div className="mt-4 flex gap-2">
  <Button type="submit" disabled={isSaving}>
    {isSaving ? "Saving..." : "Save"}
  </Button>
  <Button variant="secondary" onClick={closeForm}>
    Cancel
  </Button>
</div>
```

disabled is not one of the props Button names, so it lands in ...rest and reaches the real <button>, where the browser greys it out and ignores clicks. That is the rest spread earning its keep: Button never had to know that disabled exists.

LMS

- Dashboard
- Teachers**
- Students
- Courses
- Enrollments
- Lessons
- Assignments
- Submissions
- Results
- Accounts

Teachers

The people who teach courses.

New teacher

Name: Shirin Sultana

Email: shirin@lms.test

Subject: User Interface Design

☒ Active

Save **Cancel**

4 teachers **+ Add teacher**

ID	NAME	EMAIL	SUBJECT	ACTIVE	ACTION
1	Imran Hossain	imran@lms.test	Web Development	Yes	

Figure 10.1 The Teachers form: Input, Checkbox and two Buttons, all from the shared folder.

CHECKPOINT Run `npm run dev` in `frontend/`, open `http://localhost:5180`, log in as an admin (only admins may add teachers), go to Teachers, click "Add teacher", fill in Subject and hit Save. For the moment the request is in flight the button reads "Saving..." and cannot be clicked twice. If you can double-click your way into creating two teachers, the `disabled` prop is not reaching the button.

IconButton: a button that is only an icon

The little pencil and bin icons in the Action column are buttons too, but they need no padding for text and no background until you hover.

frontend/src/components/IconButton.jsx

```
const looks = {
  plain: "rounded-md p-2 text-slate-600 hover:bg-slate-100",
  danger: "rounded-md p-2 text-red-600 hover:bg-red-50",
};

export default function IconButton({
  variant = "plain",
  type = "button",
  className = "",
  children,
  ...rest
}) {
  return (
    <button type={type} className={looks[variant] + " " + className} {...rest}>
      {children}
    </button>
  );
}
```

That is the same shape as `Button`, line for line. Only the two strings in `looks` differ.

PROP	TYPE	DEFAULT	WHAT IT DOES
<code>variant</code>	<code>"plain"</code> <code>"danger"</code>	<code>"plain"</code>	Grey icon, or red icon
<code>type</code>	string	<code>"button"</code>	Same submit protection as <code>Button</code>
<code>className</code>	string	<code>""</code>	Extra classes
<code>children</code>	JSX	none	The icon element
<code>...rest</code>	anything	none	<code>onClick</code> , <code>aria-label</code> and so on

The icons come from **lucide-react**, a package of ready-made icon components listed in `frontend/package.json`. You import the ones you want by name and use them as tags. These two sit inside the Action cell of the Teachers table:

`frontend/src/pages/Teachers.jsx`

```
<IconButton
  onClick={() => openFormForEditing(teacher)}
  aria-label="Edit"
>
  <Pencil size={14} />
</IconButton>

<IconButton
  variant="danger"
  onClick={() => askToDelete(teacher)}
  aria-label="Delete"
>
  <Trash2 size={14} />
</IconButton>
```

`aria-label` is what a screen reader announces. A button whose only content is a picture has no text to read out, so without the label a blind user hears "button" and nothing else. It costs four words. Always add it.

COMMON MISTAKE

Writing `onClick={openFormForEditing(teacher)}` without the arrow. That calls the function immediately while the table is being drawn, once per row, and opens the edit form for the last teacher in the list the moment the page loads. `onClick` wants a function to call later, so you have to wrap it: `onClick={() => openFormForEditing(teacher)}`.

Alert: the red and green banners

Most pages stack two of these near the top. One shows API errors in red. One shows "Teacher added." in green. Twelve of the thirteen pages render at least one, they should look the same everywhere, and they should disappear completely when there is nothing to say.

```
// frontend/src/components/Alert.jsx (top)
import { CircleCheck, CircleAlert } from "lucide-react";

const looks = {
  error: "border-red-200 bg-red-50 text-red-700",
  success: "border-green-200 bg-green-50 text-green-700",
  info: "border-slate-200 bg-slate-50 text-slate-600",
};

const icons = {
  error: <CircleAlert size={16} className="mt-0.5 shrink-0" />,
  success: <CircleCheck size={16} className="mt-0.5 shrink-0" />,
  info: null,
};
```

Two lookup objects this time, keyed by the same three names. `icons` holds actual JSX in a plain object, which is allowed, because JSX is just a value like any other. `info` maps to `null`, and rendering `null` in React draws nothing, so the info variant simply has no icon.

Now the component:

```
// frontend/src/components/Alert.jsx (bottom)
export default function Alert({ variant = "error", className = "", children }) {
  if (!children) {
    return null;
  }

  return (
    <div
      className={
        "mb-4 flex items-start gap-2 whitespace-pre-line rounded-md border px-3 py-2 text-sm " +
        looks[variant] +
        " " +
        className
      }
    >
      {icons[variant]}
      <span>{children}</span>
    </div>
  );
}
```

The `if (!children) return null;` at the top is the important part. A component that returns `null` puts nothing at all on the page. Because of those three lines, a page can render `<Alert>{error}</Alert>` unconditionally and never think about it again: when `error` is the empty string, the banner

is not there; the moment `error` gets a message, the banner appears. No `{error && ...}` juggling repeated on twelve pages.

That is why the Teachers page can be this blunt:

```
frontend/src/pages/Teachers.jsx

<Alert>{error}</Alert>
<Alert variant="success">{notice}</Alert>
```

Note the default: `variant = "error"`, so the plain `<Alert>` with no variant is the red one. Errors are the common case, so they got the default.

`whitespace-pre-line` in the class list is there for a specific reason. When Django REST Framework rejects several fields at once, the `readError` function in `api.js` joins the complaints with a newline character:

```
// frontend/src/api.js (last line of readError)
return parts.join("\n");
```

Browsers normally collapse newlines in HTML into a single space, which would run every complaint into one long line. `whitespace-pre-line` tells the browser to honour them. `items-start` keeps the icon aligned to the first line rather than floating in the vertical middle of a three-line message, and `shrink-0` on the icon stops a long message from squashing it.

PROP	TYPE	DEFAULT	WHAT IT DOES
<code>variant</code>	<code>"error"</code> <code>"success"</code> <code>"info"</code>	<code>"error"</code>	Colour set and which icon, if any
<code>className</code>	string	<code>""</code>	Extra classes
<code>children</code>	string or JSX	none	The message. Empty means the banner is not rendered at all

NOTE

`variant="info"` is defined in `Alert.jsx` but no page in this project uses it yet. It is there so that the next person who needs a neutral grey notice does not have to invent one.

Checkbox: a tick box with its label attached

Only two pages need this one, Teachers and Students, both for the "Active" flag.

frontend/src/components/Checkbox.jsx

```
export default function Checkbox({ label, className = "", ...rest }) {
  return (
    <label
      className={
        "flex items-center gap-2 text-sm text-slate-700 " + className
      }
    >
      <input type="checkbox" className="h-4 w-4" {...rest} />
      {label}
    </label>
  );
}
```

The whole thing is wrapped in a `<label>`, and the `<input>` sits inside it. That is not decoration. When a checkbox lives inside its own label, the browser makes the label text clickable too, so users can hit the word "Active" instead of aiming at a 16-pixel square. You get that for free just by nesting; no `htmlFor` or `id` needed.

`flex items-center gap-2` puts the box and the word on one line with a small space between them.

PROP	TYPE	DEFAULT	WHAT IT DOES
<code>label</code>	string	none	The text beside the box
<code>className</code>	string	<code>""</code>	Extra classes, usually spacing
<code>...rest</code>	anything	none	<code>checked</code> and <code>onChange</code> go to the real <code><input></code>

From the Teachers form:

frontend/src/pages/Teachers.jsx

```
<Checkbox
  label="Active"
  className="mt-4"
  checked={isActive}
  onChange={(event) => setIsActive(event.target.checked)}
/>
```

COMMON MISTAKE

Writing `event.target.value` here out of habit. A checkbox has a `value`, but it is not the tick state. The tick state is `event.target.checked`, and it is a true/false, which is exactly what the API's `is_active` field wants.

Select: the dropdown

A dropdown is a text box with a list inside it. It reuses the same two style strings from `styles.js`, so it lines up with the `Input` next to it in a two-column grid.

frontend/src/components/Select.jsx

```
import { inputBox, labelText } from "../styles.js";

export default function Select({
  label,
  placeholder,
  className = "",
  children,
  ...rest
}) {
  return (
    <label className={className}>
      <span className={labelText}>{label}</span>
      <select className={inputBox} {...rest}>
        {placeholder && <option value="">{placeholder}</option>}
        {children}
      </select>
    </label>
  );
}
```

Two new things here.

`children` is the list of `<option>` tags, which the page builds from data it fetched. `Select` does not care what they are; it just puts them inside the `<select>`.

`{placeholder && <option value="">{placeholder}</option>}` is React's standard way of saying "only if". In JavaScript, `A && B` gives you `B` when `A` has a real value, and gives you `A` when it does not. So if `placeholder` is a string, you get the `<option>`; if `placeholder` was never passed, you get `undefined`, and React draws nothing for `undefined`. The result is a grey "Choose a teacher..." first row that exists only when the page asked for one.

That placeholder option has `value=""` deliberately. An empty value means "nothing chosen yet", and because the field is marked `required` the browser refuses to submit the form while it is still selected.

PROP	TYPE	DEFAULT	WHAT IT DOES
<code>label</code>	string	none	Grey label above the dropdown
<code>placeholder</code>	string	none	If given, adds an empty first option with this text
<code>className</code>	string	<code>""</code>	Extra classes
<code>children</code>	JSX	none	The <code><option></code> list
<code>...rest</code>	anything	none	<code>value</code> , <code>onChange</code> , <code>required</code> go to the real <code><select></code>

From the Courses form, where the options are built from the teacher list fetched alongside the courses:

frontend/src/pages/Courses.jsx

```

<Select
  label="Teacher"
  placeholder="Choose a teacher..."
  required
  value={teacherId}
  onChange={(event) => setTeacherId(event.target.value)}
>
  {teachers.map((teacher) => (
    <option key={teacher.id} value={teacher.id}>
      {teacher.name}
    </option>
  ))}
</Select>

```

`.map()` walks the array and returns a new array of `<option>` elements. `key` is a prop React insists on for any list item; it uses it to tell rows apart when the list changes. A database id is the ideal key because it is unique and it never changes.

The Accounts page shows the other way to use `Select`: with hand-written options and no placeholder, because the role always has a value.

frontend/src/pages/Accounts.jsx

```
<Select
  label="Role"
  required
  value={role}
  onChange={(event) => setRole(event.target.value)}
>
  <option value="student">Student ☐ read only, may hand work in</option>
  <option value="teacher">
    Teacher ☐ course material, enrollments and grading
  </option>
  <option value="admin">Admin ☐ everything, including accounts</option>
</Select>
```

COMMON MISTAKE

A `<select>` always hands you back a **string**, even when the option values were numbers. Send that straight to the API and Django rejects it. The Courses page converts it at save time, and the comment in the file says so:

```
// frontend/src/pages/Courses.jsx (inside handleSave)
// A <select> always hands back a string. The API wants the integer id.
const values = { title, description, teacher: Number(teacherId) };
```

The same trap appears in reverse when opening the edit form. `course.teacher` is a number, `<select value>` wants a string, so the page converts the other way with `setTeacherId(String(course.teacher));`.

Textarea: the multi-line box

Same idea as `Input`, for descriptions, feedback and submitted work.

frontend/src/components/Textarea.jsx

```
import { inputBox, labelText } from "../styles.js";

export default function Textarea({ label, rows = 3, className = "", ...rest }) {
  return (
    <label className={"block " + className}>
      <span className={labelText}>{label}</span>
      <textarea rows={rows} className={inputBox} {...rest} />
    </label>
  );
}
```

Two differences from `Input`. First, `rows = 3` is a named prop with a default, so most pages get a three-line box without asking, and a page that wants more can say so. Second, the wrapping label always gets `block` in front of whatever `className` the page passed. A `<label>` is inline by default, which would let it sit next to whatever came before it; `block` forces it onto its own line, which is what a full-width text area always wants.

PROP	TYPE	DEFAULT	WHAT IT DOES
<code>label</code>	string	none	Grey label above the box
<code>rows</code>	number	<code>3</code>	How many lines tall
<code>className</code>	string	<code>""</code>	Extra classes, added after <code>block</code>
<code>...rest</code>	anything	none	<code>value</code> , <code>onChange</code> , <code>required</code> go to the real <code><textarea></code>

The Submissions page is the only one that overrides the height, because a student's answer needs room:

frontend/src/pages/Submissions.jsx

```
<Textarea
  label="Content"
  rows={5}
  className="mt-4"
  required
  value={content}
  onChange={(event) => setContent(event.target.value)}
/>
```

Note `rows={5}` in braces, not `rows="5"` in quotes. It is a number, so it gets braces.

Table: the shell, not the contents

This is where the split between "shared" and "page-specific" gets interesting. Every table in the app has the same outer frame: a horizontally scrolling wrapper, an uppercase grey header row, thin dividing lines. But the cells are different on every page. The Teachers table prints `teacher.subject`. The Courses table has to look up a teacher name. Both Teachers and Students turn `is_active` into "Yes" or "No".

So `Table` takes over the frame and the header, and hands the rows back to the page.

```

// The header row is built from a list of column names. The body rows stay in
// the pages themselves, because each one formats its cells differently.
export default function Table({ columns, children }) {
  return (
    <div className="overflow-x-auto p-4">
      <table className="w-full text-left text-sm">
        <thead>
          <tr className="border-b border-slate-200 text-xs uppercase text-slate-500">
            {columns.map((column, index) => (
              <th key={index} className="px-3 py-2 font-medium">
                {column}
              </th>
            ))}
          </tr>
        </thead>

        <tbody>{children}</tbody>
      </table>
    </div>
  );
}

```

Reading it from the outside in: a `<div>` with `overflow-x-auto`, so a wide table gets its own sideways scrollbar instead of pushing the whole page sideways. Inside that, a `<table>` at full width. Inside that, `<thead>` holding one `<tr>`, whose `<th>` cells are generated by mapping over the `columns` array. Then `<tbody>{children}</tbody>`, which is the hole the page fills.

`columns.map((column, index) => ...)` uses the second argument that `.map()` provides, the position in the array. It is used as the `key`. Using an array position as a key is normally discouraged, because rows that move around confuse React. Here it is fine: column headers are a fixed list written by hand in the page, and they never reorder.

PROP	TYPE	WHAT IT DOES
<code>columns</code>	array of strings	One header cell per entry, in order
<code>children</code>	JSX	The <code><tr></code> rows for the body

Now the page side. The columns go in as a plain array, and the rows are mapped from the API data straight between the tags:

```

<Table columns={["ID", "Name", "Email", "Subject", "Active", "Action"]} >
  {teachers.map((teacher) => (
    <tr
      key={teacher.id}
      className="border-b border-slate-100 hover:bg-slate-50"
    >
      <td className="px-3 py-2 text-slate-700">{teacher.id}</td>
      <td className="px-3 py-2 text-slate-700">{teacher.name}</td>
      <td className="px-3 py-2 text-slate-700">{teacher.email}</td>
      <td className="px-3 py-2 text-slate-700">{teacher.subject}</td>
      <td className="px-3 py-2 text-slate-700">
        {teacher.is_active ? "Yes" : "No"}
      </td>
      <td className="px-3 py-2">
        <div className="flex gap-1">
          {mayEdit ? (
            <>
              <IconButton
                onClick={() => openFormForEditing(teacher)}
                aria-label="Edit"
              >
                <Pencil size={14} />
              </IconButton>

              <IconButton
                variant="danger"
                onClick={() => askToDelete(teacher)}
                aria-label="Delete"
              >
                <Trash2 size={14} />
              </IconButton>
            </>
          ) : (
            <span className="px-2 text-slate-400">&mdash;</span>
          )}
        </div>
      </td>
    </tr>
  ))}
</Table>

```

`mayEdit` comes from `canWrite(user?.role, "teacher")` in `permissions.js`. Only an admin gets `true` for teachers, so everyone else sees a dash in the Action column instead of the two icons. The role rules are Chapter 9's subject; here the point is only that the page decides what goes in the cell, not `Table`.

See Figure 0.4.

The Enrollments page passes a shorter list, and its cells do a lookup instead of printing a field:

```
frontend/src/pages/Enrollments.jsx
```

```
<Table columns=["ID", "Student", "Course", "Enrolled", "Action"]>
```

The one rule to respect: the number of names in `columns` must match the number of `<td>` cells in each row, or the header and the body drift out of alignment. `Table` cannot check this for you.

**COMMON
MISTAKE**

Adding a column name to the array and forgetting the matching `<td>`, or the other way round. The table does not crash. It just quietly puts every value under the wrong heading, which is much worse. Count them both when you edit.

CHECKPOINT

Squeeze your browser window narrow on the Submissions page, whose `columns` array has six entries. The table should grow its own horizontal scrollbar while the sidebar and the page heading stay put. That is `overflow-x-auto` in `Table.jsx` doing its job.

ConfirmDialog: the pop-up that replaces `window.confirm`

Deleting a course deletes its lessons, its assignments and its enrollments too. That is not a click you want a user to make by accident. Every delete in this app goes through a confirmation step first.

The browser has a built-in one, `window.confirm("Are you sure?")`, and it is one line. This project does not use it, for four reasons.

It cannot be styled at all: it is drawn by the operating system, not by your page, and it looks nothing like the rest of the app. It cannot show a custom button label, so the confirm button always says "OK" instead of "Delete teacher". It **blocks the whole browser tab** while it is open, freezing every timer and every pending network response until the user clicks. And it hands you back a plain true/false with no way to show a "Deleting..." state while the request to the server is in flight.

So this project has its own, and the file opens by saying exactly that:

```
// frontend/src/components/ConfirmDialog.jsx (top)
import { useEffect } from "react";
import { TriangleAlert, X } from "lucide-react";
import Button from "../Button.jsx";
import IconButton from "../IconButton.jsx";

// Replaces window.confirm(), which cannot be styled and blocks the whole tab
// while it is open. `open` decides whether it shows; the page owns that state.
export default function ConfirmDialog({
  open,
  title = "Are you sure?",
  message,
  confirmLabel = "Delete",
  isWorking = false,
  onConfirm,
  onCancel,
}) {
```

Note the first two imports of other components in this folder. `ConfirmDialog` builds its buttons out of `Button` and `IconButton` rather than writing fresh `<button>` tags, so the dialog's Cancel button is character-for-character the same Cancel button as the one on the form behind it. Components using components is normal and good.

Note also `onConfirm` and `onCancel`. Those props are **functions**. The page passes in what should happen; the dialog decides when to call it. This is the standard React arrangement: the child owns the appearance, the parent owns the consequences.

THE ESCAPE KEY LISTENER

Users expect Escape to close a pop-up. The native `window.confirm` did that for free. This one has to arrange it.

```
// frontend/src/components/ConfirmDialog.jsx (the effect)
// Escape closes it, the way the native dialog did.
useEffect(() => {
  if (!open) {
    return;
  }

  function handleKeyDown(event) {
    if (event.key === "Escape") {
      onCancel();
    }
  }

  window.addEventListener("keydown", handleKeyDown);
  return () => window.removeEventListener("keydown", handleKeyDown);
}, [open, onCancel]);
```

`useEffect` is a React **hook**, which is the name React gives to its own built-in functions that start with `use`. This one is for doing things that are not "return some markup": setting timers, talking to the network, and here, listening for a key.

There is a real problem it is solving. A keypress on the Escape key does not land on the dialog, because the dialog is not focused. It lands on the window. So the dialog has to ask the window to tell it about every keypress, using `window.addEventListener`, and then check whether the key was Escape.

The two easy-to-miss details:

`if (!open) return;` bails out before attaching anything. The dialog component exists on the page all the time, even when it is invisible, so without this guard a closed dialog would still be listening.

The `return () => window.removeEventListener(...)` at the end is a **cleanup function**. React calls it before the effect runs again and when the component is removed. Without it, every time the dialog opened it would add another listener and never take one away. Open and close the dialog twenty times and Escape would call `onCancel` twenty times at once. This kind of leak is invisible in a small app and painful in a big one.

The array at the end, `[open, onCancel]`, is the **dependency list**. It tells React "re-run this effect when either of these two values changes". Since `open` is in the list, opening the dialog attaches the listener and closing it runs the cleanup.

RENDERING NOTHING WHEN CLOSED

frontend/src/components/ConfirmDialog.jsx

```
if (!open) {
  return null;
}
```


Same trick as `Alert`. This is why every page can leave `<ConfirmDialog ... />` sitting at the bottom of its JSX permanently and simply flip `open`.

NOTE

The order here matters and cannot be swapped. The `useEffect` call has to come **before** the `if (!open) return null`. React requires every hook to be called on every render, in the same order, every time. Move the `return null` above the `useEffect` and React will complain the first time the dialog opens. That is why the effect has its own internal `if (!open) return;` instead of just not being called.

THE BACKDROP, AND CLICKING OUTSIDE TO CLOSE

frontend/src/components/ConfirmDialog.jsx

```
return (  
  <div  
    className="fixed inset-0 z-50 flex items-center justify-center bg-slate-900/40 p-4"  
    onClick={onCancel}  
  >
```

That outer `<div>` is the backdrop, the dark sheet over the page. `fixed inset-0` pins it to all four edges of the browser window regardless of scrolling. `z-50` lifts it above everything else on the page. `bg-slate-900/40` is dark slate at 40 percent opacity, so you can still see the table underneath. `flex items-center justify-center` centres the white card in the middle of the screen, both across and down.

`onClick={onCancel}` on the backdrop is what makes clicking outside the card close the dialog.

Which immediately creates a problem, and the next few lines solve it.

frontend/src/components/ConfirmDialog.jsx

```
<div  
  role="dialog"  
  aria-modal="true"  
  className="w-full max-w-sm rounded-lg border border-slate-200 bg-white p-5 shadow-  
lg"  
  // Clicking the card itself must not reach the backdrop's onClick.  
  onClick={(event) => event.stopPropagation()}  
>
```

Browser clicks **bubble**: after the element you actually clicked has handled the click, the browser offers it to that element's parent, then that parent's parent, all the way up. So a click on the white card also counts as a click on the backdrop containing it. Without a stop, clicking the word "Delete teacher" inside the card would bubble out to the backdrop and close the dialog you were reading.

`event.stopPropagation()` stops the climb. The click is handled at the card and never reaches the backdrop. The confirm and cancel buttons inside the card still work normally, because their own `onClick` fires first, before any bubbling starts.

`role="dialog"` and `aria-modal="true"` tell screen readers this is a pop-up demanding an answer, not just another box on the page.

THE HEADER AND THE MESSAGE

frontend/src/components/ConfirmDialog.jsx

```
<div className="mb-3 flex items-start justify-between gap-3">
  <div className="flex items-center gap-2">
    <span className="flex h-8 w-8 items-center justify-center rounded-full bg-red-50 text-red-600">
      <TriangleAlert size={16} />
    </span>
    <h2 className="text-sm font-semibold text-slate-900">{title}</h2>
  </div>

  <IconButton onClick={onCancel} aria-label="Close">
    <X size={16} />
  </IconButton>
</div>

<p className="mb-5 text-sm text-slate-600">{message}</p>
```

The warning triangle sits in a pale red circle. `h-8 w-8` makes a 32-pixel square, `rounded-full` turns a square into a circle, and the flex centring puts the icon in its middle. `justify-between` on the outer row pushes the X to the far right.

That is now the third way out of this dialog: Escape, click the backdrop, or click the X. All three call the same `onCancel`.

THE BUTTONS, THE BUSY LABEL, AND THE DISABLED STATE

frontend/src/components/ConfirmDialog.jsx

```
<div className="flex justify-end gap-2">
  <Button variant="secondary" onClick={onCancel} disabled={isWorking}>
    Cancel
  </Button>
  <Button
    variant="danger"
    onClick={onConfirm}
    disabled={isWorking}
    className="disabled:opacity-60"
  >
    {isWorking ? "Deleting..." : confirmLabel}
  </Button>
</div>
</div>
</div>
);
}
```

`isWorking` is the prop that `window.confirm` could never give you. Deleting means an HTTP request to the Django server, and that takes time. During that time:

Both buttons get `disabled={isWorking}`, so the user cannot fire a second delete request for the same row, and cannot cancel out from under a request that has already gone.

The danger button's label swaps to "Deleting...", so the user can see why nothing has happened yet.

`disabled:opacity-60` fades the button while it is disabled. `disabled:` is another Tailwind variant, applying only in that state.

`confirmLabel` defaults to `"Delete"`. No page in this project overrides it, because every use is a delete, but the prop exists so the next thing you build ("Archive", "Publish", "Sign out everywhere") does not need a copy of this file.

PROP	TYPE	DEFAULT	WHAT IT DOES
<code>open</code>	boolean	none	<code>true</code> shows the dialog, <code>false</code> renders nothing
<code>title</code>	string	"Are you sure?"	Bold heading beside the warning triangle
<code>message</code>	string	none	The sentence explaining what will be lost
<code>confirmLabel</code>	string	"Delete"	Text on the red button when idle
<code>isWorking</code>	boolean	<code>false</code>	While true, both buttons disable and the red one says "Deleting..."
<code>onConfirm</code>	function	none	Called when the red button is clicked
<code>onCancel</code>	function	none	Called by Escape, the backdrop, the X, and Cancel

THE PAGE SIDE OF THE DIALOG

Now look at what the Teachers page has to hold to drive this. Two pieces of state:

frontend/src/pages/Teachers.jsx

```
// The row waiting on the confirm dialog, or null when it is closed.
const [teacherToDelete, setTeacherToDelete] = useState(null);
const [isDeleting, setIsDeleting] = useState(false);
```

`teacherToDelete` does double duty. It holds the row being deleted, so the message can name the person, and its being non-null is what opens the dialog. One piece of state instead of two.

The bin icon just stores the row:

frontend/src/pages/Teachers.jsx

```
function askToDelete(teacher) {
  setTeacherToDelete(teacher);
}
```

And confirming does the work:

frontend/src/pages/Teachers.jsx

```
async function confirmDelete() {
  setIsDeleting(true);

  try {
    await teachersApi.remove(teacherToDelete.id);
    setError("");
    setNotice("Teacher deleted.");
    setTeacherToDelete(null);
    reload();
  } catch (problem) {
    setError(problem.message);
    setTeacherToDelete(null);
  } finally {
    setIsDeleting(false);
  }
}
```

`setIsDeleting(true)` at the top is what flips the button to "Deleting...". `setTeacherToDelete(null)` closes the dialog, and it happens on both the success path and the failure path, so a rejected delete closes the pop-up and leaves the red `Alert` explaining why. `finally` runs either way, so the busy flag always gets cleared.

Then the tag itself, the last thing in the page's JSX:

frontend/src/pages/Teachers.jsx

```
<ConfirmDialog
  open={teacherToDelete !== null}
  title="Delete teacher"
  message={`Delete ${teacherToDelete?.name} || "this teacher"? Their courses go too. This
  cannot be undone.`}
  isWorking={isDeleting}
  onConfirm={confirmDelete}
  onCancel={() => setTeacherToDelete(null)}
/>
```

The backticks make a **template string**, where `${...}` drops a value into the middle of text. The `?.` in `teacherToDelete?.name` is **optional chaining**: if `teacherToDelete` is `null`, the whole expression is `undefined` instead of crashing. That matters because this line is evaluated on every render, including all the renders where the dialog is closed and `teacherToDelete` is `null`. The `||` "this teacher" then supplies a fallback so the sentence still reads properly.

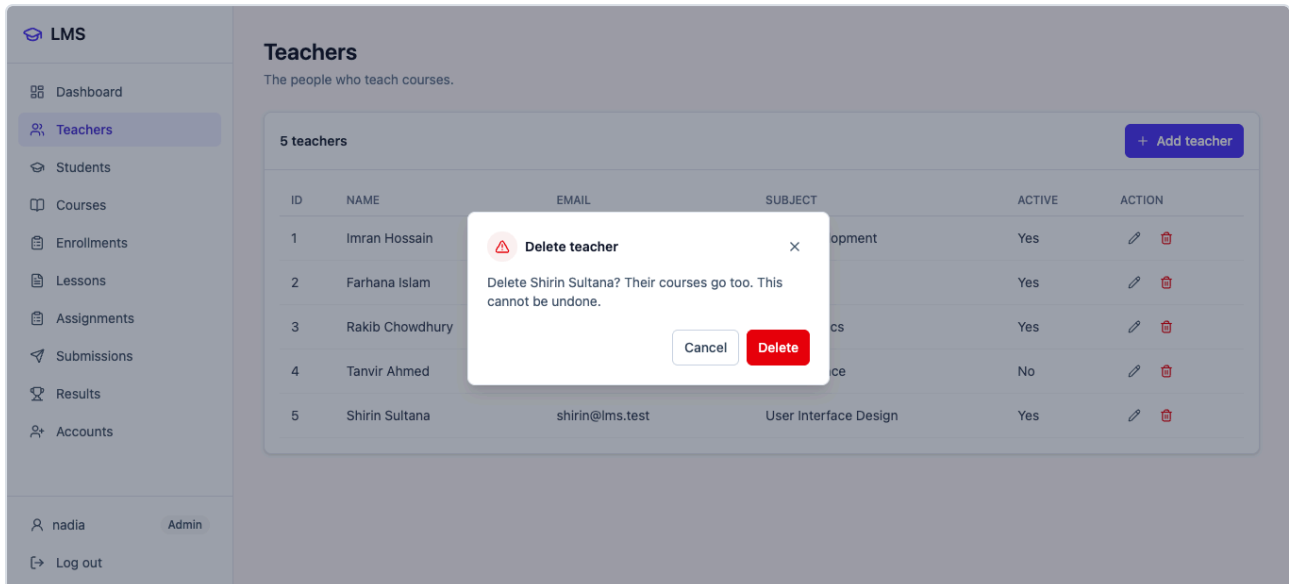


Figure 10.2 ConfirmDialog naming the exact teacher, with Cancel and Delete built from Button.

Every page writes its own warning, naming what else will be destroyed:

frontend/src/pages/Courses.jsx

```
message={`Delete ${courseToDelete?.title}? Its lessons, assignments and enrollments go too. This cannot be undone.`}
```

frontend/src/pages/Lessons.jsx

```
message={`Delete ${lessonToDelete?.title}? Its assignments go too. This cannot be undone.`}
```

Those sentences are true. They describe the `on_delete=models.CASCADE` rules in `backend/backend/models.py` from the backend chapters: `Course.teacher`, `Lesson.course`, `Assignment.course`, `Assignment.lesson` and `Enrollment.course` are all set to cascade, which means deleting the parent row deletes the children with it. A generic "Are you sure?" would not have told the user that deleting one teacher takes their courses with them.

CHECKPOINT On the Teachers page, click a bin icon. The dialog should name the actual teacher in its sentence. Now press Escape: it closes. Open it again and click the dark area outside the card: it closes. Open it a third time and click on the card's own heading text: it stays open. If that third click closes it, `event.stopPropagation()` is missing or on the wrong element.

ProtectedRoute

`ProtectedRoute.jsx` also lives in this folder, but it is not a piece of user interface. It is the gate that decides whether a page may be shown at all, and it is covered in full in Chapter 9 alongside the auth context and the role rules.

index.js: the barrel file

Twelve components live in twelve separate files. The Teachers page needs eight of them. Without any help, the top of that page would read:

```
// what Teachers.jsx would need without a barrel file
import Alert from "../components/Alert.jsx";
import Button from "../components/Button.jsx";
import Checkbox from "../components/Checkbox.jsx";
import ConfirmDialog from "../components/ConfirmDialog.jsx";
import IconButton from "../components/IconButton.jsx";
import Input from "../components/Input.jsx";
import PageHeader from "../components/PageHeader.jsx";
import Table from "../components/Table.jsx";
```

Eight lines, and `../components/` typed eight times, repeated across the twelve pages that use these components. So the folder has one more file whose only job is to re-export everything from one place. This kind of file is called a **barrel**.

frontend/src/components/index.js

```
export { default as Alert } from "./Alert.jsx";
export { default as Button } from "./Button.jsx";
export { default as Checkbox } from "./Checkbox.jsx";
export { default as ConfirmDialog } from "./ConfirmDialog.jsx";
export { default as Div } from "./Div.jsx";
export { default as IconButton } from "./IconButton.jsx";
export { default as Input } from "./Input.jsx";
export { default as PageHeader } from "./PageHeader.jsx";
export { default as ProtectedRoute } from "./ProtectedRoute.jsx";
export { default as Select } from "./Select.jsx";
export { default as Table } from "./Table.jsx";
export { default as Textarea } from "./Textarea.jsx";
```

Read one line: `export { default as Alert } from "./Alert.jsx"` means "take the default export out of `Alert.jsx`, give it the name `Alert`, and export it again from here." No import statement is needed; `export ... from` does both steps at once. Nothing else is in the file. It defines no component of its own.

Now the real top of the Teachers page is one statement:

frontend/src/pages/Teachers.jsx

```
import {
  Alert,
  Button,
  Checkbox,
  ConfirmDialog,
  IconButton,
  Input,
  PageHeader,
  Table,
} from "../components/index.js";
```

The braces mean these are **named** imports, matching the names the barrel exported. The individual files use `export default`, so importing directly from `Alert.jsx` would take no braces; importing from `index.js` takes braces because the barrel gave everything a name. That asymmetry catches everyone once.

Pages that need less write less. Login needs three:

frontend/src/pages/Login.jsx

```
import { Alert, Button, Input } from "../components/index.js";
```

Dashboard needs two:

frontend/src/pages/Dashboard.jsx

```
import { Alert, PageHeader } from "../components/index.js";
```

And `App.jsx` reaches into the same barrel for the router gate:

frontend/src/App.jsx

```
import { ProtectedRoute } from "../components/index.js";
```

The second thing the barrel buys you is freedom to reorganise. If you later split the folder into `components/forms/` and `components/layout/`, you change the paths inside `index.js` and every page keeps working untouched.

TIP

When you add a thirteenth component, add its line to `index.js` in the same alphabetical order. The list is sorted, and a sorted list is a list you can scan. Forgetting the line is the most common cause of `The requested module does not provide an export named 'X'` in this project.

COMMON MISTAKE

Typing `from "../components"` and leaving off `/index.js`. Every import in this codebase names the file with its extension, `.js` or `.jsx`, and so should yours. Following one consistent rule means you never have to stop and wonder whether a given path will resolve.

The whole folder at a glance

FILE	RENDERS	KEY PROPS
<code>styles.js</code>	nothing, exports two class strings	<code>inputBox</code> , <code>labelText</code>
<code>Div.jsx</code>	a <code><div></code>	<code>className</code> , <code>children</code>
<code>PageHeader.jsx</code>	<code><h1></code> plus <code><p></code>	<code>title</code> , <code>subtitle</code>
<code>Alert.jsx</code>	coloured banner, or nothing	<code>variant</code> , <code>children</code>
<code>Button.jsx</code>	<code><button></code> with text	<code>variant</code> , <code>type</code> , <code>children</code>
<code>IconButton.jsx</code>	<code><button></code> with an icon	<code>variant</code> , <code>children</code> , <code>aria-label</code>
<code>Input.jsx</code>	label plus <code><input></code> , eye button on passwords	<code>label</code> , plus anything an input takes
<code>Checkbox.jsx</code>	tick box plus its text	<code>label</code> , <code>checked</code> , <code>onChange</code>
<code>Select.jsx</code>	label plus <code><select></code>	<code>label</code> , <code>placeholder</code> , <code>children</code>
<code>Textarea.jsx</code>	label plus <code><textarea></code>	<code>label</code> , <code>rows</code>
<code>Table.jsx</code>	scroll box, <code><table></code> , header row	<code>columns</code> , <code>children</code>
<code>ConfirmDialog.jsx</code>	pop-up over the page, or nothing	<code>open</code> , <code>message</code> , <code>isWorking</code> , <code>onConfirm</code> , <code>onCancel</code>
<code>ProtectedRoute.jsx</code>	its children, or a redirect	<code>role</code> , <code>children</code> (Chapter 9)
<code>index.js</code>	nothing, re-exports the rest	none

Two patterns cover the whole folder. Either the component wraps one HTML element and forwards everything else with `...rest` (`Input`, `Select`, `Textarea`, `Checkbox`, `Button`, `IconButton`, `Div`), or it owns a bit of structure and leaves a hole for the page to fill with `children` (`Table`, `Alert`, `ConfirmDialog`). Once you can spot which of the two a file is doing, you can read any of them in under a minute.

TIP

Twelve components, none longer than eighty lines, and not one `useState` among them. The only hooks in the folder are the `useEffect` for the Escape key in `ConfirmDialog` and the two that `ProtectedRoute` needs to read the login state. That is deliberate. State lives in the pages; the components only draw. Keeping it that way is what makes them safe to use on twelve pages at once.

What you have now

- A clear idea of what a React component is: a capital-letter function returning JSX, usable as a tag
- Props, destructured in the function signature, and `children` for whatever sits between the tags
- The `...rest` spread pattern, so a component forwards `type`, `required`, `disabled`, `onChange` and anything else without listing them
- `styles.js` holding the two class strings that `Input`, `Select` and `Textarea` all share, so a look change is a one-line edit
- The `looks` object pattern behind the `variant` prop on `Button`, `IconButton` and `Alert`
- `Alert` and `ConfirmDialog` both returning `null` when they have nothing to show, so pages can render them unconditionally
- A confirm dialog that closes on Escape, on a backdrop click and on the X, that stops clicks inside the card from bubbling out, and that disables both buttons and says "Deleting..." while the delete request is in flight
- `Table` owning the frame and the header row while each page supplies its own `<tr>` cells
- One import line per page, thanks to the barrel at `frontend/src/components/index.js`

The Page Pattern, Built Once: Students

This is the chapter to read twice.

The app has eight resource pages: Teachers, Students, Courses, Enrollments, Lessons, Assignments, Submissions and Results. They are not eight different designs. They are the same page, typed out eight times with different nouns in it. Once you can build one of them without looking, you can build the other seven in an afternoon.

We are going to build Students from an empty file.

Two words come up constantly from here on. The **backend** is the Django program, the part that keeps the database and answers questions over the network. The **frontend** is the React app that runs in your browser and draws the screen. They are separate programs that talk to each other.

Students is the page the project treats as the reference copy. Its own comments spell out the six steps, and every other page follows them in the same order:

1. state for the rows, the form, and the errors
2. `useEffect()` reads the list from the API when the page opens
3. `reload()` re-runs that after a save or a delete
4. `handleSave()` creates a new row, or updates the one being edited
5. `askToDelete()` opens the confirm dialog, `confirmDelete()` runs on agree
6. the JSX: banners, form, table, confirm dialog

JSX is the HTML-looking markup you write inside a React file. It looks like HTML but it lives in a `.js` or `.jsx` file and can hold JavaScript values inside curly braces.

Here is the finished page, so you know where we are going.

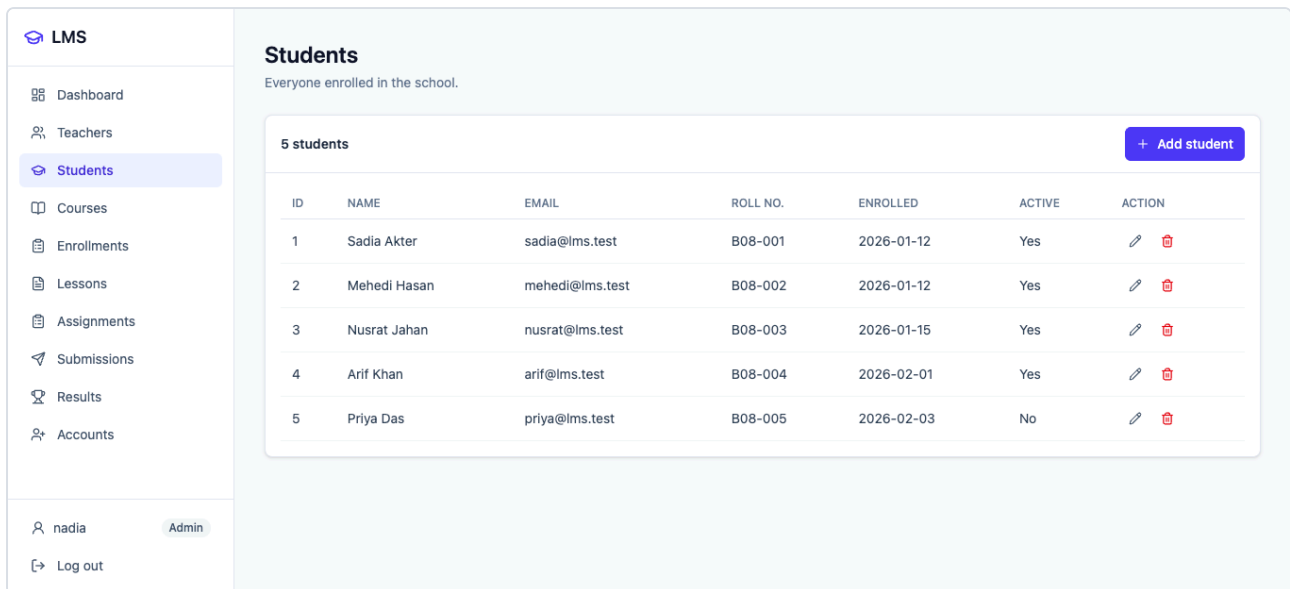


Figure 11.1 The Students page: a red or green banner slot at the top, an Add button, and one row per student.

What already exists before we start

You built these in earlier chapters. The Students page only uses them; it does not define them.

THING WE IMPORT	WHERE IT LIVES	WHAT IT GIVES US
<code>studentsApi</code>	<code>frontend/src/api.js</code>	the four calls that talk to Django
<code>useAuth</code>	<code>frontend/src/auth.js</code>	the signed-in user, including their role
<code>useFlash</code>	<code>frontend/src/flash.js</code>	a short message that clears itself
<code>canCreate</code> , <code>canWrite</code>	<code>frontend/src/permissions.js</code>	may this role add / change rows?
<code>Alert</code> , <code>Button</code> , <code>Table</code> , ...	<code>frontend/src/components/index.js</code>	the small shared UI pieces

`studentsApi` is worth a second look, because the whole page is built on these four calls. In `api.js` it is made by one **factory function**, meaning a function whose job is to build and hand back another value rather than do work itself:

```
// frontend/src/api.js (already written, shown here for reference)
export const studentsApi = resource("student");
```

That single line produces four functions. This is what each one sends to the server:

YOU WRITE	THE BROWSER SENDS	TO THIS URL
<code>studentsApi.list()</code>	GET	<code>http://127.0.0.1:8001/api/student/</code>
<code>studentsApi.create(values)</code>	POST	<code>http://127.0.0.1:8001/api/student/</code>
<code>studentsApi.update(id, values)</code>	PUT	<code>http://127.0.0.1:8001/api/student/5/</code>
<code>studentsApi.remove(id)</code>	DELETE	<code>http://127.0.0.1:8001/api/student/5/</code>

An **API call** here means: the browser sends a message over the network to the Django program, and Django sends an answer back. GET means "give me the data". POST means "here is a new row". PUT means "replace this row". DELETE means "remove this row".

The rows themselves come from `StudentSerializer` on the backend. A **serializer** is the Django class that decides which fields of a database row go out over the network and which ones the server will accept coming in. `StudentSerializer` lists exactly these:

FIELD	TYPE	REQUIRED BY THE SERVER?
<code>id</code>	number	filled in by the database
<code>name</code>	text	no (<code>blank=True, null=True</code>)
<code>email</code>	text	no
<code>enrollment_date</code>	date, <code>YYYY-MM-DD</code>	yes
<code>is_active</code>	true / false	no, defaults to true
<code>roll_number</code>	text	no

NOTE

The field is called `roll_number` with an underscore, because that is what Python and the database call it. In React we will hold it in a variable called `rollNumber`, because that is the JavaScript habit. The two names have to be joined up by hand at exactly one place, in `handleSave()`. Watch for it later.

Create the file

Open a terminal (the black window where you type commands), go to the frontend folder, and make an empty file:

```
cd "/Users/tausif/Projects/Ostad/Batch 08/lms/frontend"
touch src/pages/Students.jsx
```

`touch` makes an empty file if it does not exist, and does nothing to the contents if it does. On Windows PowerShell use `New-Item -ItemType File src/pages/Students.jsx` instead.

The route to this file already exists from the routing chapter, in `frontend/src/App.jsx`:

```
// frontend/src/App.jsx (already written)
<Route path="/students" element={<Students />} />
```

So the moment this file exports a working component, the address `http://localhost:5180/students` will show it.

The imports

Every file starts by saying what it needs. Type these lines at the top of the empty file:

`frontend/src/pages/Students.jsx`

```
import { useEffect, useState } from "react";
import { Pencil, Plus, Trash2, X } from "lucide-react";
import { studentsApi } from "../api.js";
import { useAuth } from "../auth.js";
import { useFlash } from "../flash.js";
import { canCreate, canWrite } from "../permissions.js";
```

Line by line:

- `useState` and `useEffect` are the two React **hooks** we need. A hook is a function whose name starts with `use`, which React lets you call at the top of a component to give that component a memory or a behaviour.
- `lucide-react` is the icon set listed in `frontend/package.json`. `Pencil`, `Plus`, `Trash2` and `X` are four small drawings, used as React components.
- `../api.js` means "go up one folder, then open `api.js`". We are in `src/pages/`, so `..` is `src/`.
- `useAuth` gives us the signed-in user. `useFlash` gives us the self-clearing green message. `canCreate` and `canWrite` answer permission questions.

Then the shared UI pieces, imported in one block from the components folder:

```
// frontend/src/pages/Students.jsx (continuing the imports)
import {
  Alert,
  Button,
  Checkbox,
  ConfirmDialog,
  IconButton,
  Input,
  PageHeader,
  Table,
} from "../components/index.js";
```

`components/index.js` is a list of re-exports: it imports each component from its own file and exports it again, so other files have one place to import from. Without it this page would need eight separate import lines pointing at eight separate files. That is the only reason that file exists.

COMMON MISTAKE

Writing `import { studentsApi } from "../api"` without `.js`. Vite (the tool that runs the frontend while you work) will usually still find it, but every other file in this project writes the extension. Mixed styles make searching the codebase harder. Keep the `.js`.

The map at the top of the file

Now the comment block and the function that holds everything else. Copy the comment too. It is the map of the file, and it is the reason this page is the one you were told to read first.

```
// frontend/src/pages/Students.jsx (after the imports)
// Read this page first. All eight resource pages are the same six steps:
//   1. state for the rows, the form, and the errors
//   2. useEffect()    reads the list from the API when the page opens
//   3. reload()       re-runs that effect after a save or a delete
//   4. handleSave()   creates a new row, or updates the one being edited
//   5. askToDelete() / confirmDelete() opens the dialog, then deletes
//   6. the JSX: banners, form, table, confirm dialog
export default function Students() {
```

`export default function Students()` declares a **component**: a function that returns a description of a screen. `export default` means "this file's main thing is this function", which is what lets `App.jsx` write `import Students from "../pages/Students.jsx"`.

Everything from here on goes inside those curly braces, in the order shown.

Step 1 - state

WHAT STATE IS, AND WHY YOU NEVER JUST ASSIGN A VARIABLE

React draws the screen by calling your component function and using whatever it returns. If you want the screen to change, React has to call the function again. It only knows to do that if you tell it, and `useState` is how you tell it.

This does nothing at all:

```
// WRONG. Not from the project. Never write this.  
let students = [];  
students = await studentsApi.list(); // the screen does not change
```

The variable holds the new data, but React was never told, so it never re-draws, so the table stays empty. Worse, the next time React does re-draw for some other reason, the function runs again from the top and `students` goes back to `[]`.

`useState` fixes both problems. It stores the value outside the function call, and its setter tells React to run the function again.

```
// frontend/src/pages/Students.jsx (step 1, the rows)  
// 1. state  
const [students, setStudents] = useState([]);  
const [isLoading, setIsLoading] = useState(true);  
const [error, setError] = useState('');
```

`useState([])` hands back two things in an array: the current value, and a function that replaces it. The square brackets on the left pull them apart and name them. The value passed to `useState` is only the starting value, used on the very first draw.

- `students` starts as `[]`, an empty list, because the page has not asked the server anything yet.
- `error` starts as `''`, an empty string, meaning "no error to show".
- `isLoading` starts as `true`.

Why loading is a piece of state. Talking to a server takes time. Somewhere between a few milliseconds and a few seconds pass between "the page opened" and "the rows arrived". During that gap `students` is `[]`, which looks exactly like "there are no students". Those are two different facts and the user deserves to be told which one is true. So the page keeps a separate true/false value: `isLoading` starts `true`, and is set to `false` once the answer comes back, whether it worked or failed. In step 6 it decides between showing `Loading...` and showing `3 students`.

THE GREEN BANNER

```
// frontend/src/pages/Students.jsx (step 1, continued)
// A short "Saved" line that clears itself after a few seconds.
const [notice, setNotice] = useFlash();
```

`useFlash` is a **custom hook**: a function of your own that calls React's hooks inside it. It behaves like `useState("")`, with one addition. Its own `useEffect` starts a 4 second timer whenever the message is non-empty, and clears the message when the timer fires. That is why no code in this page ever has to remember to take the green banner down.

PERMISSIONS DECIDE WHETHER BUTTONS EXIST

```
// frontend/src/pages/Students.jsx (step 1, continued)
// The server enforces this too. Hiding the buttons just keeps the page
// honest about what will actually work.
const { user } = useAuth();
const mayCreate = canCreate(user?.role, "student");
const mayEdit = canWrite(user?.role, "student");
const [isSaving, setIsSaving] = useState(false);
```

`useAuth()` returns an object holding `user`, `isLoggedIn`, `login` and `logout`. The curly braces on the left pick out just `user`.

`user?.role` uses the **optional chaining** operator `?.`. It means "if `user` is null, produce nothing instead of crashing". `user` can briefly be null, so plain `user.role` would throw an error and blank the page.

The string `"student"` is the **resource name**, the label this project uses for one kind of row. Look it up in `permissions.js`:

```
// frontend/src/permissions.js (already written)
const WRITERS = {
  teacher: [ADMIN],
  student: [ADMIN],
  course: [ADMIN, TEACHER],
  enrollment: [ADMIN, TEACHER],
  lesson: [ADMIN, TEACHER],
  assignment: [ADMIN, TEACHER],
  submission: [ADMIN, TEACHER],
  results: [ADMIN, TEACHER],
};
```

Only an admin may write students. So for an admin `mayCreate` and `mayEdit` are both `true`; for a teacher or a student both are `false`. Those two values are **booleans**, the type that can only ever be true or false. In step 6 they decide whether the Add button and the pencil and trash icons get drawn at

all. Not greyed out. Not hidden with CSS. Never created.

NOTE

This is decoration, not security. `frontend/src/permissions.js` says so in its first comment: the server is what actually enforces the rules, and a mistake in this file is a cosmetic bug rather than a hole. A curious person can still send a POST to `/api/student/` by hand, and Django will still refuse them, because `StudentListCreateView` in `backend/backend/views.py` carries `permission_classes = [AdminWrites]`.

`isSaving` is another loading flag, this one for the Save button. It stops a user pressing Save four times and creating four students.

THE DELETE STATE

```
// frontend/src/pages/Students.jsx (step 1, continued)
// The row waiting on the confirm dialog, or null when it is closed.
const [studentToDelete, setStudentToDelete] = useState(null);
const [isDeleting, setIsDeleting] = useState(false);
```

`studentToDelete` holds a whole student object, not just an id. Holding the object means the dialog can say "Delete Rahim Uddin?" instead of "Delete row 7?". `null` means the dialog is closed. This one piece of state is both the switch and the payload.

THE FORM FIELDS, ONE PIECE OF STATE EACH

```
// frontend/src/pages/Students.jsx (step 1, continued)
const [name, setName] = useState("");
const [email, setEmail] = useState("");
const [rollNumber, setRollNumber] = useState("");
const [enrollmentDate, setEnrollmentDate] = useState("");
const [isActive, setIsActive] = useState(true);
```

Five fields, five pieces of state. Each one starts empty, except `isActive` which starts `true`, matching the Django default `is_active = models.BooleanField(default=True)` in `backend/backend/models.py`.

This is the **controlled input** idea, and it is worth being precise about. In plain HTML a text box owns its own contents; you ask it what it holds when you need to know. In React it works the other way round. The box is told what to show, and it reports every keystroke back:

```
<Input
  label="Name"
  value={name}
  onChange={(event) => setName(event.target.value)}
/>
```

`value={name}` means the box always displays whatever `name` currently holds. `onChange` fires on each keystroke; `event.target.value` is the text now in the box; `setName` stores it, React re-draws, and the box displays the new value. The state is the single source of truth, and the box is just a window onto it.

This pays off in three places in this page. Opening the form for editing is just five `set` calls, and the boxes fill themselves in. Clearing the form is five more, with empty strings. And `handleSave` can build the object to send from state alone, without reading anything back out of the boxes on screen:

```
const values = {
  name,
  email,
  roll_number: rollNumber,
  enrollment_date: enrollmentDate,
  is_active: isActive,
};
```

Five separate pieces of state, gathered into one object at the moment of sending. That one object is what goes to Django. Some codebases keep all fields in a single state object from the start. This project keeps them apart while typing and joins them at the end, which makes each `onChange` a one-liner.

IS THE FORM OPEN, AND IS IT ADDING OR EDITING?

```
// frontend/src/pages/Students.jsx (step 1, continued)
const [formIsOpen, setFormIsOpen] = useState(false);
const [editingId, setEditingId] = useState(0);
```

`formIsOpen` decides whether the form is on screen. It starts `false`, so the page opens as a plain table.

`editingId` is the important one. It answers "which student is this form about?".

- `0` means the form is adding a brand new student.
- any other number is the `id` of the student being changed.

That single number steers three things: the heading text, whether `handleSave` calls `create` or `update`, and which id `update` is given.

COMMON MISTAKE

Mixing up the two "nothing" markers in this file. `editingId` uses the number `0` for "not editing anything". `studentToDelete` uses `null` for "not deleting anything". They are different types on purpose, because one is compared with `editingId === 0` and the other with `studentToDelete !== null`. If you write `editingId === null` the page will always take the update branch and every new student will fail.

THE RELOAD COUNTER

```
// frontend/src/pages/Students.jsx (step 1, last part)
// Bumping this re-runs the effect below. Saving and deleting call reload()
// so the table shows what the server now holds.
const [reloadCount, setReloadCount] = useState(0);

function reload() {
  setReloadCount((count) => count + 1);
}
```

This is step 3, and it lives here in the file because a function belongs next to the state it changes. Type it now; the full explanation is a few pages down, after you have seen the effect it drives.

Step 2 - read the list when the page opens

The page needs to fetch its rows exactly once, when it first appears. Not on every keystroke, not on every re-draw. `useEffect` is the tool for that: it runs code *after* React has drawn, and lets you say when it should run again.

```
// frontend/src/pages/Students.jsx (step 2)
// 2. read the list when the page opens. GET /api/student/ returns a plain
// array, not {count, results}, because this backend has no pagination.
useEffect(() => {
  async function load() {
    try {
      setStudents(await studentsApi.list());
      setError('');
    } catch (problem) {
      setError(problem.message);
    } finally {
      setIsLoading(false);
    }
  }

  load();
}, [reloadCount]);
```

Pagination means a server handing back one page of rows at a time instead of all of them. This backend does not switch it on: `backend/lms/settings.py` sets no `DEFAULT_PAGINATION_CLASS`, so every list comes back whole.

Five things are happening here. Take them one at a time.

The dependency array. The `[reloadCount]` at the very end is the list of values React watches. After every draw, React compares each value in that list with what it was last time. If nothing changed, the effect is skipped. If something changed, the effect runs again.

- `[]`, an empty array, means "nothing is watched, so run once when the page opens and never again".
- `[reloadCount]` means "run once when the page opens, and again any time `reloadCount` becomes a different number".

That second form is what gives us the reload trick in step 3. If you forget the array entirely and write `useEffect(() => { ... })`, the effect runs after *every* draw. Since the effect calls `setStudents`, which causes a draw, which runs the effect, which calls `setStudents`, your browser will fetch the student list forever.

**COMMON
MISTAKE**

An endless fetch loop from a missing dependency array. If the Network tab in your browser's developer tools shows hundreds of requests to `student/` and your laptop fan comes on, this is why. Check that the effect ends with `}`, `[reloadCount];` and not `});`.

"Runs once when the page opens" means once per visit to the page, not once per browser session. Click Teachers then click back on Students and the effect runs again, because React threw the old Students component away and made a new one.

The async function inside the effect. Fetching takes time, so `studentsApi.list()` hands back a **promise**: an object meaning "the answer is not here yet, but it is coming". `await` pauses inside that function until the answer arrives. `await` is only allowed inside a function marked `async`. The effect's own function cannot be `async`, because React expects it to return either nothing or a cleanup function, and an `async` function always returns a promise. So the pattern is: declare `async function load()` inside, then call `load()` on the next line. Every page in this project does exactly this.

**COMMON
MISTAKE**

Writing `useEffect(async () => { ... }, [])`. It looks tidier and it partly works, but React will treat the returned promise as a cleanup function and complain. Declare-then-call is the fix.

`setStudents(await studentsApi.list())`. Read it inside out. `studentsApi.list()` fires the GET request. `await` waits for the parsed answer. `setStudents(...)` stores it and asks React to re-draw. Because this backend has no pagination turned on, the answer is a plain array like `[{...}, {...}]`,

and it can go straight into state. If pagination were on you would get `{count: 12, results: [...]}` and you would have to pull `.results` out first.

try / catch / finally. This is the shape of every API call in the app.

- `try` holds the risky work.
- `catch (problem)` runs only if something inside `try` threw an error. `api.js` throws a real `Error` object with a readable `message` on it, so `problem.message` is already a sentence a human can read. It goes into `error` state.
- `finally` runs either way, whether the try succeeded or the catch caught. Turning off `isLoading` belongs there, because the loading text has to stop even when the request failed.

Where does the message end up? `setError(problem.message)` puts it in state, and step 6 draws `<Alert>{error}</Alert>` at the top of the page. `Alert` returns `null` when its children are empty, so an empty `error` draws nothing at all. There is no "show the banner" call anywhere. Setting the string is showing the banner.

The messages themselves come from `api.js`. If Django is not running you get `Could not reach the server. Is Django running on http://127.0.0.1:8001?`. If Django refuses you, `readError()` in `api.js` pulls out the `detail` line Django sent; for a student or a teacher trying to change a student row that line is `Only an admin can change teacher and student records.`, the `message` set on the `AdminWrites` class in `backend/backend/permissions.py`. If a field is rejected you get `enrollment_date: This field is required.`

See Figure 5.1.

Notice `setError("")` on the success path. Without it, an error from a previous attempt would stay on screen after a later attempt worked.

Step 3 - reload()

Look again at the three lines from step 1:

```
// frontend/src/pages/Students.jsx (step 3)
const [reloadCount, setReloadCount] = useState(0);

function reload() {
  setReloadCount((count) => count + 1);
}
```

The problem it solves: after you save a new student, the table on screen is out of date. It still shows what the server held a moment ago. You could try to patch the array in state by hand, but then the screen and the database can drift apart, and you would have to duplicate the server's own logic for ids and defaults.

The simpler answer is to ask the server again. But the fetching code lives inside the effect, and you cannot call an effect directly. So instead we change a value the effect is watching. `reloadCount` goes from 0 to 1, React notices the dependency changed, and the effect runs a second time. `handleSave` and `confirmDelete` both end with `reload()`.

`setReloadCount((count) => count + 1)` passes a function rather than a number. React calls it with the current value and stores what comes back. Compare it with `setReloadCount(reloadCount + 1)`, which reads a `reloadCount` that was captured when this draw started. In this page either version happens to work, but the function form is always correct, so it is the one to learn.

CHECKPOINT After you finish the page, open the browser's developer tools with F12, go to the Network tab, and filter for `student`. Loading `/students` shows one GET to `student/`. Save a student and you will see a POST, immediately followed by a second GET. That second GET is `reload()` doing its job.

The three form helpers

These sit between step 2 and step 4 in the file. They are the only places that change `editingId`.

```
// frontend/src/pages/Students.jsx (opening a blank form)
function openEmptyForm() {
  setName('');
  setEmail('');
  setRollNumber('');
  setEnrollmentDate('');
  setIsActive(true);
  setEditingId(0);
  setFormIsOpen(true);
}
```

Clear all five fields, set `editingId` to 0 so `handleSave` will create, then open the form. Clearing matters: without it, opening Add straight after an Edit would show the previous student's details, and saving would quietly create a copy of them.

```
// frontend/src/pages/Students.jsx (opening a filled form)
function openFormForEditing(student) {
  setName(student.name ?? '');
  setEmail(student.email ?? '');
  setRollNumber(student.roll_number ?? '');
  setEnrollmentDate(student.enrollment_date ?? '');
  setIsActive(student.is_active);
  setEditingId(student.id);
  setFormIsOpen(true);
}
```

The whole student object arrives as an argument, straight from the table row that was clicked.

`??` is the **nullish coalescing** operator: "use the left side, unless it is null or undefined, in which case use the right side". It is here because the Django model marks `name`, `email` and `roll_number` as `blank=True, null=True`, so the API really can send `"name": null`. Putting `null` into `value` on a text box makes React switch that box from controlled to uncontrolled and print a warning. `?? ""` turns any null into an empty string first.

`is_active` gets no `??` because it is a plain `BooleanField` and is always sent as true or false.

```
// frontend/src/pages/Students.jsx (closing the form)
function closeForm() {
  setFormIsOpen(false);
}
```

Closing does not clear the fields. It does not need to, because whichever function opens the form next fills every one of them.

Step 4 - save

One function handles both creating and updating. `editingId` decides which.

```
// frontend/src/pages/Students.jsx (step 4, the first half)
// 4. create, or update the row being edited. Student.enrollment_date is a
// normal editable field here; it is Enrollment.enrollment_date that the
// server fills in for you.
async function handleSave(event) {
  event.preventDefault();
  setIsSaving(true);

  const values = {
    name,
    email,
    roll_number: rollNumber,
    enrollment_date: enrollmentDate,
    is_active: isActive,
  };
}
```

`event.preventDefault()` is not optional. A `<form>` element's built-in behaviour is to send itself to a URL and reload the whole page, which is how the web worked before JavaScript. A full reload would throw away all your state, build the component again from scratch, and wipe the message you were about to show. `preventDefault()` cancels that so your own code can handle the submit.

**COMMON
MISTAKE**

Leaving out `event.preventDefault()`. The symptom is odd: the page seems to flicker and reset, the green banner appears for a split second and vanishes, and the browser address bar picks up a `?` on the end. The save may even have worked. The reload is what you are seeing.

`values` is where the naming worlds meet. `name` and `email` use the shorthand form, because the JavaScript variable and the server field are already spelled the same. `roll_number: rollNumber` and `enrollment_date: enrollmentDate` and `is_active: isActive` translate JavaScript names into the names `StudentSerializer` expects. Send `rollNumber` by mistake and Django ignores it, so the field silently saves as empty.

The comment above the function is worth reading twice. `Student.enrollment_date` is a normal `DateField` you can type into. `Enrollment.enrollment_date`, on the Enrollments page, is `auto_now_add=True`, which means the server sets it when the row is first created and ignores anything you send. Same column name, two different models, two different rules.

```
// frontend/src/pages/Students.jsx (step 4, the second half)
try {
  if (editingId === 0) {
    await studentsApi.create(values);
    setNotice("Student added.");
  } else {
    await studentsApi.update(editingId, values);
    setNotice("Student updated.");
  }
  setError("");
  closeForm();
  reload();
} catch (problem) {
  setError(problem.message);
} finally {
  setIsSaving(false);
}
}
```

The same try / catch / finally shape as the effect.

The `if` is the whole create-or-update decision, in one line. `editingId === 0` means the form was opened by `openEmptyForm`, so POST a new row. Otherwise PUT over the existing row with that id.

The four lines after the `if` only run when nothing threw:

- `setNotice(...)` puts text in the flash message, which draws the green banner and starts its own 4 second timer.
- `setError("")` clears any old red banner.

- `closeForm()` folds the form away.
- `reload()` bumps the counter, the effect runs again, the table catches up.

If the call threw, none of those run. The form stays open with the user's typing still in it, the red banner explains what Django objected to, and they can fix it and press Save again. That behaviour is free, purely because those lines are inside `try` after the `await`.

`finally` sets `isSaving` back to `false` no matter what. This is the one that bites people who skip `finally`: put `setIsSaving(false)` only on the success path and a single failed save leaves the button disabled and reading `Saving...` forever, with no way out but a page refresh.

TIP

`create` sends POST and `update` sends PUT. PUT means "replace this row with exactly these fields". That is why `values` always carries all five fields, even the ones the user did not touch. Send a partial object with PUT and the missing fields are treated as blank.

Step 5 - delete, in two halves

Deleting is destructive, so it is deliberately split. The trash icon does not delete anything. It only asks.

```
// frontend/src/pages/Students.jsx (step 5, the ask)
// 5. the trash button only opens the dialog. ConfirmDialog calls the second
// function once the user has actually agreed.
function askToDelete(student) {
  setStudentToDelete(student);
}
```

One line. It stores the whole student object. Storing it does three jobs at once: `studentToDelete !== null` opens the dialog, `studentToDelete?.name` fills in the message, and `studentToDelete.id` is the id to delete when the user agrees.

```
// frontend/src/pages/Students.jsx (step 5, the act)
async function confirmDelete() {
  setIsDeleting(true);

  try {
    await studentsApi.remove(studentToDelete.id);
    setError("");
    setNotice("Student deleted.");
    setStudentToDelete(null);
    reload();
  } catch (problem) {
    setError(problem.message);
    setStudentToDelete(null);
  } finally {
    setIsDeleting(false);
  }
}
```

`setStudentToDelete(null)` appears in both branches, which is on purpose. On success the dialog closes because the job is done. On failure it also closes, so the red banner underneath is visible rather than hidden behind the grey backdrop.

`isDeleting` goes to the dialog as `isWorking`. Inside `ConfirmDialog.jsx` it disables both buttons and changes the confirm label to `Deleting...`, so a slow network cannot produce two DELETE requests for the same row.

The full round trip, in order:

1. User clicks the trash icon on a row. `onClick={() => askToDelete(student)}` runs.
2. `studentToDelete` is now that student, so `open={studentToDelete !== null}` is `true` and the dialog appears.
3. User clicks Cancel, clicks the X in the corner, presses Escape, or clicks the grey area behind the card. All four call `onCancel`, which is `() => setStudentToDelete(null)`, and the dialog disappears. Nothing was sent.
4. Or the user clicks Delete. `onConfirm` is `confirmDelete`, and only now does a DELETE request leave the browser.

NOTE

`ConfirmDialog` exists because `window.confirm()` cannot be styled and blocks the whole browser tab while it is open. The comment at the top of `frontend/src/components/ConfirmDialog.jsx` says exactly that. The Escape key is handled by a `useEffect` inside the dialog that adds a `keydown` listener and removes it again in its cleanup function, the small function an effect returns so React can undo what the effect set up.

Step 6 - the JSX

Everything above was logic. Now the part that describes the screen. It all lives inside one `return`.

HEADING AND BANNERS

```
// frontend/src/pages/Students.jsx (step 6, top of the return)
// 6. the JSX
return (
  <div>
    <PageHeader title="Students" subtitle="Everyone enrolled in the school." />

    <Alert>{error}</Alert>
    <Alert variant="success">{notice}</Alert>
  </div>
)
```

Two `Alert`s, always written into the JSX, usually invisible. The first thing `Alert.jsx` does inside its function is `if (!children) { return null; }`, and returning `null` from a component means "draw nothing". So an empty string produces no element, no gap, no border. The first `Alert` has no `variant`, and the default in `Alert.jsx` is `error`, which is the red styling. The second passes `variant="success"` for green.

Because the flash message clears itself, the green banner is genuinely temporary. Nothing in `Students.jsx` ever hides it.

THE FORM, DRAWN ONLY WHEN OPEN

```
// frontend/src/pages/Students.jsx (step 6, the form opens)
{formIsOpen && (
  <form
    onSubmit={handleSave}
    className="mb-6 rounded-lg border border-slate-200 bg-white p-4 shadow-sm"
  >
    <div className="mb-4 flex items-center justify-between">
      <h2 className="text-sm font-semibold text-slate-800">
        {editingId === 0 ? "New student" : "Edit student"}
      </h2>
      <IconButton onClick={closeForm}>
        <X size={16} />
      </IconButton>
    </div>
  </form>
)}
```

`{formIsOpen && ( ... )}` is the standard React way to draw something conditionally. In JavaScript, `false && anything` evaluates to `false`, and React draws nothing for `false`. When `formIsOpen` is `true`, the expression evaluates to the JSX on the right, and that is what gets drawn.

`onSubmit={handleSave}` on the `<form>` catches both the Save button and the Enter key inside any text box, so keyboard users get the same behaviour for free.

`{editingId === 0 ? "New student" : "Edit student"}` is a ternary: condition, then value-if-true, then value-if-false. The same `editingId` that steers `handleSave` also writes the heading, so the two can never disagree.

THE FIELDS

```
// frontend/src/pages/Students.jsx (step 6, the four text fields)
<div className="grid gap-4 sm:grid-cols-2">
  <Input
    label="Name"
    value={name}
    onChange={(event) => setName(event.target.value)}
  />

  <Input
    label="Email"
    type="email"
    value={email}
    onChange={(event) => setEmail(event.target.value)}
  />

  <Input
    label="Roll number"
    value={rollNumber}
    onChange={(event) => setRollNumber(event.target.value)}
  />

  <Input
    label="Enrollment date"
    type="date"
    required
    value={enrollmentDate}
    onChange={(event) => setEnrollmentDate(event.target.value)}
  />
</div>
```

Four controlled inputs, all the same shape: `value` in, `onChange` out.

`Input.jsx` takes `label` and `className` of its own and passes everything else straight through to the real `<input>` with `{...rest}`. That is why `type="email"`, `type="date"` and `required` work here without `Input` knowing about them. `type="date"` gives you the browser's own date picker, and it produces `YYYY-MM-DD`, which is exactly the format Django's `DateField` wants.

NOTE

Only Enrollment date is marked `required`, and that matches the model exactly. In `backend/backend/models.py` the `Student` fields `name`, `email` and `roll_number` are all `blank=True, null=True`, while `enrollment_date = models.DateField()` has no such escape. The form asks for what the server insists on and no more.

The checkbox needs a different pair of **props**, meaning the named values you pass into a component from outside:

```
// frontend/src/pages/Students.jsx (step 6, the checkbox)
<Checkbox
  label="Active"
  className="mt-4"
  checked={isActive}
  onChange={(event) => setIsActive(event.target.checked)}
/>
```

`checked` rather than `value`, and `event.target.checked` rather than `event.target.value`. A checkbox's `value` is a string that has nothing to do with whether it is ticked.

**COMMON
MISTAKE**

Writing `value={isActive}` and `event.target.value` on the checkbox. React will not stop you. The box will simply never respond to clicks, because `checked` is never being set.

THE BUTTONS

```
// frontend/src/pages/Students.jsx (step 6, form buttons)
<div className="mt-4 flex gap-2">
  <Button type="submit" disabled={isSaving}>
    {isSaving ? "Saving..." : "Save"}
  </Button>
  <Button variant="secondary" onClick={closeForm}>
    Cancel
  </Button>
</div>
</form>
)}
```

Save carries `type="submit"`, which is what makes the form's `onSubmit` fire. `disabled={isSaving}` greys it out during the request, and the label changes to `Saving...` at the same moment, from the same piece of state.

Cancel has no `type`, and that is safe here for a specific reason: `Button.jsx` declares `type = "button"` as its default. A plain HTML `<button>` inside a form defaults to `type="submit"`, so a Cancel button written by hand would submit the form. The shared component quietly protects you from that.

The `)}` on the last line closes the `{formIsOpen && (` from earlier.

THE TABLE CARD AND THE ADD BUTTON

```
// frontend/src/pages/Students.jsx (step 6, the card header)
<div className="rounded-lg border border-slate-200 bg-white shadow-sm">
  <div className="flex items-center justify-between border-b border-slate-200 px-4
py-3">
    <h2 className="text-sm font-semibold text-slate-800">
      {isLoading ? "Loading..." : `${students.length} students`}
    </h2>
    {mayCreate && (
      <Button onClick={openEmptyForm}>
        <Plus size={14} />
        Add student
      </Button>
    )}
  </div>
```

This is where `isLoading` finally earns its keep: `Loading...` while waiting, then the real count. The backticks are a **template literal**, a string that can hold values inside `${ }`, and `${students.length}` drops the row count into the middle of it.

`{mayCreate && ( ... )}` is the permission gate. For an admin, `mayCreate` is `true` and the button is drawn. For a teacher or a student it is `false` and the button never enters the page. Not hidden. Not disabled. Absent.

THE ROWS

```
// frontend/src/pages/Students.jsx (step 6, the table)

<Table
  columns={[
    "ID",
    "Name",
    "Email",
    "Roll no.",
    "Enrolled",
    "Active",
    "Action",
  ]}
>
  {students.map((student) => (
    <tr
      key={student.id}
      className="border-b border-slate-100 hover:bg-slate-50"
    >
```

`Table` takes the column names as an array and builds the header row. Everything you put between the tags becomes the `<tbody>`, because each page formats its cells differently.

`students.map(...)` is the heart of the table. `.map()` takes an array and a function, runs the function on every item, and gives back a new array of the results. Feed it 3 student objects and you get back 3 `<tr>` elements. React draws an array of elements one after another. Zero students gives an empty array, which draws nothing, which is exactly right.

The `key` prop, properly. `key={student.id}` is not decoration and React will warn in the console if you leave it out. Here is what it is for.

When the list changes, React does not throw the table away and rebuild it. It compares the new list with the old one and touches only what differs. Without keys, React can only match by position: old first row against new first row, and so on. Delete the student at the top of a 3 row table and React sees "row 1 changed from Rahim to Karim, row 2 changed from Karim to Fatima, row 3 disappeared" and rewrites three rows instead of removing one.

Positional matching also mixes up anything the browser owns rather than React: which text box has the cursor, how far a row is scrolled, whether a row is mid-animation. That state stays with the position and ends up attached to a different student's data.

With `key={student.id}`, React matches by identity. Delete Rahim and React sees that keys 5 and 9 are still present and key 3 is gone, so it removes one row and leaves the others untouched.

**COMMON
MISTAKE**

Using the array position as the key, as in `students.map((student, index) => <tr key={index}>)`. That is the same as having no key at all, because position is exactly what React was already using. Use the database id. It is unique and it belongs to the row, not to where the row happens to sit today.

THE CELLS

```
// frontend/src/pages/Students.jsx (step 6, the data cells)
<td className="px-3 py-2 text-slate-700">{student.id}</td>
<td className="px-3 py-2 text-slate-700">{student.name}</td>
<td className="px-3 py-2 text-slate-700">{student.email}</td>
<td className="px-3 py-2 text-slate-700">
  {student.roll_number}
</td>
<td className="px-3 py-2 text-slate-700">
  {student.enrollment_date}
</td>
<td className="px-3 py-2 text-slate-700">
  {student.is_active ? "Yes" : "No"}
</td>
```

Note the field names inside the braces. They are the Django names with underscores, `roll_number` and `enrollment_date` and `is_active`, because these objects came straight from the API and were never renamed.

`{student.is_active ? "Yes" : "No"}` matters. React draws nothing at all for a true/false value, so `{student.is_active}` on its own would leave the cell blank for every student, active or not. The ternary turns true and false into words.

THE ACTION CELL

```
// frontend/src/pages/Students.jsx (step 6, the action cell)

<td className="px-3 py-2">
  <div className="flex gap-1">
    {mayEdit ? (
      <>
        <IconButton
          onClick={() => openFormForEditing(student)}
          aria-label="Edit"
        />
        <Pencil size={14} />
      </IconButton>

        <IconButton
          variant="danger"
          onClick={() => askToDelete(student)}
          aria-label="Delete"
        />
        <Trash2 size={14} />
      </IconButton>
    ) : (
      <span className="px-2 text-slate-400">&mdash;</span>
    )}
  </div>
</td>
</tr>
)}}
</Table>
</div>
```

A ternary this time rather than `&&`, because there is something to show in the other case: a grey dash, so the column is not disturbingly empty for read-only users.

`<>` and `</>` are a **fragment**, a wrapper that groups two elements without adding a real element to the page. A ternary branch must produce one thing, and we have two buttons, so they need grouping. A `<div>` would work but would add a box the layout does not want.

`onClick={() => openFormForEditing(student)}` needs the arrow function, the short `() => ...` way of writing a function. Writing `onClick={openFormForEditing(student)}` would call the function immediately, while the row is being drawn, for every row, and hand its return value to `onClick`. The arrow wraps it into "a function that, when clicked, calls this with this student".

Each row's arrow function remembers its own `student`, which is how the pencil on row 3 knows to edit student 3.

`aria-label="Edit"` gives the icon-only button a name for screen readers, which would otherwise announce it as an unlabelled button.

`))}` closes the `.map()` callback.

THE DIALOG

```
// frontend/src/pages/Students.jsx (step 6, the end of the file)
<ConfirmDialog
  open={studentToDelete !== null}
  title="Delete student"
  message={`Delete ${studentToDelete?.name || "this student"}? Their enrollments and
submissions go too. This cannot be undone.`}
  isWorking={isDeleting}
  onConfirm={confirmDelete}
  onCancel={() => setStudentToDelete(null)}
/>
</div>
);
}
```

The dialog sits at the bottom of the JSX, outside the table, because it covers the whole screen when open. Where it sits in the file does not matter for it; `ConfirmDialog.jsx` uses `fixed inset-0 z-50` to float above everything.

`open={studentToDelete !== null}` turns the stored object into a true/false switch.

`studentToDelete?.name` needs the `?.` for a real reason. When the dialog is closed, `studentToDelete` is `null`, but this `message` line still runs on every draw. Plain `studentToDelete.name` would crash the page the instant it loaded. `|| "this student"` covers the other case, a student row whose `name` is null or empty in the database.

The warning text is specific to this page, and it is true: in `backend/backend/models.py`, `Enrollment` and `Submission` both point at `Student` with `on_delete=models.CASCADE`, which tells the database to delete those rows too when the student goes. The Teachers page carries a different warning, because deleting a teacher takes their courses with it the same way.

The finished file is 307 lines, which is too long to print in one block. You have now seen every line of it in order.

Prove it works

Start both halves of the app. Backend in one terminal:

```
cd "/Users/tausif/Projects/0stad/Batch 08/lms/backend"
python manage.py runserver 8001
```

Frontend in another:

```
cd "/Users/tausif/Projects/Ostad/Batch 08/lms/frontend"
npm run dev
```

`frontend/vite.config.js` sets the port to 5180, so open `http://localhost:5180/students` and sign in as an admin. Remember that this app logs you in with a phone number, not a username and not an email.

CHECKPOINT The heading shows `Loading...` for a moment, then `N students`. The Add student button is on the right. The table has seven columns. If the row count is 0 and there is no red banner, the page is working and the database is simply empty. Click Add student, fill in a name and a date, press Save, and the row appears.

CHECKPOINT Watch the green banner after saving. Do nothing, and about four seconds later it vanishes on its own. That is the `setTimeout` inside `useFlash` in `frontend/src/flash.js`, not anything in this page.

CHECKPOINT Stop the Django terminal with Ctrl+C, then refresh `/students`. You get the red banner reading `Could not reach the server. Is Django running on http://127.0.0.1:8001?`. That message is thrown by the `catch` around `fetch` in `api.js`, travels as `problem.message`, lands in `error` state, and is drawn by the first `<Alert>`. Start Django again and refresh to clear it.

CHECKPOINT Sign out and sign in as a student. The rows still load, because reading is open to any signed-in account. But the Add student button is gone, and every row's Action column shows a grey dash instead of the pencil and trash. Right-click and choose Inspect: the buttons are not in the HTML at all. That is `mayCreate` and `mayEdit` from `canCreate(user?.role, "student")` and `canWrite(user?.role, "student")`.

The same page again, with different nouns

Now open `frontend/src/pages/Teachers.jsx` and read it top to bottom. It carries a six-step comment block of its own, the same state, the same effect, the same `reload()`, the same `handleSave`, the same two-part delete.

See Figure 0.4.

Here is every difference between the two files:

STUDENTS	TEACHERS
<code>import { studentsApi } from "../api.js";</code>	<code>import { teachersApi } from "../api.js";</code>
<code>export default function Students()</code>	<code>export default function Teachers()</code>
comment block opens <code>// Read this page first.</code> All eight resource pages are the same six steps:	comment block opens <code>// Every resource page</code> follows the same six steps:
numbered step comments sit in the body	one comment inside the effect instead: <code>// The list endpoint is not paginated, so this is a plain array.</code>
<code><PageHeader title="Students" subtitle="Everyone enrolled in the school." /></code>	<code><PageHeader title="Teachers" subtitle="The people who teach courses." /></code>
<code>canCreate(user?.role, "student")</code>	<code>canCreate(user?.role, "teacher")</code>
<code>studentToDelete</code>	<code>teacherToDelete</code>
five fields: name, email, rollNumber, enrollmentDate, isActive	four fields: name, email, subject, isActive
<code>required</code> sits on the Enrollment date input	<code>required</code> sits on the Subject input
the <code>values</code> object is spread over five lines	<code>const values = { name, email, subject, is_active: isActive }; on one line</code>
<code>Table columns=["ID", "Name", "Email", "Roll no.", "Enrolled", "Active", "Action"]</code>	<code>Table columns=["ID", "Name", "Email", "Subject", "Active", "Action"]</code>
<code>setNotice("Student added.")</code>	<code>setNotice("Teacher added.")</code>
dialog warns <code>Their enrollments and submissions go too.</code>	dialog warns <code>Their courses go too.</code>

That is the whole list. Names, labels and columns change. The logic does not.

See Figure 10.1.

See Figure 8.1.

See Figure 10.2.

See Figure 6.1.

When the row only carries an id

Students and Teachers carry no pointers to other tables, so their pages fetch one list each. The other six pages are not so lucky.

Look at what the API sends for a course:

```
{"id": 4, "title": "Intro to Python", "description": "...", "teacher": 3}
```

"teacher": 3 is a **foreign key**: a pointer to row 3 of the teacher table. The serializer sends the number, not the name, because `CourseSerializer` lists `'teacher'` in its fields and takes DRF's default behaviour for it. Nobody wants a table column that reads 3.

There is no setting on this backend to expand it, so the join happens in the browser. `Courses.jsx` fetches the teacher list as well. An **endpoint** is one address on the server, like `/api/course/`, so this effect is talking to two of them:

```
// frontend/src/pages/Courses.jsx (step 2, two endpoints instead of one)
useEffect(() => {
  async function load() {
    try {
      const [courseRows, teacherRows] = await Promise.all([
        coursesApi.list(),
        teachersApi.list(),
      ]);
      setCourses(courseRows);
      setTeachers(teacherRows);
      setError('');
    } catch (problem) {
      setError(problem.message);
    } finally {
      setIsLoading(false);
    }
  }

  load();
}, [reloadCount]);
```

`Promise.all([a, b])` fires both requests at the same moment and waits for both to finish. Two `await`s on separate lines would work, but the second request would not start until the first came back, doubling the wait.

Then a small helper turns an id into a name:

```
// frontend/src/pages/Courses.jsx (the join)
// A course sends {"teacher": 3}, not the teacher's name. There is no way to
// ask the API to expand it, so the teacher list is fetched as well and the
// two are joined here in JavaScript.
function findTeacherName(teacherId) {
  const teacher = teachers.find((item) => item.id === teacherId);
  if (teacher) {
    return teacher.name;
  }
  return "Unknown";
}
```

`.find()` walks the array and returns the first item for which the test is true, or `undefined` if none matched. The `if` guard is what stops a missing teacher crashing the page: it shows `Unknown` instead.

The cell then reads:

```
// frontend/src/pages/Courses.jsx (using the join in a cell)
<td className="px-3 py-2 text-slate-700">
  {findTeacherName(course.teacher)}
</td>
```

The same `teachers` array also fills the `<Select>` in the form, where each `<option>` shows a teacher's name but carries their id as its `value`:

```
// frontend/src/pages/Courses.jsx (the same list, as a dropdown)
{teachers.map((teacher) => (
  <option key={teacher.id} value={teacher.id}>
    {teacher.name}
  </option>
))}
```

COMMON MISTAKE

`.find()` uses `===`, which compares type as well as value, so `3 === "3"` is false. A `<select>` always hands back its value as a **string**. That is why `Courses.jsx` writes `setTeacherId(String(course.teacher))` when filling the form, and `teacher: Number(teacherId)` when sending it. Skip either conversion and you get a teacher column full of `Unknown`, or a Django error saying the field must be a valid integer.

`Enrollments.jsx` does the same thing twice over, with `findStudentName` and `findCourseTitle`, and fetches three lists in one `Promise.all`.

LMS

Dashboard

Teachers

Students

Courses

Enrollments

Lessons

Assignments

Submissions

Results

Accounts

nadia

Admin

Log out

Enrollments

Which student is taking which course.

6 enrollments

+ Add enrollment

ID	STUDENT	COURSE	ENROLLED	ACTION
1	Sadia Akter	React Fundamentals	2026-08-04	
2	Mehedi Hasan	React Fundamentals	2026-08-04	
3	Nusrat Jahan	React Fundamentals	2026-08-04	
4	Sadia Akter	Django REST API	2026-08-04	
5	Arif Khan	Django REST API	2026-08-04	
6	Mehedi Hasan	Relational Databases	2026-08-04	

Figure 11.2 Enrollments. The Student and Course columns are ids from the API, turned into names in the browser.

The six steps, as a recipe

Apply this to any of the other seven pages. Keep the order; it is the order the code sits in the file.

- State.** In this order: the rows array, `isLoading`, `error`, `useFlash` for the notice, `useAuth` plus `mayCreate` and `mayEdit`, `isSaving`, the `xToDelete` object and `isDeleting`, one piece of state per form field, `formIsOpen`, `editingId` starting at `0`, and `reloadCount` with its `reload()` function. If the rows carry foreign keys, add one state array per related list and a `findXName` helper for each.
- The effect.** `useEffect(() => { async function load() {...} load(); }, [reloadCount]);` with `try / catch (problem) / finally`. `setRows` on success, `setError("")` on success, `setError(problem.message)` on failure, `setIsLoading(false)` in `finally`. Use `Promise.all` if there is more than one list to fetch.
- reload().** `setReloadCount((count) => count + 1)`. Nothing more.
- The three form helpers plus handleSave.** `openEmptyForm` clears every field and sets `editingId` to `0`. `openFormForEditing(row)` fills every field with `?? ""` on any nullable text field and sets `editingId` to `row.id`. `closeForm` just closes. `handleSave` starts with `event.preventDefault()` and `setIsSaving(true)`, builds one `values` object using the exact server field names, then `if (editingId === 0)` create else update, then notice, `setError("")`, `closeForm()`, `reload()`.
- The two delete functions.** `askToDelete(row)` only stores the row. `confirmDelete()` awaits `remove(xToDelete.id)`, sets the notice, clears `xToDelete` on both success and failure, calls `reload()`, and clears `isDeleting` in `finally`.
- The JSX, in this order.** `PageHeader`; the two `Alert`s; `{formIsOpen && (<form onSubmit={handleSave}>...)}>` with a ternary heading and controlled inputs; the card whose header shows `isLoading ? "Loading..." : count` and the `{mayCreate && ...}` Add button; the `<Table>`

with `rows.map((row) => <tr key={row.id}>)`; the Action cell gated on `mayEdit`; and the `<ConfirmDialog>` last, with `open={xToDelete !== null}` and `?.` in the message.

The eight pages at a glance

PAGE	FILE	API OBJECTS IT IMPORTS	PERMISSION RESOURCE STRING	EXTRA LISTS FETCHED FOR JOINS
Teachers	<code>pages/Teachers.jsx</code>	<code>teachersApi</code>	<code>"teacher"</code>	none
Students	<code>pages/Students.jsx</code>	<code>studentsApi</code>	<code>"student"</code>	none
Courses	<code>pages/Courses.jsx</code>	<code>coursesApi</code> , <code>teachersApi</code>	<code>"course"</code>	teachers
Enrollments	<code>pages/Enrollments.jsx</code>	<code>coursesApi</code> , <code>enrollmentsApi</code> , <code>studentsApi</code>	<code>"enrollment"</code>	students, courses
Lessons	<code>pages/Lessons.jsx</code>	<code>coursesApi</code> , <code>lessonsApi</code>	<code>"lesson"</code>	courses
Assignments	<code>pages/Assignments.jsx</code>	<code>assignmentsApi</code> , <code>coursesApi</code> , <code>lessonsApi</code>	<code>"assignment"</code>	courses, lessons
Submissions	<code>pages/Submissions.jsx</code>	<code>assignmentsApi</code> , <code>studentsApi</code> , <code>submissionsApi</code>	<code>"submission"</code>	assignments, students
Results	<code>pages/Results.jsx</code>	<code>assignmentsApi</code> , <code>resultsApi</code> , <code>studentsApi</code> , <code>submissionsApi</code>	<code>"results"</code>	submissions, students, assignments

Two details in that table catch people out.

NOTE

The Results row uses `"results"`, plural, in both the API path and the permission string. Every other resource is singular. `api.js` even flags it: `// Note the naming: /submission/ is singular but /results/ is plural.` Write `canWrite(user?.role, "result")` and `WRITERS["result"]` is undefined, so `canWrite` falls back to `[]` and hides the buttons from everyone, including admins.

TIP

Submissions is the one page where `canCreate` and `canWrite` disagree. `permissions.js` has a special case: a student may create a submission but not edit one afterwards. So on that page `mayCreate` is true for a student while `mayEdit` is false, meaning the Add button appears but the row icons do not. The server matches it: `SubmissionWrites` in `backend/backend/permissions.py` allows a student's POST and refuses everything else. Every other page has the two flags in agreement.

What you have now

- A complete `frontend/src/pages/Students.jsx`, built from an empty file in the six steps its own comments describe.
- A working mental model of `useState`: the value plus a setter, and why assigning to an ordinary variable never changes the screen.
- Controlled inputs, where `value` and `onChange` make state the single source of truth, and one `values` object assembled at save time with the server's own field names.
- `useEffect` with a dependency array, an `async function load()` declared and called inside it, and `try` / `catch` / `finally` around every API call.
- The `reloadCount` trick that re-runs the effect after a save or a delete, so the table matches the database.
- `editingId === 0` as the single switch between POST and PUT.
- `.map()` with `key={row.id}`, and a clear answer to what breaks without it.
- A two-part delete: `askToDelete` only stores the row, `confirmDelete` is the only place a DELETE request is sent.
- `mayCreate` and `mayEdit` deciding whether buttons are drawn at all, backed by the server that actually enforces the rules.
- A recipe and a lookup table you can point at any of the other seven pages.

The Other Pages

Now I have everything verified. Here is the corrected chapter.

Chapter 12 - The Other Pages

Chapter 11 built the Teachers page from nothing. By the end of it you had a shape that repeats: state at the top, a `reloadCount` that re-runs the fetch, a form that doubles as "add" and "edit", a table, a confirm dialog, and two role flags that decide which buttons appear.

Ten files are left. Six of them reuse that shape almost exactly. If this chapter re-taught all of it six times it would be forty pages of the same code with the nouns swapped. So it does not. For each page here you get only the parts that are *new*, and a clear statement of which parts you already know how to write.

Here is the whole chapter in one table. The right-hand column is the only reason each page needs its own section.

PAGE	FILE	WHAT IS NEW
Courses	<code>Courses.jsx</code>	One dropdown; turning a teacher id into a teacher name
Enrollments	<code>Enrollments.jsx</code>	Two dropdowns; a date the server sets, so it is in the table but not the form
Lessons	<code>Lessons.jsx</code>	Nothing at all. Proof the pattern holds
Assignments	<code>Assignments.jsx</code>	A dropdown filtered by another dropdown; date-and-time conversion
Submissions	<code>Submissions.jsx</code>	A role rule where "may add" and "may edit" are different answers
Results	<code>Results.jsx</code>	A one-to-one rule, and a 400 error turned into a sentence
Dashboard	<code>Dashboard.jsx</code>	Eight counts, no counting endpoint
Accounts	<code>Accounts.jsx</code>	Admin only; creates a login; does not list logins
Profile	<code>Profile.jsx</code>	Two forms on one page, with different consequences
NotFound	<code>NotFound.jsx</code>	The catch-all route

`Students.jsx` is not in that list. It is the Teachers page with a roll number and an enrollment date in place of the subject, so there is nothing new in it to explain.

Make the files now so the imports in `App.jsx` have something to point at. From the `frontend` folder:

```
# run this from lms/frontend
cd src/pages
touch Courses.jsx Enrollments.jsx Lessons.jsx Assignments.jsx
touch Submissions.jsx Results.jsx Dashboard.jsx
touch Accounts.jsx Profile.jsx NotFound.jsx
```

NOTE

An empty `.jsx` file will make Vite complain the moment `App.jsx` imports it, because a route needs a component and an empty file exports nothing. Build the pages in the order of this chapter and the errors clear one by one. If you want silence in the meantime, put `export default function Courses() { return null; }` in each file as a placeholder.

Two words are used constantly from here on. An **endpoint** is one address on the server that you can call, like `/api/course/`. A **serializer** is the Django class that decides which fields go out in the JSON an endpoint sends, and which fields it will accept coming in.

12.1 Courses: one dropdown, and an id that has to become a name

Open the API in a browser and look at one course. This is what the server sends:

```
{ "id": 1, "title": "Django Basics", "description": "...", "teacher": 3 }
```

`"teacher": 3` is a number. It is the `id` of a row in the teacher table. There is no name anywhere in that response.

That is not an accident, and it is not something you can ask the API to change from the frontend. Look at the serializer that produced it:

backend/backend/serializers.py

```
class CourseSerializer(serializers.ModelSerializer):
    class Meta:
        model = Course
        fields = ['id', 'title', 'description', 'teacher']
```

`teacher` is a `ForeignKey` on the model. A **foreign key** is a column that stores the id of a row in another table. That is the link between a course and its teacher. `ModelSerializer` renders a foreign key as the plain id unless you tell it otherwise, and nothing here tells it otherwise.

So the page has two jobs the Teachers page never had:

1. Show a **name** in the Teacher column of the table, when all it has is the number 3.
2. Offer a **list of teachers to pick from** in the form, because a human is not going to type "3".

Both jobs need the same thing: the full list of teachers, in the browser, at the same time as the list of courses.

FETCHING TWO LISTS AT ONCE

The Teachers page fetched one list. This page fetches two. Both are needed before the table can render anything sensible, and neither depends on the other, so they go out together:

```
// frontend/src/pages/Courses.jsx (inside useEffect)
const [courseRows, teacherRows] = await Promise.all([
  coursesApi.list(),
  teachersApi.list(),
]);
setCourses(courseRows);
setTeachers(teacherRows);
```

A **promise** is JavaScript's word for a result that has not arrived yet. `coursesApi.list()` gives you one straight away; the actual rows turn up later. `Promise.all` takes a list of promises and gives back a single promise that finishes when all of them have. Both network requests leave the browser at the same moment and the code waits once, not twice. Written the slow way it would be `await coursesApi.list()` then `await teachersApi.list()`, which means the second request does not even start until the first has come back.

The `[courseRows, teacherRows]` on the left is **array destructuring**: `Promise.all` hands back an array of results in the same order as the promises you gave it, and this line names the first one `courseRows` and the second `teacherRows`.

Here is the whole effect, so you can see where those lines sit:

frontend/src/pages/Courses.jsx

```
useEffect(() => {
  async function load() {
    try {
      const [courseRows, teacherRows] = await Promise.all([
        coursesApi.list(),
        teachersApi.list(),
      ]);
      setCourses(courseRows);
      setTeachers(teacherRows);
      setError("");
    } catch (problem) {
      setError(problem.message);
    } finally {
      setIsLoading(false);
    }
  }

  load();
}, [reloadCount]);
```

The `[reloadCount]` at the end is the same dependency array from Chapter 11. When `reload()` bumps the number, both lists are fetched again.

JOINING THE TWO LISTS

Now the browser holds `courses` (each with a `teacher` number) and `teachers` (each with an `id` and a `name`). Matching one to the other is a short function:

frontend/src/pages/Courses.jsx

```
// A course sends {"teacher": 3}, not the teacher's name. There is no way to
// ask the API to expand it, so the teacher list is fetched as well and the
// two are joined here in JavaScript.
function findTeacherName(teacherId) {
  const teacher = teachers.find((item) => item.id === teacherId);
  if (teacher) {
    return teacher.name;
  }
  return "Unknown";
}
```

`.find()` walks the array and hands back the first item where the test is true, or `undefined` if nothing matches. Joining two lists like this is a normal thing to do in a frontend and has a name: a **client-side join**.

The `return "Unknown"` matters more than it looks. Suppose somebody deletes a teacher in another browser tab while your Courses page is still showing an old copy of the data. `.find()` then returns `undefined`, and reading `teacher.name` off `undefined` crashes the whole page with "Cannot read properties of undefined". A missing name should print the word Unknown, not blank the screen.

COMMON MISTAKE

`.find((item) => item.id === teacherId)` uses `===`, which compares type as well as value. `3 === 3` is true; `3 === "3"` is false. Every id that arrives from the API is a real number, so this works. The moment you pass in a string from a form field it silently finds nothing and every row says "Unknown". That is the whole reason for the `Number()` conversion two sections down.

NOTE

The parameter is called `teacherId` and there is also a state variable called `teacherId` for the form. Inside this function, the parameter wins and hides the state variable. That is legal JavaScript and it is what the file really does, but it is the kind of thing that makes a reader pause. If it bothers you, rename the parameter; nothing else refers to it.

Using it in the table is one call per row:

```
// frontend/src/pages/Courses.jsx (inside the table body)
<td className="px-3 py-2 text-slate-700">
  {findTeacherName(course.teacher)}
</td>
```

THE DROPDOWN

The form needs a `<select>`. The `Select` component from Chapter 10 takes a `placeholder` and renders it as a first `<option>` with an empty value, then whatever `<option>` elements you pass as children:

```
// frontend/src/pages/Courses.jsx (inside the form)
<Select
  label="Teacher"
  placeholder="Choose a teacher..."
  required
  value={teacherId}
  onChange={(event) => setTeacherId(event.target.value)}
>
  {teachers.map((teacher) => (
    <option key={teacher.id} value={teacher.id}>
      {teacher.name}
    </option>
  ))}
</Select>
```

Read the two halves of an `<option>` separately. `value={teacher.id}` is what gets submitted. `{teacher.name}` is what the human reads. That split is the whole trick: the person picks "Ayesha Rahman" and the form receives `3`.

`required` plus the empty-valued placeholder is what makes the browser refuse to submit until a real teacher is chosen. The placeholder's value is `""`, and `required` on a `<select>` means "not the empty value".

STRINGS IN, NUMBERS OUT

An HTML form field only ever holds text. Even a `<select>` whose options were built from numbers gives you back a string. So `teacherId` in state is `"3"`, not `3`.

The API wants the number. Convert it at the moment of saving:

```
// frontend/src/pages/Courses.jsx (inside handleSave)
// A <select> always hands back a string. The API wants the integer id.
const values = { title, description, teacher: Number(teacherId) };
```

And when you open the form to edit an existing course, the traffic runs the other way. The course holds `teacher: 3`, a number, but the `<select>` compares its `value` prop against option values as text. Feed it a number and it matches nothing and shows the placeholder, which looks like the course has no teacher:

frontend/src/pages/Courses.jsx

```
function openFormForEditing(course) {  
  setTitle(course.title);  
  setTeacherId(String(course.teacher));  
  setDescription(course.description);  
  setEditingId(course.id);  
  setFormIsOpen(true);  
}
```

`String(course.teacher)` turns `3` into `"3"`.

**COMMON
MISTAKE**

Forgetting `String()` here is the single most common bug on every page in this chapter. The symptom is specific and worth memorising: you click the pencil on a row, the form opens with the text fields filled in correctly, and the dropdown sits on "Choose a teacher...". Nothing errors. You just cannot see why it is empty.

Everything else on this page is Chapter 11 with `course` in place of `teacher`: the same `formIsOpen` and `editingId` state, the same `handleSave` that branches on `editingId === 0`, the same `ConfirmDialog`.

One line in that dialog is worth reading, because it tells the truth about the database:

frontend/src/pages/Courses.jsx

```
message={`Delete ${courseToDelete?.title}? Its lessons, assignments and enrollments go too.  
This cannot be undone.`}
```

That is not a scare message someone invented. Every model that points at `Course` declares `on_delete=models.CASCADE`, which means "when the course goes, take these with it". Deleting one course really does remove its lessons, and the assignments under those lessons, and the submissions under those assignments. A confirm dialog that does not say so is lying by omission.

LMS

Dashboard

Teachers

Students

Courses

Enrollments

Lessons

Assignments

Submissions

Results

Accounts

nadia

Admin

Log out

Courses

Every course, and who teaches it.

3 courses

+ Add course

ID	TITLE	DESCRIPTION	TEACHER	ACTION
1	React Fundamentals	Components, props, state and hooks from scrat...	Imran Hossain	
2	Django REST API	Build and secure a JSON API with Django REST ...	Farhana Islam	
3	Relational Databases	Tables, keys, joins and normal forms.	Farhana Islam	

Figure 12.1 The Courses page. The Teacher column shows names, joined in the browser from the separate teacher list.

CHECKPOINT Add a course, choose a teacher, save. The new row should show that teacher's name, not a number. Now open the Teachers page in a second browser tab, delete that teacher, come back to Courses and refresh. The course is gone as well, because `Course.teacher` also cascades. That is the delete message proving itself.

12.2 Enrollments: two dropdowns, and a date you cannot type

An enrollment is a join between a student and a course. It carries almost no data of its own:

backend/backend/models.py

```
class Enrollment(models.Model):
    student = models.ForeignKey(Student, on_delete=models.CASCADE)
    course = models.ForeignKey(Course, on_delete=models.CASCADE)
    enrollment_date = models.DateField(auto_now_add=True)
```

Two foreign keys and a date. That gives the page three things to do differently.

THREE REQUESTS, TWO JOINS

Both foreign keys arrive as ids, so both need a list to join against. That is three lists in flight at once:

frontend/src/pages/Enrollments.jsx

```
// Three endpoints, because the enrollment rows only carry ids.
useEffect(() => {
  async function load() {
    try {
      const [enrollmentRows, studentRows, courseRows] = await Promise.all([
        enrollmentsApi.list(),
        studentsApi.list(),
        coursesApi.list(),
      ]);
      setEnrollments(enrollmentRows);
      setStudents(studentRows);
      setCourses(courseRows);
      setError("");
    } catch (problem) {
      setError(problem.message);
    } finally {
      setIsLoading(false);
    }
  }

  load();
}, [reloadCount]);
```

And two lookup helpers instead of one:

frontend/src/pages/Enrollments.jsx

```
function findStudentName(studentId) {
  const student = students.find((item) => item.id === studentId);
  if (student) {
    return student.name;
  }
  return "Unknown";
}

function findCourseTitle(courseId) {
  const course = courses.find((item) => item.id === courseId);
  if (course) {
    return course.title;
  }
  return "Unknown";
}
```

Same shape as `findTeacherName`. Nothing new.

The two dropdowns sit side by side in the form. `sm:grid-cols-2` is the Tailwind class that puts them in two columns once the screen is wide enough, and stacks them on a phone:

```
// frontend/src/pages/Enrollments.jsx (inside the form)
<div className="grid gap-4 sm:grid-cols-2">
  <Select
    label="Student"
    placeholder="Choose a student..."
    required
    value={studentId}
    onChange={(event) => setStudentId(event.target.value)}
  >
    {students.map((student) => (
      <option key={student.id} value={student.id}>
        {student.name}
      </option>
    ))}
  </Select>

  <Select
    label="Course"
    placeholder="Choose a course..."
    required
    value={courseId}
    onChange={(event) => setCourseId(event.target.value)}
  >
    {courses.map((course) => (
      <option key={course.id} value={course.id}>
        {course.title}
      </option>
    ))}
  </Select>
</div>
```

`AUTO_NOW_ADD`, AND WHY THERE IS NO DATE FIELD IN THE FORM

Look again at that model line:

```
# backend/backend/models.py (one line from Enrollment)
enrollment_date = models.DateField(auto_now_add=True)
```

`auto_now_add=True` tells Django: when this row is **created**, set this column to right now, and never let anything set it again. Not on an update, and not from incoming data. The field is effectively owned by the server.

The serializer lists it, so it comes back in every response:

```
backend/backend/serializers.py
```

```
class EnrollmentSerializer(serializers.ModelSerializer):
    class Meta:
        model = Enrollment
        fields = ['id', 'student', 'course', 'enrollment_date']
```

But `ModelSerializer` reads `auto_now_add` off the model and quietly marks that field **read-only**. Read-only means it goes out in responses and is ignored in requests. Send `{"enrollment_date": "1999-01-01"}` and you will not get an error. You will get a `201 Created` and a row dated today. The value simply evaporates.

That is why the page has no date input, and the comment in `handleSave` says exactly that:

```
// frontend/src/pages/Enrollments.jsx (inside handleSave)
// enrollment_date is auto_now_add on the server. Sending it is silently
// ignored, so it is shown in the table but never in the form.
const values = { student: Number(studentId), course: Number(courseId) };
```

Two fields go up. Three fields come down. The date is displayed like any other column:

```
// frontend/src/pages/Enrollments.jsx (inside the table body)
<td className="px-3 py-2 text-slate-700">
    {enrollment.enrollment_date}
</td>
```

No formatting is needed. A `DateField` serialises to `"2026-08-04"`, which is already readable. (A `DateTimeField` is a different story, and you will meet it on the next page but one.)

COMMON MISTAKE

Adding an "Enrolled on" date input to this form because it feels incomplete without one. It will look like it works. Users will fill it in. Nothing will ever be saved, and nobody will get an error message telling them so. A field the server refuses to accept should not exist in the form.

TIP

Two settings are worth telling apart. `auto_now_add=True` sets the value once, at creation, and is what `enrollment_date` and `submitted_at` use. `auto_now=True` overwrites the value on **every** save, which is what you want for a "last modified" column. Neither can be set from an API request.

The delete message uses both lookups, so it reads like a sentence about people rather than about rows:

frontend/src/pages/Enrollments.jsx

```
message={
  enrollmentToDelete
    ? `Take ${findStudentName(enrollmentToDelete.student)} off
    ${findCourseTitle(enrollmentToDelete.course)}? This cannot be undone.`
    : ""
}
```

The `enrollmentToDelete ? ... : ""` guard is there because `ConfirmDialog` receives this prop on every render, including the renders when nothing is selected and `enrollmentToDelete` is `null`. Reading `.student` off `null` throws. The Courses page dodged the same problem with `courseToDelete?.title`, using **optional chaining** (`?.` returns `undefined` instead of throwing when the thing on the left is null). Here two properties are read and passed into functions, so a plain guard is clearer.

See Figure 11.2.

CHECKPOINT Add an enrollment. The Enrolled column should show today's date without you typing anything. Now click the pencil on that row, change only the course, and save. The date does not move, because `auto_now_add` fires on creation and never again.

12.3 Lessons: the shortest page in the book

A lesson belongs to a course. That is the whole model:

backend/backend/models.py

```
class Lesson(models.Model):
    title = models.CharField(max_length=100)
    description = models.TextField()
    course = models.ForeignKey(Course, on_delete=models.CASCADE)
```

Two text fields and one foreign key. Which means `Lessons.jsx` is `Courses.jsx` with the nouns swapped: teacher becomes course, `findTeacherName` becomes `findCourseTitle`, and the second list fetched is courses instead of teachers.

That is worth pausing on. This is the third page and there is genuinely nothing new in it. When you can look at a model, count "two plain fields and one foreign key", and know without opening the file exactly what the page contains, the pattern has done its job.

The differences from Courses, in full. The fetch:

frontend/src/pages/Lessons.jsx

```
// There is no filtering on this API, so "lessons in course 3" is not a
// request you can make. Both lists download in full and are joined here.
useEffect(() => {
  async function load() {
    try {
      const [lessonRows, courseRows] = await Promise.all([
        lessonsApi.list(),
        coursesApi.list(),
      ]);
      setLessons(lessonRows);
      setCourses(courseRows);
      setError("");
    } catch (problem) {
      setError(problem.message);
    } finally {
      setIsLoading(false);
    }
  }

  load();
}, [reloadCount]);
```

The lookup:

frontend/src/pages/Lessons.jsx

```
function findCourseTitle(courseId) {
  const course = courses.find((item) => item.id === courseId);
  if (course) {
    return course.title;
  }
  return "Unknown";
}
```

The values sent on save:

```
// frontend/src/pages/Lessons.jsx (inside handleSave)
const values = { title, description, course: Number(courseId) };
```

And the delete message, which names what cascades away from a lesson rather than from a course:

frontend/src/pages/Lessons.jsx

```
message={`Delete ${lessonToDelete?.title}? Its assignments go too. This cannot be undone.`}
```

That is it. The imports, the state, the effect, the form, the table, the dialog and the two role flags all have the same shape as the page before, with `lesson` where `course` used to be. The role flags on this page read `canCreate(user?.role, "lesson")` and `canWrite(user?.role, "lesson")`.

NOTE

That comment about filtering is a real limit of this backend.

`LessonListCreateView` is a plain `ListCreateAPIView` with `queryset = Lesson.objects.all()` and no filter backend, so `/api/lesson/?course=3` returns every lesson and ignores the query string. With a few dozen lessons that is fine. With ten thousand it would not be, and the fix is on the server, not in React.

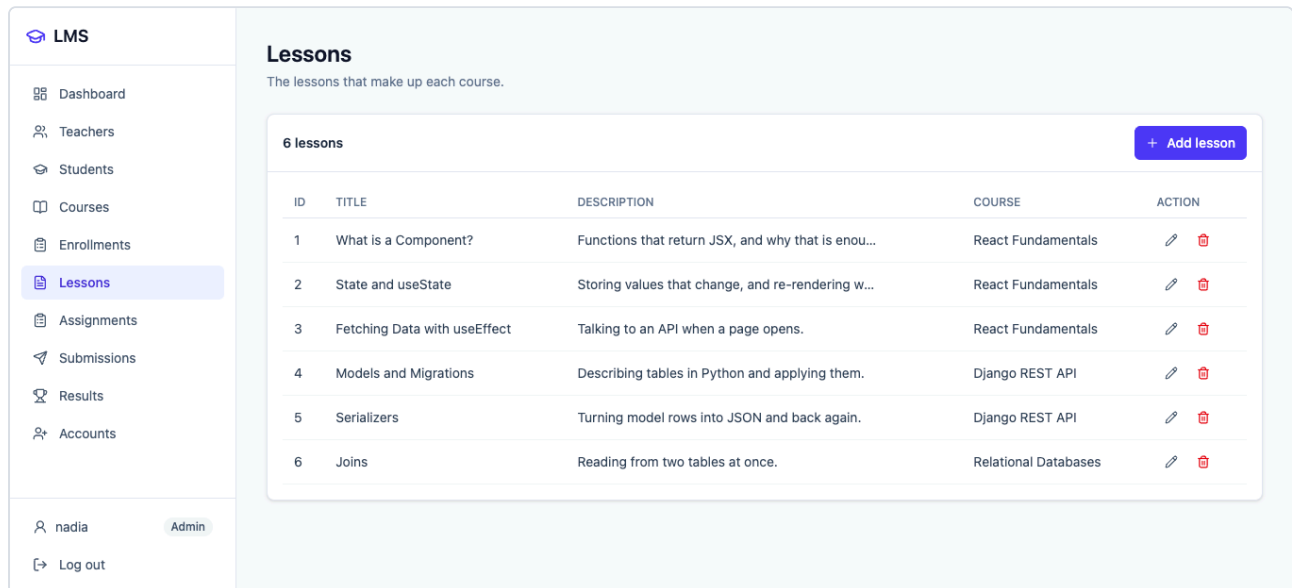


Figure 12.2 The Lessons page. Same table, same form, same buttons. Only the nouns changed.

CHECKPOINT

Copy `Courses.jsx`, rename the file to `Lessons.jsx`, and do a find-and-replace of `teacher` to `course` and `course` to `lesson` in the right places. If your version behaves like the screenshot on the first run, you have understood the pattern properly. If it does not, the mismatch will teach you more than reading the file would have.

12.4 Assignments: the two genuinely hard bits

This is the page where things stop being mechanical. An assignment points at a course *and* a lesson, and it carries a deadline:

backend/backend/models.py

```
class Assignment(models.Model):
    title = models.CharField(max_length=100)
    description = models.TextField()
    lesson = models.ForeignKey(Lesson, on_delete=models.CASCADE)
    course = models.ForeignKey(Course, on_delete=models.CASCADE)
    due_date = models.DateTimeField()
```

Two problems come out of those five lines, and neither is obvious until you hit it.

PROBLEM ONE: THE LESSON DROPDOWN SHOULD NOT OFFER EVERY LESSON

The form has a Course select and a Lesson select. Every lesson already belongs to a course. So once someone has chosen "Django Basics", offering them a lesson from "React Fundamentals" is offering them a way to create a nonsense row: an assignment attached to course 1 and lesson 9, where lesson 9 lives in course 2.

The database will not stop you. Both foreign keys are valid on their own; nothing in the model says they have to agree. This has to be handled in the form.

The fix is one expression, computed fresh on every render:

frontend/src/pages/Assignments.jsx

```
// An assignment points at both a course and a lesson, so once a course is
// chosen only that course's lessons are worth offering.
const lessonsToOffer = courseId
  ? lessons.filter((lesson) => lesson.course === Number(courseId))
  : lessons;
```

Take that apart slowly, because there are three ideas packed into it.

It is not state. There is no `useState` for `lessonsToOffer` and no `useEffect` that fills it in. It is a plain `const` sitting in the component body, which means it is recalculated from scratch every single time the component renders. And the component re-renders whenever `courseId` changes, because `setCourseId` is a state setter. So the filtered list can never be out of date with the selected course. Storing it in state would mean keeping two things in sync by hand, and one of them would eventually be wrong.

The ternary handles "nothing chosen yet". `courseId ? A : B` reads as "if `courseId` has a value, use A, otherwise use B". When the form first opens, `courseId` is `""`, which JavaScript treats as false, so `lessonsToOffer` is the whole `lessons` array. Without that branch, an empty course would filter down to zero lessons and the Lesson dropdown would be mysteriously blank before the user has done anything wrong.

`Number(courseId)` again. `lesson.course` is a number from the API. `courseId` is a string from the `<select>`. `9 === "9"` is false, so without the conversion the filter matches nothing and every course looks like it has no lessons.

Rendering it is a one-word change from the other dropdowns. It maps `lessonsToOffer`, not `lessons`:

```
// frontend/src/pages/Assignments.jsx (inside the form)
<Select
  label="Lesson"
  placeholder="Choose a lesson..."
  required
  value={lessonId}
  onChange={(event) => setLessonId(event.target.value)}
>
  {lessonsToOffer.map((lesson) => (
    <option key={lesson.id} value={lesson.id}>
      {lesson.title}
    </option>
  ))}
</Select>
```

Note that the full `lessons` array is still in state and still used. `findLessonTitle` searches it so the table can name a lesson from any course, including one you are not currently filtered to. Filtering the *options* is not the same as throwing the data away.

Now the follow-up question that catches everyone. Suppose the user picks course "Django Basics", then picks lesson "Models and Migrations", then changes their mind and switches the course to "React Fundamentals". `lessonsToOffer` immediately recalculates and no longer contains "Models and Migrations". But `lessonId` is still set to that lesson's id.

A `<select>` whose `value` matches none of its options renders as blank. It looks empty. But the state behind it is not empty, it still holds a lesson from the old course. `required` sees a non-empty value and lets the form submit. You would save exactly the mismatched row the filter was built to prevent, and the user would swear they never chose that lesson.

So changing the course has to clear a stale lesson. That is what this function is for, and it is why the Course select calls a named handler instead of `setCourseId` directly:

frontend/src/pages/Assignments.jsx

```
// Changing the course clears a lesson that no longer belongs to it.
function handleCourseChange(nextCourseId) {
  setCourseId(nextCourseId);

  const lesson = lessons.find((item) => item.id === Number(lessonId));
  if (lesson && lesson.course !== Number(nextCourseId)) {
    setLessonId('');
  }
}
```

Read the condition carefully. It only clears when there **is** a currently selected lesson (`lesson` is truthy) **and** that lesson belongs to a different course than the new one. If you re-pick the same course, or if no lesson was selected yet, nothing is cleared and the user does not lose work they were about to keep.

It searches `lessons`, the full array, not `lessonsToOffer`. It has to: the lesson it is looking for is by definition one that the new filter would exclude.

Wiring it up:

```
// frontend/src/pages/Assignments.jsx (inside the form)
<Select
  label="Course"
  placeholder="Choose a course..."
  required
  value={courseId}
  onChange={(event) => handleCourseChange(event.target.value)}
/>
```

COMMON MISTAKE

Filtering the options but forgetting to clear the selection. This bug is invisible in every quick test, because you only see it if you choose a lesson and *then* go back and change the course. It survives to production more often than almost any other form bug.

PROBLEM TWO: `DATETIME-LOCAL` AND TIME ZONES

Now the deadline. `due_date` is a `DateTimeField`, so it has a date *and* a clock time, and this is where a whole class of "the due date is six hours wrong" bugs lives.

Three facts, each simple on its own:

Fact one: the server works in UTC.

```
backend/lms/settings.py
```

```
TIME_ZONE = 'UTC'  
...  
USE_TZ = True
```

`USE_TZ = True` means Django stores every datetime as a **timezone-aware** value, and the reference point is UTC. **UTC** (Coordinated Universal Time) is the world's zero line. Every local time is UTC plus or minus some offset. Dhaka is UTC+6. New York is UTC-5 in winter. Storing in UTC means one moment has exactly one stored value regardless of where the person who typed it was standing.

So the API sends a due date like this:

```
{ "due_date": "2026-08-15T23:59:00Z" }
```

The trailing `Z` means UTC. The `T` just separates the date part from the time part.

Fact two: `<input type="datetime-local">` **has no time zone at all.** That is what "local" means in the name. It shows the user their own wall clock, it wants its `value` in the exact format `YYYY-MM-DDTHH:mm`, and the string it gives back has no `Z` and no offset:

```
2026-08-16T05:59
```

That string means "five fifty-nine in the morning, wherever the user is". It is not a moment in time until you know where they were.

Fact three: those two are six hours apart in Dhaka, and they are the same instant. `23:59 UTC` on the 15th is `05:59` local on the 16th at UTC+6.

Now you can see the trap. The obvious thing to do with `"2026-08-15T23:59:00Z"` is chop it to sixteen characters and hand it to the input:

```
// WRONG - do not write this  
return text.slice(0, 16); // "2026-08-15T23:59"
```

That puts 11:59 PM on the 15th in front of a Dhaka user, when the deadline they set was 5:59 AM on the 16th. Wrong day, wrong time, and it looks plausible enough that nobody notices for weeks.

Here is the conversion the project actually uses, going from API to input box:

```
// <input type="datetime-local"> wants "YYYY-MM-DDTHH:mm" in local time, but
// the API sends UTC ("2026-08-15T23:59:00Z"). Slicing the string straight
// off would show the UTC clock time, so the offset is taken off first.
function toDateTimeInputValue(text) {
  if (!text) {
    return "";
  }
  const moment = new Date(text);
  const offsetInMs = moment.getTimezoneOffset() * 60 * 1000;
  return new Date(moment.getTime() - offsetInMs).toISOString().slice(0, 16);
}
```

Follow it with real numbers, for a browser set to Dhaka (UTC+6), given `"2026-08-15T23:59:00Z"`:

STEP	VALUE	WHAT HAPPENED
<code>new Date(text)</code>	the instant <code>23:59 UTC</code>	<code>Z</code> is understood, so this is unambiguous
<code>.getTimezoneOffset()</code>	<code>-360</code>	minutes to add to local to reach UTC. Dhaka is ahead, so it is negative
<code>offsetInMs</code>	<code>-21600000</code>	that is minus six hours in milliseconds
<code>.getTime() - offsetInMs</code>	the instant plus 6 hours	subtracting a negative adds
<code>new Date(...)</code>	the instant <code>05:59 UTC</code> on the 16th	a deliberately "wrong" moment
<code>.toISOString()</code>	<code>"2026-08-16T05:59:00.000Z"</code>	always prints in UTC, which is the point
<code>.slice(0, 16)</code>	<code>"2026-08-16T05:59"</code>	exactly the format the input wants

The trick in the middle is worth naming. `toISOString()` will only ever print UTC. So to make it print your *local* wall clock, you shift the instant by your own offset first, print it as UTC, and the digits come out matching the clock on your wall. The intermediate `Date` object is a throwaway that represents no real moment. It exists only so that `toISOString()` prints the right characters.

`.slice(0, 16)` keeps `YYYY-MM-DDTHH:mm` and drops the `:00.000Z`. The input rejects seconds and milliseconds unless you ask for them with a `step` attribute.

Now the other direction, at save time. The input has given you `"2026-08-16T05:59"` with no zone marker:

```
// frontend/src/pages/Assignments.jsx (inside handleSave)
const values = {
  title,
  description,
  course: Number(courseId),
  lesson: Number(lessonId),
  // The input gives local time. Send UTC so the server stores the
  // moment that was actually meant.
  due_date: due.toISOString(),
};
```

where `due` came from one line higher up:

```
// frontend/src/pages/Assignments.jsx (inside handleSave)
const due = new Date(dueDate);
```

This direction needs no offset arithmetic, because of a rule in JavaScript that happens to work in your favour: `new Date()` given a date-and-time string **with no zone** interprets it in the browser's local zone. So `"2026-08-16T05:59"` on a Dhaka machine becomes the instant `23:59 UTC` on the 15th. Then `toISOString()` prints that instant in UTC, and you send `"2026-08-15T23:59:00.000Z"`, which is the same moment the user meant.

COMMON MISTAKE

Sending the raw input string, `due_date: dueDate`, DRF will accept `"2026-08-16T05:59"` without complaint. With `USE_TZ = True` and `TIME_ZONE = 'UTC'`, Django reads a value with no offset as being UTC already, so it stores `05:59 UTC`. Your Dhaka user set a 5:59 AM deadline and the system now believes it is due at 11:59 AM their time. Six hours of free extra time, and no error anywhere to tell you.

TIP

To see this for yourself, change your computer's time zone, reload the page, and look at an existing assignment. The Due column should show a *different* clock time, because the same instant is a different wall clock in a different place. That is not a bug. That is what storing in UTC buys you.

The table has its own, much simpler formatter:

frontend/src/pages/Assignments.jsx

```
function showDateAndTime(text) {  
  if (!text) {  
    return "";  
  }  
  return new Date(text).toLocaleString();  
}
```

`toLocaleString()` formats a date the way the reader's own machine formats dates, in their own zone, in their own language. It is the right tool for *displaying* a moment and the wrong tool for filling in an input, because its output format changes from computer to computer and `datetime-local` accepts exactly one format.

ONE MORE SMALL THING: WHERE THE DATE CHECK LIVES

`toISOString()` throws a `RangeError` if the `Date` is invalid, which happens when `dueDate` is empty or garbage. That thrown error has to land somewhere useful, so the whole thing sits inside the `try`:

```
// frontend/src/pages/Assignments.jsx (inside handleSave)  
try {  
  // toISOString() throws on an unparseable date, so this has to happen  
  // inside the try or the button would stay stuck on "Saving...".  
  const due = new Date(dueDate);  
  if (Number.isNaN(due.getTime())) {  
    throw new Error("Enter a due date.");  
  }  
  
  // ... build `values`, then create or update ...  
} catch (problem) {  
  setError(problem.message);  
} finally {  
  setIsSaving(false);  
}
```

`new Date("nonsense")` does not throw. It gives you an Invalid Date, whose `.getTime()` is `NaN` ("not a number"). `Number.isNaN()` is the way to test for that, since `NaN === NaN` is famously false.

The reason this sits *inside* the `try` is the `finally`. `setIsSaving(true)` runs before the block; `setIsSaving(false)` runs in `finally`, which executes whether the block succeeded or threw. Move the date check above the `try` and a bad date escapes the whole structure: no error banner, no `finally`, and the Save button says "Saving..." forever with nothing happening.

LMS

Dashboard

Teachers

Students

Courses

Enrollments

Lessons

Assignments

Submissions

Results

Accounts

nadia

Admin

Log out

Assignments

Work set against a lesson.

3 assignments

+ Add assignment

ID	TITLE	COURSE	LESSON	DUE	ACTION
1	Build a Profile Card	React Fundamentals	What is a Component?	8/20/2026, 11:00:00 PM	
2	Counter with useState	React Fundamentals	State and useState	8/27/2026, 11:00:00 PM	
3	Write Your First Serializer	Django REST API	Serializers	9/3/2026, 11:00:00 PM	

Figure 12.3 The Assignments page. The Lesson dropdown only lists lessons from the chosen course, and Due shows local time.

CHECKPOINT Create two courses with lessons in each. Open the assignment form, pick the first course, pick one of its lessons, then switch the course. The Lesson box should reset to "Choose a lesson..." and offer only the second course's lessons. Save an assignment for 11:00 PM tonight, then reopen it with the pencil. The input must still read 11:00 PM. If it reads some other hour, the conversion is only running in one direction.

12.5 Submissions: where "can add" and "can edit" split apart

A submission is a piece of work handed in:

backend/backend/models.py

```
class Submission(models.Model):
    assignment = models.ForeignKey(Assignment, on_delete=models.CASCADE)
    student = models.ForeignKey(Student, on_delete=models.CASCADE)
    submitted_at = models.DateTimeField(auto_now_add=True)
    content = models.TextField()
```

Structurally you have already seen all of it. Two foreign keys means two dropdowns and two lookup helpers, exactly like Enrollments. `submitted_at` is `auto_now_add`, so it is in the table and not in the form, exactly like `enrollment_date`:


```
// frontend/src/pages/Submissions.jsx (inside handleSave)
// submitted_at is auto_now_add, so the server sets it and ignores
// anything sent for it. It appears in the table but not in the form.
const values = {
  assignment: Number(assignmentId),
  student: Number(studentId),
  content,
};
```

The one display detail: `submitted_at` is a `DateTimeField`, not a `DateField`, so the raw value is `"2026-08-04T09:12:44.123456Z"`. That is not something to show a person. This page uses the same formatter as Assignments:

frontend/src/pages/Submissions.jsx

```
function showDateAndTime(text) {
  if (!text) {
    return "";
  }
  return new Date(text).toLocaleString();
}
```

And because `content` is a `TextField` that can hold an entire essay, the Content column is cut short:

frontend/src/pages/Submissions.jsx

```
function shorten(text) {
  if (!text) {
    return "";
  }
  if (text.length > 60) {
    return text.slice(0, 60) + "...";
  }
  return text;
}
```

That is a different technique from the truncation on Courses and Lessons, which used the CSS classes `block max-w-xs truncate` to let the browser cut the text with an ellipsis. CSS truncation keeps the full text in the page, so it is still searchable and copyable; `shorten()` genuinely removes it. Both are in this project. Neither is wrong. (Results uses `shorten()` too, on its Feedback column.)

THE ROLE RULE THAT IS NOT LIKE THE OTHERS

Now the interesting part. Every page so far has used the same two flags with the same answer:

frontend/src/pages/Lessons.jsx

```
const mayCreate = canCreate(user?.role, "lesson");  
const mayEdit = canWrite(user?.role, "lesson");
```

For teachers, students, courses, enrollments, lessons, assignments and results, those two are always the same value. A role that can add can also edit. Submissions is the one place where they differ:

frontend/src/pages/Submissions.jsx

```
const mayCreate = canCreate(user?.role, "submission");  
const mayEdit = canWrite(user?.role, "submission");
```

For a student, `mayCreate` is `true` and `mayEdit` is `false`.

Here is where that comes from:

frontend/src/permissions.js

```
// May this role add a new row? Same as canWrite everywhere except submissions,  
// which a student is allowed to hand in but not to edit afterwards.  
export function canCreate(role, resource) {  
  if (resource === "submission" && role === STUDENT) {  
    return true;  
  }  
  return canWrite(role, resource);  
}
```

The whole role system is one special case wide. `canCreate` is `canWrite` with a single exception bolted on the front.

And the server carries the identical exception:

```

class SubmissionWrites(RoleWritePermission):
    """Same as above, except a student may hand work in.

    A student cannot edit or delete a submission, because nothing links a
    Student row to a login account – so the server has no way to tell whose
    work it is. Handing in is the one write that is safe without that link.
    """

    roles_that_may_write = (Profile.ADMIN, Profile.TEACHER)
    message = "Students can hand work in, but cannot change or remove a submission."

    def has_permission(self, request, view):
        if (
            request.method == "POST"
            and request.user
            and request.user.is_authenticated
            and role_of(request.user) == Profile.STUDENT
        ):
            return True

        return super().has_permission(request, view)

```

Read the docstring, because it explains a real design limit rather than a rule someone preferred. In this database, `Student` and `User` are separate tables with no link between them. A `Student` row has a name and an email; a `User` row is a login. Nothing joins them. So when a signed-in student sends a `PUT`, the server genuinely cannot tell whether that submission is theirs or someone else's. "You may only edit your own" is not a rule it is able to check. Refusing all edits from students is the honest answer.

`POST` is different. Creating a row does not require knowing who owns anything, so it is safe to allow.

In the page, this comes out as: a student sees the "Add submission" button, and sees a dash in the Action column of every row instead of pencil and bin icons:

```
// frontend/src/pages/Submissions.jsx (inside the table body)
<div className="flex gap-1">
  {mayEdit ? (
    <
      <IconButton
        onClick={() => openFormForEditing(submission)}
        aria-label="Edit"
      />
      <Pencil size={14} />
    </IconButton>

    <IconButton
      variant="danger"
      onClick={() => askToDelete(submission)}
      aria-label="Delete"
    />
    <Trash2 size={14} />
  ) : (
    <span className="px-2 text-slate-400">&mdash;</span>
  )}
</div>
```

NOTE

Hiding buttons is decoration. The comment at the top of every page in this chapter says so out loud: "The server enforces this too. Hiding the buttons just keeps the page honest about what will actually work." A student who opens the browser console and sends a `PUT` by hand gets a 403 from `SubmissionWrites`, with the message "Students can hand work in, but cannot change or remove a submission." If the only check were the hidden button, that request would have gone through.

LMS

Dashboard

Teachers

Students

Courses

Enrollments

Lessons

Assignments

Submissions

Results

Accounts

nadia

Admin

Log out

Submissions

Work handed in by students.

3 submissions

+ Add submission

ID	ASSIGNMENT	STUDENT	CONTENT	SUBMITTED	ACTION
1	Build a Profile Card	Sadia Akter	https://github.com/sadia/profile-card - card renders with pr...	8/4/2026, 11:26:01 AM	
2	Build a Profile Card	Mehedi Hasan	https://github.com/mehedi/profile-card - used a nested compo...	8/4/2026, 11:26:01 AM	
3	Counter with useState	Sadia Akter	https://github.com/sadia/counter - increment and reset both ...	8/4/2026, 11:26:01 AM	

Figure 12.4 The Submissions page. Content is cut at 60 characters and Submitted is a server-set timestamp.

CHECKPOINT Sign in as a student. The Submissions page shows the "Add submission" button *and* a dash in every Action cell. That combination appears on no other page in the app. Add a submission and it saves; there is no way to change it afterwards.

12.6 Results: grading each submission exactly once

A result is a grade attached to a piece of submitted work. One line of the model decides the shape of this whole page:

backend/backend/models.py

```
class Results(models.Model):
    submission = models.OneToOneField(Submission, on_delete=models.CASCADE)
    score = models.FloatField()
    feedback = models.TextField()
```

`OneToOneField`, not `ForeignKey`. A foreign key allows many rows to point at the same target; a one-to-one allows exactly one. In the database it is a foreign key with a **unique** constraint on it. So a submission can be graded once and only once, and the second attempt is rejected by the database itself.

NOTE

The model class is called `Results`, plural, and the URL is `/api/results/` while every other resource is singular (`/api/submission/`, `/api/course/`). The frontend keeps that spelling all the way through: `api.js` exports `resultsApi = resource("results")`, and the permission lookups on this page are `canCreate(user?.role, "results")` and `canWrite(user?.role, "results")`. Get the plural wrong in `permissions.js` and `canWrite` returns `false` for everybody, so the buttons vanish for admins too.

MAKING A SUBMISSION ID READABLE

Before the one-to-one rule, a smaller problem. The Submission column would otherwise show a number, and "4" tells a teacher nothing. Naming it takes three lists and a bit of work:

frontend/src/pages/Results.jsx

```
// "Submission #4" on its own says nothing useful, so the student and the
// assignment are looked up and folded into the label.
function describeSubmission(submissionId) {
  const submission = submissions.find((item) => item.id === submissionId);
  if (!submission) {
    return `Submission #${submissionId}`;
  }

  const student = students.find((item) => item.id === submission.student);
  const assignment = assignments.find(
    (item) => item.id === submission.assignment,
  );

  const who = student ? student.name : "Unknown student";
  const what = assignment ? assignment.title : "Unknown assignment";
  return `#${submission.id} - ${who}, ${what}`;
}
```

This is a **two-hop join**. A result points at a submission; that submission points at a student and at an assignment. So the page has to fetch four lists to render one column:

```
// frontend/src/pages/Results.jsx (inside useEffect)
const [resultRows, submissionRows, studentRows, assignmentRows] =
  await Promise.all([
    resultsApi.list(),
    submissionsApi.list(),
    studentsApi.list(),
    assignmentsApi.list(),
  ];
```

Every hop falls back to readable text instead of crashing: `Submission #4` if the submission is gone, "Unknown student" or "Unknown assignment" if either of those is. The output looks like `#4 – Nusrat Jahan, Week 1 Exercise`.

The same function labels the dropdown options *and* the table cells, so a teacher reads the identical string in both places.

LEAVING OUT SUBMISSIONS THAT ARE ALREADY GRADED

If a submission already has a result, offering it in the "New result" dropdown is offering a guaranteed failure. So it is filtered out:

frontend/src/pages/Results.jsx

```
// Results.submission is one-to-one: a submission can only be graded once,  
// and a second attempt comes back as a 400. Already-graded submissions are  
// left out of the "new result" dropdown so that is hard to walk into.  
const gradedIds = results  
  .filter((result) => result.id !== editingId)  
  .map((result) => result.submission);  
  
const submissionsToOffer = submissions.filter(  
  (submission) => !gradedIds.includes(submission.id),  
);
```

Two steps. First, collect the submission ids that already have a result: `.filter()` keeps some results, `.map()` turns each kept result into just its `submission` number, so `gradedIds` is an array like `[1, 4, 7]`. Then keep only the submissions whose id is **not** in that array. `.includes()` asks "is this value in the array", and the `!` flips it.

The subtle piece is `result.id !== editingId` in the first filter, and it is there for editing.

Picture grading submission 4, then clicking the pencil on that result to fix the score.

`openFormForEditing` sets `submissionId` to `"4"`. But submission 4 is graded, so without that condition it would be excluded from `submissionsToOffer`, the `<select>` would have no option with value `"4"`, and the dropdown would render blank. The user would be looking at an edit form that had apparently forgotten which submission it was editing.

`editingId` is `0` while adding, and no result has id `0`, so during an add the condition is true for every result and nothing is excluded from `gradedIds`. While editing result 5, that one result is skipped, so its own submission stays available and the dropdown displays correctly. One comparison, two behaviours.

TIP

`gradedIds` and `submissionsToOffer` are both plain `const` declarations in the component body, recalculated on every render, exactly like `lessonsToOffer` on Assignments. That is why they are correct the instant `reload()` brings fresh results back. Anything derived from state should be computed, not stored.

AND WHEN SOMEONE GETS THERE ANYWAY

Filtering the dropdown makes a duplicate hard. It does not make it impossible: two teachers grading at once, or a stale page that has not reloaded, will still produce a `POST` for a submission that was graded thirty seconds ago.

The database refuses. DRF turns the unique constraint into a `400 Bad Request` with a body like:

```
{ "submission": ["results with this submission already exists."] }
```

`readError()` in `api.js` flattens that into the string `submission: results with this submission already exists.` Truthful, and not something you would show a teacher. So this page rewrites it:

```
// frontend/src/pages/Results.jsx (the catch in handleSave)
} catch (problem) {
  // The one-to-one clash comes back as "submission: results with this
  // submission already exists", which reads badly. Say it plainly.
  if (problem.message.includes("already exists")) {
    setError("That submission has already been graded. Edit its result instead.");
  } else {
    setError(problem.message);
  }
} finally {
  setIsSaving(false);
}
```

The `else` branch matters as much as the `if`. Only the one error that has a better wording gets rewritten; everything else passes through untouched. A page that swallowed every error into one friendly sentence would hide the next bug you need to see.

COMMON MISTAKE

Matching on the exact full error string instead of `.includes("already exists")`. DRF builds that wording from your model's name, so it changes if you rename the model, and the `readError()` prefix depends on the field name. Matching a distinctive fragment survives that; matching the entire sentence breaks quietly the first time anything changes, and the user is back to reading `submission: results with this submission already exists.`

One last small thing, the score input:


```
// frontend/src/pages/Results.jsx (inside the form)
<Input
  label="Score"
  type="number"
  step="0.01"
  required
  value={score}
  onChange={(event) => setScore(event.target.value)}
/>
```

`score` is a `FloatField` on the model, so decimals are allowed. `type="number"` gives phones a numeric keypad and puts the little up-down arrows on desktop. `step="0.01"` is what permits `87.5`; the default step for a number input is `1`, and without this the browser refuses to submit anything with a fraction. The value still arrives as a string, so `handleSave` converts it: `score: Number(score)`.

The delete message earns a read too:

frontend/src/pages/Results.jsx

```
message={
  resultToDelete
    ? `Delete the result for ${describeSubmission(resultToDelete.submission)}? The
      submission can then be graded again. This cannot be undone.`
    : ""
}
```

"The submission can then be graded again" is the practical consequence of the one-to-one rule running backwards. Delete the result and that submission reappears in the dropdown. That is genuinely useful information at the moment someone is deciding whether to click Delete.

LMS

Dashboard

Teachers

Students

Courses

Enrollments

Lessons

Assignments

Submissions

Results

Accounts

nadia

Admin

Log out

Results

Scores and feedback for submitted work.

2 results

+ Add result

ID	SUBMISSION	SCORE	FEEDBACK	ACTION
1	#1 — Sadia Akter, Build a Profile Card	92	Clean props, good naming. Add a default photo for missing im...	
2	#2 — Mehedi Hasan, Build a Profile Card	85	Works well. The avatar component could take a size prop.	

Figure 12.5 The Results page. Each row names the student and the assignment behind the submission id.

CHECKPOINT Grade a submission. Click "Add result" again and open the Submission dropdown: the one you just graded is gone from the list. Now click the pencil on the result you created. The dropdown shows that same submission, correctly selected. Both behaviours come from the single `result.id !== editingId` comparison.

12.7 Dashboard: eight numbers, and where they come from

The dashboard is the first screen after login. It shows one tile per resource, each with a count, each a link to the matching page.

There is no `/api/counts/` endpoint on this backend. Nothing on the server will tell you how many courses exist. So the dashboard does the only thing available: fetch all eight lists and measure their lengths.

The tiles are data, not markup. One array describes the whole grid:

frontend/src/pages/Dashboard.jsx

```
// One tile per resource. `api` is the module from api.js the count comes from.
const tiles = [
  {
    address: "/teachers",
    text: "Teachers",
    api: teachersApi,
    icon: <Users size={16} className="text-indigo-500" />,
  },
  {
    address: "/students",
    text: "Students",
    api: studentsApi,
    icon: <GraduationCap size={16} className="text-indigo-500" />,
  },
  // ...six more, one per resource: courses, enrollments, lessons,
  // assignments, submissions, results
];
```

The array sits *outside* the component function, at module level. That means it is built once when the file is first loaded, not rebuilt on every render. It is allowed to live there because nothing in it depends on props or state.

Look at what a tile holds. `address` is where clicking goes. `text` is the label. `icon` is a ready-made React element. And `api` is not a string or a URL, it is the actual object exported from `api.js`, the same `teachersApi` that the Teachers page calls. Putting a function-holding object inside data like this is normal JavaScript, and it means the fetching loop never has to map a name to a call.

That loop is short because of it:

frontend/src/pages/Dashboard.jsx

```
// There is no counts endpoint, so every list is fetched and measured.
useEffect(() => {
  async function load() {
    try {
      const lists = await Promise.all(tiles.map((tile) => tile.api.list()));

      const nextCounts = {};
      tiles.forEach((tile, index) => {
        nextCounts[tile.text] = lists[index].length;
      });

      setCounts(nextCounts);
      setError('');
    } catch (problem) {
      setError(problem.message);
    } finally {
      setIsLoading(false);
    }
  }

  load();
}, []);
```

`tiles.map((tile) => tile.api.list())` calls `.list()` on each of the eight api objects and collects the eight promises. `Promise.all` waits for all of them. Eight requests, in parallel, one wait.

The `lists` array comes back in the same order as `tiles`, which is what makes the second loop safe: `lists[index]` is guaranteed to be the response for `tiles[index]`. `.forEach()` with its second argument, the index, builds an object like `{ Teachers: 4, Students: 12, Courses: 3, ... }` keyed by the tile's `text`.

The `[]` at the end of the effect is an empty dependency array. The dashboard has nothing to reload, so it fetches once when the page appears and never again.

NOTE

This is eight HTTP requests to draw eight numbers, and it downloads every row of every table to count them. For a class-sized LMS that is fine and it needs no server changes. If this were a real deployment with tens of thousands of rows, the fix would be a single endpoint on the Django side returning `{"teachers": 4, "students": 12, ...}` from `.count()` queries, which never load the rows at all. Knowing that a shortcut has a limit is different from the shortcut being wrong.

Rendering is one `.map()` over the same array:

```
frontend/src/pages/Dashboard.jsx
```

```
{tiles.map((tile) => (  
  <Link  
    key={tile.address}  
    to={tile.address}  
    className="rounded-lg border border-slate-200 bg-white p-4 shadow-sm hover:border-indigo-300"  
  >  
    <div className="flex items-center justify-between">  
      <span className="text-sm font-medium text-slate-500">  
        {tile.text}  
      </span>  
      {tile.icon}  
    </div>  
    <p className="mt-2 text-3xl font-semibold text-slate-900">  
      {isLoading ? "-" : (counts[tile.text] ?? 0)}  
    </p>  
  </Link>  
)})
```

Each tile is a `<Link>`, so the whole card is clickable, not just a small link inside it. `to={tile.address}` uses react-router, which swaps the page without a full browser reload.

The number line handles two states at once. While loading, it prints a dash rather than `0`, because a real zero and "not known yet" should not look identical. Once loaded, `counts[tile.text] ?? 0` reads the count, and `??` (the **nullish coalescing** operator, which means "use the left value unless it is null or undefined") substitutes `0` if the key is missing. That guard is what stops the tiles rendering `undefined` on the first paint, before the effect has finished.

The heading greets whoever is signed in:

```
frontend/src/pages/Dashboard.jsx
```

```
<PageHeader  
  title={`Welcome back, ${user?.username ?? "there"}`}  
  subtitle="A quick count of everything in the system."  
>
```

`user` comes from `useAuth()`, which reads the session saved at login. `user?.username ?? "there"` covers the moment before the session has been read: "Welcome back, there" is odd but harmless, while a crash on `undefined.username` is not.

See Figure 0.3.

CHECKPOINT Load the dashboard and watch it: all eight tiles show a dash, then all eight fill in at once. They fill together because `Promise.all` waits for the slowest of the eight. Add a course, click Dashboard in the sidebar, and the Courses tile goes up by one, because navigating back mounts the component again and the effect runs again.

12.8 Accounts: creating logins, admin only

Everything so far edits records: a `Teacher` row is a name, an email and a subject. None of it is a login. A teacher listed on the Teachers page cannot sign in unless somebody also creates a `User` account for them.

The Accounts page is where that happens, and it is for admins only.

THREE LOCKS, NOT ONE

The page is protected in three separate places, and it is worth seeing all three because each catches a different thing.

Lock one, the sidebar link is not rendered:

frontend/src/Sidebar.jsx

```
// `onlyFor` keeps a link out of the sidebar for other roles. The route and
// the API check the role again; this is only about not offering it.
{
  address: "/accounts",
  text: "Accounts",
  icon: <UserPlus size={16} />,
  onlyFor: ADMIN,
},
```

Lock two, the route itself:

frontend/src/App.jsx

```
{/* Admins only, and the API refuses everyone else regardless. */}
<Route
  path="/accounts"
  element={
    <ProtectedRoute role="admin">
      <Accounts />
    </ProtectedRoute>
  }
/>
```

That is a second `ProtectedRoute` nested inside the one wrapping the whole sidebar. The outer one requires a token; the inner one requires the admin role. Type `/accounts` in the address bar as a student and `ProtectedRoute` sends you straight back to the dashboard with `<Navigate to="/" replace />`.

Lock three, the API:

backend/backend/views.py

```
class RegisterView(generics.CreateAPIView):
    """Create an account. Admins only.

    There is no public sign-up: a student does not make their own account, an
    admin makes it for them and picks the role. Anonymous callers get a 401,
    signed-in teachers and students a 403.

    If there is no admin left to do this, `python manage.py createsuperuser`
    still works, and a superuser counts as an admin.
    """

    queryset = User.objects.all()
    serializer_class = RegisterSerializer
    permission_classes = [IsAdmin]
```

Locks one and two are convenience. Lock three is the one that actually holds. Delete the React app entirely and `POST /api/register/` still refuses a non-admin.

The call carries the admin's own token, which is what makes lock three possible:

frontend/src/api.js

```
// Creating an account is an admin job, so this call carries the admin's own
// token. There is no public sign-up.
export function register(details) {
  return request("/register/", {
    method: "POST",
    body: JSON.stringify(details),
  });
}
```

Compare that with `login()`, which passes `auth: false` to deliberately leave the header off. `register()` does not, so `request()` attaches `Authorization: Bearer <token>` and the server can see who is asking.

NOTE

There used to be a public `/register` page. It is gone, and `App.jsx` keeps a redirect so an old bookmark does not land on a 404: `<Route path="/register" element={<Navigate to="/login" replace />} />`. The `replace` means the redirect does not leave an entry in the browser history, so pressing Back does not bounce you round in a loop.

THE FORM, AND WHAT THE SERIALIZER DEMANDS

The page is a plain form. No table, no `reloadCount`, no `editingId`, no confirm dialog. It has seven fields and submits once:

```
// frontend/src/pages/Accounts.jsx (inside handleSubmit)
await register({
  username,
  email,
  password,
  phone,
  role,
  first_name: firstName,
  last_name: lastName,
});
```

Note the renaming on the last two lines. React state uses `firstName` because that is the JavaScript convention; Django uses `first_name` because that is the Python and database convention. The object literal translates between them at the boundary.

What the server requires comes straight from the serializer:

FIELD	REQUIRED?	RULE ON THE SERVER
<code>username</code>	yes	Must not already exist (<code>validate_username</code>)
<code>email</code>	yes	Declared <code>EmailField(required=True)</code> , so the format is checked
<code>password</code>	yes	Run through Django's password validators, write-only
<code>phone</code>	yes	Must not already exist (<code>validate_phone</code>), write-only
<code>first_name</code>	yes	Declared <code>CharField(required=True)</code>
<code>last_name</code>	no	Comes from the base <code>User</code> model, blank is fine
<code>role</code>	no	One of admin, teacher, student. Defaults to student

`first_name` being required is easy to miss, because on Django's own `User` model it is optional. The serializer overrides it:

backend/backend/serializers.py

```
class RegisterSerializer(serializers.ModelSerializer):
    phone = serializers.CharField(required=True, write_only=True)
    first_name = serializers.CharField(required=True)
    email = serializers.EmailField(required=True)
    # Only an admin can reach this endpoint, so letting the caller pick the
    # role is safe. Left out, the account is a student.
    role = serializers.ChoiceField(
        choices=Profile.ROLE_CHOICES,
        required=False,
        default=Profile.STUDENT,
    )
```

The form matches, field for field. `Last name` is the only `Input` in the file without `required` on it:

```
// frontend/src/pages/Accounts.jsx (two of the seven fields)
<Input
  label="First name"
  required
  value={firstName}
  onChange={(event) => setFirstName(event.target.value)}
/>

<Input
  label="Last name"
  value={lastName}
  onChange={(event) => setLastName(event.target.value)}
/>
```

`write_only=True` on `phone` and `password` means they are accepted on the way in and never sent back out. That is obviously right for a password. For the phone it is a smaller thing but the same instinct.

THE ROLE SELECT

Three roles, with the permission each one gets spelled out in the option text:


```
// frontend/src/pages/Accounts.jsx (inside the form)
<Select
  label="Role"
  required
  value={role}
  onChange={(event) => setRole(event.target.value)}
>
  <option value="student">Student ☐ read only, may hand work in</option>
  <option value="teacher">
    Teacher ☐ course material, enrollments and grading
  </option>
  <option value="admin">Admin ☐ everything, including accounts</option>
</Select>
```

Two details separate this from every other `Select` in the app. There is no `placeholder`, and `role` starts as `"student"` rather than `""`. So the dropdown is never empty and the safest role is preselected. If an admin rushes the form and never touches this field, they create a student, which is the choice that can do the least damage.

The three values must match `Profile.ROLE_CHOICES` on the server exactly. `role` is a `ChoiceField`, so `"Teacher"` with a capital T would come back as a 400.

PASSWORD RULES THAT WILL REJECT A SUBMISSION

`validate_password` in the serializer runs Django's configured validators:

backend/backend/serializers.py

```
def validate_password(self, value):
    # Run Django's own password rules, the same ones the reset flow uses.
    validate_password(value)
    return value
```

Which validators those are is set in settings:

backend/lms/settings.py

```
AUTH_PASSWORD_VALIDATORS = [
    {
        'NAME': 'django.contrib.auth.password_validation.UserAttributeSimilarityValidator',
    },
    {
        'NAME': 'django.contrib.auth.password_validation.MinimumLengthValidator',
    },
    {
        'NAME': 'django.contrib.auth.password_validation.CommonPasswordValidator',
    },
    {
        'NAME': 'django.contrib.auth.password_validation.NumericPasswordValidator',
    },
]
```

In plain terms, a password is rejected if it:

- is too much like the username, email or name on the same account
- is shorter than 8 characters
- appears in Django's built-in list of common passwords
- is all digits

So `12345678` fails twice over: it is common and it is numeric. `password` fails as common. `nusrat` fails on length and on similarity to the username. Every one of those comes back as a 400 with a readable sentence, which `readError()` flattens and the red `Alert` displays.

COMMON MISTAKE

Testing this page with `test1234` or `admin123` and reading the rejection as a bug in your form. It is not. Django is doing exactly what it was configured to do. Pick something like `Winter-Term-2026` and it goes straight through.

AFTER A SUCCESSFUL CREATE

The admin stays signed in as themselves:

frontend/src/pages/Accounts.jsx

```
// Deliberately not logged in as the new person: the admin who filled
// this in stays signed in as themselves.
setNotice(
    `Account created for ${username} as a ${role}. They log in with the phone number
    ${phone}.`,
);
clearForm();
```

That is a real decision, not an oversight. A public sign-up page logs you in as the account it just made, because that account is yours. Here it belongs to somebody else. Swapping the admin's session for the new user's would be wrong every time.

The success banner is set to stay on screen longer than elsewhere:

frontend/src/pages/Accounts.jsx

```
const [notice, setNotice] = useFlash(8000);
```

Every other page calls `useFlash()` with no argument and gets the 4000 millisecond default. This message contains a phone number the admin has to read and pass on to a real person, so it gets eight seconds.

`clearForm()` then empties all seven fields, ready for the next account, and resets the role back to `"student"`.

THE HONEST LIMITATION

This page creates accounts. It does not list them, search them, edit them, deactivate them or delete them.

Which means there is no screen in this app for: seeing who has an account, promoting a student to teacher, fixing a typo in somebody else's email, or resetting a colleague's password. All of that is Django admin, at `http://127.0.0.1:8001/admin/`, where the `Profile` model has the `role` field on it and the list page lets you edit that role directly. The Profile page says so out loud when an admin is looking:

frontend/src/pages/Profile.jsx

```
{user?.role === "admin" && (
  <p className="mt-6 max-w-4xl text-sm text-slate-500">
    Changing somebody else's details, or anybody's role, is
    done from the Django admin site.
  </p>
)}
```

A small app leaning on Django admin for the rare administrative jobs is a reasonable trade. The important part is that the app tells you where to go instead of leaving you hunting for a button that does not exist.

See Figure 0.5.

CHECKPOINT As an admin, create a teacher account. Read the phone number off the green banner, log out, and sign in with that phone and password. The sidebar should now show no Accounts link, and the Teachers page should show no Add button, because a teacher may not change teacher records. Log back in as the admin afterwards.

12.9 Profile: your own account, in two halves

Everyone gets this page. It is the only route inside the sidebar with no role condition at all:

frontend/src/App.jsx

```
{/* Your own account. Every role gets this one. */}  
<Route path="/profile" element={<Profile />} />
```

The way in is the username at the bottom of the sidebar, which is a `<Link to="/profile">`.

TWO FORMS, BECAUSE THEY ARE TWO DIFFERENT JOBS

Look at the file's opening comment:

frontend/src/pages/Profile.jsx

```
// Everyone gets this page, whatever their role. Two separate forms, because  
// they are two different jobs with different rules: your details save on  
// their own, and a password change needs the old password and signs you out.
```

There are two `<form>` elements side by side in a grid, and everything about them is separate. Two submit handlers. Two "saving" flags. Two error messages. They even hit different endpoints: `PATCH /api/profile/` and `POST /api/change-password/`.

You can see the split in the state:

frontend/src/pages/Profile.jsx

```
const [detailsError, setDetailsError] = useState("");  
const [detailsNotice, setDetailsNotice] = useFlash();  
const [isSavingDetails, setIsSavingDetails] = useState(false);  
  
const [currentPassword, setCurrentPassword] = useState("");  
const [newPassword, setNewPassword] = useState("");  
const [confirmPassword, setConfirmPassword] = useState("");  
  
const [passwordError, setPasswordError] = useState("");  
const [isSavingPassword, setIsSavingPassword] = useState(false);
```

Two error variables, not one, so a rejected password does not paint a red banner over the details form the user has just saved successfully. The `Alert` for each sits inside its own form.

TIP

One `<form>` per submit action is an HTML rule, not a style preference. A form submits all its fields together and fires one `onSubmit`. Two independent Save buttons in one `<form>` means guessing which one was pressed, and pressing Enter in any text field fires whichever button comes first in the markup.

READING THE ACCOUNT FROM THE SERVER

The saved session in `localStorage` holds only three things: id, username and role. That is what `login()` stored after the login response. It does not have your email or your phone. So the page fetches:

frontend/src/pages/Profile.jsx

```
// Read the account from the server rather than from the saved session: the
// session only keeps id, username and role.
useEffect(() => {
  async function load() {
    try {
      const data = await fetchProfile();
      setUsername(data.user.username ?? '');
      setRole(data.user.role ?? '');
      setFirstName(data.user.first_name ?? '');
      setLastName(data.user.last_name ?? '');
      setEmail(data.user.email ?? '');
      setPhone(data.user.phone ?? '');
      setDetailsError('');
    } catch (problem) {
      setDetailsError(problem.message);
    } finally {
      setIsLoading(false);
    }
  }

  load();
}, []);
```

Note `data.user.username`, not `data.username`. The view wraps the account in a `user` key:

```
# backend/backend/views.py (inside ProtectedView)
def get(self, request):
    return Response({
        'message': 'successfully fetched this user',
        'user': self._user_data(request.user)
    })
```

Every `?? ""` is there because a React input must never receive `undefined` as its `value`. If it does, React treats the input as **uncontrolled**, which means React stops tracking what is typed in it, and it logs a warning about switching from uncontrolled to controlled the moment a real value shows up. `last_name` is the one most likely to be empty, and `phone` comes back as `None` for any account with no Profile row.

WHAT YOU CAN CHANGE AND WHAT YOU CANNOT

Four fields are editable. Two are shown as read-only text.

The badge at the top displays your username and role without inputs:

frontend/src/pages/Profile.jsx

```
<div className="mb-4 flex flex-wrap items-center gap-2 rounded-md bg-slate-50 px-3 py-2 text-sm text-slate-600">
  <span>
    Signed in as <strong>{username}</strong>
  </span>
  <span className="rounded-full bg-white px-2 py-0.5 text-xs font-medium text-slate-600">
    {roleLabel(role)}
  </span>
</div>
```

`roleLabel()` from `permissions.js` maps `"admin"` to `"Admin"` and anything unknown to `"No role"`.

That is not just a frontend decision. The serializer's docstring lists exactly what is missing and why:

backend/backend/serializers.py

```
class ProfileUpdateSerializer(serializers.Serializer):
    """What you are allowed to change about yourself.

    Not on this list, on purpose:
        role      - changing your own would be a promotion. Admins only, in
                    Django admin.
        username  - it is how an admin finds you; the phone is what you log in
                    with, so there is nothing to gain by renaming yourself.
        password  - has its own endpoint, because it needs the old one first.
    """

    first_name = serializers.CharField(required=False, allow_blank=True, max_length=150)
    last_name = serializers.CharField(required=False, allow_blank=True, max_length=150)
    email = serializers.EmailField(required=False)
    phone = serializers.CharField(required=False, max_length=20)
```

Four fields in that class. Four inputs on the page. If `role` were merely hidden in React, anyone could send `{"role": "admin"}` with curl and promote themselves. It is absent from the serializer, so DRF discards it before the view ever sees it.

The view is equally narrow:

backend/backend/views.py

```
def patch(self, request):
    serializer = ProfileUpdateSerializer(
        data=request.data,
        context={'request': request},
    )
    serializer.is_valid(raise_exception=True)
    serializer.save()

    request.user.refresh_from_db()
    return Response({
        'message': 'Your details have been saved.',
        'user': self._user_data(request.user)
    })
```

There is no id in the URL and no id in the body. `serializer.save()` reads `self.context["request"].user` and edits that account and no other. There is no way to aim this endpoint at somebody else.

Saving is one call:

frontend/src/pages/Profile.jsx

```
await updateProfile({
  first_name: firstName,
  last_name: lastName,
  email,
  phone,
});
setDetailsError("");
setDetailsNotice("Your details have been saved.");
```

`updateProfile` sends `PATCH`, not `PUT`. **PATCH** means "change the fields I sent"; **PUT** means "replace the whole thing with what I sent". Every field on `ProfileUpdateSerializer` is `required=False`, and `serializer.save()` uses `if field in data` before setting anything, so an absent field is genuinely left alone rather than blanked.

CHANGING YOUR PHONE CHANGES YOUR LOGIN

This is the one thing on the page that surprises people, so the form says it in plain words:

frontend/src/pages/Profile.jsx

```
<p className="mt-2 text-xs text-slate-500">
  Your phone number is what you log in with. Change it and the
  new one is what you type next time.
</p>
```

`LoginView` looks an account up by phone, not by username or email. So the Phone box on this page is not contact information, it is your username in the sense that matters. Change it, log out, and typing the old number gets you "Invalid phone or password".

The server protects the one thing it can:

backend/backend/serializers.py

```
def validate_phone(self, value):
    value = value.strip()
    if not value:
        raise serializers.ValidationError(
            "Your phone number is how you sign in, so it cannot be blank."
        )

    me = self.context["request"].user
    if Profile.objects.filter(phone=value).exclude(user=me).exists():
        raise serializers.ValidationError(
            "Another account already uses that phone number."
        )
    return value
```

`.exclude(user=me)` is the important part. Without it, saving the form without touching the phone would find your own row, decide the number is taken, and refuse. `phone` is `unique=True` on the model, so "taken by somebody else" has to be distinguished from "taken by you".

Email gets a similar check, for a specific reason:

backend/backend/serializers.py

```
def validate_email(self, value):
    # Password reset looks an account up by email, so two accounts sharing
    # one would make that ambiguous.
    me = self.context["request"].user
    if User.objects.filter(email__iexact=value).exclude(pk=me.pk).exists():
        raise serializers.ValidationError(
            "Another account already uses that email address."
        )
    return value
```


`email__iexact` is a case-insensitive match, so `Nusrat@example.com` collides with `nusrat@example.com`. Django's `User.email` is not unique by default; this makes it unique here because the password reset flow needs to find exactly one account from an address.

The read-only note sits just above the Save button:

frontend/src/pages/Profile.jsx

```
<p className="mt-4 text-xs text-slate-500">
  Your username and your role are set by an admin, so they are
  not editable here.
</p>
```

THE PASSWORD FORM, AND WHY IT SIGNS YOU OUT

Three fields, all `type="password"`, all required. The server checks all three:

backend/backend/serializers.py

```
def validate_current_password(self, value):
    # Knowing the old password is what makes this yours to change. Without
    # it, a borrowed browser tab would be enough to take the account.
    if not self.context["request"].user.check_password(value):
        raise serializers.ValidationError("That is not your current password.")
    return value
```

That comment is the whole argument for asking. You are already signed in; the token proves that. But a token proves the browser is yours, not that the person at the keyboard is. Someone who walks up to an unlocked laptop could otherwise change the password and lock out the owner. Requiring the old password makes that a different, harder attack.

Then the checks that need to see all three fields at once:

backend/backend/serializers.py

```
def validate(self, attrs):
    if attrs["new_password"] != attrs["confirm_password"]:
        raise serializers.ValidationError({
            "confirm_password": "The two new passwords do not match."
        })

    if attrs["new_password"] == attrs["current_password"]:
        raise serializers.ValidationError({
            "new_password": "The new password has to be different from the old one."
        })

    validate_password(attrs["new_password"], user=self.context["request"].user)
    return attrs
```

`validate_password` here is the same set of rules the Accounts page runs, with one addition: passing `user=` switches on the similarity check against *your* username, email and name.

NOTE

A per-field method like `validate_current_password` can only see one field. Comparing two fields needs `validate(self, attrs)`, which runs after all the individual ones and receives everything. That is a DRF convention worth remembering; it is the same reason `PasswordResetConfirmSerializer` compares its two passwords in `validate` too.

Now the part users find alarming until it is explained. On success, the page signs you out:

frontend/src/pages/Profile.jsx

```
await changePassword({
  current_password: currentPassword,
  new_password: newPassword,
  confirm_password: confirmPassword,
});

// The old token stays technically valid until it expires, so the honest
// thing is to end the session here and make them sign in with the new
// password.
//
// The message is left where the sign-out redirect cannot lose it. Being
// dumped on the login screen with no explanation reads like a failure,
// when in fact the change worked.
leaveNoticeForLogin("Password changed. Please log in again.");
logout();
navigate("/login", { replace: true });
```

The reason is a property of the token format. A **JWT** (JSON Web Token) is self-contained: it carries its own claims and its own signature, and the server checks it by verifying the signature, not by looking it up in a database. Which means there is nothing to delete to make one stop working. It is valid until its expiry time, full stop.

The Django side does what it can:

```
# backend/backend/views.py (inside ChangePasswordView.post)
# Refresh tokens issued earlier are retired. The access token already
# in the caller's hands cannot be revoked – JWTs are not looked up on
# each request – so the frontend signs you out and makes you log in
# again with the new password.
blacklist_user_tokens(user)
```

Refresh tokens *are* stored, so they can be blacklisted. The short-lived access token in the browser cannot be. Dropping the session on the frontend is the remaining honest move: the browser throws away the token it is holding, so the account is genuinely reachable only with the new password.

Which leaves a user-experience problem. Signing out redirects to `/login`, and a login page that appears with no explanation looks like something failed, when in fact the change succeeded. Router state cannot carry a message through that redirect, and the comment in `flash.js` says precisely why:

frontend/src/flash.js

```
// A message that has to survive both a page change and a sign-out.  
//  
// Router state cannot carry this. Signing out re-renders whatever page you  
// were on inside ProtectedRoute, which fires its own redirect to /login and  
// replaces the history entry – throwing away any state that came with it. So  
// the message goes somewhere the redirect cannot touch, and the login page  
// takes it exactly once.  
export function leaveNoticeForLogin(message) {  
  sessionStorage.setItem(HANDOFF_KEY, message);  
}
```

`sessionStorage` survives navigation within the tab and is cleared when the tab closes. The Login page reads it with `takeNoticeLeftForLogin()`, which removes the key as it reads, so the message shows once and does not reappear on the next visit.

One last detail in the handler. Look at where `setIsSavingPassword(false)` is:

frontend/src/pages/Profile.jsx

```
} catch (problem) {  
  setPasswordError(problem.message);  
  setIsSavingPassword(false);  
}
```

In the `catch`, not in a `finally`. Every other save in this book uses `finally`. Here, the success path navigates away and unmounts the component, so setting state afterwards would be pointless at best. Only the failure path still has a form on screen with a button that needs to come back to life.

See Figure 0.6.

CHECKPOINT Change your first name and save. The green "Your details have been saved." appears and fades after four seconds. Now change your password. You land on the login page with "Password changed. Please log in again." in a banner. Sign in with the new password. Refresh the login page before signing in and the banner is gone, because `takeNoticeLeftForLogin` removed it the first time it was read.

12.10 NotFound: the route that catches everything else

The smallest file in the project:

frontend/src/pages/NotFound.jsx

```
export default function NotFound() {
  return (
    <div className="flex h-screen w-screen items-center justify-center">
      <div className="p-10 border border-slate-300 rounded shadow">
        <h1 className="text-4xl font-semibold text-slate-900">404</h1>
        <p className="mt-2 text-lg text-slate-600">Page Not Found</p>
      </div>
    </div>
  );
}
```

No state, no props, no imports. **404** is the HTTP status code for "there is no such page", and it is recognisable enough to use as a heading on its own.

What makes it work is where it sits in the route list:

frontend/src/App.jsx

```
<Route path="*" element={<NotFound />} />
</Routes>
```

`path="*" matches any address. React Router does not simply take the first match; it scores every route and picks the most specific one, and a bare * scores lowest by design. So /courses still reaches the Courses page even though * also matches it, and /corses (a typo) reaches this.`

Two things about its position are deliberate.

It is last. Not because the router requires it, but because a human reading `App.jsx` should find the catch-all at the bottom where they expect it.

It is outside the `ProtectedRoute` block. Every real page is nested inside the route that renders `<Sidebar />`. This one is not. So a 404 fills the whole window with no sidebar around it, which is the reason for `h-screen w-screen` on the outer div. It also means an unknown address shows this page rather than bouncing you to `/login`, which is more truthful: the problem is the address, not your session.

See Figure 0.7.

CHECKPOINT Visit `http://localhost:5180/nope` while signed in. You get the 404 card, centred, with no sidebar. Now visit `http://localhost:5180/register`. You are sent to the login page instead, because that specific redirect route beats the catch-all. (5180 is the port set in `vite.config.js`. If Vite told you a different number when it started, use that one.)

What you have now

- A Courses page that joins a teacher id to a teacher name in the browser, and a dropdown that shows names while submitting ids
- An Enrollments page with two dropdowns and a date column the server fills in, proving that `auto_now_add` fields belong in the table and never in the form
- A Lessons page that needed nothing new at all, which is the clearest evidence the pattern holds
- An Assignments page whose Lesson dropdown narrows to the chosen course, clears itself when the course changes, and converts between local time and UTC in both directions so a deadline means the same moment everywhere
- A Submissions page where a student may hand work in but not edit it, matching `SubmissionWrites` on the server exactly
- A Results page that hides already-graded submissions, keeps the one being edited visible, and turns a unique-constraint 400 into a sentence a teacher can act on
- A Dashboard that fetches eight lists in parallel and measures them, because there is no counting endpoint
- An Accounts page locked three ways, that creates a login with a role and tells you the phone number the new person will sign in with
- A Profile page with independent details and password forms, where changing your phone changes your login and changing your password ends your session on purpose
- A 404 page on a catch-all route, rendered outside the sidebar so it fills the window
- Every route in `App.jsx` now pointing at a real file, with nothing left importing an empty module

The Whole Codebase, File by File

Twelve chapters ago this project was an empty folder. Now Git is tracking 68 files, and there are a few more on disk that Git is told to ignore. This chapter is the map. Nothing new gets built here. Instead you get three things:

1. The whole folder tree, with a note on every line.
2. A table for each folder saying what each file is for and which chapter built it.
3. Two traces that follow a single click all the way from the browser to MySQL and back.

Keep a finger in this chapter while you read the rest of the book. When a chapter says "open `backend/backend/serializers.py`" and you cannot remember which of the two `backend` folders that is, this is where you look it up.

The chapter map used in the tables

The last column of every table below names the chapter that builds that file. Here is what those numbers mean.

CH	TITLE
1	What we are building
2	Installing the tools
3	Starting the Django project
4	Models and migrations
5	The Django admin
6	Serializers, views and URLs
7	Accounts, login and JSON Web Tokens
8	Starting the React app
9	Routes, the sidebar and the shared components
10	api.js, logging in, and keeping the session
11	The eight resource pages
12	Roles, permissions and your own profile
13	The whole codebase, file by file (this chapter)

The whole tree

Here is every file in the project that you would ever open by hand. Five folders are left out, because nothing in them is written by a person and none of them belongs in Git: `node_modules/` (the downloaded JavaScript libraries), `__pycache__/` (Python's compiled cache), `backend/.venv/` (the Python virtual environment), `frontend/dist/` (the packaged frontend), and `.git/` (Git's own storage).

```

lms/
├── .DS_Store
├── .gitignore
├── README.md
├── docs/
│   ├── guide/
│   │   ├── lms-guide.html
│   │   ├── lms-guide.pdf
│   │   ├── lms-auth-guide.html
│   │   ├── lms-auth-guide.pdf
│   │   └── images/
│   │       ├── ss-login.png
│   │       ├── ss-dashboard.png
│   │       └── ... 24 files in all
│   └── slides/
│       ├── lms-auth-slides.html
│       └── lms-auth-slides.pdf
├── backend/
│   └── manage.py
...
├── requirements.txt
├── lms/
URLs
│   ├── __init__.py
package.
│   ├── settings.py
│   ├── urls.py
│   ├── wsgi.py
│   └── asgi.py
├── backend/
│   ├── __init__.py
package.
│   ├── apps.py
│   ├── models.py
│   ├── serializers.py
│   ├── views.py
│   ├── urls.py
│   ├── permissions.py
│   ├── admin.py
│   └── tests.py
in it.
│   └── migrations/
│       ├── __init__.py
package.
│   ├── 0001_initial.py
│   ├── 0002_course_enrollment_lesson_assignment_submission_and_more.py
│   └── 0003_profile_role.py

```

the whole project, one Git repository
 # macOS folder junk. Not part of the app.
 # what Git must never commit
 # how to run the project, in one page

 # this book

 # the companion volume on accounts

 # the screenshots printed in both books

 # the class deck for the same subject

 # everything Python
 # the command you type: runserver, migrate,

 # the six Python packages this needs
 # the project package: settings and the root

 # empty. Marks the folder as a Python

 # database, apps, CORS, JWT, email
 # /admin/ and /api/ and nothing else
 # entry point for a normal production server
 # entry point for an async production server
 # the one Django app. All the real code.
 # empty. Marks the folder as a Python

 # the app's name, so Django can find it
 # the nine database tables
 # turns models into JSON and back
 # one class per endpoint
 # which URL reaches which view
 # who is allowed to change what
 # registers every model in Django admin
 # the stub Django made. Nothing was written

 # the history of every table change
 # empty. Marks the folder as a Python

 # Student, Teacher, Profile
 # adds the `role` column


```

└─ 0004_existing_accounts_become_admins.py # changes data, not table shape
└─ frontend/                               # everything JavaScript
    ├── .gitignore                         # node_modules, dist, logs
    ├── README.md                         # OUT OF DATE. Describes an older version.
    ├── package.json                      # the libraries and the four npm scripts
    └── package-lock.json                 # the exact version of every library. Never
edit.
    ├── vite.config.js                   # React plugin, Tailwind plugin, port 5180
    ├── eslint.config.js                 # the code checker's rules
    ├── index.html                       # the single HTML page the whole app lives in
    ├── public/
    │   └─ favicon.svg                   # the little icon in the browser tab
    └─ src/
        ├── main.jsx                     # the first file that runs
        ├── index.css                     # one line: @import "tailwindcss";
        ├── App.jsx                       # the list of addresses
        └── Sidebar.jsx                   # the left menu (collapsible) and the page
frame
    ├── api.js                           # every call to Django, in one file
    └── auth.js                           # the auth context object and the useAuth()
hook
    ├── AuthContext.jsx                  # AuthProvider: who is logged in
    ├── permissions.js                   # decides which buttons to show for a role
    ├── flash.js                         # the green message that clears itself
    ├── components/
    │   ├── index.js                     # one import line for all of the below
    │   ├── styles.js                     # two shared Tailwind class strings
    │   ├── Alert.jsx                     # the red and green banners
    │   ├── Button.jsx                     # primary, secondary, danger
    │   ├── Checkbox.jsx                   # a checkbox with a label
    │   ├── ConfirmDialog.jsx             # "Delete this?" instead of window.confirm()
    │   ├── Div.jsx                       # a plain div wrapper
    │   ├── IconButton.jsx                 # the small pencil and bin buttons
    │   └── Input.jsx                     # a labelled text box, with a show-password
eye
    ├── PageHeader.jsx                   # the title and subtitle at the top of a page
    ├── ProtectedRoute.jsx               # no token, no page
    ├── Select.jsx                        # a labelled dropdown
    ├── Table.jsx                         # the header row; pages supply the body rows
    └── Textarea.jsx                      # a labelled multi-line box
    └─ pages/
        ├── Login.jsx                     # phone and password
        ├── Dashboard.jsx                 # the eight counting tiles
        ├── Teachers.jsx                  # list, add, edit, delete teachers
        ├── Students.jsx                  # the most heavily commented resource page
        ├── Courses.jsx                   # courses, with a teacher dropdown
        ├── Enrollments.jsx               # links a student to a course
        └── Lessons.jsx                   # lessons inside a course

```

```

└─ Assignments.jsx      # assignments, with a due date
└─ Submissions.jsx     # handed-in work
└─ Results.jsx         # a score and feedback per submission
└─ Accounts.jsx        # admin only: create a login
└─ Profile.jsx         # your own details and password
└─ NotFound.jsx        # the 404 page

```

TIP

You do not have to trust that list. Run `git ls-files` from the project root and Git prints every file it is tracking, one per line — 98 of them, most of that being the screenshots under `docs/guide/images/`. The tree above is deliberately shorter than that list in one place only: it names the images folder rather than all 24 files in it.

NOTE

There really are two folders called `backend`. The outer one, `lms/backend/`, is just a folder on your disk holding the Python side. The inner one, `lms/backend/backend/`, is the Django **app** (a Django app is one bundle of models, views and URLs that Django loads as a unit), and its name is the string `'backend'` in `INSTALLED_APPS`. That is why every import in the Python code reads `from backend.views import ...` and why the database tables are called `backend_teacher`, `backend_course` and so on.

Repo root

FILE	WHAT IT IS FOR	CHAPTER
<code>README.md</code>	How to start MySQL, Django and Vite; the endpoint table; the list of backend quirks. Read it before anything else.	1
<code>.gitignore</code>	Tells Git to skip <code>__pycache__/</code> , <code>db.sqlite3</code> , <code>.venv/</code> , <code>.env</code> and <code>frontend/node_modules/</code> .	2
<code>.DS_Store</code>	A file macOS drops into every folder you open in Finder. Nothing in the app reads it.	not used
<code>docs/</code>	<code>guide/</code> holds this book and the companion volume on accounts, as HTML and PDF, with the screenshots in <code>guide/images/</code> . <code>slides/</code> holds the class deck for the same subject. None of it is part of the running app.	13

backend/

FILE	WHAT IT IS FOR	CHAPTER
<code>manage.py</code>	The command-line front door for Django. Everything you type starts <code>python manage.py ...</code> . It sets <code>DJANGO_SETTINGS_MODULE</code> to <code>lms.settings</code> and hands the rest to Django.	3
<code>requirements.txt</code>	The six Python packages: <code>Django==5.2.7</code> , <code>djangorestframework==3.15.2</code> , <code>djangorestframework-simplejwt==5.4.0</code> , <code>django-cors-headers==4.9.0</code> , <code>mysqlclient==2.2.7</code> , <code>PyJWT==2.10.1</code> .	2
<code>lms/</code>	The project package: settings and the root URL list.	3
<code>backend/</code>	The one Django app, where all the real code lives.	3
<code>.venv/</code>	The virtual environment: a private copy of Python and the six packages. Created by you, never committed.	2

backend/lms/

This folder is the project's control room. It is created by `django-admin startproject lms` and holds no business logic at all.

FILE	WHAT IT IS FOR	CHAPTER
<code>__init__.py</code>	Empty. Its presence is what makes the folder importable as <code>lms</code> .	3
<code>settings.py</code>	Every switch: <code>INSTALLED_APPS</code> , <code>MIDDLEWARE</code> , <code>CORS_ALLOWED_ORIGINS</code> , <code>REST_FRAMEWORK</code> , <code>SIMPLE_JWT</code> , <code>DATABASES</code> (MySQL, read from environment variables with defaults), password validators, the email settings, and <code>DEFAULT_AUTO_FIELD</code> .	3, and 7 for <code>SIMPLE_JWT</code> , 8 for CORS
<code>urls.py</code>	Twenty-three lines. Sends <code>/admin/</code> to Django admin and <code>/api/</code> into <code>backend/urls.py</code> .	6
<code>wsgi.py</code>	The entry point a normal production web server would use. Untouched.	3
<code>asgi.py</code>	The entry point an async production web server would use. Untouched.	3

The whole of `lms/urls.py` that matters is these four lines, and it is worth memorising because every request in the book passes through them:

backend/lms/urls.py

```
urlpatterns = [  
    path('admin/', admin.site.urls),  
    path('api/', include('backend.urls')),  
]
```

NOTE

`settings.py` still says "Generated by 'django-admin startproject' using Django 6.0.6" at the top, and the first two migration files say "Generated by Django 6.0.6" too. (The third one says 5.2.7, and the fourth has no such line, because a person wrote it.) The project now runs on Django 5.2.7, the version pinned in `requirements.txt`. Those comments are a leftover from the machine the files were first made on. The setting `DEFAULT_AUTO_FIELD = 'django.db.models.BigAutoField'` near the bottom of `settings.py` exists for exactly that reason: without it, Django 5.2 would want to rewrite every primary key that Django 6 had already created.

backend/backend/

This is the app: nine Python files, plus the `migrations/` folder.

FILE	WHAT IT IS FOR	CHAPTER
<code>__init__</code> <code>___.py</code>	Empty. Makes the folder a Python package.	3
<code>apps.py</code>	Five lines. <code>class BackendConfig(AppConfig): name = 'backend'</code> . Django reads this to learn the app's label.	3
<code>models.py</code>	Nine classes, one per table: <code>Profile</code> , <code>Teacher</code> , <code>Student</code> , <code>Course</code> , <code>Enrollment</code> , <code>Lesson</code> , <code>Assignment</code> , <code>Submission</code> , <code>Results</code> . Every relationship in the app is declared here.	4
<code>migrations/</code>	The record of how those tables came to be. Own table below.	4
<code>admin.py</code>	Registers all nine models with <code>@admin.register(...)</code> so they appear at <code>http://127.0.0.1:8001/admin/</code> . <code>ProfileAdmin</code> has <code>list_editable = ['role']</code> , which is how you promote somebody to admin.	5
<code>serializers.py</code>	Fourteen serializer classes. Eight plain <code>ModelSerializer</code> s for the resources, plus <code>RegisterSerializer</code> , <code>LoginSerializer</code> , <code>ProfileUpdateSerializer</code> , <code>ChangePasswordSerializer</code> and the two password-reset ones.	6, and 7 and 12 for the account ones
<code>views.py</code>	Twenty-two view classes. Sixteen are short generic views for the eight resources, two per resource; the other six are the hand-written account views.	6, and 7 and 12 for the account ones
<code>urls.py</code>	Twenty-two <code>path(...)</code> lines mapping every URL under <code>/api/</code> to a view.	6
<code>permissions.py</code>	<code>role_of()</code> , <code>IsAdmin</code> , <code>RoleWritePermission</code> , and its three subclasses <code>AdminWrites</code> , <code>TeachingStaffWrites</code> and <code>SubmissionWrites</code> . This file is what actually enforces roles.	12
<code>tests.py</code>	Created by <code>startapp</code> , never filled in. Three lines, all of it the stub Django wrote.	not used

A **generic view** is a view class Django REST Framework already wrote for you: you say which rows it works on and which serializer to use, and it supplies the rest. The eight resource views are all the same shape because of that. Here is the pair for teachers, copied whole, so you can see how little there is to them:

backend/backend/views.py

```
class TeacherListCreateView(generics.ListCreateAPIView):
    queryset = Teacher.objects.all()
    serializer_class = TeacherSerializer
    permission_classes = [AdminWrites]

class TeacherRetrieveUpdateDestroyAPIView(generics.RetrieveUpdateDestroyAPIView):
    queryset = Teacher.objects.all()
    serializer_class = TeacherSerializer
    permission_classes = [AdminWrites]
```

Three lines each. `ListCreateAPIView` gives you `GET` (list them all) and `POST` (make one). `RetrieveUpdateDestroyAPIView` gives you `GET`, `PUT`, `PATCH` and `DELETE` for a single row picked out by its id.

backend/backend/migrations/

A **migration** is a small Python file describing one change to the database shape. Django writes them for you when you run `makemigrations`, and applies them when you run `migrate`. They run in numbered order, and Django remembers in a table called `django_migrations` which ones it has already done.

FILE	WHAT IT DOES	CHAPTER
<code>__init__.py</code>	Empty. Makes the folder a package. Django will not find migrations without it.	4
<code>0001_initial.py</code>	Creates <code>Student</code> , <code>Teacher</code> and <code>Profile</code> . At this point <code>Profile</code> has only <code>user</code> and <code>phone</code> .	4
<code>0002_course_enrollment_lesson_assignment_submission_and_more.py</code>	Creates <code>Course</code> , <code>Enrollment</code> , <code>Lesson</code> , <code>Assignment</code> , <code>Submission</code> and <code>Results</code> , with all their foreign keys. Django built that long name out of the models it found.	4
<code>0003_profile_role.py</code>	Adds one column: <code>role</code> , a <code>CharField(max_length=10)</code> with choices <code>admin</code> , <code>teacher</code> , <code>student</code> , defaulting to <code>student</code> .	12
<code>0004_existing_accounts_become_admins.py</code>	Changes data, not table shape. Runs <code>Profile.objects.update(role="admin")</code> once, so people who registered before roles existed are not suddenly locked out.	12

Migration `0004` is the only one in the project that a person wrote by hand rather than `makemigrations` generating it. It uses `migrations.RunPython`, which lets you run ordinary Python during a migration:

backend/backend/migrations/0004_existing_accounts_become_admins.py

```
def make_existing_accounts_admins(apps, schema_editor):  
    """Everyone who already had an account keeps full access.  
  
    New accounts default to `student`, but the people who registered before  
    roles existed were able to do everything, so locking them out on upgrade  
    would be a surprise. Promote them once, here.  
    """  
    Profile = apps.get_model("backend", "Profile")  
    Profile.objects.update(role="admin")
```

**COMMON
MISTAKE**

Deleting a migration file because it has a long ugly name. Do not. Django tracks migrations by filename in the `django_migrations` table. Remove `0002_course_enrollment_lesson_assignment_submission_and_more.py` and the next `migrate` will try to create tables that already exist, and fail.

frontend/

FILE	WHAT IT IS FOR	CHAPTER
<code>package.json</code>	The six runtime libraries (<code>react</code> , <code>react-dom</code> , <code>react-router-dom</code> , <code>lucide-react</code> , <code>tailwindcss</code> , <code>@tailwindcss/vite</code>), the dev tools, and the four scripts: <code>dev</code> , <code>build</code> , <code>lint</code> , <code>preview</code> .	8
<code>package-lock.json</code>	The exact version of every library and of every library those libraries need. <code>npm install</code> writes it. Never edit it by hand.	8
<code>vite.config.js</code>	Eighteen lines. Turns on the React plugin and the Tailwind plugin, and sets <code>server.port</code> to <code>5180</code> .	8
<code>index.html</code>	The one real HTML page. It has an empty <code><div id="root"></code> and a <code><script type="module" src="/src/main.jsx"></code> . Everything you see is put inside that div by React.	8
<code>eslint.config.js</code>	Rules for <code>npm run lint</code> . Has one custom rule so that imported components and icons are not reported as unused.	8
<code>.gitignore</code>	Keeps <code>node_modules</code> , <code>dist</code> and log files out of Git.	8
<code>README.md</code>	Out of date. See the warning below.	not used
<code>public/favicon.svg</code>	The browser-tab icon. Vite copies anything in <code>public/</code> to the site root as-is.	8
<code>src/</code>	All the code you wrote. Own tables below.	8 onward
<code>dist/</code>	The output of <code>npm run build</code> . See the section below.	not used
<code>node_modules/</code>	The downloaded libraries. Created by <code>npm install</code> , never committed.	8

COMMON MISTAKE

`frontend/README.md` describes an earlier version of this app that had no server. It tells you to edit `src/data.js` to add a teacher, and lists a page called `Register.jsx` . Neither file exists any more. Data comes from the Django API now, and there is no public sign-up page. The README at the **project root**, `lms/README.md` , is the current one. If the two disagree, the root one wins.

frontend/src/

Nine files sit directly in `src/` . These are the app's plumbing: everything that is not a single screen and not a small reusable widget.

FILE	WHAT IT IS FOR	CHAPTER
<code>main.jsx</code>	Thirteen lines, and the first thing that runs. It calls <code>createRoot(...)</code> on the <code>#root</code> div and wraps <code><App /></code> in <code><BrowserRouter></code> and <code><AuthProvider></code> .	8
<code>index.css</code>	One line: <code>@import "tailwindcss";</code> . That single line is the whole styling setup.	8
<code>App.jsx</code>	The list of addresses. Every <code><Route></code> in the app is here, including the redirect from <code>/register</code> to <code>/login</code> and the <code>*</code> route that shows the 404 page.	9
<code>Sidebar.jsx</code>	The left menu, the signed-in user's name and role badge, the Log out button, and the <code><Outlet /></code> where each page is drawn. Filters the <code>Accounts</code> link out for non-admins, and collapses to an icon-only strip, remembering that in <code>localStorage</code> .	9
<code>api.js</code>	Every <code>fetch</code> in the app. <code>request()</code> at the centre, the auth calls, and the <code>resource()</code> factory that builds <code>list</code> , <code>create</code> , <code>update</code> and <code>remove</code> for each of the eight resources.	10
<code>auth.js</code>	Fourteen lines: <code>AuthContext</code> and the <code>useAuth()</code> hook. Kept apart from <code>AuthContext.jsx</code> so that file exports only a component, which is what Vite's fast refresh (the feature that updates the page as you type without losing your place) needs.	10
<code>AuthContext.jsx</code>	<code>AuthProvider</code> : holds <code>user</code> and <code>isLoggedIn</code> , provides <code>login</code> and <code>logout</code> , reads the saved session on first render, and listens for the <code>lms:unauthorised</code> event.	10
<code>permissions.js</code>	A copy of the backend's role table used only to decide which buttons to draw. Exports <code>ADMIN</code> , <code>TEACHER</code> , <code>STUDENT</code> , <code>canWrite()</code> , <code>canCreate()</code> and <code>roleLabel()</code> .	12
<code>flash.js</code>	<code>useFlash()</code> , the green message that clears itself after four seconds, plus <code>leaveNoticeForLogin()</code> and <code>takeNoticeLeftForLogin()</code> , which carry one message across a sign-out.	11

NOTE

`permissions.js` in the frontend and `permissions.py` in the backend hold the same table twice, on purpose. The comment at the top of the JavaScript file says it plainly: "The server is what actually enforces them, so a mistake here is a cosmetic bug, not a hole." If you add a resource, change both.

frontend/src/components/

Small pieces used by many pages. None of them fetches anything; none of them knows about roles. They take props (values passed in from outside, written like HTML attributes) and draw HTML.

FILE	WHAT IT IS FOR	CHAPTER
<code>index.js</code>	Twelve re-export lines, so a page can write one import instead of twelve.	9
<code>styles.js</code>	Two exported strings, <code>inputBox</code> and <code>labelText</code> , shared by <code>Input</code> , <code>Select</code> and <code>Textarea</code> .	9
<code>Alert.jsx</code>	The banner. Three looks: <code>error</code> (red, the default), <code>success</code> (green), <code>info</code> (grey). Returns <code>null</code> when there is no message, so an empty banner never takes up space.	9
<code>Button.jsx</code>	Three looks: <code>primary</code> (indigo), <code>secondary</code> (outlined) and <code>danger</code> (red). Defaults to <code>type="button"</code> so a button inside a form does not submit it by accident.	9
<code>Checkbox.jsx</code>	A checkbox with its label, wrapped in a <code><label></code> so clicking the text ticks the box.	9
<code>ConfirmDialog.jsx</code>	The delete confirmation. Closes on Cancel, on Escape, and on a click outside the card. Shows "Deleting..." and disables both buttons while the request is running.	11
<code>Div.jsx</code>	A plain <code>div</code> that passes its <code>className</code> and children through.	9
<code>IconButton.jsx</code>	The small square buttons that hold an icon. Two looks: <code>plain</code> and <code>danger</code> .	9
<code>Input.jsx</code>	A labelled text box. Everything it does not use, it spreads onto the <code><input></code> with <code>{...rest}</code> , so <code>required</code> , <code>value</code> and <code>onChange</code> all work. <code>type="password"</code> takes a second branch that adds the eye button for showing the password.	9
<code>PageHeader.jsx</code>	The <code><h1></code> title and grey subtitle at the top of every page.	9
<code>ProtectedRoute.jsx</code>	Redirects to <code>/login</code> when there is no token, remembering where you were headed. With <code>role="admin"</code> it also redirects a non-admin back to <code>/</code> .	10
<code>Select.jsx</code>	A labelled dropdown. Draws an extra empty <code><option></code> when you pass <code>placeholder</code> .	9
<code>Table.jsx</code>	Builds the header row from a list of column names. The body rows stay in each page, because every page formats its cells differently.	9
<code>Textarea.jsx</code>	A labelled multi-line box, three rows by default.	9

frontend/src/pages/

One file per screen. Thirteen of them.

FILE	WHAT IT IS FOR	CHAPTER
<code>Login.jsx</code>	Phone and password. Calls <code>login()</code> from the auth context, then navigates to wherever you were trying to go.	10
<code>Dashboard.jsx</code>	Eight tiles with counts. There is no counts endpoint (no single address that just returns the numbers), so it fetches all eight lists with <code>Promise.all</code> and measures their lengths.	11
<code>Students.jsx</code>	The reference resource page, 307 lines, with a comment on every one of the six steps. Read this one first.	11
<code>Teachers.jsx</code>	The same six steps for teachers, in 267 lines. The trace later in this chapter walks through it line by line.	11
<code>Courses.jsx</code>	Adds a teacher dropdown, so it loads <code>/api/course/</code> and <code>/api/teacher/</code> together and joins them in JavaScript.	11
<code>Enrollments.jsx</code>	Two dropdowns, student and course. <code>enrollment_date</code> is set by the server, so it is shown but never edited.	11
<code>Lessons.jsx</code>	Lessons plus a course dropdown.	11
<code>Assignments.jsx</code>	The longest page, 365 lines. Narrows the lesson dropdown to the chosen course, and converts between UTC and the browser's local clock for the <code>datetime-local</code> box.	11
<code>Submissions.jsx</code>	Handed-in work. This is the one page where a student sees an Add button, because <code>SubmissionWrites</code> lets a student POST.	11, 12
<code>Results.jsx</code>	Score and feedback. Leaves already-graded submissions out of the dropdown, because <code>Results.submission</code> is one-to-one.	11
<code>Accounts.jsx</code>	Admin only. Creates a login and picks its role. Does not list existing accounts; that is a Django admin job.	12
<code>Profile.jsx</code>	Two separate forms: your details, and a password change that signs you out.	12
<code>NotFound.jsx</code>	Nine lines. A box saying 404.	9

CHECKPOINT Count the pages yourself. Run `ls frontend/src/pages` and you should see exactly thirteen `.jsx` files, matching the table above. If you see a `Register.jsx`, you are looking at an older copy of the project.

Two things on disk that the app never reads

Neither of these is in Git, both can be sitting in your working copy, and neither is used when you run the app.

backend/db.sqlite3

This is the database file Django makes when it is left on its own default engine — a stray from a `migrate` run before `DATABASES` was edited, or from a copy of the project that predates MySQL. It is not in Git, and nothing reads it. `settings.py` points at MySQL and only MySQL:

backend/lms/settings.py

```
DATABASES = {
    'default': {
        'ENGINE': 'django.db.backends.mysql',
        'NAME': os.getenv('DB_NAME', 'lms_db'),
        'USER': os.getenv('DB_USER', 'lms_user'),
        'PASSWORD': os.getenv('DB_PASSWORD', 'lms_user_pw'),
        'HOST': os.getenv('DB_HOST', '127.0.0.1'),
        'PORT': os.getenv('DB_PORT', '3306'),
    }
}
```

`ENGINE` says `django.db.backends.mysql`. There is no second entry and no SQLite fallback. The root `.gitignore` lists `db.sqlite3`, which is why a fresh clone does not carry one. If you find one anyway, you can delete it and the app will not notice. And if you are hunting for your data and cannot find it, it is in MySQL, in the database named `lms_db`.

frontend/dist/

This folder is what `npm run build` produces: the whole React app squeezed into one JavaScript file and one CSS file, ready to be copied onto a real web server. It currently holds `index.html`, `favicon.svg` and `assets/index-B_6RSYlw.js` plus `assets/index-da4n1Hff.css`.

While you are developing, none of it is used. `npm run dev` starts Vite's own server on port 5180, which reads your source files directly and rebuilds them as you type. The `dist/` folder is a snapshot from whenever somebody last ran the build. It is stale the moment you edit a file in `src/`.

`frontend/.gitignore` lists `dist`, so it is not committed either.

COMMON MISTAKE

Editing something in `frontend/src/`, seeing no change, and going looking in `dist/`. The dev server never reads `dist/`. If the page is not updating, look at the terminal running `npm run dev` for an error, and at the browser console for a red message.

Follow one click end to end: adding a teacher

This is the payoff. You are logged in as an admin, you are on the Teachers page, you have typed a name and a subject, and you click **Save**. Here is every file that click touches, in order.

See Figure 10.1.

1. THE FORM CALLS `HANDLESAVE`

The form element in `Teachers.jsx` names the function that runs on submit:

frontend/src/pages/Teachers.jsx

```
<form
  onSubmit={handleSave}
  className="mb-6 rounded-lg border border-slate-200 bg-white p-4 shadow-sm"
>
```

The Save button inside it is `<Button type="submit" disabled={isSaving}>`. Clicking it fires the form's submit event, which React routes to `handleSave`.

2. `HANDLESAVE` STOPS THE BROWSER AND GATHERS THE VALUES

frontend/src/pages/Teachers.jsx

```
async function handleSave(event) {
  event.preventDefault();
  setIsSaving(true);

  const values = { name, email, subject, is_active: isActive };
```

`event.preventDefault()` stops the browser's built-in behaviour, which would be to reload the whole page. `setIsSaving(true)` re-renders the button as "Saving..." and disables it, so a double click cannot create two teachers.

The four values come from the four pieces of **state** the form's boxes are wired to. State is a value a component remembers between redraws. Note the key names: `is_active` with an underscore, because that is what Django calls the field. The JavaScript variable is `isActive`, the JSON key is `is_active`.

3. `HANDLESAVE` PICKS CREATE OVER UPDATE, AND CALLS THE API

frontend/src/pages/Teachers.jsx

```
try {
  if (editingId === 0) {
    await teachersApi.create(values);
    setNotice("Teacher added.");
  } else {
    await teachersApi.update(editingId, values);
    setNotice("Teacher updated.");
  }
}
```

`editingId` is `0` when the form was opened by `openEmptyForm()`, and the row's id when it was opened by `openFormForEditing(teacher)`. Since we clicked "Add teacher", it is `0`, so `create` runs.

4. TEACHERSAPI.CREATE BUILDS THE REQUEST

`teachersApi` is made in `api.js` by a small factory function. A factory is a function that returns a ready-made object, so the same four calls can be stamped out for every resource instead of being typed eight times:

frontend/src/api.js

```
function resource(path) {
  return {
    list: () => request(`${path}/`),
    create: (values) =>
      request(`${path}/`, { method: "POST", body: JSON.stringify(values) }),
```

and:

frontend/src/api.js

```
export const teachersApi = resource("teacher");
```

So `teachersApi.create(values)` becomes `request("/teacher/", { method: "POST", body: '{"name": "...", "email": "...", "subject": "...", "is_active": true}' })`. `JSON.stringify` turns the JavaScript object into a text string, because that is all an HTTP body can carry.

5. REQUEST() ADDS THE HEADERS AND THE TOKEN

frontend/src/api.js

```
async function request(path, options = {}) {
  const { auth = true, ...rest } = options;

  const headers = { ...(rest.headers ?? {}) };
  if (rest.body !== undefined) {
    headers["Content-Type"] = "application/json";
  }

  const token = getToken();
  if (auth && token) {
    headers.Authorization = `Bearer ${token}`;
  }
}
```

A **header** is a short labelled line of extra information sent alongside a request. Two of them get added here. `Content-Type: application/json` tells Django that the body is JSON and not a form. `Authorization: Bearer eyJhbGci...` is the access token, read out of `localStorage` by `getToken()`. Without that second header Django has no idea who you are and will refuse the request.

6. `FETCH` SENDS IT

frontend/src/api.js

```
response = await fetch(BASE_URL + path, { ...rest, headers });
```

`BASE_URL` is `import.meta.env.VITE_API_URL ?? "http://127.0.0.1:8001/api"`. Nobody has set `VITE_API_URL`, so the URL is `http://127.0.0.1:8001/api/teacher/` and the method is `POST`.

Because your page is served from `http://localhost:5180` and the request goes to `http://127.0.0.1:8001`, that is a **cross-origin** request: the two addresses are not the same, so the browser treats the server as a stranger. It first sends a small `OPTIONS` request asking Django whether it is allowed. That question is called a **preflight**. `corsheaders.middleware.CorsMiddleware`, listed second in `MIDDLEWARE` (the list of small pieces of code every request passes through), answers yes because `http://localhost:5180` is in `CORS_ALLOWED_ORIGINS`. Then the real `POST` goes out.

COMMON MISTAKE

Vite reports port 5181 because 5180 was already busy, and every request suddenly fails with a CORS error in the console. `settings.py` only allows 5180 and 5173. Either free up 5180 or add the new port to `CORS_ALLOWED_ORIGINS` and restart Django.

7. DJANGO'S ROOT URL LIST STRIPS `/API/`

backend/lms/urls.py

```
path('api/', include('backend.urls')),
```

The path is `/api/teacher/`. The first segment matches, so Django hands the rest, `teacher/`, to `backend/urls.py`.

8. THE APP'S URL LIST PICKS THE VIEW

backend/backend/urls.py

```
path('teacher/', TeacherListCreateView.as_view(), name='teacher-list'),
```

`teacher/` matches exactly, and there is nothing left over, so this line wins rather than `teacher/<int:pk>/` on the next line. `TeacherListCreateView.as_view()` is the function Django will now call.

9. DRF WORKS OUT WHO YOU ARE

Before your view runs, Django REST Framework authenticates the request using whatever is in `DEFAULT_AUTHENTICATION_CLASSES`:

backend/lms/settings.py

```
"DEFAULT_AUTHENTICATION_CLASSES": (  
    "rest_framework_simplejwt.authentication.JWTAuthentication",  
),
```

`JWTAuthentication` reads the `Authorization: Bearer ...` header, checks the token's signature against `SECRET_KEY`, checks it has not expired, pulls the user id out of it, loads that `User` from the database, and sets `request.user`. If any of that fails, DRF returns `401` and your view never runs.

10. `ADMINWRITES.HAS_PERMISSION` DECIDES

backend/backend/views.py

```
class TeacherListCreateView(generics.ListCreateAPIView):  
    queryset = Teacher.objects.all()  
    serializer_class = TeacherSerializer  
    permission_classes = [AdminWrites]
```

DRF now calls `has_permission` on every class in `permission_classes`. `AdminWrites` does not define its own; it inherits the one from `RoleWritePermission`:

backend/backend/permissions.py

```
def has_permission(self, request, view):  
    if not request.user or not request.user.is_authenticated:  
        return False  
  
    if request.method in SAFE_METHODS:  
        return True  
  
    return role_of(request.user) in self.roles_that_may_write
```

`SAFE_METHODS` is DRF's tuple of methods that only read: `GET`, `HEAD`, `OPTIONS`. Ours is `POST`, so it falls through to the last line. `AdminWrites` sets `roles_that_may_write = (Profile.ADMIN,)`, which is the tuple `("admin",)`.

11. `ROLE_OF` LOOKS UP YOUR ROLE

backend/backend/permissions.py

```
def role_of(user):
    if not user or not user.is_authenticated:
        return None

    if user.is_superuser:
        return Profile.ADMIN

    profile = Profile.objects.filter(user=user).only("role").first()
    if profile is None:
        return Profile.STUDENT

    return profile.role
```

One extra query hits MySQL here, reading the `role` column off your `Profile` row. It comes back as `"admin"`, `"admin"` is in `(("admin",))`, so `has_permission` returns `True` and the view runs.

If you had been signed in as a teacher, `has_permission` would return `False`, DRF would answer `403 Forbidden` with the class's `message` in the body, and steps 12 to 16 would never happen:

backend/backend/permissions.py

```
message = "Only an admin can change teacher and student records."
```

12. `TEACHERSERIALIZER` VALIDATES THE JSON

backend/backend/serializers.py

```
class TeacherSerializer(serializers.ModelSerializer):
    class Meta:
        model = Teacher
        fields = ['id', 'name', 'email', 'subject', 'is_active']
```

`ListCreateAPIView` builds `TeacherSerializer(data=request.data)` and calls `is_valid()`. Because it is a `ModelSerializer`, the rules come straight from the model: `subject` has no `blank=True`, so it is required; `email` is an `EmailField`, so `not-an-email` is rejected; `name` and `subject` are capped at 100 characters. `id` is read-only, because it is the primary key, the number that identifies the row.

If anything fails, DRF answers `400` with a body like `{"subject": ["This field may not be blank."]}` and no row is created.

13. `SERIALIZER.SAVE()` REACHES THE MODEL

A valid serializer calls `Teacher.objects.create(**validated_data)`, which builds one instance of the model class you wrote in Chapter 4:

backend/backend/models.py

```
class Teacher(models.Model):

    name = models.CharField(max_length=100, blank=True, null=True)
    email = models.EmailField(blank=True, null=True)
    subject = models.CharField(max_length=100)
    is_active = models.BooleanField(default=True)
```

14. THE ORM WRITES TO MYSQL

Saving that instance makes Django's **ORM** (the layer that turns Python objects into SQL) produce roughly this statement and send it down the `mysqlclient` connection configured in `settings.py`:

```
INSERT INTO backend_teacher (name, email, subject, is_active) VALUES (...);
```

The table is called `backend_teacher` because Django names tables `<app label>_<model name in lower case>`, and the app label is `backend`. MySQL assigns the next auto-increment `id` and hands it back. Django sets it on the object.

CHECKPOINT Open a MySQL client and run `SELECT * FROM backend_teacher ORDER BY id DESC LIMIT 1;` against `lms_db`. The row you just added through the browser is there, with the id the browser is about to display.

15. DRF SENDS `201 CREATED` BACK

The serializer is re-read to build the response body, which now includes the id MySQL generated:

```
{"id": 7, "name": "Rina Ahmed", "email": "rina@example.com", "subject": "Physics",
"is_active": true}
```

The status code is `201`, which means "created", not the usual `200`.

16. `REQUEST()` UNWRAPS THE RESPONSE

Back in the browser:

frontend/src/api.js

```
if (response.status === 204) {
  return null;
}

const text = await response.text();
let body = null;
if (text) {
  try {
    body = JSON.parse(text);
  } catch {
    body = null;
  }
}

if (!response.ok) {
  throw new Error(readError(body, response));
}

return body;
```

201 is not 204, so it reads the text and parses it. `response.ok` is `true` for anything in the 200s, so no error is thrown and the parsed object is returned. `await teachersApi.create(values)` in step 3 now resolves.

17. THE PAGE SHOWS A GREEN BANNER AND RELOADS THE LIST

frontend/src/pages/Teachers.jsx

```
await teachersApi.create(values);
setNotice("Teacher added.");
} else {
  await teachersApi.update(editingId, values);
  setNotice("Teacher updated.");
}
setError("");
closeForm();
reload();
```

`setNotice` comes from `useFlash()` in `flash.js`, so the green banner will take itself down after four seconds. `closeForm()` sets `formIsOpen` to `false`, which removes the form from the screen.

18. `RELOAD()` RE-RUNS THE EFFECT

frontend/src/pages/Teachers.jsx

```
function reload() {  
  setReloadCount((count) => count + 1);  
}
```

That is the whole trick. `reloadCount` is a number nothing displays. Changing it re-renders the component, and because it is the dependency of the effect below (the list in the square brackets that says "run this again whenever one of these changes"), the effect runs again:

frontend/src/pages/Teachers.jsx

```
useEffect(() => {  
  async function load() {  
    try {  
      // The list endpoint is not paginated, so this is a plain array.  
      setTeachers(await teachersApi.list());  
      setError('');  
    } catch (problem) {  
      setError(problem.message);  
    } finally {  
      setIsLoading(false);  
    }  
  }  
}  
  
load();  
}, [reloadCount]);
```

`teachersApi.list()` is a `GET` to the same `/api/teacher/`, hitting the same view, but this time `has_permission` returns `True` at the `SAFE_METHODS` line without ever looking up your role. The response is a plain array of teacher objects, which goes into `teachers` state.

19. REACT REDRAW THE TABLE

`setTeachers` changes state, so React re-renders. The `teachers.map(...)` inside `<Table>` now produces one more `<tr>`, and your new teacher appears. Above it, `<Alert variant="success">{notice}</Alert>` has a message for the first time, so the green banner renders.

See Figure 8.1.

20. THE SAVE BUTTON COMES BACK

frontend/src/pages/Teachers.jsx

```
    } finally {  
      setIsSaving(false);  
    }  
  }
```

`finally` runs whether the save worked or threw, so the button is re-enabled either way. Four seconds later the `useFlash` timer clears `notice`, `<Alert>` sees empty children and returns `null`, and the green banner disappears on its own.

That is twenty steps across nine files, for one click. Every other create, update and delete in the app follows the same path with different names.

CHECKPOINT Open your browser's developer tools, go to the Network tab, and add a teacher. You should see three entries in a row: an `OPTIONS` to `teacher/` (the CORS preflight), a `POST` to `teacher/` returning `201`, and a `GET` to `teacher/` returning `200`. Click the `POST` and look at the Headers tab to see `Authorization: Bearer ...` going out, and the Response tab to see the new `id` coming back.

A shorter trace: logging in

Adding a teacher assumed you already had a token. Here is where the token comes from. Seven steps.

1. The login form submits. `Login.jsx` has `<form onSubmit={handleSubmit}>`, and `handleSubmit` clears any old error and calls the context:

frontend/src/pages/Login.jsx

```
try {  
  // The backend looks you up by phone number, not by username.  
  await login(phone, password);  
  navigate(goBackTo, { replace: true });  
} catch (problem) {  
  setError(problem.message);  
} finally {  
  setIsSaving(false);  
}
```

2. `login` in the auth provider calls the API. `login` came out of `useAuth()`, which reads the context that `AuthProvider` supplies:

frontend/src/AuthContext.jsx

```
async function login(phone, password) {  
  const data = await loginRequest(phone, password);
```

`loginRequest` is the `login` function from `api.js`, renamed on import so it does not clash with the local `login`.

3. **api.js posts to `/login/` with no token.** This is the one call in the app that deliberately turns authentication off, because you do not have a token yet:

frontend/src/api.js

```
export function login(phone, password) {  
  return request("/login/", {  
    method: "POST",  
    body: JSON.stringify({ phone, password }),  
    auth: false,  
  });  
}
```

4. **Django routes it to `LoginView`.** `/api/login/` goes through `lms/urls.py` to `backend/urls.py`, where the first line matches:

backend/backend/urls.py

```
path('login/', LoginView.as_view(), name='login'),
```

5. **`LoginView.post` finds you by phone and checks the password.** This view is hand-written rather than generic, because looking somebody up by their `Profile.phone` is not something DRF can guess:

backend/backend/views.py

```
try:  
    profile = Profile.objects.get(phone=phone)  
    user = profile.user  
except Profile.DoesNotExist:  
    return Response({'error': 'Invalid phone or password'},  
status=status.HTTP_400_BAD_REQUEST)  
  
if not user.check_password(password):  
    return Response({'error': 'Invalid phone or password'},  
status=status.HTTP_400_BAD_REQUEST)
```

Both failures give the same sentence on purpose, so nobody can use the error message to find out which phone numbers are registered. `check_password` scrambles what you typed with the same one-way recipe used when the password was set, then compares the two scrambled results; the real password is never stored.

6. Tokens are made and sent back. `get_tokens_for_user` asks `simplejwt` for a pair:

backend/backend/views.py

```
def get_tokens_for_user(user):
    """Helper to get JWT tokens for a user."""
    refresh = RefreshToken.for_user(user)
    return {
        'refresh': str(refresh),
        'access': str(refresh.access_token),
    }
```

and the response wraps them together with a message, your id, username and role. Note the shape: the tokens are **nested** under a `tokens` key.

7. The token lands in localStorage. Back in `AuthContext.jsx`:

frontend/src/AuthContext.jsx

```
// The tokens are nested. data.access does not exist.
const nextUser = {
  id: data.user_id,
  username: data.username,
  role: data.role,
};
saveSession(data.tokens.access, nextUser);

setUser(nextUser);
setIsLoggedIn(true);
```

`saveSession` is in `api.js` and does the actual writing:

frontend/src/api.js

```
export function saveSession(token, user) {
  localStorage.setItem(TOKEN_KEY, token);
  localStorage.setItem(USER_KEY, JSON.stringify(user));
}
```

with `TOKEN_KEY = "lms_access_token"` and `USER_KEY = "lms_user"`. From this moment every call in step 5 of the teacher trace finds a token and attaches it.

Then `setIsLoggedIn(true)` re-renders everything under the provider, `ProtectedRoute` stops redirecting, and `navigate(goBackTo, { replace: true })` from step 1 sends you to the dashboard, or back to whichever page you originally asked for.

CHECKPOINT After logging in, open developer tools, go to Application (Chrome) or Storage (Firefox), pick Local Storage, and select `http://localhost:5180`. You should see two keys: `lms_access_token` holding a long string with two dots in it, and `lms_user` holding something like `{"id":1,"username":"tausif","role":"admin"}`. Delete `lms_access_token` and refresh the page: you are back at the login screen.

NOTE The refresh token is thrown away. `saveSession` is only ever given `data.tokens.access`. There is no refresh endpoint on this backend, so a refresh token would have nothing to refresh against. That is why `SIMPLE_JWT["ACCESS_TOKEN_LIFETIME"]` is set to eight hours instead of the library's five-minute default.

Glossary

Every technical word this book uses, in one sentence each.

API - a way for one program to ask another program for data or to change data, without a human clicking anything; here, the React app asks the Django app.

endpoint - one specific address in an API that does one job, like

`http://127.0.0.1:8001/api/teacher/`.

JSON - a plain-text way of writing data that both Python and JavaScript understand, made of `{ }` objects, `[ ]` lists, numbers, strings and `true` / `false`.

HTTP method - the verb on a request that says what you want done: `GET` to read, `POST` to create, `PUT` to replace, `PATCH` to change part, `DELETE` to remove.

status code - a three-digit number in the reply saying how it went; this app uses `200` fine, `201` created, `204` deleted with no body, `400` your data was wrong, `401` no valid token, `403` your role is not allowed, `404` no such thing.

CORS - a browser rule that blocks a page on one address from calling a server on another address unless that server says it is allowed; `CORS_ALLOWED_ORIGINS` in `settings.py` is where this app says it.

JWT - JSON Web Token, a long signed string that carries who you are, so the server can trust it without looking anything up in a session table.

access token - the JWT you attach to every request as `Authorization: Bearer ...`; in this app it lasts eight hours and lives in `localStorage` under `lms_access_token`.

refresh token - a longer-lived token whose only job is to fetch a new access token; this backend issues one but has no endpoint to redeem it, so the frontend ignores it.

migration - a numbered Python file describing one change to the database shape, applied by `python manage.py migrate`, like `0003_profile_role.py` adding the `role` column.

ORM - Object Relational Mapper, the layer that lets you write `Teacher.objects.all()` in Python instead of writing `SELECT * FROM backend_teacher` in SQL.

model - a Python class that stands for one database table, with one class attribute per column; `Teacher` in `models.py` is a model.

serializer - the piece that turns a model instance into JSON on the way out, and checks incoming JSON before it becomes a model on the way in; `TeacherSerializer` is one.

view - the code that handles one endpoint: it decides who is allowed in, runs the serializer, and returns a response; `TeacherListCreateView` is a view.

queryset - a description of which rows a view works with, written in the ORM; `Teacher.objects.all()` is the queryset on the teacher views.

primary key - the column that gives every row a unique id; Django adds it automatically as `id`, a number MySQL counts up on its own.

foreign key - a column holding another table's primary key, which is how rows point at each other; `Course.teacher` is a foreign key to `Teacher`, and the API sends it as a plain number like `3`.

one-to-one - a foreign key with a uniqueness rule, so each row on each side can only be paired once; `Results.submission` is one-to-one, which is why a submission can only be graded once.

cascade delete - `on_delete=models.CASCADE` on a foreign key, meaning that deleting the parent row deletes the children too; deleting a teacher here deletes their courses, and then those courses' lessons.

virtual environment - a private folder holding its own copy of Python and its own installed packages, so this project's Django 5.2.7 cannot clash with another project's; it lives in `backend/.venv/`.

package manager - the tool that downloads and installs libraries for you: `pip` on the Python side reading `requirements.txt`, `npm` on the JavaScript side reading `package.json`.

npm script - a named shortcut listed under `"scripts"` in `package.json` and run with `npm run <name>`; this project has `dev`, `build`, `lint` and `preview`.

component - a JavaScript function that returns some screen, which you then use like an HTML tag; `Button` in `components/Button.jsx` is used as `<Button>`.

prop - a value passed into a component from outside, written like an HTML attribute; in `<Alert variant="success">` the prop `variant` is `"success"`.

state - a value a component remembers between renders and can change, created with `useState`; `const [name, setName] = useState("")` in `Teachers.jsx` is state.

hook - a function whose name starts with `use` that lets a component tap into React's features; this project uses `useState`, `useEffect`, `useContext`, and its own `useAuth` and `useFlash`.

JSX - the HTML-looking syntax written inside JavaScript files, which Vite compiles into real function calls; it is why `.jsx` files can contain `<div>` in the middle of code.

context - React's way of handing a value to every component below it without passing it down through every layer; `AuthContext` carries the signed-in user to every page inside `AuthProvider`.

route - a rule pairing one address with one component, declared as `<Route path="/teachers" element={<Teachers />} />` in `App.jsx`.

controlled input - a form box whose displayed text comes from state and whose `onChange` writes back to that state, so React always knows what is in it; every `<Input>` in this app is one.

localStorage - a small store the browser keeps per website, surviving refreshes and restarts, holding text only; this app keeps `lms_access_token` and `lms_user` there.

environment variable - a setting read from outside the code so it can differ per machine without editing files; `DB_PASSWORD` on the Django side and `VITE_API_URL` on the Vite side are both read this way, each with a default in the code.

What you have now

- A complete file tree of the project, with a one-line note on every folder and file.
- A table per folder saying what each file does and which chapter builds it.
- The knowledge that a stray `backend/db.sqlite3` and the built `frontend/dist/` can sit in your working copy without ever being read by the running app, and that `frontend/README.md` describes an older, server-less version.
- A twenty-step trace of "add a teacher", from the `onSubmit` on the form to the `INSERT` in MySQL and back to the green banner.
- A seven-step trace of logging in, ending with the access token in `localStorage`.
- A glossary of the thirty-two terms this book leans on.

Running It, Breaking It, Fixing It

You have built the whole thing. The Django project, the models, the serializers, the views, the permissions, the React pages, the sidebar, the login screen. This chapter is about living with it.

Three things happen here. First, you learn the exact start-up routine, so that getting the app on screen stops being a guessing game. Second, you walk the app end to end and put real data in it, in the order the database will accept. Third, and this is the longest part, you meet the errors. Not made-up errors. The actual red text this project produces, with the actual fix next to it.

Read the troubleshooting section even when nothing is broken. Recognising an error message is most of the skill of fixing it.

14.1 The three things that must be running

An app like this is not one program. It is three programs that talk to each other over the network on your own machine.

#	PROGRAM	LIVES ON	JOB
1	MySQL	port 3306	Stores the rows. Everything else forgets; this remembers.
2	Django	http://127.0.0.1:8001	The API. Reads and writes MySQL, checks tokens, checks roles.
3	Vite	http://localhost:5180	Serves the React pages to your browser.

An **API** is the part of a program that other programs talk to instead of people. Django here serves no web pages of its own for the app; it answers requests with data. A **token** is the long string of characters Django hands you when you log in, which your browser then sends back with every later request as proof of who you are.

The chain runs one way. The browser talks to Vite for the page, then the page talks to Django for the data, then Django talks to MySQL for the rows. Break any link and everything downstream of it fails.

NOTE

"Port" is just a numbered door on your computer. One program can hold one door at a time. `127.0.0.1` and `localhost` both mean "this machine, not the internet". They are two names for the same address, which is why the CORS list in `backend/lms/settings.py` has to name both. **CORS** stands for Cross-Origin Resource Sharing, and it is a browser safety rule: a page loaded from one address is not allowed to read a reply from a different address unless that other server says the address is fine. `CORS_ALLOWED_ORIGINS` is Django's list of the addresses it says are fine.

Open **three separate terminal windows** and leave all three open. Each one holds a program that keeps running. Do not close them, do not run something else in them.

TERMINAL 1: MYSQL

MySQL is usually installed as a background service, so this is often already running before you sit down. Start it the way it was installed on your machine:

```
# macOS, installed with Homebrew
brew services start mysql
```

```
# Linux with systemd
sudo systemctl start mysql
```

On Windows, or if you installed XAMPP or MySQL Workbench, start the server from that program's control panel.

Then, **once ever**, create the database and the user this project expects. Open a MySQL shell as an administrator (`mysql -u root -p`) and run exactly this:

```
-- run once, in a MySQL shell
CREATE DATABASE lms_db;
CREATE USER 'lms_user'@'127.0.0.1' IDENTIFIED BY 'lms_user_pw';
GRANT ALL PRIVILEGES ON lms_db.* TO 'lms_user'@'127.0.0.1';
```

Those three names are not decoration. They are the defaults in `backend/lms/settings.py`:

```
# backend/lms/settings.py (the DATABASES block)
DATABASES = {
    'default': {
        'ENGINE': 'django.db.backends.mysql',
        'NAME': os.getenv('DB_NAME', 'lms_db'),
        'USER': os.getenv('DB_USER', 'lms_user'),
        'PASSWORD': os.getenv('DB_PASSWORD', 'lms_user_pw'),
        'HOST': os.getenv('DB_HOST', '127.0.0.1'),
        'PORT': os.getenv('DB_PORT', '3306'),
    }
}
```

`os.getenv('DB_NAME', 'lms_db')` reads an environment variable named `DB_NAME`, and falls back to `'lms_db'` when there is none. An environment variable is a value you set in the terminal before running a command. So if your MySQL uses different credentials you do not edit this file, you set the variables instead:

```
# only if your MySQL differs from the defaults
export DB_USER=root
export DB_PASSWORD=my_root_password
```

COMMON MISTAKE

Creating the user as `'lms_user'@'localhost'` instead of `'lms_user'@'127.0.0.1'`. MySQL treats those as two different users. Django connects to `127.0.0.1`, finds no matching grant, and refuses you with error 1045. Copy the SQL above exactly.

CHECKPOINT

Run `mysql -u lms_user -p'lms_user_pw' -h 127.0.0.1 -e "SHOW DATABASES;"`. If `lms_db` is in the list you have printed, MySQL is up and the credentials Django will use actually work. If this fails, nothing further in this chapter will work either.

TERMINAL 2: THE DJANGO BACKEND

Every Python project of any size gets its own **virtual environment**, usually shortened to "venv". It is a private folder holding one copy of Python and only the libraries this project needs, so that installing Django 5.2 here cannot break some other project that wants Django 4. In this repo that folder is `backend/.venv`.

This is the full block for a fresh terminal. The first two lines only matter the very first time; after that you skip straight to activating.

```
# Terminal 2, from the lms folder
cd backend
python3 -m venv .venv           # first time only: build the venv
source .venv/bin/activate       # every time: step into it
pip install -r requirements.txt  # first time, and after any dependency change
python manage.py migrate
python manage.py runserver 8001
```

On Windows the activation line is different:

```
.venv\Scripts\activate
```

What each line does:

- `python3 -m venv .venv` creates the private folder.
- `source .venv/bin/activate` switches this terminal over to it. You will see `(.venv)` appear at the front of your prompt. That prefix is the only reliable sign that you are inside it.
- `pip install -r requirements.txt` installs the six libraries listed in `backend/requirements.txt`:

`backend/requirements.txt`

```
Django==5.2.7
djangorestframework==3.15.2
djangorestframework-simplejwt==5.4.0
django-cors-headers==4.9.0
mysqlclient==2.2.7
PyJWT==2.10.1
```

- `python manage.py migrate` builds the tables in `lms_db`. It works from the **migration files** in `backend/backend/migrations/`, which are small Python files describing each change ever made to the models. Running `migrate` is safe any time; when everything is already up to date it prints `No migrations to apply.`
- `python manage.py runserver 8001` starts the API and holds the terminal. Leave it.

You should see something close to this:

```
Watching for file changes with StatReloader
Performing system checks...

System check identified no issues (0 silenced).
Django version 5.2.7, using settings 'lms.settings'
Starting development server at http://127.0.0.1:8001/
Quit the server with CONTROL-C.
```

TIP

`runserver` restarts itself whenever you save a `.py` file. That is what "Watching for file changes" means. You almost never need to stop and start it by hand. The exception is `--noreload`, covered in the troubleshooting table below, which turns that off.

CHECKPOINT

Open `http://127.0.0.1:8001/api/teacher/` in your browser. You should get the Django REST Framework browsable page saying `HTTP 401 Unauthorized` and `"Authentication credentials were not provided."` That 401 is a **success**. It proves Django is up, the URL is wired, and the permission classes are doing their job. A blank page or "site can't be reached" means the server is not running.

See Figure 2.1.

TERMINAL 3: THE VITE FRONTEND

```
# Terminal 3, from the lms folder
cd frontend
npm install      # first time, and after any dependency change
npm run dev
```

There is no `venv` here. `npm install` reads `frontend/package.json` and drops everything into `frontend/node_modules`. `npm run dev` runs the `dev` script from that same file, which is just `vite`.

Vite prints:

```
VITE v8.1.5  ready in 412 ms

→ Local:   http://localhost:5180/
→ Network: use --host to expose
→ press h + enter to show help
```

Read the port number on that `Local:` line every single time. It is the line that tells you where the app actually is. The project asks for 5180:

`frontend/vite.config.js`

```
server: {
  port: 5180,
},
```

and 5180 is one of the four addresses Django will accept requests from:

backend/lms/settings.py

```
CORS_ALLOWED_ORIGINS = [  
    'http://localhost:5180',  
    'http://127.0.0.1:5180',  
    'http://localhost:5173',  
    'http://127.0.0.1:5173',  
]
```

If Vite reports any other number, the browser will block every API call. Section 14.4 has the fix.

CHECKPOINT All three terminals are open and none of them has given you back a normal prompt. Terminal 1 may have (a service runs in the background), but Terminals 2 and 3 should both look busy. Now open `http://localhost:5180` and you should land on the login page. Not the dashboard. The login page, because `ProtectedRoute` in `frontend/src/components/ProtectedRoute.jsx` sends anyone without a token to `/login`.

See Figure 0.1.

STOPPING EVERYTHING

Press `Ctrl+C` in Terminal 3, then `Ctrl+C` in Terminal 2. MySQL can stay running; it costs you very little and saves the start-up step next time.

14.2 Getting into a database that has nothing in it

You have just run `migrate` on a brand new `lms_db`. Every table exists and every table is empty. There is no account. You cannot log in, and there is no sign-up page, because this app deliberately has none:

frontend/src/App.jsx

```
{/* /register used to be a public page. There is no public sign-up any  
   more: an admin creates accounts at /accounts, inside the app. */}  
<Route path="/register" element={<Navigate to="/login" replace />} />
```

So the first account has to be made from the terminal. It takes three steps, and the second one is the one everybody skips.

STEP 1: MAKE A SUPERUSER

In Terminal 2, stop `runserver` with `Ctrl+C` (or open a fourth terminal and activate the venv there):


```
# backend/, with the venv active
python manage.py createsuperuser
```

It asks three questions:

```
Username: tausif
Email address: tausif@example.com
Password:
Password (again):
Superuser created successfully.
```

A **superuser** is Django's built-in "can do anything" account. It can sign in at `http://127.0.0.1:8001/admin/` and edit any row in any table.

STEP 2: GIVE THAT USER A PROFILE WITH A PHONE NUMBER

Here is the part that catches everyone. `createsuperuser` creates a row in Django's `auth_user` table. It does **not** create a `Profile`. And this app's login view does not look at `auth_user` at all to find you. It looks at `Profile.phone`:

```
# backend/backend/views.py (inside LoginView.post)
try:
    profile = Profile.objects.get(phone=phone)
    user = profile.user
except Profile.DoesNotExist:
    return Response({'error': 'Invalid phone or password'},
status=status.HTTP_400_BAD_REQUEST)
```

Read that carefully. The lookup starts from the phone number. No Profile means no phone number means nothing to look up, so a superuser with no Profile gets `Invalid phone or password` no matter what they type. Not because the password is wrong. Because there is no way to name that account on the login form.

The account is not short of permission, it is simply unreachable. `role_of()` in `backend/backend/permissions.py` already treats any superuser as an admin:

```
# backend/backend/permissions.py (inside role_of)
if user.is_superuser:
    return Profile.ADMIN
```

The only missing piece is the phone number to type. Start `runserver` again, then go to:

```
http://127.0.0.1:8001/admin/backend/profile/add/
```

Fill in three fields:

FIELD	WHAT TO PUT
User	pick the superuser you just made from the dropdown
Phone	any phone number you will remember, for example <code>01711110001</code>
Role	Admin

Click **Save**.

See Figure 3.2.

NOTE

Role is set to `Student` by default on that form, because `Profile.role` has `default=STUDENT`. For a superuser it makes no practical difference, since `role_of()` returns admin on `is_superuser` before it ever reads the row. Set it to Admin anyway. Six weeks from now you will be reading the Profile list to work out who is who, and a superuser labelled "Student" is a lie you will believe.

STEP 3: LOG IN WITH THE PHONE NUMBER

Go to <http://localhost:5180/login>. Type the **phone number**, not the username, not the email. Type the password you set in step 1.

You land on the dashboard, the sidebar shows your username with an "Admin" badge beside it, and because you are an admin the **Accounts** link is in the sidebar. Everyone else's account gets made from there, on a form, with no terminal involved.

See Figure 0.3.

COMMON MISTAKE

Typing the username on the login form. The placeholder on the phone field literally reads `01711110001`, and the Accounts page that made the account says "They sign in with the phone number, not the username." It still happens on the first try to almost everybody, because every other website in the world wants an email. The error you get is `Invalid phone or password`, which sounds like a wrong password and is not.

See Figure 0.2.

CHECKPOINT

You are on the dashboard, and the sidebar shows nine links plus **Accounts** at the bottom of the list. If **Accounts** is missing, your role is not admin. Go back to <http://127.0.0.1:8001/admin/backend/profile/>, fix the role in the list (it is editable straight from the list view), save, then log out and back in.

MAKING THE OTHER ACCOUNTS

Now use the app. Go to **Accounts** in the sidebar and fill in the form: first name, last name, username, email, phone, password, and a role from the dropdown. Make yourself one teacher account and one student account so you can see the app through all three sets of eyes.

The green banner tells you what to hand over:

Account created for rakib as a teacher. They log in with the phone number 01711110002.

You stay signed in as yourself. Creating an account does not switch you into it.

See Figure 0.5.

TIP

Give the accounts phone numbers that are easy to tell apart, like 01711110001 for the admin, ...0002 for the teacher, ...0003 for the student. You will be logging in and out of all three constantly while testing roles.

14.3 A guided first run: filling the app with data

Now put a full set of rows in. This is worth doing once slowly, because it teaches you the shape of the data better than any diagram.

The order is not a suggestion. Look at the foreign keys in backend/backend/models.py :

TO CREATE A	YOU MUST ALREADY HAVE	BECAUSE OF THIS LINE
Course	a Teacher	<code>teacher = models.ForeignKey(Teacher, ...)</code>
Enrollment	a Student and a Course	<code>student = ...</code> and <code>course = ...</code>
Lesson	a Course	<code>course = models.ForeignKey(Course, ...)</code>
Assignment	a Lesson and a Course	<code>lesson = ...</code> and <code>course = ...</code>
Submission	an Assignment and a Student	<code>assignment = ...</code> and <code>student = ...</code>
Results	a Submission	<code>submission = models.OneToOneField(Submission, ...)</code>

A **foreign key** is a column holding the id of a row in another table. `Course.teacher` holds a number like 3, meaning "teacher number 3". You cannot point at a teacher who does not exist, so the teacher has to be created first. In the app this shows up in a way that is easy to misread: the dropdown is empty. Not broken, empty, because there is nothing to list yet.

Work top to bottom.

1. A TEACHER

Sidebar → **Teachers** → **Add teacher**. The form has Name, Email, Subject and an Active checkbox. Only Subject is marked required (`Teacher.name` and `Teacher.email` are `blank=True, null=True` in the model), but fill all of them in. Save.

See Figure 10.1.

Expect: the form closes, a green banner reads `Teacher added.` and fades after four seconds, and a row appears with ID `1`. The header above the table changes from `0 teachers` to `1 teachers`.

See Figure 8.1.

NOTE

The grammar of "1 teachers" is not a typo you should fix in passing. It is the literal template `` ${teachers.length} teachers `` in `frontend/src/pages/Teachers.jsx`. Leave it for now and notice how many real apps have exactly this bug.

2. A STUDENT

Sidebar → **Students** → **Add student**. Name, Email, Roll number, **Enrollment date**, Active. The date is required here.

Expect: green banner `Student added.`, one row.

COMMON MISTAKE

Assuming this "Enrollment date" is the same as the "Enrolled" column on the Enrollments page. They are two different fields on two different models. `Student.enrollment_date` is the day the person joined the school, you type it, it is editable. `Enrollment.enrollment_date` is the day a student joined one course, the server stamps it, and you never see it in a form. The comment in `frontend/src/pages/Students.jsx` spells this out at the top of `handleSave()`.

3. A COURSE

Sidebar → **Courses** → **Add course**. Title, a **Teacher** dropdown, Description.

The dropdown now has your teacher in it. That list did not come from the courses endpoint. An **endpoint** is one URL the API answers on. The Courses page fetches `/api/course/` and `/api/teacher/` at the same time and joins them in JavaScript, because a course row only carries `{"teacher": 1}` and a number is not a name.

Expect: green banner `Course added.`, and the table's Teacher column shows the teacher's **name**, not the number 1.

4. AN ENROLLMENT

Sidebar → **Enrollments** → **Add enrollment**. Two dropdowns, Student and Course. That is the whole form.

Expect: green banner `Enrollment added.`, and the "Enrolled" column filled in with today's date that you never typed. The server did that:

```
// frontend/src/pages/Enrollments.jsx (inside handleSave)
// enrollment_date is auto_now_add on the server. Sending it is silently
// ignored, so it is shown in the table but never in the form.
const values = { student: Number(studentId), course: Number(courseId) };
```

`auto_now_add=True` is a setting on the model field. It tells Django to fill that column in itself at the moment the row is first created, which also makes the field read-only to anything you send.

See Figure 11.2.

5. A LESSON

Sidebar → **Lessons** → **Add lesson**. Title, a Course dropdown, Description.

Expect: green banner `Lesson added.`, one row, Course column showing the course title.

6. AN ASSIGNMENT

Sidebar → **Assignments** → **Add assignment**. Title, **Due date** (a date-and-time picker), Course, Lesson, Description.

Two things here are worth watching for.

First, pick the Course **before** the Lesson. The lesson dropdown narrows itself to the lessons of the chosen course:

frontend/src/pages/Assignments.jsx

```
// An assignment points at both a course and a lesson, so once a course is
// chosen only that course's lessons are worth offering.
const lessonsToOffer = courseId
  ? lessons.filter((lesson) => lesson.course === Number(courseId))
  : lessons;
```

Second, the due date. You type local time; the page converts it to UTC before sending, and converts back to local when you reopen the form for editing. **UTC** is the one worldwide clock that servers agree to store times in, so that a time saved in Dhaka and read in London means the same moment.

Expect: green banner `Assignment added.`, and a Due column showing the same wall-clock time you typed. The table prints it with `toLocaleString()`, which renders in your own timezone even though the server stored UTC. If the Due column is out by several hours, the conversion is the thing to look at.

CHECKPOINT With one assignment saved, open `http://127.0.0.1:8001/admin/backend/assignment/` and look at the same row in Django admin. The `due_date` shown there is in UTC, because `TIME_ZONE = 'UTC'` in settings. Seeing the same moment written two ways in two places is the clearest way to understand why the conversion in `Assignments.jsx` exists.

7. A SUBMISSION

Sidebar → **Submissions** → **Add submission**. Assignment, Student, Content.

Expect: green banner `Submission added.`, and a "Submitted" column with the current date and time you never typed. Same `auto_now_add` story as the enrollment date.

8. GRADE IT

Sidebar → **Results** → **Add result**. Submission, Score, Feedback.

The Submission dropdown does not just say "Submission #1". It shows the submission id, then the student's name, then the assignment title, assembled by `describeSubmission()` in `frontend/src/pages/Results.jsx` out of three separate lists the page loaded.

Expect: green banner `Result added.`

See Figure 12.5.

Now do the thing that teaches you something. Click **Add result** again and look at the Submission dropdown. It is empty. The one submission you have is already graded and has been filtered out on purpose:

`frontend/src/pages/Results.jsx`

```
// Results.submission is one-to-one: a submission can only be graded once,  
// and a second attempt comes back as a 400. Already-graded submissions are  
// left out of the "new result" dropdown so that is hard to walk into.  
const gradedIds = results  
  .filter((result) => result.id !== editingId)  
  .map((result) => result.submission);
```

One-to-one means the link goes both ways and only once: one submission has at most one result, and one result belongs to exactly one submission. To change a grade you use the pencil icon on the existing result, not a new one.

CHECKPOINT Click **Dashboard**. There are eight tiles: Teachers, Students, Courses, Enrollments, Lessons, Assignments, Submissions and Results. Every one of them should read `1`. If any tile reads `0`, that step did not save and you missed a red banner. The dashboard fetches all eight lists and counts them, so it is the fastest whole-system check you have.

Finally, log out and log back in as your **student** account. Walk the same pages. Every Add / Edit / Delete button is gone, replaced by a grey dash in the Action column, and **Accounts** is missing from the sidebar. The one exception is **Add submission**, which a student still gets, because `canCreate()` in `frontend/src/permissions.js` carves out that single case and `SubmissionWrites` on the server agrees with it.

See Figure 6.1.

See Figure 6.2.

14.4 Troubleshooting: real errors and real fixes

Everything on the left of these tables is text this project actually produces. Find your error, apply the fix, move on.

ERRORS FROM THE DJANGO TERMINAL

WHAT YOU SEE	WHAT IT MEANS AND WHAT TO DO
<code>Error: That port is already in use.</code>	Something already holds port 8001. Usually it is an older <code>runserver</code> you forgot about in another terminal window. Find it and <code>Ctrl+C</code> it. If you cannot find it, run <code>lsof -ti:8001 xargs kill</code> on macOS or Linux. Or move out of its way with a different port, <code>python manage.py runserver 8002</code> , but then you must also set <code>VITE_API_URL=http://127.0.0.1:8002/api</code> in <code>frontend/.env</code> , because the frontend falls back to 8001 when that variable is not set.
<code>pywatchman.SocketTimeout: timed out waiting for response</code>	Django's auto-reloader tried to use Watchman, a file-watching service, and Watchman did not answer in time. Two real fixes. Turn the reloader off: <code>python manage.py runserver --noreload</code> . The cost is that you now restart the server yourself after every Python edit. Or give Watchman longer: <code>DJANGO_WATCHMAN_TIMEOUT=20 python manage.py runserver 8001</code> . Both are in the project README.
<code>django.db.utils.OperationalError: (1045, "Access denied for user 'lms_user'@'localhost' (using password: YES)")</code>	MySQL is running, and it rejected the username or password. Look at the host in the quotes. If it says <code>@'localhost'</code> but you granted to <code>@'127.0.0.1'</code> , MySQL considers those different users. Re-run the <code>CREATE USER</code> and <code>GRANT</code> lines from section 14.1 exactly as written, or set <code>DB_USER</code> and <code>DB_PASSWORD</code> to credentials that do work.
<code>django.db.utils.OperationalError: (2002, "Can't connect to local MySQL server through socket '/tmp/mysql.sock' (2)")</code>	MySQL is not running at all. Nothing answered. Start it (<code>brew services start mysql</code> , <code>sudo systemctl start mysql</code> , or your control panel) and try again. On some setups the sibling error <code>(2003, "Can't connect to MySQL server on '127.0.0.1'")</code> appears instead. Same cause, same fix.
<code>ModuleNotFoundError: No module named 'MySQLdb'</code>	The MySQL driver is not installed in the environment you are running from . Two causes. Either you forgot <code>source .env/bin/activate</code> and are using your system Python, or you never ran <code>pip install -r requirements.txt</code> . Check for <code>(.env)</code> at the front of your prompt first, then reinstall. <code>mysqlclient==2.2.7</code> is the package that provides <code>MySQLdb</code> .
<code>pip install</code> fails while building <code>mysqlclient</code> , with lines like <code>Exception: Can not find valid pkg-config name.</code> or <code>fatal error: 'mysql.h' file not found</code>	<code>mysqlclient</code> is not pure Python. It compiles against MySQL's C header files, so those headers have to be on your machine before pip can build it. On macOS with Homebrew: <code>brew install mysql pkg-config</code> . On Debian or Ubuntu: <code>sudo apt install python3-dev default-libmysqlclient-dev build-essential pkg-config</code> . Then run <code>pip install -r requirements.txt</code> again.

TIP

When `runserver` prints a wall of red, read the **last** line first. That block is called a traceback: the list of function calls Python was inside when things went wrong, printed oldest first. The sentence that actually names your problem is at the bottom, not the top.

ERRORS FROM THE VITE TERMINAL AND THE BROWSER

WHAT YOU SEE	WHAT IT MEANS AND WHAT TO DO
Port 5180 is in use, trying another one...	Vite could not have 5180, so it quietly took 5181 (or 5182, and so on). The app loads fine. Then every API call fails, because 5181 is not in <code>CORS_ALLOWED_ORIGINS</code> . Best fix: free 5180 by closing whatever holds it, usually another Vite you left running. Second best: add the port Vite actually picked to <code>CORS_ALLOWED_ORIGINS</code> in <code>backend/lms/settings.py</code> , both the <code>localhost:</code> and the <code>127.0.0.1:</code> form, then restart Django.
In the browser console: Access to fetch at 'http://127.0.0.1:8001/api/teacher/' from origin 'http://localhost:5181' has been blocked by CORS policy: No 'Access-Control-Allow-Origin' header is present on the requested resource.	The same problem as the row above, seen from the browser's side. Django replied, but without a header saying "the page at <code>localhost:5181</code> is allowed to read this", so the browser threw the reply away before React ever saw it. Note that this is a browser rule, not a Django rule. The exact same request from the terminal with <code>curl</code> (a command-line tool that fetches a URL) succeeds. Fix the origin, or add it to the list in settings.
The red banner reads Could not reach the server. Is Django running on http://127.0.0.1:8001?	The <code>fetch()</code> call threw before any reply came back. Django is down, crashed, or was never started. Look at Terminal 2. This message comes from the <code>catch</code> around <code>fetch</code> in <code>frontend/src/api.js</code> , and it fires for network failures only, not for a 400 or a 403.
A blank white page in the browser and nothing at all in the Vite terminal	You are looking at the wrong port. Read the <code>Local:</code> line in Terminal 3 again and open that address.

The banner text is not generic. It is written into `frontend/src/api.js`:

```
// frontend/src/api.js (inside request)
let response;
try {
  response = await fetch(BASE_URL + path, { ...rest, headers });
} catch {
  throw new Error(
    "Could not reach the server. Is Django running on http://127.0.0.1:8001?",
  );
}
```

COMMON MISTAKE

Treating a CORS error as a Django bug and rewriting the view. Nothing is wrong with the view. Open the same URL directly in a browser tab, or run it through `curl`, and you will get a perfectly good reply. CORS is only ever about which **page** is allowed to read a reply, and the answer lives entirely in that one settings list.

ERRORS THE API SENDS BACK, SHOWN IN THE RED BANNER

These arrive as JSON, the plain-text format the API uses to send data, and are flattened into one readable line by `readError()` in `frontend/src/api.js`, which is why a rejected field reads `email: Enter a valid email address.` rather than `[object Object]`.

The three-digit numbers below are HTTP status codes: 400 means "the data you sent is wrong", 401 means "I do not know who you are", 403 means "I know who you are and the answer is still no".

WHAT THE BANNER SAYS	WHY, AND WHAT TO DO
Invalid phone or password	Either no <code>Profile</code> has that phone number, or the password does not match. The login view returns the same sentence for both on purpose, so that nobody can use the login form to discover which phone numbers exist. Check you typed the phone and not the username.
Everything suddenly fails with 401, and you are thrown back to <code>/login</code> with <code>Your session has expired. Please log in again.</code>	Your access token ran out. <code>ACCESS_TOKEN_LIFETIME</code> is 8 hours in <code>backend/lms/settings.py</code> , so this happens the next morning, not mid-task. There is no refresh endpoint on this backend , so nothing can renew the token quietly. <code>api.js</code> sees the 401, clears <code>localStorage</code> (the browser's own small store of saved values), fires a <code>lms:unauthorised</code> event, <code>AuthContext</code> hears it and drops the user, <code>ProtectedRoute</code> redirects. Log in again. That is the whole story, and section 14.6 explains how you would build the missing piece.
Only an admin can create accounts.	You reached <code>/api/register/</code> as a teacher or a student. That is the <code>message</code> on <code>IsAdmin</code> in <code>backend/backend/permissions.py</code> . The Accounts link is not even in your sidebar; you got here by typing the URL or by calling the API by hand. Log in as an admin.
Your role does not allow this change.	The base message on <code>RoleWritePermission</code> . In practice you will more often see one of the three specific overrides: <code>Only an admin can change teacher and student records.</code> , <code>Only a teacher or an admin can change course material and grades.</code> , or <code>Students can hand work in, but cannot change or remove a submission.</code> All of them mean the same thing: reading was fine, writing is not, for this role.
This phone number is already registered.	Raised by <code>validate_phone</code> on <code>RegisterSerializer</code> , because <code>Profile.phone</code> is <code>unique=True</code> and one phone number identifies exactly one login. Pick another number. If you genuinely want to reuse it, free it first at <code>http://127.0.0.1:8001/admin/backend/profile/</code> .
That username is already taken.	Same idea, from <code>validate_username</code> . Django's <code>User.username</code> is unique.
<code>submission: results</code> with this submission already exists.	The one-to-one rule. <code>Results.submission</code> is a <code>OneToOneField</code> , so a submission can be graded once. On the Results page you will not usually see this raw text, because <code>handleSave()</code> catches any message containing <code>already exists</code> and replaces it with <code>That submission has already been graded. Edit its result instead.</code> You will see the raw wording if you post to <code>/api/results/</code> yourself. Fix: edit the existing result with the pencil icon, or delete it and grade again.
A 400 listing several password complaints at once, for example <code>password: This password is too short. It must contain at least 8 characters. This password is too</code>	Django's four password validators from <code>AUTH_PASSWORD_VALIDATORS</code> all ran and all objected. They apply on the Accounts page, on the profile page's change-password form, and on password reset, because <code>validate_password()</code> is called in all three serializers. The fourth possible complaint is <code>The password is too similar to the username.</code> Choose a longer, less obvious password. Note the validators do not run on <code>createsuperuser</code> if you tell it to keep an insecure password anyway, which is why a superuser can have a password the Accounts form would reject.

WHAT THE BANNER SAYS	WHY, AND WHAT TO DO
common. This password is entirely numeric.	
Another account already uses that phone number. or Another account already uses that email address.	From <code>ProfileUpdateSerializer</code> when you edit your own profile. Email has to stay unique because password reset finds accounts by email.
That is not your current password.	The change-password form needs the old password, not just a live session. That is deliberate: a borrowed browser tab should not be enough to take an account over.

NOTE

A 403 with a sentence and a 400 with a field name are two different animals. 403 means "you are who you say you are, and you still may not do this" and comes from `backend/backend/permissions.py`. 400 means "the data you sent is wrong" and comes from a serializer, the class that checks incoming data field by field before it ever reaches the database. When you are debugging, open the Network tab in your browser's developer tools and read the status number before you read anything else.

ERRORS FROM `MAKEMIGRATIONS`

WHAT YOU SEE	WHAT IT MEANS AND WHAT TO DO
It is impossible to add a non-nullable field 'user' to teacher without specifying a default. This is because the database needs something to populate existing rows. followed by Please select a fix: and two numbered options	You added a required field to a model that already has rows in the database. Django has no idea what to put in that column for the rows that are already there, so it stops and asks. Option 2 quits so you can edit <code>models.py</code> , and that is nearly always the right choice while learning. Then either give the field <code>null=True</code> , <code>blank=True</code> , or give it a sensible <code>default=</code> . Re-run <code>makemigrations</code> afterwards.
<code>makemigrations</code> wants to rewrite the primary key of every single model, for no reason you can see	Already handled, do not undo it. A primary key is the id column every table gets. The existing migrations were generated on Django 6, where <code>BigAutoField</code> is the default primary key; Django 5.2 falls back to <code>AutoField</code> unless told otherwise. <code>DEFAULT_AUTO_FIELD = 'django.db.models.BigAutoField'</code> at the bottom of <code>backend/lms/settings.py</code> is what stops it. If you ever see this, check that line is still there.

COMMON MISTAKE

Deleting migration files to make an error go away. It works for about ten minutes and then leaves your database and your migration history describing two different worlds, which is far harder to untangle than the original problem. If you truly want to start over on a throwaway database, drop and recreate `lms_db` in MySQL and run `migrate` again, then redo the three steps in section 14.2.

14.5 Common beginner mistakes with this exact project

Fourteen things that will cost you an hour each if nobody warns you. All fourteen are specific to this codebase.

1. Logging in with a username. The form asks for a phone number. `LoginView` starts from `Profile.objects.get(phone=phone)`, so a username has literally nothing to match against. You get `Invalid phone or password` and spend twenty minutes doubting the password.

2. Reading `data.access` instead of `data.tokens.access`. The login view returns the tokens nested inside a `tokens` object:

```
# backend/backend/views.py (the end of LoginView.post)
return Response({
    'message': 'Login successful',
    'user_id': user.id,
    'username': user.username,
    # The frontend uses this to decide which buttons to show. It is
    # not what enforces anything: the permission classes do that.
    'role': role_of(user),
    'tokens': tokens
})
```

So `data.access` is `undefined`, you store `undefined` in `localStorage`, and every following request sends `Authorization: Bearer undefined` and gets a 401. The correct read is the one already in `AuthContext.jsx`: `saveSession(data.tokens.access, nextUser)`.

3. Expecting `data.results` from a list endpoint. This backend has no pagination configured, so `GET /api/teacher/` returns a plain JSON array. Writing `data.results.map(...)` because a tutorial elsewhere did throws `Cannot read properties of undefined (reading 'map')`. It is `data.map(...)` everywhere in this project.

4. Sending `enrollment_date` or `submitted_at` in a form. Both are `auto_now_add=True` on the model, which makes them read-only in the serializer. Sending them is silently ignored, so nothing errors and nothing happens, which is worse than an error. The tables display them; the forms never offer them. `Student.enrollment_date` is a different field and is editable, which is exactly why this trips people.

5. Forgetting the Bearer prefix. When you try a request by hand with `curl` or from the browser console, the header must be `Authorization: Bearer <token>`, with the space. `AUTH_HEADER_TYPES: ("Bearer",)` in the `SIMPLE_JWT` block is what defines that word. Sending the raw token, or `Token <token>`, gets you a 401 that looks identical to an expired session.

6. Editing `frontend/src/permissions.js` and expecting the server to change. That file decides which buttons are drawn. Nothing more. Its own first comment says so: "a mistake here is a cosmetic bug, not a hole". Add `STUDENT` to the `teacher` list and a student now sees an Add button that produces `Only an admin can change teacher and student records.` when clicked. The rules that matter are in `backend/backend/permissions.py`.

7. Trying to grade the same submission twice. `Results.submission` is a `OneToOneField`. The second attempt is a 400. The Results page hides already-graded submissions from the new-result dropdown, so you have to work at it, but you will manage it eventually by opening two tabs. Edit the existing result instead.

8. Running `npm run dev` from the wrong folder. There is no `package.json` in the repo root and none in `backend/`, so from either place npm cannot find the file that lists the scripts. You get `npm error code ENOENT` and a line ending `Could not read package.json`. The command belongs in `frontend/`, and only there.

9. Forgetting to activate the venv. Symptoms come in a set: `ModuleNotFoundError: No module named 'django'`, or `No module named 'MySQLdb'`, or `pip install` appearing to work while `runserver` still cannot find anything. The tell is your prompt. No `(.venv)` at the front, no venv. Every new terminal needs `source .venv/bin/activate` again; activation does not survive closing the window.

10. Running `migrate` against the wrong database. Django writes a `backend/db.sqlite3` the moment it is pointed at its default engine, and nothing in this project reads that file. If you set `DB_NAME` in one terminal and not another, or edit `DATABASES` to point at SQLite for a quick test, you end up with two databases and your data in whichever one you are not looking at. Before you panic that "my teachers disappeared", check which database you are actually connected to.

11. Assuming a `Teacher` row is a teacher's login. They are unrelated tables. Creating a Teacher row on the Teachers page does not create an account, and creating a teacher account on the Accounts page does not create a Teacher row. Nothing joins them. This is the single biggest conceptual gap in the project and section 14.6 opens with it.

12. Restarting Vite after changing `backend/lms/settings.py`. Wrong server. Settings belong to Django, so it is Terminal 2 that has to restart. Django usually reloads itself when a `.py` file changes, but if you started it with `--noreload` it will not, and you will keep seeing the old CORS list.

13. Expecting logging out to invalidate the token. There is no logout endpoint. `logout()` in `AuthContext.jsx` deletes the token from `localStorage` and nothing else. The token stays valid on the server until it expires, and if you had copied it beforehand it would still work. That is a real property of JWTs (JSON Web Tokens, the token format this project uses), not an oversight in the frontend.

14. Editing a role from your own profile page. `ProfileUpdateSerializer` accepts four fields: `first_name`, `last_name`, `email`, `phone`. Posting `{"role": "admin"}` is ignored, not obeyed, because the serializer never declared the field. Role changes happen in Django admin, by somebody who already has admin.

14.6 Where to go next

Five improvements to this exact codebase, smallest last. Each names the files that would change, so you can go and look at them before you start.

1. LINK `TEACHER` AND `STUDENT` TO `USER`

This is the big one, and it is the thing standing between the app you have and the app you probably imagined. Right now a `Teacher` row is a name and a subject, not a person who can log in, so the server cannot answer "whose submission is this?" or "which courses are mine?". Roles exist; ownership does not.

The change starts in `backend/backend/models.py`, adding something like a nullable `OneToOneField(User, null=True, blank=True, on_delete=models.SET_NULL)` to both `Teacher` and `Student`. Nullable matters: your existing rows have no user to point at, and a non-nullable field is exactly the `makemigrations` question from section 14.4. Then a new migration file in `backend/backend/migrations/`, then `backend/backend/serializers.py` to expose the link, then `backend/backend/permissions.py` for a check on the individual row rather than the whole table, then `get_queryset()` overrides in `backend/backend/views.py` so a student's list only contains their own rows. Once that exists, `SubmissionWrites` can stop refusing students outright and start allowing "edit your own".

2. ADD PAGINATION AND FILTERING

Every list endpoint currently downloads every row. The Dashboard fetches all eight lists on load just to count them. With two hundred students that is slow; with twenty thousand it is unusable.

Pagination means handing back one page of rows at a time. It is a few lines in the `REST_FRAMEWORK` block of `backend/lms/settings.py` (`DEFAULT_PAGINATION_CLASS` and `PAGE_SIZE`). The catch is that it changes the response shape from a plain array to `{count, next, previous, results}`, so `frontend/src/api.js` and all eight pages under `frontend/src/pages/` have to be updated together, and mistake number 3 above becomes the correct code instead of the wrong code.

Filtering means adding `django-filter` to `backend/requirements.txt`, registering it in settings, and setting `filterset_fields` on views like `LessonListCreateView` in `backend/backend/views.py`. Then `/api/lesson/?course=3` works, and the Assignments page can stop downloading every lesson in the school to show you four of them.

3. BUILD THE MISSING `/RESET-PASSWORD` PAGE

The backend half already exists. `PasswordResetRequestView` emails a link, `PasswordResetConfirmView` accepts `uid`, `token`, `new_password` and `confirm_password`, and `frontend/src/api.js` already exports `requestPasswordReset()` and `confirmPasswordReset()`. What is missing is the page the link points at.

You would add `frontend/src/pages/ResetPassword.jsx`, wire a public route for it in `frontend/src/App.jsx` next to `/login`, and read `uid` and `token` out of the query string. While you are there, look at `FRONTEND_PASSWORD_RESET_URL` in `backend/lms/settings.py`: it defaults to port **5173**, but this frontend runs on **5180**, so the emailed link points at the wrong place. Note also that `EMAIL_BACKEND` is the console backend, meaning the email prints into Terminal 2 rather than being sent. That is fine for development, and it is where you will find your test link.

4. ADD A TOKEN REFRESH ENDPOINT

`RefreshToken.for_user()` in `backend/backend/views.py` already mints a refresh token with a seven-day lifetime, and `LoginView` already returns it as `data.tokens.refresh`. Then the frontend throws it away: `saveSession(data.tokens.access, nextUser)` stores only the access token.

The endpoint itself is one line in `backend/backend/urls.py`, using `TokenRefreshView` from `rest_framework_simplejwt.views`. The interesting work is in `frontend/src/api.js`: store the refresh token in `saveSession()`, and change `handleExpiredToken()` so that instead of dumping the session it tries the refresh call once, and only gives up if that also fails. Do that and `ACCESS_TOKEN_LIFETIME` in settings can come back down from 8 hours to something sane.

5. LIST THE ACCOUNTS ON THE ACCOUNTS PAGE

`frontend/src/pages/Accounts.jsx` creates accounts and then forgets them. To see who exists, or change a role, or deactivate somebody, you leave the app for `http://127.0.0.1:8001/admin/backend/profile/`.

Making it a real page means a new list view in `backend/backend/views.py` guarded by `IsAdmin`, a serializer joining `User` and `Profile` in `backend/backend/serializers.py`, a path in `backend/backend/urls.py`, a call in `frontend/src/api.js`, and a `<Table>` under the existing form in `Accounts.jsx`. It is the same six steps as every resource page, which is the point: once you can do it once you can do it anywhere in this codebase.

What you have now

- A start-up routine you can repeat from memory: MySQL, then Django in an activated venv, then Vite, three terminals, three ports.
- The three-step path into an empty database, and a clear reason why `createsuperuser` on its own leaves you locked out of everything except `/admin/`.

- One full set of data in the app, created in foreign-key order, with a working teacher, student, course, enrollment, lesson, assignment, submission and grade.
- A troubleshooting table covering the real errors this project produces, from `That port is already in use.` through CORS blocks to `results with this submission already exists.`
- A list of fourteen specific traps, and enough understanding of why each one happens to spot the fifteenth on your own.
- Five concrete next projects, each with the files you would open first.

List of Figures

FIGURE	WHAT IT SHOWS
Figure 0.1	The login screen. Phone number and password only. No sign-up link, because accounts are handed out by an admin.
Figure 0.2	The same screen after a failed attempt. One red banner, one message, and no clue about which half was wrong.
Figure 0.3	The dashboard as an admin. Eight tiles, one per record type, each showing a live count. Clicking a tile opens that page.
Figure 0.4	The Teachers page. Title, subtitle, a count in the card header, the Add teacher button, and one row per teacher with pencil and trash buttons.
Figure 0.5	The Accounts page. One form, a Role dropdown, and a green banner after a successful save telling the admin which phone number the new person logs in with.
Figure 0.6	The profile page. Your details on the left, change password on the right, and a line saying your username and role are set by an admin.
Figure 0.7	The 404 page. A bordered card, nothing else.
Figure 2.1	The DRF browsable API page at <code>/api/teacher/</code> , refusing an anonymous visitor with a 401
Figure 3.1	The Django admin home page. Every model registered in <code>admin.py</code> appears under the BACKEND heading, alongside Django's own Groups and Users.
Figure 3.2	The Django admin "Add profile" form. Three fields: the User dropdown, the phone number you will log in with, and the Role dropdown showing Admin, Teacher and Student.
Figure 4.1	What <code>/api/teacher/</code> shows in a plain browser tab. A browser cannot attach the Authorization header, so the API answers 401 instead of the list above.
Figure 5.1	A student who forces a write through anyway gets the permission class's own message back from the API, shown here in the red banner.
Figure 6.1	The Teachers page as a student sees it. The list is there, because reading is open to any signed-in account. There is no Add button and the Action column is dashes.
Figure 6.2	The sidebar for a signed-in student. Every resource page is listed, because reading is open, but the Accounts link is gone.
Figure 8.1	The green "Teacher added." banner that <code>useFlash</code> clears on its own after four seconds.
Figure 10.1	The Teachers form: Input, Checkbox and two Buttons, all from the shared folder.
Figure 10.2	<code>ConfirmDialog</code> naming the exact teacher, with Cancel and Delete built from Button.

FIGURE	WHAT IT SHOWS
Figure 11.1	The Students page: a red or green banner slot at the top, an Add button, and one row per student.
Figure 11.2	Enrollments. The Student and Course columns are ids from the API, turned into names in the browser.
Figure 12.1	The Courses page. The Teacher column shows names, joined in the browser from the separate teacher list.
Figure 12.2	The Lessons page. Same table, same form, same buttons. Only the nouns changed.
Figure 12.3	The Assignments page. The Lesson dropdown only lists lessons from the chosen course, and Due shows local time.
Figure 12.4	The Submissions page. Content is cut at 60 characters and Submitted is a server-set timestamp.
Figure 12.5	The Results page. Each row names the student and the assignment behind the submission id.